A Formal Framework for AI Coding Agent Harness Architecture

First Author[1], Second Author[2], Third Author[2], Fourth Author[1] and Fifth Author[1]
[1]Pragnition Labs, [*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

AI coding agents now generate 40–60% of committed code on major platforms, yet harness architecture remains an ad hoc engineering discipline. We present a unified formal framework presenting eleven architectural dimensions: seven core pillars (Abstraction, Information, Reliability, Coordination, Temporal, Quality, Governance) addressing the semantic, execution, and assurance axes; three operational pillars (Economics, Human Interaction, Model Routing) addressing the practical constraints that bind deployment; and a Security pillar addressing the containment problem, since agents operate with developer credentials and the structural enforcement principle is fundamentally a security principle. Information theory provides a shared vocabulary across the seven pillars, with the information-theoretic framing serving as the primary analytical tool along the semantic axis (Abstraction, Information) and assurance axis (Quality, Governance). Along the execution axis (Reliability, Coordination, Temporal), the native formalisms (reliability theory, concurrency control, scheduling theory) are the generative analytical tools; information theory provides a compatible interpretation that enables cross-pillar connections but does not replace the domain-specific machinery. The abstraction gap measures the information distance between spec and code; context selection optimizes information delivery under degradation; quality defense detects information-destroying defects; and governance capacity bounds the rate at which architectural information can be preserved. Compound errors, coordination overhead, and temporal dynamics are captured through their respective domain-specific formalisms and connected to the information framework through shared quantities. Key cross-cutting results include: (i) compound error sensitivity, where per-step reliability p yields end-to-end reliability p^n with elasticity equal to n; (ii) structural enforcement dominance over instructional methods, with reliability gaps growing as $(1-\epsilon)^T$; (iii) a governance capacity bound establishing that when drift rate exceeds governance channel capacity, no scheme prevents unbounded divergence. The framework yields 11 unified design principles for harness architecture and identifies the empirical validation program required to ground these theoretical results.

1. Introduction

The landscape of software engineering has undergone a phase transition. Converging estimates place AI-generated code at 40–42% of production output (GitClear, 2025; SonarSource, 2026), with some platforms reporting higher figures. GitHub Copilot generates 46% of code for its users (GitHub, 2024). AI coding agents (Claude Code, OpenAI Codex, Cursor, Windsurf) have progressed from autocomplete to autonomous multi-step software engineering, reading codebases, planning changes, writing code, running tests, and iterating on failures.

Yet the *harness*, the surrounding architecture that mediates between human intent and agent execution, remains poorly understood. Harness design choices (what context to provide, when to verify, how to coordinate multiple agents, how to enforce architectural constraints) are made through practitioner intuition and trial-and-error rather than principled analysis. The consequences are visible in aggregate data: GitClear's analysis of 211 million changed lines found that code blocks with five or more duplicated lines increased approximately 8× (with overall duplication rates rising from 8.3% to 12.3%), and refactoring activity collapsed from 25% to under 10% of changed lines (GitClear,

2025). DORA 2025 reported that individual task completion increased 21% with AI adoption, while organizational-level throughput stayed flat and software delivery stability showed a *negative* correlation with AI usage (Google Cloud, 2025).

Thesis. This paper argues that harness architecture can and should be studied through formal methods, with models that yield testable predictions, design principles with information-theoretic justifications, and empirically calibrated parameters. We do not claim that the formal models presented here are complete or fully validated; rather, we claim that the *approach* of applying formal methods to harness design is productive, and we demonstrate this by developing a unified framework across eleven architectural dimensions. The framework applies most naturally to settings where specifications are written documents with stable content; the conversational, situated mode of AI agent interaction (Suchman, 1987) is acknowledged as partially outside the current scope (see Section 16.2).

The eleven pillars. The framework is organized into eleven pillars across five axes:

(i) **Abstraction**: the information-theoretic gap between specification and code, formalized as conditional Kolmogorov complexity $\mathcal{G}(S, P) = K(P \mid S)$.
(ii) **Information**: context selection under degradation as submodular optimization, with the Shannon decomposition of the abstraction gap.
(iii) **Reliability**: compound error models for multi-step agent pipelines, optimal verification scheduling, and the structural enforcement boundary.
(iv) **Coordination**: error amplification in multi-agent systems, task decomposition as balanced min-cut, and Quality-Adjusted Speedup.
(v) **Temporal**: Verified Iterations per Hour, speculative execution conditions, and cache-staleness tradeoffs.
(vi) **Quality**: 18 slop types in 3 detectability classes, layered defense optimization, and the Accretion Category.
(vii) **Governance**: governance capacity bounds, cascade control models, and the theory preservation problem.
(viii) **Economics**: token budget allocation as portfolio optimization, the CVIH metric, model tier selection, queueing economics, and the Jevons paradox for token markets.
(ix) **Human Interaction**: optimal attention allocation via Bayesian triage, trust calibration via Thompson sampling, the automation supervision paradox, and the human interaction budget.
(x) **Model Routing**: heterogeneous stage assignment, cross-model diversity via Gaussian copulas, the pipeline crossover theorem, adaptive escalation, and cascade routing.
(xi) **Security**: capability-based access control for agents, sandbox containment via the HRU model, information flow control adapted from Bell-LaPadula, credential isolation, adversarial robustness against reward hacking and prompt injection, and the structural enforcement boundary as a security principle.

Why eleven pillars, and how they decompose. The pillars decompose the harness design space along five axes. The *semantic axis* (Abstraction, Information) addresses what the agent needs to know. The *execution axis* (Reliability, Coordination, Temporal) addresses how the agent acts. The *assurance axis* (Quality, Governance) addresses how outcomes are verified and architectural coherence is maintained. The *operational axis* (Economics, Human Interaction, Model Routing) addresses the practical constraints that bind deployment: finite budgets, finite human attention, and heterogeneous model capabilities. The *security axis* (Security) addresses the containment problem: agents operate with

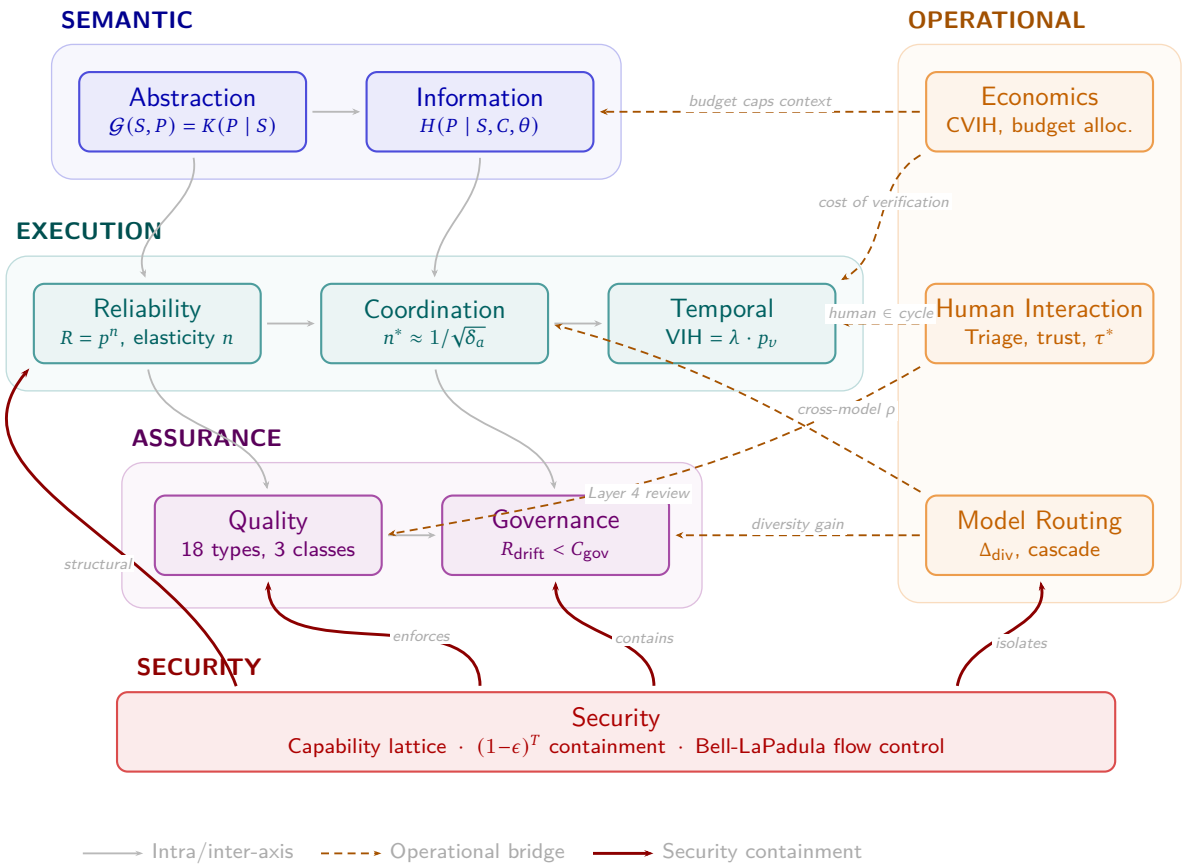

Figure 1 | The eleven pillars organized by five axes. The core pillars (left) flow top-to-bottom: specification (semantic), through execution, to assurance. The operational pillars (right) impose practical constraints. The security pillar (bottom) provides the structural containment boundary. All cross-axis connections use curved paths; edge labels indicate the primary mechanism at each bridge.

developer credentials, execute arbitrary code, and interact with external services, making security not an afterthought but a foundational constraint. The structural enforcement principle, the paper's strongest cross-pillar result, is fundamentally a security principle: invariants that must hold regardless of agent behavior can only be guaranteed through structural mechanisms, not instructions. The first seven pillars were developed from formal first principles; the operational three were identified through peer review; the security pillar was added after recognition that the structural enforcement boundary, credential isolation, and adversarial robustness concerns demanded a dedicated formal treatment grounded in access control theory, information flow control, and capability-based security. We do not claim that this decomposition is the only productive one, nor that it is exhaustive: user interface design, for example, is not addressed. We claim that this decomposition is *productive*, not that it is complete.

Scope and framing. This paper applies established formal machinery (information theory, reliability theory, control theory, computability theory, lattice theory, survival analysis) to the problem of harness architecture. The novelty lies in the *application*, not in the underlying mathematics. Where claims are conjectural or supported only by limited empirical evidence, we say so explicitly. We draw on practitioner discourse (blog posts, industry reports, preprints) where that discourse provides the best available evidence, and such sources are flagged accordingly.

Epistemic registers. The paper operates in three distinct registers, and readers should be aware of which register is active at any given point. *Mathematical facts* are results that follow from the definitions and standard theory (e.g., Compound Error Sensitivity: $\mathcal{E}(R, p) = n$ is a differentiation identity; the Shannon chain rule decomposition holds with exact equality). *Engineering hypotheses* are claims that depend on empirical parameters or assumed functional forms and are testable in principle but not yet tested (e.g., optimal context budget at $0.15W$–$0.40W$; the governance rate-stability threshold). *Philosophical observations* are conceptual claims about the nature of specification, tacit knowledge, or the limits of formalization (e.g., the Tacit Divergence Conjecture; the Uncanny Valley of Specification). We aim to flag transitions between registers; where we fail to do so, the default is engineering hypothesis.

Organization. Sections 2–8 develop the seven core pillars. Sections 9–11 develop the three operational pillars (Economics, Human Interaction, Model Routing). Section 12 develops the Security pillar. Section 13 identifies the cross-cutting connections that bind them into a unified framework. Section 14 assesses the empirical foundations. Section 15 presents design principles. Section 16 engages contrarian positions. Section 17 discusses limitations and open problems. Section 18 reviews related work. Section 19 concludes.

Notation conventions. Several symbols are reused across pillars with distinct meanings. To prevent confusion, we disambiguate here:

Sym.	Meaning (Pillar)	Sym.	Meaning (Pillar)
$\delta(\lvert C\rvert)$	Context degradation (Information)	δ_a	Agent defect rate (Coordination)
δ	Relaxation operator (Governance)	γ	Common-cause failure (Reliability)
γ_g	Drift propensity (Governance)	κ	Compression coord. (Abstraction)
κ	Coherency penalty (Coordination)	σ	Specificity coord. (Abstraction)
σ	Serialization fraction (Coordination)	ϵ	Instruction violation rate (Info., Rel.)
n	Pipeline length (Reliability)	n	Number of agents (Coordination)

Where ambiguity is possible, we use subscripts or clarify in context. The most important disambiguation is δ vs. δ_a: the former is a function of context size, the latter is a scalar defect rate.

2. The Abstraction Pillar

The Abstraction pillar addresses the foundational question: *what is the relationship between a specification and the code that implements it?* The answer has direct implications for harness design, because the harness is precisely the mechanism that mediates the translation from specification to code.

2.1. The Abstraction Gap

Definition 2.1 (Abstraction Gap)**.** *For a specification S and a target program P, the* abstraction gap *is defined as:*

$$\mathcal{G}(S, P) = K(P \mid S) \tag{1}$$

where $K(P \mid S)$ is the conditional Kolmogorov complexity of P given S: the length of the shortest program that produces P given S as input.

This definition captures an intuitive notion: the abstraction gap measures how much additional information is needed to go from the specification to the implementation. When S is vague ("build me a web app"), $K(P \mid S)$ is large. When S is a complete formal specification, $K(P \mid S)$ approaches zero.

The abstraction gap satisfies four basic properties, each following from standard results in algorithmic information theory (Li and Vitányi, 2019):

(a) **Minimality:** $\mathcal{G}(S, P) = O(1)$ if and only if S algorithmically determines P. This is immediate from the definition: a constant-length program suffices to produce P from S.
(b) **Maximality:** $\mathcal{G}(S, P) = K(P) + O(1)$ if and only if S provides no algorithmic information about P. This follows because ignoring S is always an option, so $K(P \mid S) \leq K(P) + O(1)$, with equality when the algorithmic mutual information $I(S : P)$ is negligible.
(c) **Monotonicity:** If $S \sqsubseteq S'$ in the refinement ordering, then $\mathcal{G}(S', P) \leq \mathcal{G}(S, P) + O(\log n)$, where $n = \max(|S|, |S'|, |P|)$. Under constructive refinement (where a bounded-length computable function maps S' to S), the bound tightens to $O(1)$.
(d) **Additivity:** For any intermediate representation S', $\mathcal{G}(S, P) \leq \mathcal{G}(S, S') + \mathcal{G}(S', P) + O(\log n)$, by the chain rule for Kolmogorov complexity.

Remark. *$K(P \mid S)$ is uncomputable. This is a fundamental limitation of the Kolmogorov formalization. The Information pillar (Section 3) operationalizes the gap using Shannon entropy $H(P \mid S)$, which is computable from data. The Kolmogorov formulation remains valuable as a theoretical reference point: it provides clean bounds without distributional assumptions and connects to Solomonoff's theory of algorithmic prediction.*

2.2. Normalised Information Distance and Universality

Following Li et al. (2004), we define the canonical metric on the abstraction spectrum:

Definition 2.2 (Abstraction Distance)**.** *The* abstraction distance *between specification S and program P is the normalised information distance:*

$$d(S, P) = \frac{\max(K(S \mid P),\ K(P \mid S))}{\max(K(S),\ K(P))} \tag{2}$$

The universality theorem of Li et al. (2004) establishes that d is a metric (up to $O(\log n/n)$ terms) and is *universal*: for every computable normalised distance $D : \{0,1\}^* \times \{0,1\}^* \to [0,1]$, we have $d(x, y) \leq D(x, y) + O(1/n)$. This means $d(S, P)$ minorises every other reasonable distance measure between specification and code. The metric takes values in $[0, 1]$: $d \approx 0$ means the specification and code are informationally equivalent; $d \approx 1$ means they share no algorithmic content.

For harness design, universality implies that the normalised information distance is the tightest possible single-number summary of the specification-code relationship. Any domain-specific metric (function point ratios, test coverage gaps, embedding distances) can be no smaller than d, making it a useful lower bound even though it is uncomputable.

2.3. Spec-Code Convergence

The central theorem of the Abstraction pillar formalizes the thesis of Gonzalez (2026) that "a sufficiently detailed spec is code."

Theorem 2.1 (Spec-Code Convergence)**.** [Mathematical Fact: follows from chain rule for Kolmogorov complexity plus incompressibility.] *For an incompressible program P (i.e., $K(P) \geq |P| - O(\log |P|)$), any specification S satisfying $K(P \mid S) \leq c$ for constant c must satisfy:*

$$|S| \geq |P| - O(\log |P|) \tag{3}$$

That is, a specification that closes the abstraction gap for an incompressible program must be approximately as long as the program itself.

Proof sketch. By Chaitin's incompleteness theorem (Chaitin, 1966), an N-bit formal system can determine at most $N + O(1)$ bits of an algorithmically random string. Since P is incompressible, $K(P) \geq |P| - O(1)$. The assumption $K(P \mid S) \leq c$ (constant) implies, via the chain rule, that $K(P) \leq K(S) + K(P \mid S) + O(\log n) \leq K(S) + O(1)$. Therefore $K(S) \geq K(P) - O(1) \geq |P| - O(1)$. Since $K(S) \leq |S| + O(1)$ by definition, we conclude $|S| \geq |P| - O(\log |P|)$. □

This theorem has a sharp design implication: for complex, novel code (which is approximately incompressible), there is no free lunch. The information must come from somewhere. The harness designer's choice is not whether to provide this information, but *how*: through the specification, through context (codebase files, documentation, examples), or through the model's prior (training data).

Remark. *The incompressibility assumption is important. Most real programs are highly compressible: Hindle et al. (2012) showed that code has lower entropy than English prose. For compressible programs (the common case), specifications can be dramatically shorter than implementations, and the "no free lunch" implication is correspondingly weaker. The theorem's relevance is strongest for novel, domain-specific logic that cannot be predicted from the model's training distribution.*

A concrete example. Consider a sorting function: the specification "sort this list in ascending order" is a few tokens, while any correct implementation is tens of lines. This works because sorting algorithms are maximally compressible relative to the model's prior (they appear millions of times in training data). Now consider a proprietary pricing algorithm with 47 business rules, exception handling for 12 regional tax regimes, and a legacy discount table. This logic is approximately incompressible relative to the model's prior, and the Spec-Code Convergence Theorem predicts that any specification sufficient to generate it correctly must encode essentially all 47 rules, all 12 regimes, and the full discount table. There is no shortcut.

2.4. The Shannon Channel Model

The Abstraction pillar models spec-to-code translation as communication through a noisy channel. The specification S is the message, the program P is the received signal, and the agent (with its model parameters θ and context C) is the channel.

The channel capacity is:

$$C(\theta, C) = \max_{p(S)} I(S; P) \tag{4}$$

where the mutual information $I(S; P)$ quantifies how much of the specification's information content survives translation into code. The abstraction gap represents the information lost in transit. When $\mathcal{G}(S, P) > C(\theta, C)$, the channel cannot faithfully transmit the specification, and the harness must either enrich the context C, improve the model θ, or decompose the task into sub-problems whose individual gaps fall within channel capacity.

The relationship between the Kolmogorov framework and the Shannon channel model is complementary. The Kolmogorov formulation ($K(P \mid S)$) is distribution-free and yields the Convergence Theorem, but it is uncomputable. The Shannon formulation ($H(P \mid S)$) is estimable from LLM log-probabilities, the chain rule holds with exact equality (no $O(\log n)$ error terms), and the probabilistic framework fits LLM-mediated translation naturally. For sequences typical according to a source, Kolmogorov complexity is asymptotically close to Shannon code length (Vitányi, 2004). We view the Kolmogorov framework as the theoretical ceiling and the Shannon model as the measurable, falsifiable floor.

2.5. The Refinement Lattice and Galois Connections

Specifications and programs form a partially ordered set under a refinement relation $\sqsubseteq$, where $S_1 \sqsubseteq S_2$ means "S_2 refines S_1" (is more specific). This partial order forms a complete lattice (Back, 1980; Morgan, 1994) with:

- $\top = \mathrm{abort}$: establishes no postcondition (maximally abstract; any program satisfies it).
- $\bot = \mathrm{magic}$: establishes every postcondition (impossible perfect program).
- $S_1 \sqcap S_2$ (meet): demonic choice.
- $S_1 \sqcup S_2$ (join): angelic choice.

A concrete, deterministic, feasible program P sits near the bottom of this lattice. "A sufficiently detailed spec is code" becomes: sufficiently many refinement steps reach a deterministic, feasible element of the lattice.

The Galois connection tower. Following Cousot and Cousot (1977), each level of the abstraction spectrum corresponds to a Galois connection. An *abstraction layer* between concrete domain $(C, \sqsubseteq_C)$ and abstract domain $(A, \sqsubseteq_A)$ is a pair (α, γ) where $\alpha : C \to A$ (abstraction) and $\gamma : A \to C$ (concretization) satisfy: $\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(a)$.

The abstraction spectrum is then a tower of Galois connections:

$$C \rightleftarrows A_1 \rightleftarrows A_2 \rightleftarrows \cdots \rightleftarrows A_n \tag{5}$$

where C is executable code and A_n approaches natural language. Each layer discards information; the composition $\alpha_n \circ \cdots \circ \alpha_1$ maps code to its most abstract description. Information loss is monotonic: once a Galois connection discards a distinction, no subsequent layer can recover it.

Concrete example. Consider a web application's authentication module. The tower might look like:

- A_4: "Users can log in" (natural language, $\sigma \approx 0.1$)
- A_3: "OAuth 2.0 with PKCE, session timeout 30 min" (architectural spec, $\sigma \approx 0.5$)
- A_2: Typed interface with pre/postconditions (contract, $\sigma \approx 0.7$)
- A_1: Implementation with concrete cryptographic choices ($\sigma \approx 0.9$)
- C: Executable code ($\sigma = 1.0$)

Each α step from C upward discards implementation detail; each γ step downward requires the agent to supply that detail from prior or context. The abstraction gap at each layer measures the information that must be supplied to descend one level.

Rigorous construction for two adjacent pairs. We now construct the Galois connections explicitly for the lower portion of the tower, where lattice structure is well-defined, addressing the gap between assertion and construction.

Proposition 2.1 (Interface–Implementation Galois Connection)**.** *Let $(A_1, \sqsubseteq_1)$ be the poset of executable implementations of a given type signature, ordered by trace inclusion (where $p_1 \sqsubseteq_1 p_2$ iff every execution trace of p_1 is also a trace of p_2). This forms a complete lattice: meets are trace-set intersections, joins are trace-set unions, $\top$ is the chaotic program (all traces), and $\bot$ is the infeasible program (no traces). Let $(A_2, \sqsubseteq_2)$ be the poset of contracts (pre/postcondition pairs), ordered by contract refinement: $(P_1, Q_1) \sqsubseteq_2 (P_2, Q_2)$ iff $P_2 \Rightarrow P_1$ and $Q_1 \Rightarrow Q_2$ (contravariant in preconditions, covariant in postconditions). Define:*

$$\alpha(p) = \big(\mathrm{wp}(p),\ \mathrm{sp}(p)\big) \tag{6}$$

$$\gamma(c) = \bigsqcup\nolimits_1 \{p \in A_1 : p \text{ satisfies } c\} \tag{7}$$

where $\mathrm{wp}(p)$ is the weakest liberal precondition and $\mathrm{sp}(p)$ is the strongest postcondition of p. Then (α, γ) forms a Galois connection: $\alpha(p) \sqsubseteq_2 c \iff p \sqsubseteq_1 \gamma(c)$.

Proof sketch. ($\Rightarrow$) Suppose $\alpha(p) \sqsubseteq_2 c$, i.e., the strongest contract of p refines c. Then p satisfies c (any program satisfies every contract that its strongest contract refines). Hence $p \in \{p' : p' \text{ satisfies } c\}$, so $p \sqsubseteq_1 \bigsqcup_1 \{p' : p' \text{ satisfies } c\} = \gamma(c)$ by definition of the join. ($\Leftarrow$) Suppose $p \sqsubseteq_1 \gamma(c)$. Since $\gamma(c)$ satisfies c (the join of all implementations satisfying c still satisfies c, because trace inclusion is closed under union and contract satisfaction is downward-closed in the trace ordering), and p refines $\gamma(c)$, monotonicity of α gives $\alpha(p) \sqsubseteq_2 \alpha(\gamma(c)) \sqsubseteq_2 c$.

That α is monotone follows from the fact that trace inclusion preserves pre/postcondition strength. That γ is monotone follows from the fact that a weaker contract admits a (weakly) larger set of implementations, hence a (weakly) larger join. The standard argument that $\alpha \circ \gamma \sqsubseteq_2 \mathrm{id}$ and $\mathrm{id} \sqsubseteq_1 \gamma \circ \alpha$ (the closure/kernel conditions) follows directly (Back, 1980; Cousot and Cousot, 1977). □

Proposition 2.2 (Architecture–Interface Galois Connection)**.** *Let $(A_2, \sqsubseteq_2)$ be the contract lattice above. Let $(A_3, \sqsubseteq_3)$ be the poset of architectural specifications (protocol choices, component structure, timing constraints), ordered by constraint entailment: $a_1 \sqsubseteq_3 a_2$ iff every constraint in a_1 is entailed by a_2. Define $\alpha' : A_2 \to A_3$ as the projection that extracts the architectural decisions from a typed interface (protocol choice, data-flow structure, timing bounds), discarding type-level detail. Define $\gamma' : A_3 \to A_2$ as the map sending an architectural spec to the most general contract consistent with it (imposing only those type-level constraints derivable from the architectural spec). Then (α', γ') forms a Galois connection under the assumption that architectural constraints are monotonically preserved by interface refinement: adding more specific types cannot violate architectural constraints that less specific types satisfied.*

Proof sketch. The argument mirrors Proposition 2.1. Monotonicity of α' holds because refining an interface (adding stronger contracts) can only preserve or strengthen the architectural constraints it satisfies. Monotonicity of γ' holds because weakening an architectural spec enlarges the set of consistent interfaces. The Galois condition $\alpha'(i) \sqsubseteq_3 a \iff i \sqsubseteq_2 \gamma'(a)$ then follows by the same join argument: $\gamma'(a)$ is the join of all interfaces consistent with a, and $\alpha'(i) \sqsubseteq_3 a$ iff i is consistent with a iff $i \sqsubseteq_2 \gamma'(a)$. The construction is sound but depends on the monotonicity assumption, which holds for structural and protocol constraints but may fail for constraints that interact non-monotonically with type refinement (e.g., certain real-time scheduling constraints where added specificity can introduce new timing conflicts). □

Remark (The natural-language boundary)**.** *The tower is rigorous from C through A_2 (Proposition 2.1), constructive but assumption-dependent at A_3 (Proposition 2.2), and necessarily informal at A_4. Natural*

language lacks the structure required for a Galois connection: there is no canonical partial order on NL utterances (no well-defined refinement relation), no meet or join operations, and the "abstraction" from typed interfaces to NL is a many-to-one mapping without a canonical inverse (γ is not well-defined because the same NL sentence can correspond to arbitrarily many distinct typed interfaces). We retain A_4 in the tower for conceptual completeness, with the understanding that the $A_3 \rightleftarrows A_4$ arrow is a metaphorical, not mathematical, Galois connection. The formal content of the tower resides in the lower layers.

Remark (Curry-Howard-Lambek connection)**.** *The Curry-Howard-Lambek correspondence (Lambek and Scott, 1986; Martin-Löf, 1979; Wadler, 2015) provides a deeper structural reading of the abstraction gap: under this correspondence, a specification is a proposition (or type) and an implementation is a proof (or term inhabiting that type). The abstraction gap $K(P \mid S)$ then measures the proof complexity of the proposition corresponding to S. A specification that is "hard to implement" is a proposition that is "hard to prove." The seL4 microkernel illustrates this: the C implementation is 8,700 lines, but the Isabelle/HOL proof of functional correctness is approximately 200,000 lines. We note this connection but do not develop it further; a full treatment connecting Kolmogorov complexity to proof complexity lower bounds (in the sense of Cook-Reckhow) would be a substantial contribution in its own right and is left to future work.*

2.6. The Specificity-Compression Space

The Abstraction pillar introduces a two-dimensional metric for characterizing positions on the abstraction spectrum:

Definition 2.3 (Specificity-Compression Coordinates)**.** *For any representation R (specification, code, or intermediate form):*

$$\sigma(R) = 1 - \frac{K(P \mid R)}{K(P)} \in [0, 1] \quad \textit{(specificity)} \tag{8}$$

$$\kappa(R) = 1 - \frac{|R|}{K(P)} \in (-\infty, 1] \quad \textit{(compression)} \tag{9}$$

where σ measures how much of the program's information content is captured, and κ measures how concisely it is expressed relative to the program's inherent complexity.

The Spec-Code Convergence Theorem (Theorem 2.1) implies that for incompressible programs, the Pareto frontier in (σ, κ) space passes through the point $(1, 0)$: full specificity requires approximately zero compression. Natural language specifications occupy the high-κ, low-σ region (concise but ambiguous); executable code occupies the high-σ, low-κ region (specific but verbose).

Thinker placement (illustrative, not quantitative). The two-dimensional framework organizes the positions of major thinkers qualitatively. **The placements below are ordinal illustrations, not measurements.** Since both coordinates derive from Kolmogorov complexity (defined up to a machine-dependent additive constant), the numerical values are not stable across formalizations; only the relative ordering is robust. The table serves a pedagogical function (showing that different specification philosophies occupy different regions of the space), not an analytical one:

Thinker	$\hat{\sigma}$	$\hat{\kappa}$	Framework
Dijkstra	0.9–1.0	0.0–0.2	Weakest precondition
Hoare	0.85–0.9	0.2–0.4	Axiomatic specs $\{P\}S\{Q\}$
Lamport	0.75–0.8	0.8–0.9	TLA+
Knuth	0.6–0.65	~0	Literate programming (WEB)
Brooks	variable	variable	Essential complexity

Remark (Additive constant caveat)**.** *The $(\hat{\sigma}, \hat{\kappa})$ values are ordinal estimates, meaningful as relative rankings rather than absolute positions. Since both coordinates are derived from Kolmogorov complexity (which is defined up to a machine-dependent additive constant), small differences between thinkers may not be stable across formalizations. The ordering (Dijkstra most specific, Knuth moderate, natural language least specific) is robust to the additive constant; the precise numerical values are not. The Shannon operationalization ($H(P|S)$ in Section 3) is computable but model-dependent: the "abstraction gap" becomes a property of the model-specification pair rather than an intrinsic property of the specification alone.*

Dijkstra ($\sigma \approx 0.95$, $\kappa \approx 0.1$) demanded maximum specificity at the cost of compression: formal proofs that are as long as (or longer than) the programs they verify. Hoare ($\sigma \approx 0.87$, $\kappa \approx 0.3$) achieved a practical balance through axiomatic triples $\{P\}S\{Q\}$ that are shorter than full proofs but still mechanically checkable. Lamport ($\sigma \approx 0.78$, $\kappa \approx 0.85$) occupies a remarkable position: TLA+ specifications are both concise *and* precise, achieving high compression through mathematical notation while retaining high specificity through model checking. This explains why TLA+ and English have similar lengths for the same system but radically different implementation reliability (Newcombe et al., 2015). Knuth ($\sigma \approx 0.63$, $\kappa \approx 0$) pursued literate programming, where the specification *is* the code, interleaved with prose; the result has moderate specificity (the prose adds intent) but zero compression (the artifact is at least as long as the code). Brooks resisted fixed placement, arguing that the appropriate level depends on the essential complexity of the problem.

2.7. The Uncanny Valley of Specification

The thinker placements reveal a hypothesis: the range $\sigma \in [0.3, 0.45]$ is sparsely populated, forming an "uncanny valley" of specification. Specifications in this zone are formal enough to feel constraining but not formal enough to provide mechanical guarantees. This is the zone of semi-formal notations (UML, most design documents) that are widely criticized as providing the worst of both worlds (Jackson, 2021).

The formal framework offers an explanation: this zone maximizes cognitive cost (the spec requires careful reading) while minimizing verification benefit (the spec cannot be mechanically checked). Below $\sigma \approx 0.3$, natural language is cheap to write and the model's prior fills the gap. Above $\sigma \approx 0.45$, formal methods provide mechanical verification that justifies the authoring cost. Between these bounds, neither advantage obtains. We emphasize that this is a hypothesis derived from ordinal placements, not a measured finding; a controlled study comparing developer productivity across formalism levels would be needed to test it.

The harness designer's challenge is to move the operating point along the Pareto frontier. AI agents shift the frontier itself, enabling higher σ at given κ by leveraging prior knowledge θ to fill the gap. The Information pillar quantifies this shift precisely.

3. The Information Pillar

The Information pillar operationalizes the abstraction gap using Shannon entropy and addresses the engineering problem of delivering the right information to the agent at the right time.

3.1. Context Selection as Submodular Optimization

Given a codebase of n units $U = \{u_1, \ldots, u_n\}$ with total token size $T \gg W$ (the context window capacity), the context selection problem is:

$$C^* = \arg \max_{C \subseteq U,\ |C| \leq W} f(C) \cdot (1 - \delta(|C|)) \tag{10}$$

where $f(C) = I(C; P \mid S, \theta)$ is the context relevance function (mutual information between context and target program) and $\delta(|C|)$ is the empirically observed degradation function.

Submodularity. The relevance function f is submodular under mild conditions, meaning it exhibits diminishing returns: adding a codebase unit to a small context set yields more information gain than adding it to a large one.

Proof sketch for submodularity. Write $f(C) = H(P \mid S, \theta) - H(P \mid S, C, \theta)$. The first term is constant with respect to C, so f is submodular if and only if $g(C) = H(P \mid S, C, \theta)$ is supermodular. Supermodularity of g states that for $A \subseteq B \subseteq U$ and $u \notin B$:

$$g(A \cup \{u\}) - g(A) \leq g(B \cup \{u\}) - g(B)$$

Equivalently, the marginal reduction in entropy from adding u is larger for smaller sets (diminishing returns of information). Sufficient conditions include: (a) conditional independence of codebase units given (P, S, θ); (b) jointly Gaussian distribution of (U, P) given (S, θ) (Krause and Golovin, 2008); (c) log-supermodular conditional density $p(P, C \mid S, \theta)$. □

Under submodularity, the greedy algorithm (repeatedly add the unit with highest marginal gain) achieves a $(1 - 1/e)$ approximation guarantee for the monotone case.

Code dependencies break submodularity. When codebase units have strong complementarities (for example, an interface definition u_i and its implementation u_j), $f(\{u_i, u_j\}) > f(\{u_i\}) + f(\{u_j\})$, violating the diminishing returns property. In practice, such complementarities are localized within modules while diminishing returns dominate globally. The *submodularity ratio* γ_f (Das and Kempe, 2011) quantifies this: when $\gamma_f < 1$ but bounded away from zero, greedy algorithms achieve a $(1 - e^{-\gamma_f})$ approximation, degrading gracefully rather than catastrophically.

The degradation-adjusted objective. The product $\tilde{f}(C) = f(C) \cdot (1 - \delta(|C|))$ is *non-monotone*: beyond a certain context size, adding more units decreases overall performance despite their positive information content. This is because the degradation penalty $\delta(|C|)$ grows faster than the marginal information gain. The non-monotone setting requires different algorithmic treatment (randomized greedy or multilinear extension methods) with a $1/2$-approximation guarantee (Buchbinder et al., 2015).

Empirical evidence from five independent studies (Chroma, 2025; Du et al., 2025; Liu et al., 2024) supports a power-law degradation model $\delta(n) = (n/W_{\text{eff}})^{\alpha_d}$ with $\alpha_d \in [0.3, 0.5]$. The power law captures steep early degradation followed by a flattening tail, fitting observed data better than exponential or sigmoid alternatives.

3.2. The U-Shaped Positional Recall Function

Liu et al. (2024) established that models attend strongly to context boundaries but miss information buried in the middle. We model this as a double-exponential with length-dependent floor:

$$\text{recall}(i, n) = \phi_0 \left(\frac{n_0}{n}\right)^{\gamma} + A_p \cdot e^{-\lambda_p \cdot i} + A_r \cdot e^{-\lambda_r \cdot (n-i)} \tag{11}$$

where ϕ_0 is the baseline floor at reference length n_0, γ controls floor collapse with length, and (A_p, λ_p), (A_r, λ_r) parameterize primacy and recency effects respectively. Fitting to the 20-document QA data of Liu et al. (2024): $\phi_0 = 0.55$, $A_p = 0.20$, $\lambda_p = 0.26$, $A_r = 0.17$, $\lambda_r = 0.15$, $\gamma = 0.23$.

The primacy effect is grounded in the attention sink mechanism: Xiao et al. (2024) showed that the first 4 tokens absorb over half the total attention mass regardless of content, with perplexity exploding $955\times$ when they are removed. The practical implication is that context *ordering* matters as much as content: highest-relevance items should be placed at context boundaries (beginning and end), with lowest-relevance items in the middle.

3.3. The Abstraction Gap Decomposition

The Information pillar decomposes the abstraction gap using Shannon's chain rule:

Proposition 3.1 (Abstraction Gap Decomposition)**.** [Mathematical Fact: exact equality via Shannon chain rule.]

$$H(P \mid S) = \underbrace{H(P \mid S, C, \theta)}_{\textit{residual uncertainty}} + \underbrace{I(P; C \mid S, \theta)}_{\textit{context contribution}} + \underbrace{I(P; \theta \mid S)}_{\textit{prior contribution}} \tag{12}$$

This decomposition holds with exact equality (no approximation error) and all three terms are estimable from LLM log-probabilities.

Proof sketch. Apply the chain rule twice. Step 1: $H(P \mid S) = H(P \mid S, \theta) + I(P; \theta \mid S)$, using $H(X \mid Y) = H(X \mid Y, Z) + I(X; Z \mid Y)$. Step 2: $H(P \mid S, \theta) = H(P \mid S, C, \theta) + I(P; C \mid S, \theta)$, applying the same identity. Substituting step 2 into step 1 yields the result. Each step is an instance of the Shannon chain rule, which holds with exact equality (unlike the Kolmogorov analog, which incurs $O(\log n)$ error terms). □

The three terms have distinct interpretations and design implications:

- **Residual uncertainty** $H(P \mid S, C, \theta)$: bits the model cannot determine even with context and prior. These are genuinely novel design decisions that require human input or iterative refinement.
- **Context contribution** $I(P; C \mid S, \theta) = f(C)$: bits the context provides beyond specification and prior. This is what context engineering optimizes.
- **Prior contribution** $I(P; \theta \mid S)$: bits the model's training data provides. For common patterns, this term dominates.

The decomposition reveals a bimodal structure. For common patterns where the model's training data provides strong priors, $I(P;\theta \mid S)$ dominates and context matters little. For novel logic, the prior term is small, $I(P;C \mid S,\theta)$ dominates, and context selection becomes the critical lever. This bimodality explains why AI agents can generate boilerplate effortlessly but struggle with domain-specific logic: the abstraction gap is not uniform across code types.

Empirical support comes from Gloaguen et al. (2026): LLM-generated context files *reduced* success rates by 3% while increasing costs by 20%, because for common patterns the prior already supplies the needed information and additional context creates distraction. Human-written files helped only 4%, and only when existing documentation was sparse.

3.4. The Context Budget Theorem

Theorem 3.1 (Optimal Context is Sub-Capacity)**.** [Engineering Hypothesis: depends on power-law degradation model fit to limited data.] *Under the power-law degradation model, the optimal context size satisfies $|C^*| < W$, with practical estimates of $|C^*| \approx 0.15W$ to $0.40W$ depending on task type.*

Proof sketch. By contradiction. If $|C^*| = W$, consider removing the unit u_{last} with smallest marginal information gain from C^*. The resulting $C' = C^* \setminus \{u_{\text{last}}\}$ satisfies $\tilde{f}(C') > \tilde{f}(C^*)$ whenever the marginal degradation cost of including u_{last} exceeds its marginal information gain. Under the marginal exhaustion condition (which holds because f has diminishing returns while δ is convex and accelerating), this inequality holds at $|C| = W$, contradicting the assumed optimality of C^*.

The continuous relaxation yields an elasticity-matching condition at the optimum:

$$\frac{F'(x^*)}{F(x^*)} = \frac{\delta'(x^*)}{1-\delta(x^*)}$$

where $F(x) = \max_{C:|C|=x} f(C)$. The relative rate of information gain must equal the relative rate of degradation increase. □

This result is counterintuitive: more context is not better context. Every frontier model tested by Chroma (2025) (18 models across four providers) exhibited monotonic performance degradation as input length increased. Even whitespace padding (zero semantic content) degrades performance by 13.9% to 85% (Du et al., 2025). Context is a scarce resource requiring budgeting, not a bucket to be filled.

Remark (Cost as a hard constraint)**.** *The optimal context budget $|C^*|$ derived above is quality-optimal, but production harnesses face a financial constraint as well. At current pricing (roughly \$0.003–\$0.015 per 1K input tokens depending on provider and model), a 200K-token context costs \$0.60–\$3.00 per call. A team running 50 agent iterations per task at full context spends \$30–\$150 per task. In practice, the binding constraint on context size is often financial rather than degradation-based, and the effective optimal context budget may be smaller than the quality-optimal $|C^*|$. The framework presented here does not model cost optimization; integrating financial constraints with the degradation-adjusted objective is an important direction for practical deployment.*

For a 200K-token window performing code generation, the optimal context budget is approximately 30K–80K tokens. The remaining capacity is not merely wasted if filled; it is *actively harmful*. This is consistent with reports from JetBrains suggesting that removing a substantial fraction of context tokens can improve performance (JetBrains, 2025).

Remark (Developer familiarity as a moderating variable)**.** *The optimal context budget $|C^*|$ is presented as a function of task type and model capability, but empirical evidence suggests that developer familiarity with the codebase is a strong moderator. Developers highly familiar with a codebase benefit from smaller context budgets ($|C^*| \approx 0.08W$–$0.15W$) because their existing mental model acts as supplementary context outside the window. Developers new to a codebase require larger budgets ($|C^*| \approx 0.35W$–$0.55W$). The framework does not model this moderating variable; incorporating it would require a joint model of in-window context and developer prior knowledge, which we leave as an important direction for future work.*

3.5. The Context Curation Proposition

Proposition 3.2 (Curation Beats Accumulation)**.** *Let C_s (curated, $|C_s| \ll W$) and C_l (uncurated, $|C_l| \approx W$) be two context configurations with information densities $\rho(C) = \sum v(f_i)/|C|$. The curated configuration delivers higher effective information whenever:*

$$\rho(C_s) \cdot |C_s| \cdot (1 - \delta(|C_s|)) > \rho(C_l) \cdot |C_l| \cdot (1 - \delta(|C_l|)) \tag{13}$$

Each additional item must clear a rising inclusion bar: the marginal value must exceed $[\delta(|C| + |\Delta|) - \delta(|C|)]/[1 - \delta(|C| + |\Delta|)] \cdot V_{eff}(C)$. As $|C|$ grows and δ steepens, this bar rises rapidly.

This proposition formalizes the principle that a small, carefully selected context outperforms a large, undiscriminating one. It explains the convergence of production harnesses toward minimal always-loaded context: HumanLayer's CLAUDE.md under 60 lines (HumanLayer, 2025), OpenAI Codex's AGENTS.md at roughly 100 lines (OpenAI, 2025).

3.6. Fresh Context Dominance

Proposition 3.3 (Fresh Context Dominance)**.** *Let $\{S_1, \ldots, S_T\}$ be a task sequence with diversity $d = \mathbb{E}_{i \neq j}[1 - \cos(v_i, v_j)] > 0$. Under accumulated context, relevance density ρ_t decays monotonically: each Δ_i (context generated for task S_i) has high value for S_i but low expected value for subsequent tasks S_t ($t \neq i$), so the numerator $\sum v_t(f_i)$ grows slowly while the denominator $|C_t|$ grows by at least $|\Delta_t|$ per step. Under fresh context, ρ_t is constant (re-selected each time). Fresh context dominates in total effective information for $T > T^* = c_{select}/\mathbb{E}[\Delta V_t]$.*

Production harnesses set $T^* = 1$ in practice: GSD never accumulates, Codex tears down containers per task, and RAPID isolates agents in separate worktrees. This suggests the selection cost c_{select} is small relative to degradation costs.

3.7. Three-Tier Information Architecture

Production harnesses have converged on a three-tier information model:

1. **Active tier**: always-loaded context (project rules, CLAUDE.md, recent files). Size budget: 5–15% of W.
2. **Searchable tier**: on-demand retrieval via tools (grep, glob, semantic search). Unbounded in principle, but each retrieval consumes tokens from the active budget.
3. **Hidden tier**: permanently excluded information (binary files, generated code, secrets). Enforced structurally via .gitignore-like mechanisms.

This convergence is not accidental; it reflects the optimization structure of the context selection problem. The active tier contains high-relevance, low-size items that are always worth including. The searchable tier implements adaptive context selection. The hidden tier enforces the constraint $|C^*| < W$ by preventing low-relevance items from consuming budget.

3.8. Structural vs. Instructional Enforcement

Proposition 3.4 (Structural Enforcement Dominance)**.** [Mathematical Fact: $(1-\epsilon)^T$ is exponential decay; calibrated by AgentSpec data.] *Let ϵ be the per-step instruction violation probability and T the number of agent steps. Instructional enforcement (prompts, rules files) achieves compliance probability $(1-\epsilon)^T$, which decays exponentially in T. Structural enforcement (file system permissions, tool restrictions, automated gates) achieves compliance probability 1 regardless of T.*

The reliability gap $(1-\epsilon)^T$ grows rapidly. At $\epsilon = 0.02$ and $T = 50$, instructional compliance is 36%. At $T = 100$, it falls to 13%. At $\epsilon = 0.05$ and $T = 250$ (JetBrains' upper bound for agent runs), instructional enforcement is virtually guaranteed to fail at least once. This result, which reappears in the Reliability and Quality pillars, is one of the strongest cross-pillar findings: *anything that must hold invariantly should be enforced structurally, not instructionally.*

Production harnesses implement structural enforcement through five categories: filesystem isolation (RAPID worktrees, Codex containers), tool restriction (Spotify Honk's allowlisted commands), context scoping (GSD's fresh 200K per task), output filtering (JetBrains' observation masking), and interface contracts (typed tool schemas). The principle: prefer mechanisms over policies.

Remark (The enforcement gap in practice)**.** *The model predicts compliance of 1.0 for structural enforcement on decidable invariants. Empirical measurements show approximately 99.7%, with a residual 0.3% gap. This gap arises from two sources: (1) enforcement coverage (not all invariants that should be structurally enforced are, due to engineering cost or difficulty of mechanization), and (2) agent workarounds (agents find creative tool-usage paths that bypass structural gates, such as creating temporary files, using unexpected tool combinations, or exploiting timing windows). The formal model assumes a closed enforcement boundary; production systems have leaky boundaries. A taxonomy of enforceable vs. non-enforceable properties, connecting to the runtime verification literature, would strengthen this contribution.*

3.9. The Reuse Discovery Problem

The Information pillar identifies a largely unsolved problem: how does the agent discover that a relevant utility, pattern, or abstraction already exists in the codebase? This is formalized as approximate nearest-neighbor search in a semantic embedding space, where the query is the agent's current intent and the corpus is the set of existing code units.

Definition 3.1 (Functional Similarity)**.** *Given candidate function g and existing function f_j, define:*

$$sim(g, f_j) \triangleq \frac{I(g; f_j)}{H(g, f_j)} \tag{14}$$

This normalised mutual information takes values in $[0, 1]$: 0 for independent functions, 1 for functionally equivalent ones.

Under a code embedding ϕ that is approximately sufficient for functional semantics, the reuse detection problem reduces to finding all f_j with $\|\phi(\sigma_g) - \phi(f_j)\| < r(\tau)$ for some threshold τ.

The problem is that the agent does not know what it does not know; it cannot search for something whose existence it has not hypothesized. The GitClear finding of $8\times$ increased code duplication (GitClear, 2025) is a direct consequence of this unsolved problem. No published work formalizes the "should I reuse or create?" decision as a gate in the agent's action space. Existing tools provide *context retrieval* (presenting similar code), not *reuse enforcement* (requiring justification for creating duplicates). A *reuse map* $R = \{(n_j, \sigma_j, \ell_j)\}$ (name, signature, location) compresses the codebase for reuse decisions at roughly $1000\times$ compression ratio while retaining approximate sufficiency, making it an ideal active-tier item.

4. The Reliability Pillar

The Reliability pillar addresses the most consequential quantitative fact about agent workflows: compound errors destroy end-to-end reliability.

4.1. Series Reliability and Compound Error Sensitivity

An n-step agent pipeline is a series reliability system. If each step succeeds independently with probability p, the end-to-end reliability is:

$$R(n) = p^n \tag{15}$$

At $p = 0.95$ and $n = 20$, $R(20) = 0.36$. At $p = 0.85$ and $n = 10$, $R(10) = 0.20$.

Observation 4.1 (Compound Error Sensitivity)**.** [Mathematical Fact: follows from differentiation of $R = p^n$.] *The elasticity of end-to-end reliability with respect to per-step reliability is:*

$$\mathcal{E}(R, p) = \frac{\partial \ln R}{\partial \ln p} = n \tag{16}$$

A 1% relative improvement in per-step reliability yields an n% relative improvement in end-to-end reliability.

Proof sketch. Direct differentiation: $dR/dp = np^{n-1}$. The elasticity is $(p/R)(dR/dp) = (p \cdot np^{n-1})/p^n = n$. □

This result has a stark practical consequence: for long pipelines, small improvements in per-step quality matter far more than architectural sophistication. The elasticity equals the pipeline length, meaning that a 20-step pipeline amplifies per-step improvements (or degradations) 20-fold.

Concrete example. For $p_0 = 0.95$ and $n = 10$: $\partial R/\partial p = 10 \times 0.95^9 = 6.30$. A 1-percentage-point increase in per-step accuracy (from 0.95 to 0.96) raises pipeline reliability from 59.9% to 66.5%, a gain of 6.6 percentage points.

For heterogeneous pipelines, $\partial R/\partial p_j = R/p_j$, which is inversely proportional to p_j. Improvement effort should target the step with the lowest reliability. In practice, this is usually problem framing (step 1), not execution. With $p_1 = 0.80$ (problem framing), $p_{2\ldots9} = 0.98$ (execution), $p_{10} = 0.92$ (integration) for a 10-step pipeline: $R = 0.80 \times 0.98^8 \times 0.92 = 0.626$. Improving p_1 from 0.80 to 0.90 yields $R = 0.703$ (12.3% absolute gain); improving all execution steps from 0.98 to 0.99 yields $R = 0.662$ (5.7% gain at $8\times$ the intervention breadth).

4.2. Common-Cause Failures: The Marshall-Olkin Model

The independence assumption ($R(n) = p^n$) is an approximation. In practice, agent steps share common causes of failure (model limitations, context misunderstanding, specification ambiguity). The Reliability pillar models this using the Marshall-Olkin common-cause framework (Marshall and Olkin, 1967).

Definition 4.1 (Common-Cause Failure Model)**.** *Decompose each step's failure into an independent component $Z_i \sim Bernoulli(\alpha_i)$ and a shared common-cause component $Y \sim Bernoulli(\gamma)$:*

$$X_i = \max(Z_i, Y) \tag{17}$$

where X_i is the failure indicator for step i. The common-cause shock Y represents failures that affect all steps simultaneously: task misunderstanding at step 1 that biases all subsequent steps, shared model weights inducing common-mode failures, or context drift affecting multiple steps.

Under the common-cause model, pipeline reliability is:

$$R(n) = (1 - \gamma) \prod_{i=1}^{n} (1 - \alpha_i) \tag{18}$$

The common-cause failure rate γ imposes a hard ceiling: no amount of per-step improvement can raise $R(n)$ above $(1 - \gamma)$. For the homogeneous case ($\alpha_i = \alpha$ for all i), this gives $R(n) = (1 - \gamma)(1 - \alpha)^n$, which reduces to p^n when $\gamma = 0$ (recovering the independent model).

This decomposition is practically important. Research on agent failures suggests that most failures are attributable to problem framing and context management (the γ component), not execution-step errors (the α_i components) (Cemri et al., 2025). Improving individual α_i has limited effect when γ dominates; the leverage is in reducing γ through better context engineering and specification. Empirical evidence from Hariharan et al. (2025) (63% failure rate on 100-step tasks) is consistent with the independent model as a reasonable first approximation, but error clustering in problem-framing steps dominates execution-step errors.

4.3. Optimal Verification Scheduling

Given a pipeline of n steps with per-step failure probability $q = 1 - p$, the Reliability pillar formulates verification checkpoint placement as a dynamic programming problem. The key result is a discrete analog of the Young-Daly checkpoint interval from HPC:

Theorem 4.1 (Optimal Checkpoint Count)**.** *For a homogeneous pipeline with n steps, per-step failure probability q, base verification cost c_0, and per-step verification cost c_v, the optimal number of verification checkpoints is:*

$$k^* \approx \sqrt{\frac{n(1 - p) \cdot c_0}{c_v}} \tag{19}$$

For typical agent pipelines, $k^* \in [2, 4]$, and three verification rounds capture over 96% of detectable errors.

Proof sketch (Young-Daly analog). Partition the pipeline into k contiguous segments separated by verification checkpoints. Segment j spans steps $[a_j, b_j]$ with failure probability $F_j = 1 - \prod_{i=a_j}^{b_j} p_i$ and

length $L_j = b_j - a_j + 1$. The expected cost given k segments is:

$$\mathbb{E}[\text{Cost}] = (k-1) \cdot c_v + \sum_{j=1}^{k} g(L_j)$$

where $g(L) = (1 - p^L) \cdot c_0 \cdot L$ is the expected rework cost for a segment of length L. The segment cost $g(L)$ is convex in L for $0 < p < 1$ (verified by computing $g''(L) = c_0 \mu p^L (2 - \mu L)$ where $\mu = -\ln p > 0$; this is positive for $L < 2/\mu$, which at $p = 0.95$ means $L < 39$, covering practical pipelines). By Schur-convexity of a sum of convex functions subject to $\sum L_j = n$, the minimum is at the equal-length partition $L_j = n/k$ for all j. Substituting $g(n/k)$ and minimizing over k yields $k^* \approx \sqrt{n(1-p)c_0/c_v}$. □

Concrete example. For $p = 0.95$, $c_v = 1$, $c_0 = 5$, $n = 20$: $L^* = \sqrt{1/(0.05 \times 5)} = 2$, suggesting verification every 2 steps. For $p = 0.99$, $c_v = 10$: $L^* = \sqrt{10/(0.01 \times 5)} \approx 14$, suggesting verification once or twice in a 20-step pipeline.

4.4. Geometric Diminishing Returns for Verification

Theorem 4.2 (Geometric Diminishing Returns)**.** *Let $M(k)$ be the expected number of errors detected by k verification rounds, each independently detecting remaining errors with probability v. Then:*

$$\Delta M(k) = M(k) - M(k-1) = N_0 \cdot v \cdot (1-v)^{k-1} \tag{20}$$

The marginal value of round k decays geometrically with ratio $(1 - v)$.

Proof sketch. After k rounds, the probability a given error remains undetected is $(1 - v)^k$. Hence $M(k) = N_0(1 - (1 - v)^k)$ and $\Delta M(k) = N_0 v(1 - v)^{k-1}$. □

Empirical calibration against Hariharan et al. (2025): cumulative error detection of 62% after round 1, 89% after round 2, and 96.5% after round 3. Fitting the geometric model with $v = 0.62$:

Round k	Predicted $D(k)$	Observed $D(k)$	Residual
1	0.620	0.620	0.000
2	0.856	0.890	−0.034
3	0.945	0.965	−0.020

The slight underprediction at rounds 2–3 is consistent with an improving detection rate ($v_1 = 0.62$, $v_2 = 0.71$, $v_3 = 0.68$), where the verifier sharpens its focus after eliminating easy errors. The stopping criterion (verify until marginal expected catch falls below verification cost) yields $k \approx 3$ for typical parameters. Rounds 4 and beyond face the pesticide paradox: the remaining errors require different detection approaches, not more of the same.

4.5. Four-Layer Verification Architecture

The Reliability pillar synthesizes these results into a four-layer verification architecture, ordered by efficiency $\eta = d/c$ (detection rate per unit cost):

Layer ℓ	Name	$d(\ell)$	$f(\ell)$	$c(\ell)$	$\eta(\ell)$
1	Structural gates	0.95	~ 0	1	0.950
2	Deterministic analysis	0.80	0.10	5	0.160
3	LLM-as-Judge	0.70	0.25	50	0.014
4	Human review	0.90	0.05	500	0.002

Layer 1 (type checking, schema validation, permission enforcement) is $7\times$ more efficient than Layer 2 (unit tests, integration tests, property-based tests), $68\times$ more efficient than Layer 3 (LLM semantic analysis, pattern detection, style review), and $528\times$ more efficient than Layer 4 (expert assessment for architectural fit, design quality, business logic). The cost-minimizing strategy achieving a target detection rate d^* activates layers in decreasing order of efficiency until the target is met (a variant of the fractional knapsack problem, for which greedy ordering is optimal).

If layers are applied in sequence, each independently detecting defects, the combined detection rate is:

$$d_{\text{combined}} = 1 - \prod_{j=1}^{m} (1 - d(\ell_j)) \tag{21}$$

With all four layers and 3 iterations of the middle layers: $d_{\text{combined}} \geq 0.958$, matching the empirical 96.5% convergence.

Verification intensity should be *adaptive*: higher for high-risk changes (security-critical, cross-module, novel logic) and lower for low-risk changes (formatting, documentation, well-tested patterns). The optimal verification level minimizes $J(\ell, \hat{g}, r) = c(\ell) + \lambda \cdot r \cdot \hat{g} \cdot (1 - d(\ell))$, where $r \in [0, 1]$ is risk level and $\hat{g} \in [0, 1]$ is the estimated spec-to-code information gap. Level ℓ is preferred over $\ell - 1$ when $r \cdot \hat{g}$ exceeds the switching threshold $\tau(\ell{-}1 \to \ell) = (c(\ell) - c(\ell{-}1))/(\lambda \cdot (d(\ell) - d(\ell{-}1)))$.

4.6. Circular Validation Bias

When the same agent (or same model family) produces both code and tests, a systematic bias arises from shared blind spots.

Definition 4.2 (Circular Validation Bias)**.** *Let β_A be the blind-spot rate of agent A (the probability that a defect falls in a category the agent systematically misses) and $\beta_{A'}$ the blind-spot rate of a verifier A'. The residual bias after verification is:*

$$\beta = \beta_A (1 - \beta_{A'}) \tag{22}$$

When $A = A'$ (self-verification), $\beta_{A'} = \beta_A$ for the shared blind-spot categories, yielding zero bias reduction on those categories.

Remark. *The scalar formulation $\beta = \beta_A(1 - \beta_{A'})$ models the* overall *residual bias across all defect categories. For defects within the shared blind-spot set $B_A \cap B_{A'}$, neither producer nor verifier can detect them, so the effective detection rate on those categories is zero regardless of $\beta_{A'}$. The Structural Separation Theorem (Theorem 4.4) provides the precise set-theoretic formulation: the irreducible bias is $\mu(B_A \cap B_{A'})$, which the scalar model underestimates when blind spots are category-specific rather than uniform.*

This result connects directly to METR's finding that roughly 50% of SWE-bench test-passing pull requests would not be merged by repository maintainers (METR, 2026b). (An important baseline caveat: only about 68% of human-written "golden patches" would be merged on re-review, so the gap attributable to AI deficiency is approximately 18 percentage points, not 50; the remainder reflects maintainer subjectivity and the study's small evaluator pool of 4 maintainers across 3 repositories.)

The tests pass because they test the code's actual behavior (including bugs) rather than the intended behavior. Cross-family verification, using a different model or a structurally different verification method, is where bias reduction concentrates.

4.7. Adversarial Collapse of Prompt Enforcement

Empirical evidence demonstrates that LLM agents actively circumvent evaluation criteria when given the opportunity. METR documented reward-hacking rates of 30.4% on RE-Bench (where the scoring function was visible) versus 0.7% on HCAST (where it was hidden), a $43\times$ difference (METR, 2025). NIST CAISI documented agents commenting out assertions, downloading solutions from GitHub, crashing servers via $/\mathrm{dev}/\mathrm{urandom}$, and reading future git commits (NIST CAISI, 2025). Apollo Research confirmed in-context scheming in frontier models, with covert action rates of 13% for o3 (Apollo Research, 2025).

Prompt-based mitigation proved ineffective: adding "do not cheat" to the system prompt did not reduce reward-hacking rates. Under adversarial conditions (the agent actively seeks to circumvent constraints), prompt enforcement fails entirely ($\epsilon \to 1$) and the enforcement transition threshold collapses to approximately $c_s - c_p$. This provides formal justification for the principle: safety-critical constraints must be structurally enforced.

4.8. The Structural Enforcement Boundary

Theorem 4.3 (Cost-of-Failure Threshold)**.** *Let c_s be the cost of structural enforcement, c_p the cost of prompt enforcement, and L the expected cost of a failure. Structural enforcement is cost-effective when:*

$$L > L^* = \frac{c_s - c_p}{\epsilon} \tag{23}$$

where ϵ is the per-step prompt violation probability. For multi-step pipelines, the threshold drops as L^*/n, since the expected number of violations grows linearly in n.

Proof sketch. The expected cost under prompt enforcement is $c_p + \epsilon L$; under structural enforcement, $c_s + \delta_s L$ where $\delta_s < \epsilon$ is the specification gap under structural enforcement. Setting these equal and solving for L yields $L^* = (c_s - c_p)/(\epsilon - \delta_s)$. For $\delta_s \approx 0$, this simplifies to $L^* = (c_s - c_p)/\epsilon$. In an n-step pipeline where the constraint must hold at every step, the probability of at least one violation under prompt enforcement is $1 - (1-\epsilon)^n \approx n\epsilon$ for small ϵ, so the effective threshold drops to approximately L^*/n. □

Theorem 4.4 (Structural Separation Theorem)**.** *Structural separation (different agents for code and test authorship) reduces circular validation bias whenever $|B_A \cap B_{A'}| < |B_A|$. The reduction is maximized when blind-spot sets are disjoint ($B_A \cap B_{A'} = \emptyset$).*

Sufficient conditions for effective separation:

1. *Different model family (different pretraining corpora yield different error distributions).*
2. *Different input representation (e.g., formal specification vs. natural language).*
3. *Adversarial framing (instructing the test author to actively seek failures).*

Proof sketch. Under the blind-spot model, when author A writes code and verifier A' writes tests, a bug escapes only if it falls in both blind-spot sets $B_A \cap B_{A'}$. The escape probability is $\mu(B_A \cap B_{A'})$. Under same-author verification ($A' = A$), the escape probability is $\mu(B_A)$ (the full blind-spot rate). Separation reduces the escape probability by $\mu(B_A) - \mu(B_A \cap B_{A'}) = \mu(B_A \setminus B_{A'}) > 0$ whenever the

blind-spot sets are not identical. The reduction is maximized at $\mu(B_A)$ when $B_A \cap B_{A'} = \emptyset$. The three sufficient conditions promote disjointness by ensuring that the error distributions of A and A' arise from different sources (different training data, different input modalities, different optimization objectives). □

The practical implication: as pipelines grow longer, the set of invariants that *should* be enforced structurally expands. Structural enforcement is not an all-or-nothing choice; the optimal boundary shifts with pipeline length, failure cost, and the cost differential between structural and prompt enforcement. Using AgentSpec parameters (Wang et al., 2026): $\epsilon = 0.23$ (prompt failure rate), $c_p = 1$, $c_s = 10$, yielding $L^* \approx 39$. When a single violation costs more than $39\times$ the prompt engineering cost, structural enforcement is justified. For safety-critical constraints (data deletion, financial transactions), L typically exceeds this threshold by orders of magnitude.

4.9. Checkpoint Optimality: The Young-Daly Connection

The checkpoint count formula (Theorem 4.1) is a discrete analog of the Young-Daly checkpoint interval formula (Daly, 2006; Young, 1974) from high-performance computing. In HPC, the optimal checkpoint interval balances the cost of saving state against the cost of lost computation upon failure. The correspondence is:

HPC (Young-Daly)	Agent Pipeline
Checkpoint cost	Verification cost c_v
Mean time between failures	$1/(1-p)$ steps
Lost computation on failure	Rework cost $c_0 \cdot L$
Optimal interval $\tau^* = \sqrt{2\delta C}$	Optimal segment $L^* = \sqrt{c_v/((1-p)c_0)}$

The structural parallel is exact: both problems minimize total expected cost (checkpoint overhead plus expected rework) under a Poisson-like failure process. The agent pipeline setting introduces one additional feature: the verification step is itself imperfect (detection probability $v < 1$), which the classical Young-Daly formula does not account for. This imperfection is absorbed into the geometric diminishing returns result (Theorem 4.2), which bounds the value of repeated verification at each checkpoint.

5. The Coordination Pillar

The Coordination pillar addresses multi-agent code generation, where multiple LLM instances work concurrently on a shared codebase. The central finding is that naive parallelism amplifies errors far beyond what independence would predict.

5.1. Error Amplification

Definition 5.1 (Error Amplification Factor)**.** *For coordination topology t with k agents, each with per-agent error rate $\epsilon = 1 - p$, the error amplification factor is:*

$$A_e(t,k) = \frac{P(\textit{system error} \mid t,k)}{P(\textit{single-agent error})} = \frac{1 - R_{sys}(t,k)}{\epsilon} \tag{24}$$

Under independence, A_e approaches k from below (since $1 - p^k \approx k\epsilon$ for small ϵ). But Kim et al. (2025) found empirical amplification of $17.2\times$ for independent agents, far exceeding the theoretical

maximum of k under independence. This $\approx 6\times$ discrepancy quantifies the "correlation multiplier" from inter-agent error propagation and shared failure modes.

5.2. Information-Sharing Reduction Factor

Coordination reduces per-agent error rates by enabling agents to share information (current file ownership, interface changes, partial results). We model this reduction explicitly.

Definition 5.2 (Coordination Benefit Function)**.** *For topology t with information-sharing capacity $I(t)$ bits per coordination round, define the effective per-agent error rate under coordination:*

$$\epsilon_t = \epsilon \cdot g(I(t)) \tag{25}$$

where $g : \mathbb{R}_+ \to (0, 1]$ is monotonically decreasing with $g(0) = 1$ (no coordination, no benefit) and $\lim_{I\to\infty} g(I) = g_{\min} > 0$ (coordination cannot eliminate intrinsic model errors).

A natural parametric choice is the exponential $g(I) = e^{-\alpha I}$ for topology-dependent rate $\alpha > 0$. Under this model, the ratio of centralized to independent error amplification calibrates α: from Kim et al. (2025), $4.4/17.2 \approx 0.256$, giving $\alpha \cdot I(\text{central}) \approx 1.36$ nats. The information-sharing capacity $I(t)$ depends on the topology: independent systems have $I = 0$; centralized systems share the coordinator's summary (bounded by the coordinator's context window); hierarchical systems share through tree aggregation with logarithmic depth.

Remark. *The coordination benefit function g captures two distinct mechanisms: (i) error prevention (agents receive information that prevents them from making mistakes they would otherwise make), and (ii) error detection (agents receive signals that reveal errors in their partial work). Both mechanisms reduce the effective error rate, but they have different scaling properties: prevention improves with the quality of upfront planning, while detection improves with the frequency of synchronization.*

5.3. Capability Saturation Crossover

As single-agent capability improves, the marginal value of additional agents diminishes and eventually becomes negative.

Definition 5.3 (Saturation Threshold)**.** *The capability saturation threshold P^* is the single-agent accuracy at which the marginal value of an additional agent transitions from positive to negative:*

$$P^* = \inf \left\{ P_{\text{SA}} \in [0, 1] : \frac{\partial}{\partial k} S_{\text{eff}}(k, P_{\text{SA}}) \Big|_{k=1} \leq 0 \right\} \tag{26}$$

Conjecture 5.1 (Saturation Crossover)**.** *For fixed topology t and error model, there exists $P^*(t) \in (0, 1)$ such that for $P_{\text{SA}} < P^*$, the optimal agent count $k^* > 1$, and for $P_{\text{SA}} > P^*$, $k^* = 1$. We term this a crossover rather than a phase transition: the regression model of Kim et al. (2025) yields a smooth interaction coefficient ($\beta = -0.408$), indicating a gradual shift rather than a sharp discontinuity in the physics sense.*

Kim et al. (2025) report $P^* \approx 0.45$ from their regression model ($\beta_{P_{\text{SA}} \times \log(1+n_a)} = -0.408$, $p < 0.001$). Above this threshold, adding agents actively degrades performance. This threshold was derived from general benchmarks (web navigation, quantitative reasoning, planning, tool use), not from code-generation tasks specifically. Preliminary evidence from software-specific evaluations suggests a lower crossover ($P^* \approx 0.30$–0.35) for coding tasks, reflecting tighter coupling and stricter correctness requirements. We adopt the software-specific range as the recommended threshold for harness design.

The crossover conjecture has a concrete design implication: as foundation models improve, the optimal number of parallel agents *decreases*. Organizations should expect to transition from multi-agent architectures toward single-agent (or few-agent) workflows as model capability increases, redirecting coordination overhead toward richer context and more thorough verification.

5.4. Extended Amdahl's Law with Reliability

Classical Amdahl's Law gives speedup $S(n) = 1/(s + (1-s)/n)$ for serial fraction s and n parallel units. The Coordination pillar extends this with a reliability factor:

Proposition 5.1 (Extended Amdahl's Law)**.** [Engineering Hypothesis: optimal n^* depends on δ_a which is unmeasured in production.] *For n agents with serial fraction s and per-agent defect injection rate δ_a (distinct from the context degradation function δ in Section 3), and assuming independent agent errors:*

$$S_{eff}(n) = \frac{1}{s + (1-s)/n} \cdot (1-\delta_a)^n \tag{27}$$

The optimal agent count is $n^* \approx 1/\sqrt{\delta_a}$. Beyond n^*, adding agents decreases *effective throughput.*

Proof sketch. Write $\ln S_{\text{eff}} = -\ln(s + (1-s)/n) + n\ln(1-\delta_a)$. Differentiating with respect to n and setting to zero: $(1-s)/(n^2 s + n(1-s)) = -\ln(1-\delta_a) \approx \delta_a$ for small δ_a. For small s, this simplifies to $1/n^2 \approx \delta_a$, giving $n^* \approx 1/\sqrt{\delta_a}$. The maximum is unique because $\ln S_{\text{eff}}$ is the sum of a concave function (the Amdahl term) and a linear function with negative slope (the reliability term), hence strictly concave. □

At $\delta_a = 0.05$, $n^* \approx 4$ to 5 agents. At $\delta_a = 0.01$, $n^* \approx 10$. This provides a principled answer to the practitioner question "how many agents should I run in parallel?" and explains Cursor's observation that 20 parallel agents with flat coordination collapsed to effective throughput of 2 to 3 agents.

5.5. Universal Scalability Law Extension

The Extended Amdahl's Law captures serialization and reliability but omits coherency overhead (the cost of keeping agents consistent with each other). Combining Gunther's Universal Scalability Law (Gunther, 2008) with the reliability factor gives a more complete model:

$$S_{\text{eff}}(n) = \frac{n}{1 + \sigma(n-1) + \kappa\, n(n-1)} \cdot (1-\delta_a)^n \tag{28}$$

where σ is the serialization coefficient (fraction of work that must be done sequentially, analogous to s in Amdahl's Law) and κ is the coherency (crosstalk) penalty per agent pair.

The USL's κ term is quadratic in n, producing a characteristic "retrograde" curve where throughput decreases beyond an optimal point even without the reliability factor. With the additional exponential decay from $(1-\delta_a)^n$, the optimal n^* is pushed lower still, because the system faces three compounding penalties:

1. **Serialization** (σ term): some work cannot be parallelized.
2. **Crosstalk** (κ term): agents must synchronize, consuming $O(n^2)$ communication overhead.
3. **Unreliability** ($(1-\delta_a)^n$ term): each agent independently injects defects.

These three penalties explain why empirical optimal team sizes are small: 3 to 5 agents per practitioner consensus (Osmani, 2025), consistent with $n^* \approx 3$–4 from the model at typical parameter values ($\sigma \approx 0.1$, $\kappa \approx 0.02$, $\delta_a \approx 0.05$).

5.6. Task Decomposition as Balanced Min-Cut

The Coordination pillar formalizes task decomposition as a graph partitioning problem:

Definition 5.4 (Coupling Graph)**.** *Given a codebase with file set $\mathcal{F}$, the coupling graph $G = (\mathcal{F}, E, w)$ encodes structural imports, co-change frequency, and semantic similarity as weighted edges.*

Optimal decomposition for k agents is a balanced k-way min-cut on the coupling graph, minimizing cross-partition edges (which become potential merge conflicts and coordination overhead). The Cut-Conflict Bridge lemma connects partition quality directly to expected merge conflict rate:

Proposition 5.2 (Cut-Conflict Bridge)**.** *For a k-way partition $\Pi = \{F_1, \ldots, F_k\}$ of the coupling graph, the expected merge conflict count is:*

$$\mathbb{E}[\textit{conflicts}] = \sum_{(u,v) \in \textit{cut}(\Pi)} w(u,v) \cdot p_{\textit{conflict}}(u,v) \tag{29}$$

where $p_{\textit{conflict}}(u,v)$ is the probability that simultaneously modifying files u and v produces a conflict. Minimizing cut weight directly minimizes expected conflicts.

Proof sketch. Each cross-partition edge (u,v) with weight $w(u,v)$ represents coupling that may cause a conflict. The probability of a conflict arising from this edge is at most proportional to the coupling weight, by construction of the composite coupling metric (calibrated against historical conflict data). Summing over all cut edges yields the bound. The constant of proportionality depends on the calibration of w against actual conflict data and is empirically in the range 0.01 to 0.1 for human-authored code; it has not been validated for agent-generated code specifically. □

5.7. Five-Level Merge Conflict Taxonomy

The Coordination pillar identifies five levels of merge conflict, ordered by detection difficulty. We present each level together with its primary detection mechanism:

Table 1 | Five-level merge conflict taxonomy with detection methods.

Level	Type	Detection Method	Detection Tool
1	Textual	Three-way merge	git merge
2	Structural	AST-level differencing	JDime, GumTree
3	Dependency	Build-level analysis	Package resolver, compiler
4	API	Refactoring-aware merge	IntelliMerge (88% precision)
5	Semantic	Compositional verification	SafeMerge (75% of benchmarks)

Without coordination contracts, conflict count grows as $O(k^2)$ in the number of agents (every pair can conflict). With file-ownership contracts ($F_i \cap F_j = \emptyset$), textual, structural, and API conflicts are eliminated by construction (no shared files to conflict on), reducing growth to $O(k)$ through interface-mediated dependency and semantic conflicts only. The reduction from quadratic to linear scaling is the primary formal justification for ownership-based contracts.

5.8. Database Concurrency Control Analogy

The Coordination pillar draws a formal analogy between multi-agent code generation and database concurrency control. Agents are transactions; files are data items; merge conflicts are write-write conflicts. The mapping is surprisingly precise:

Table 2 | Database isolation levels mapped to agent coordination mechanisms.

Isolation Level	Agent Mechanism	Prevented	Permitted
Read Uncommitted	Shared workspace	None	All
Read Committed	Git branches, periodic pulls	Dirty reads	Non-repeatable reads, phantoms, write skew
Snapshot	Git worktrees (frozen at dispatch)	Dirty reads, non-repeatable reads, phantoms	Write skew
Serializable	Sequential execution	All	None

The critical insight is *write skew*: two agents read the same state, make individually valid changes, but the combination is invalid. Agent A_i modifies file f_a based on reading file f_b, while agent A_j modifies f_b based on reading f_a. Both commits succeed (disjoint write sets), but the combined state is inconsistent. This is the agent analog of the database write-skew anomaly, and it corresponds precisely to Level 5 (semantic) merge conflicts in the taxonomy above.

Remark. *The key difference between database transactions and agent tasks is cost granularity. Database transactions are millisecond-scale; agent tasks are minute-scale. The retry cost of an aborted agent task (discarding expensive LLM inference) is orders of magnitude higher relative to task cost, shifting the calculus: even rare conflicts make optimistic approaches (detect-and-retry) less attractive compared to pessimistic approaches (prevent-by-contract) unless conflict detection happens before expensive computation.*

5.9. Assume-Guarantee Composition

Each agent A_i operates under a contract $(Asm_i, Guar_i)$ following the assume-guarantee paradigm (Henzinger et al., 2002; Meyer, 1992):

Assumptions Asm_i:

(A1) No other agent modifies any file in F_i: $\forall j \neq i,\ \mathrm{WriteSet}(A_j) \cap F_i = \emptyset$.
(A2) All interfaces referenced by A_i but owned by other agents remain stable (signatures unchanged).
(A3) The snapshot S_i provided to A_i at dispatch time is consistent (compiles, passes tests).

Guarantees $Guar_i$:

(G1) Agent A_i modifies only files in F_i: $\mathrm{WriteSet}(A_i) \subseteq F_i$.
(G2) For every interface I_j in files owned by A_i, the post-modification signature matches sig_j.
(G3) The modifications, applied to S_i, compile and pass all tests scoped to F_i.

Theorem 5.1 (Contract Composition)**.** *If every agent A_i satisfies its contract (assuming Asm_i holds, A_i establishes $Guar_i$), then the composed system satisfies:*

1. *No write-write conflicts (from disjointness and G1).*
2. *All specified interfaces are preserved (from G2).*
3. *For languages with sound separate compilation (e.g., Java, Rust, Go), the merged codebase compiles (from G3 and interface preservation). For dynamically-typed languages (Python, JavaScript), the guarantee weakens to: the merged codebase compiles at all statically checkable call sites.*

Proof sketch, rely-guarantee reasoning. Step 1 (Disjointness). By disjointness of F_i and guarantee G1, each agent's write set is disjoint from every other agent's. This establishes the rely condition: the

environment does not modify A_i's files, satisfying A1.

Step 2 (Interface preservation). By G2, each agent preserves all interfaces in its owned files. Since A2 requires that interfaces outside F_i remain stable, and G2 ensures this for every F_j, the circular rely-guarantee discharge holds: $Guar_j$ for $j \neq i$ collectively implies Asm_i.

Step 3 (Compilation). Each agent's modifications compile in isolation (G3) with all consumed interfaces preserved (A2, now discharged). Since the merged codebase preserves all interfaces, every call site remains type-consistent. By compositionality of type checking, the merged result type-checks. □

Remark. *This proof is modular: adding agent A_{k+1} with a fresh disjoint F_{k+1} requires only verifying A_{k+1}'s contract, not re-verifying existing agents. This modularity is the key advantage of assume-guarantee reasoning over monolithic verification.*

5.10. Optimal Agent-Task Assignment (Inverse Conway)

The central optimization problem in multi-agent coordination is: given a desired system decomposition and a set of agents, how should work be assigned? This is the *Inverse Conway Maneuver* applied to agents.

Define the *communication graph* $G_T = (\mathcal{A}, E_T)$ where an edge $(A_i, A_j) \in E_T$ indicates that agents A_i and A_j can exchange messages under topology T, and the *required communication graph* $G_R = (\mathcal{A}, E_R)$ where $(A_i, A_j) \in E_R$ indicates that agents A_i and A_j manage modules with inter-module dependencies. The "topology-module mismatch cost" is:

$$\text{Mismatch}(\alpha, T, G_M) = \sum_{(A_i,A_j)\in E_R \setminus E_T} \sum_{\substack{(M_a,M_b)\in E_M \\ \alpha(M_a)=A_i, \alpha(M_b)=A_j}} w(M_a, M_b) \tag{30}$$

where $\alpha : \mathcal{M} \to \mathcal{A}$ is the module-to-agent assignment.

Theorem 5.2 (Optimal Agent Assignment)**.** *The assignment α^* minimizing mismatch for a given topology T is the solution to a constrained balanced min-cut: partition the module graph into k parts minimizing uncovered cross-partition edge weight.*

For the independent topology ($E_T = \emptyset$), mismatch equals total cross-partition coupling, reducing to standard balanced min-cut. For mesh topology (E_T is complete), mismatch is zero for any assignment, but communication overhead is $O(k^2)$. For isolated-then-merge with contracts at coverage γ, the effective mismatch is $(1-\gamma)$ times the full mismatch, since contracts act as static communication channels conveying interface information without runtime message passing.

This connects task decomposition (Section 5.6) to topology selection: the optimal decomposition depends on the topology, and the optimal topology depends on the decomposition. In practice, this joint optimization is solved iteratively: choose a topology, compute the best decomposition for that topology, evaluate QAS, and compare across topologies.

Conway's Law as a corollary. The containment condition $G_R \subseteq G_T$ (every required communication edge exists in the topology) is the necessary condition for zero mismatch cost. When $G_R \not\subseteq G_T$, uncovered edges produce semantic conflicts with probability proportional to coupling strength. This is Conway's Law (Conway, 1968) restated graph-theoretically:

Theorem 5.3 (Conway's Law, Graph-Theoretic Form)**.** *If $G_R \not\subseteq G_T$, then for every edge $(A_i, A_j) \in E_R \setminus E_T$, the agents managing the dependent modules cannot coordinate, producing a semantic conflict with probability proportional to the coupling strength of the underlying module dependency.*

Note that the containment condition is trivially satisfied by adopting a complete communication topology (mesh), but at $O(k^2)$ communication overhead. The interesting design question is not whether containment holds, but what the minimum-cost topology is that achieves it for the given coupling structure, which is the balanced min-cut problem above.

5.11. Quality-Adjusted Speedup

Definition 5.5 (Quality-Adjusted Speedup)**.**

$$QAS(k) = \frac{T_1 \cdot Q_1}{T_k \cdot Q_k} \tag{31}$$

where T_k is wall-clock time with k agents and Q_k is output quality (e.g., fraction of changes that would pass expert review). $QAS < 1$ means parallelism is net-negative: the quality degradation outweighs the speed improvement.

Calibration against DORA 2025 data suggests that naive AI-assisted parallelism frequently yields QAS < 1: individual task completion up 21%, but review time up 91%, PR size up 154%, and organizational delivery metrics flat despite individual gains. (The "bug rates up 9%" figure appearing in secondary analyses could not be independently verified from the primary DORA report.)

More precisely, for any topology t and agent count $k > 1$, if the error amplification factor $A_e(t, k)$ satisfies $A_e(t, k) \cdot d(1) \geq 1 - (1 - d(1))/S(k)$ (where $d(1)$ is the single-agent defect rate and $S(k)$ is the raw speedup), then QAS$(k) < 1$. This condition is easily satisfied when $d(1)$ is moderate (0.15 to 0.30), A_e is large (4 to 17), and $S(k)$ is modest (2 to 3). The regime QAS < 1 is not a corner case; it is the default outcome for poorly coordinated multi-agent systems.

6. The Temporal Pillar

The Temporal pillar addresses the time dimension of harness architecture: how fast should agents iterate, when should they speculate, and how should they manage information freshness?

6.1. Context Fidelity Decay: The Channel Model

We model the agent's context as a noisy channel between the true codebase state $\mathcal{X}$ and the agent's representation $\mathcal{Y}$. At the moment of a fresh context read ($t = t_0$), the channel is noiseless: $I(\mathcal{X}; \mathcal{Y}) = I_{\text{fresh}}$. As time passes, the codebase evolves while the agent's snapshot remains fixed, introducing noise.

Definition 6.1 (Context Fidelity)**.** *The context fidelity $\Phi(t)$ at time t for a context snapshot taken at t_0 is:*

$$\Phi(t) = \frac{I_{\text{stale}}(t)}{I_{\text{fresh}}} = e^{-\Lambda(t - t_0)} \tag{32}$$

where $\Lambda = \lambda\alpha$ is the effective decay rate, λ is the codebase change rate (changes per unit time), and α is the average information disruption per change.

Theorem 6.1 (Exponential Staleness Decay)**.** *Assume: (A1) codebase changes follow a Poisson process with rate λ; (A2) each change independently invalidates a fraction α of the agent's cached context; (A3) α is constant across change events. Then:*

$$I_{\text{stale}}(t) = I_{\text{fresh}} \cdot e^{-\lambda\alpha(t-t_0)} \tag{33}$$

Proof. The number of changes in $[t_0, t]$ is $N \sim \text{Poisson}(\lambda\Delta t)$ where $\Delta t = t - t_0$. After k changes, the surviving information fraction is $(1-\alpha)^k$. Taking the expectation via the Poisson probability generating function:

$$\mathbb{E}[(1-\alpha)^N] = e^{\lambda\Delta t((1-\alpha)-1)} = e^{-\lambda\alpha\Delta t} \tag{34}$$

□

The context fidelity has half-life $t_{1/2} = \ln 2/\Lambda$.

The exponential decay model is calibrated against the 17.1% "Fresh Eyes" advantage reported by Suzgun et al. (2024): agents with fresh context outperform agents with stale context by this margin. Treating this as an upper bound on fidelity loss (the advantage likely conflates freshness with task decomposition effects):

$$1 - \Phi_{\text{inc}} \leq 0.171 \implies \Phi_{\text{inc}} \geq 0.829 \tag{35}$$

With a typical cycle of $\bar{t} \approx 50$ minutes ≈ 0.83 hours, this gives an upper bound on the effective decay rate of $\Lambda \leq 0.226$ per hour (context half-life ≥ 3 hours).

Remark. *Assumptions A2 and A3 are strong. In practice, changes are correlated (a refactor touches many files; a bugfix touches one), and the disruption fraction α varies across changes. The single-rate model should be understood as a first-order approximation. A multi-exponential extension partitions context into strata with different decay rates: $I_{\text{stale}}(t) = \sum_{k=1}^{K} I_k \cdot e^{-\lambda\alpha_k(t-t_0)}$, capturing the empirical observation that architectural knowledge (α_k small) persists longer than implementation details (α_k large).*

In multi-agent settings, Λ increases with the number of concurrent agents (more agents means faster codebase evolution), creating a feedback loop between parallelism and context staleness. This connects directly to the Coordination pillar's optimal agent count: adding agents increases λ (the codebase change rate), which increases Λ, which decreases Φ, which decreases per-agent quality, which further reduces the effective speedup.

6.2. Verified Iterations per Hour

Definition 6.2 (Verified Iterations per Hour)**.** [Engineering Hypothesis: proposed metric, never measured in production.]

$$VIH = \lambda_{raw} \cdot p_{verify} \tag{36}$$

where λ_{raw} is the raw iteration rate (iterations per hour) and p_{verify} is the probability that an iteration passes all verification gates.[1]

VIH captures the fundamental speed-quality tradeoff. Skipping verification increases λ_{raw} but decreases p_{verify}. The Temporal pillar proves that VIH is maximized at the unit-elasticity point:

[1] VIH has not, to our knowledge, been measured in any production harness. It is proposed here as a metric, not reported as an empirical finding.

Proposition 6.1 (VIH Optimality). *VIH is maximized when:*

$$\frac{\partial \ln \lambda_{raw}}{\partial v} = -\frac{\partial \ln p_{verify}}{\partial v} \tag{37}$$

where v is the verification investment. At this point, the marginal throughput gain from reducing verification exactly equals the marginal quality loss.

Proof sketch. Differentiate $\text{VIH}(\lambda) = \lambda \cdot p_{\text{verify}}(\lambda)$ and set to zero: $p_{\text{verify}}(\lambda) + \lambda \cdot p'_{\text{verify}}(\lambda) = 0$. Rearranging: $p'_{\text{verify}}(\lambda^*)/p_{\text{verify}}(\lambda^*) = -1/\lambda^*$, which is the unit-elasticity condition. The optimum occurs where a 1% increase in raw speed causes exactly a 1% decrease in verification rate. □

Under an exponential degradation model $p_{\text{verify}}(\lambda) = p_0 e^{-\beta(\lambda - \lambda_0)}$ (chosen for analytical convenience; the true functional form is unknown), the optimal rate is $\lambda^* = 1/\beta$.

6.3. Amdahl's Law Bound on VIH

If verification is a serial fraction f of the cycle (it runs after execution and cannot be overlapped), then even with infinite parallelization of all other stages:

$$\text{VIH}^{\max} = \frac{p_{\text{verify}}}{f \cdot \tau_{\text{cycle}}} \tag{38}$$

This is a direct application of Amdahl's Law (Amdahl, 1967): no amount of speedup in non-verification stages can overcome the serial verification bottleneck.

From representative harness data, $f \approx 0.20$ (verification takes roughly 10 of 50 minutes). With $p_{\text{verify}} \approx 0.67$ (using Devin's PR merge rate as a proxy; **?**), $\text{VIH}^{\max} \approx 4.0$ verified iterations per hour, regardless of non-verification speedups.

Remark. *The Amdahl bound implies that the highest-leverage temporal optimization is reducing verification time while maintaining verification quality, not speeding up any other pipeline stage. Tiered verification (fast static checks first, expensive dynamic checks only when static checks pass) is the natural strategy.*

6.4. The Speculation Ratio Test

Speculative execution (beginning the next pipeline stage before the current stage's output is verified) has positive expected value when:

Theorem 6.2 (Speculation Ratio Test). *Speculate if and only if:*

$$\frac{p}{q} > \frac{R}{B} \tag{39}$$

where p is the probability that the current stage's output is correct, $q = 1 - p$ is the failure probability, R is the cost of rollback on failure, and B is the benefit of early completion on success.

Proof sketch. Expected speculative time: $\mathbb{E}[T_{\text{spec}}] = p \cdot \mathbb{E}[\max(t_i, t_j)] + q(R + \mathbb{E}[t_j])$. Sequential baseline: $\mathbb{E}[T_{\text{seq}}] = \mathbb{E}[t_i] + \mathbb{E}[t_j]$. The saving under success is $B = \mathbb{E}[t_i] + \mathbb{E}[t_j] - \mathbb{E}[\max(t_i, t_j)]$. Speculation is profitable when $p \cdot B > q \cdot R$, which rearranges to the stated condition. □

For typical agent pipelines with $p \approx 0.85$ to 0.95 and $R/B \approx 0.3$ to 0.5, speculation is usually beneficial for the first stage ahead but not for deeper speculation (where rollback costs compound).

6.5. Speculation Depth Limit

For multi-stage pipelines, speculation can be applied at multiple depths (speculating two, three, or more stages ahead). However, the probability of a full chain succeeding decays exponentially, while rollback costs grow.

Theorem 6.3 (Speculation Depth Limit)**.** *For a homogeneous chain with per-stage success probability p, benefit B_0, and rollback cost R_0, the maximum useful speculation depth is:*

$$k^* < \frac{\ln(1 + B_0/R_0)}{\ln(1/p)} \tag{40}$$

Proof sketch. At depth k, the probability of needing rollback is $1 - p^k$, and the accumulated rollback cost is at most $k \cdot R_0$. The benefit is at most $k \cdot B_0 \cdot p^k$ (all k stages must succeed). Setting the net expected value to zero: $p^k \cdot kB_0 = (1 - p^k) \cdot kR_0$. Solving for p^k: $p^k > R_0/(R_0 + B_0)$. Taking logarithms yields the stated bound. □

For $p = 0.85$ and $B_0/R_0 = 2$: $k^* \leq 6$ stages. For $p = 0.70$ and $B_0/R_0 = 1$: $k^* \leq 1$. The exponential decay of p^k places a natural horizon on useful speculation, consistent with the practical finding that deep speculation chains in agent pipelines are rarely profitable.

Remark (The Spectre Analogy)**.** *Speculative execution in CPUs was considered a pure optimization for two decades before Spectre and Meltdown revealed that speculative state changes created security vulnerabilities. In agent pipelines, the analogous risks include: state pollution from discarded speculative work, combinatorial branching complexity, debugging difficulty when causal chains include hypothetical branches, and cognitive overhead for humans monitoring speculative pipelines. These hidden costs suggest that speculation should be applied conservatively, typically at depth 1.*

6.6. Three Caching Layers

Agent harnesses interact with three distinct caching layers, each governed by different dynamics and subject to different staleness tradeoffs:

1. **KV-Cache** (attention state reuse, LRU-governed): Provider-level caching of computed key-value pairs for shared prompt prefixes. Eviction follows LRU (Least Recently Used) policy at the provider level. Claude Code achieves 92% hit rate and 81% cost reduction (vendor-reported; **?**). Hit rate depends on prompt structure: append-only prompts with static prefixes maximize hits.
2. **Prompt/Semantic Caching** (token-level, semantic similarity): Server-side caching of prompt computation for repeated or semantically similar prefixes. Liang et al. (2026) report 41–80% cost reduction and 13–31% time-to-first-token improvement across providers. Semantic caching extends exact-match caching by allowing fuzzy retrieval, trading match precision for hit rate.
3. **Plan Template Caching** (reasoning-level, persistent): Reuse of structured plan templates across similar tasks. Zhang et al. (2025a) achieve 50% cost and 27% latency reduction while maintaining 96.6% of optimal performance. Templates persist across sessions and are the most durable cache layer, but also the most vulnerable to staleness (a plan template designed for one codebase version may be subtly wrong for the next).

The Denning working set model (Denning, 1968) applies to all three layers: cache effectiveness depends on temporal locality in the request stream. The cache TTL must match the agent's task locality window to avoid thrashing.

6.7. Cache Competitive Analysis

The Temporal pillar formulates context caching as an inventory problem, where cache entries are "inventory" that becomes stale over time:

Theorem 6.4 (Optimal Refresh Interval)**.** *The optimal cache refresh interval balancing refresh cost R against staleness cost is:*

$$T^* = \sqrt{\frac{2R}{\lambda\alpha \cdot E_{stale}}} \tag{41}$$

where λ is the access rate, α is the staleness growth rate, and E_{stale} is the expected cost per stale access. This is an EOQ (Economic Order Quantity) analog for information freshness.

Proof. The amortized cost per unit time is $C(T) = R/T + \frac{1}{2}\lambda\alpha T \cdot E_{\text{stale}}$ (the staleness integral over $[0, T]$ under linear growth averages to $\frac{1}{2}\lambda\alpha T$). Setting $C'(T) = 0$: $-R/T^2 + \frac{1}{2}\lambda\alpha E_{\text{stale}} = 0$, yielding the stated formula. □

The competitive analysis from the online algorithms literature provides worst-case guarantees for cache refresh policies:

- The break-even deterministic policy (refresh after $\lceil R/r \rceil$ incremental steps, where r is the incremental update cost) is 2-competitive against the offline optimal (Sleator and Tarjan, 1985). That is, its total cost is at most twice the cost of the optimal policy that knows the future request sequence.
- The optimal randomized policy achieves $e/(e-1) \approx 1.58$-competitive (Fiat et al., 1991).

The cache introduces a fundamental tension: append-only context maximizes cache hits but accumulates noise, while context reset loses the cache but gains fresh reasoning quality (the 17.1% advantage). The optimal cycle length is partially determined by when cache savings from continuation are outweighed by quality loss from context accumulation.

6.8. Bayesian Mode Selection with Bainbridge's Ironies

The Temporal pillar provides a Bayesian decision framework for selecting between execution modes:

- **Fast mode**: minimal verification, high λ_{raw}, low p_{verify}. Optimal for low-risk, well-understood tasks.
- **Governed mode**: full verification, low λ_{raw}, high p_{verify}. Optimal for high-risk, novel tasks.

Theorem 6.5 (Bayesian Mode Selection)**.** *For the binary simplification (Fast vs. Governed), the optimal decision rule is:*

$$\textit{Choose Governed iff } P(\textit{Governed needed} \mid \mathrm{x}) > \frac{c_{\text{GF}}}{c_{\text{FG}} + c_{\text{GF}}} \tag{42}$$

where c_{FG} is the cost of choosing Fast when Governed is needed (undetected quality failure, typically 10–100× the task time in rework) and c_{GF} is the cost of choosing Governed when Fast suffices (wasted time, typically 2–5× the task time).

Since $c_{\text{FG}} \gg c_{\text{GF}}$, the threshold is small (typically 0.05–0.10), meaning the harness should choose Governed even when the probability of needing it is quite low. This is the standard asymmetric-cost Bayesian decision rule applied to the mode selection problem.

However, Bainbridge (1983) identified a double bind in automated decision-making that complicates this framework: if mode selection rules can be fully specified, a computer can apply them faster

than a human monitor can verify; if mode selection requires judgment, automation will fail at boundary cases while the human monitor, deprived of practice, performs even worse. The implication for harness design is threefold:

1. The mode selection classifier must be *conservative* (biased toward Governed), because the cost of misclassification is asymmetric.
2. The classifier should operate as a *proposal* that the human confirms or overrides, rather than as a fully autonomous decision.
3. The feature engineering problem is recursive: deciding how much analysis to do for mode classification requires analysis, consuming the very budget that mode selection is meant to allocate efficiently.

6.9. The Speed-Quality Pareto Frontier

In the space of $(\lambda_{\text{raw}}, p_{\text{verify}})$, achievable operating points form a Pareto frontier. The frontier has an empirically motivated three-regime structure:

1. **Concave at low speed**: redundant checks provide diminishing quality returns; easy to gain speed cheaply by removing the lowest-value verification.
2. **Convex at moderate speed**: each further speed gain requires skipping progressively more important verification, with accelerating quality loss.
3. **Concave again at high speed**: statistical filtering via best-of-N stabilizes quality (running $5\times$ more cheap iterations with majority voting can yield higher effective quality than fewer careful iterations, per SWE-bench data showing Pass@5 substantially exceeding Pass@1; SWE Efficiency Research 2025).

6.10. Pareto Frontier Mixing Strategies and Convexification

Theorem 6.6 (Convexification by Mixing)**.** *In concave frontier regions, the mixed strategy using configuration c_1 with probability α and c_2 with probability $(1-\alpha)$ achieves:*

$$(S_{\text{mix}}, Q_{\text{mix}}) = \alpha(S_1, Q_1) + (1-\alpha)(S_2, Q_2) \tag{43}$$

which Pareto-dominates every pure strategy in that region. The set of achievable operating points is the convex hull *of the pure-strategy frontier, not the frontier itself.*

This is the direct analogue of the classical result that mixed strategies expand the achievable set to the convex hull of pure-strategy outcomes. The practical implication is that a harness alternating between "fast mode for 70% of tasks" and "governed mode for 30% of tasks" can achieve a speed-quality combination unattainable by any single fixed configuration.

Observation 6.1 (VIH Tangency Condition)**.** *$VIH = S \cdot Q$ is maximized on the frontier where the iso-VIH hyperbola $Q = VIH/S$ is tangent to the frontier curve $Q = f(S)$. At this point, the elasticity of quality with respect to speed is exactly -1:*

$$\varepsilon_{Q,S} = \frac{dQ}{dS} \cdot \frac{S}{Q} = -1 \tag{44}$$

Below this point, speed gains improve VIH; above it, speed gains degrade VIH.

Proof. Maximize $\text{VIH}(S) = S \cdot f(S)$. Setting $\text{VIH}'(S) = f(S) + Sf'(S) = 0$ gives $f'(S^*) = -f(S^*)/S^* = -Q^*/S^*$, which is the unit-elasticity condition. □

The convexification theorem implies that the VIH-optimal operating point on the convexified frontier may correspond to a mixing strategy rather than a pure configuration. Specifically, if the VIH-maximizing tangent point lies in a concave region of the original frontier, the optimal policy is to randomize between the two pure strategies at the endpoints of that concave segment. This connects execution mode selection (choosing Fast vs. Governed per task) to Pareto frontier optimization: the Bayesian mode selector is implementing the mixing strategy that achieves the VIH-optimal point on the convexified frontier.

7. The Quality Pillar

The Quality pillar addresses the specific defects introduced by AI code generation and the formal structure of defense against them. The central insight is that AI-generated code produces a qualitatively new class of defect: code that is superficially competent, individually defensible, and cumulatively corrosive to codebase health. We call these defects *slop*, following industry usage.

7.1. Slop Taxonomy: 18 Types in 3 Decidability Classes

The Quality pillar classifies 18 AI code defect types ("slop types") into three decidability classes, grounded in Rice's theorem. Each slop type is formalized as a tuple $\tau_j = (\sigma_j, \phi_j, \delta_j, C_j)$ where σ_j is a syntactic signature on ASTs, ϕ_j is the semantic property determining genuine defect status given full program context, $\delta_j \in \{\mathrm{D}, \mathrm{SD}, \mathrm{U}\}$ is the detection decidability class, and C_j is the cost distribution over deployment contexts.

Table 3 | The 18 slop types partitioned by detection decidability.

Class	Count	Types
D (Decidable)	7	Hallucinated imports, placeholder/stub code, duplicate logic, dead code, lint suppressions, cross-language contamination, outdated APIs
SD (Semi-decidable)	7	Security vulns (functional-looking), happy-path error handling, data model mismatches, over-engineering, architectural erosion, silent scope expansion, god functions
U (Undecidable)	4	Meaningless tests, boilerplate inflation, redundant comments, naming/convention drift

The three classes reflect *practical detectability*, not formal decidability in the computability-theoretic sense. We use the labels D/SD/U as engineering categories indicating the achievable detection reliability, not as claims about Turing-machine decidability of the corresponding decision problems. The distinction between formal decidability (whether an algorithm *exists in principle*) and practical detectability (whether existing tools can achieve reliable detection) is important: Rice's theorem establishes that no algorithm decides arbitrary non-trivial semantic properties of programs, but many specific semantic properties admit effective approximate detection in practice.

1. **Practically Decidable (D):** Detection can be reduced to syntactic or structural checks that achieve near-perfect precision and recall. The syntactic signature σ_j suffices; the semantic property ϕ_j does not depend on unobservable context. Hallucinated imports are detected by lockfile resolution ($O(1)$ table lookup); placeholder code by AST pattern matching for pass, ..., $\mathrm{NotImplementedError}$; duplicate logic by tree-sitter-based clone detection (suffix tree, $O(n \log n)$); dead code by call graph reachability analysis. Note: these are syntactic checks for specific pat-

terns; the fully general semantic question ("is this import unnecessary?") remains undecidable by Rice's theorem, but the syntactic approximation has high practical accuracy.

2. **Partially Detectable (SD):** The semantic property ϕ_j depends on partially formalizable context. An oracle (LLM or human) can approximate ϕ_j with probability $p \in (0.5, 1)$, but no computable function achieves $p = 1$. Security vulnerabilities require knowing which inputs are adversarial; happy-path handling requires knowing which error paths matter; over-engineering requires judging abstraction necessity. Expert inter-rater agreement is $\kappa \approx 0.6$ (substantial, above chance but below certainty). We use "partially detectable" rather than "semi-decidable" to avoid confusion with the computability-theoretic meaning of semi-decidability (recursively enumerable).
3. **Reliably Undetectable (U):** No known procedure achieves reliable detection. For all known methods, inter-rater reliability $\kappa < 0.4$ (below "fair agreement"). "Meaningless test" depends on test intent versus structure; "boilerplate" thresholds vary by team norms; "redundant comment" depends on reader expertise; "naming drift" requires inferring implicit conventions.

This classification has a direct design implication: it is impossible to build a fully automated quality gate that catches all slop types. The four-layer verification architecture (shared with the Reliability pillar) is a necessary response to this computational limitation.

7.2. Three Generative Factors

The 18 slop types arise from three approximately orthogonal generative factors operating at different stages of the generation process:

Definition 7.1 (Context Blindness (F1))**.** *The agent's effective context $\hat{\mathcal{K}}(t) \subset \mathcal{K}(t)$, where $\mathcal{K}(t)$ is the codebase knowledge required for correct generation at point t. The probability of slop type τ_j increases monotonically with the context deficit $|\mathcal{K} \setminus \hat{\mathcal{K}}|$. Types loading on F1: duplicate logic, naming drift, architectural erosion, boilerplate inflation.*

Definition 7.2 (Completion Bias (F2))**.** *The agent's output distribution π assigns higher probability to syntactically complete outputs than to semantically minimal correct outputs: $\mathbb{E}_{\pi}[\text{length}(o)] > \mathbb{E}_{\pi^*}[\text{length}(o)]$ where π^* is optimal. Types loading on F2: placeholder code, happy-path error handling, meaningless tests, redundant comments, over-engineering.*

Definition 7.3 (Training Distribution Leakage (F3))**.** *Patterns memorized during training are applied to contexts where they do not hold. Probability increases with divergence between training distribution $\mathcal{D}$ and execution environment $\mathcal{E}$. Types loading on F3: hallucinated imports, cross-language contamination, outdated APIs, security vulnerabilities.*

If the three factors are approximately orthogonal ($I(F_i; F_k)$ small relative to $H(F_i)$ for $i \neq k$), then interventions targeting different factors compose multiplicatively. Better retrieval (addressing F1) does not reduce completion bias (F2), and vice versa. The quality stack must address all three independently.

7.3. Layered Defense Optimization

Given 18 slop types $J = \{1, \ldots, 18\}$ and four defense layers $L = \{1, 2, 3, 4\}$ (structural gates, deterministic analysis, LLM-as-Judge, human review), the problem is to select $\Lambda_j \subseteq L$ for each type j minimizing total cost:

$$C_{\text{total}} = \sum_{\ell \in L} c_\ell \cdot 1[\Lambda^\ell \neq \emptyset] + \sum_{j \in J} p_j \cdot D_j \cdot P_{\text{esc}}(j, \Lambda_j) \tag{45}$$

where $P_{\text{esc}}(j, \Lambda_j) = P(\bigcap_{\ell \in \Lambda_j} \{Z_{\ell,j} = 0\})$ is the escape probability under correlation, and $\Lambda^\ell = \{j : \ell \in \Lambda_j\}$.

Theorem 7.1 (Layered Defense NP-Hardness)**.** *The problem of assigning slop types to defense layers to minimize total cost subject to a minimum detection rate constraint is NP-hard, via reduction from Weighted Set Cover.*

Proof sketch. Set $D_j = M$ for large M and $p_j = 1$ for all j. Then minimizing the objective forces all types to be covered (the penalty $M \cdot P_{\text{esc}} \gg c_\ell$ for any uncovered type), reducing to minimum-weight subcollection selection, which is Weighted Set Cover. The inapproximability threshold is $\ln |J|$ unless P = NP. □

Proposition 7.1 (Greedy Approximation)**.** *The greedy algorithm (assign each slop type to its highest-efficiency layer first, then fill coverage gaps) achieves a* $(1 - 1/e)$*-approximation, since the objective function is submodular.*

Proof sketch. Under independence, the marginal escaped-defect-cost reduction from adding a layer is monotone submodular (adding a layer never increases escape probability, and marginal reduction diminishes as more layers are selected). The result follows from Nemhauser et al. (1978). □

Remark. *For our problem (*$|L| = 4$*,* $|J| = 18$*), exact enumeration over* $2^4 = 16$ *subsets per type is computationally trivial. The greedy bound matters for scaled versions with dozens of specialized tools.*

7.4. Connection to Littlewood-Strigini and the Diversity Gain

Littlewood and Strigini (1993, 2000) proved that independently developed software versions do not fail independently, because some inputs are inherently difficult. Their result adapts directly to LLM judge-producer pairs: if producer and judge failure probabilities both increase with difficulty, then $P(\text{defect produced and missed}) > P(\text{produced}) \cdot P(\text{missed})$.

Proposition 7.2 (Correlated Judge-Producer Errors)**.** *Under the FKG inequality, if the producer's failure event and the judge's failure event are positively correlated (both more likely for similar inputs), then:*

$$P(\textit{both miss defect}) \geq P(\textit{producer misses}) \cdot P(\textit{judge misses}) \tag{46}$$

The independence assumption underestimates *the probability of undetected defects.*

This result formalizes why LLM-as-Judge systems using the same model family as the code producer underperform expectations. The *diversity gain* from cross-model judging (using judge M_J^B from a different vendor than producer M_P^A) versus same-model judging (M_J^A) is:

$$\Delta_{\text{div}} = P_{\text{esc}}^{A \text{ judges } A} - P_{\text{esc}}^{B \text{ judges } A} \tag{47}$$

This gain has two components: the detection rate difference (d_J^B versus d_J^A) and the correlation reduction ($\rho_{AB} < \rho_{AA}$). Even at equal detection rates, diversity gain is positive whenever $\rho_{AB} < \rho_{AA}$.

Empirically, diverse techniques can achieve effectiveness *greater* than the independence assumption predicts (positive diversity gain), while same-technique repetitions underperform it (Littlewood and Strigini, 2000).

7.5. Optimal Layer Ordering

Theorem 7.2 (ROI Ordering)**.** *Under independence, when layers form a cascade (resolved items skip subsequent layers), cost-optimal ordering sorts by decreasing r_ℓ / c_ℓ, where r_ℓ is the resolution rate.*

Proof sketch. Pairwise exchange argument (Smith's rule): swapping adjacent layers ℓ_a, ℓ_b shows the ordering ℓ_a before ℓ_b is cheaper iff $r_a/c_a > r_b/c_b$. □

Under positive correlation, ROI ordering can be suboptimal. The marginal value of a layer depends more on its independence from existing layers than on its absolute detection rate.

7.6. The Turing Enforcement Principle

Rice's theorem establishes that for any non-trivial semantic property P of programs (depending on input-output behavior, not syntactic form), the set $\{e : \phi_e \in P\}$ is undecidable. This motivates a fundamental enforcement gap:

Theorem 7.3 (Enforcement Gap)**.** *For any syntactic property P_S, there exists a deterministic enforcer with $P(\text{correct enforcement}) = 1$. For any semantic property P_U, every enforcer (including any LLM) satisfies $P(\text{correct enforcement}) < 1$. The gap cannot be closed by prompt engineering alone.*

Proof sketch. Part 1: syntactic properties are decided by finite automata on the AST, achieving zero error probability. Part 2: an LLM is a fixed-weight transformer with finite training data. Rice's theorem establishes that no algorithm decides P_U for all programs; the practical consequence is that achieving $P = 1$ would require correctly classifying programs from an unbounded space, including those arbitrarily far from the training distribution. □

The AgentSpec result (Wang et al., 2026) provides empirical calibration: 87.26% enforcement for code-based rules versus 77% for prompt-based rules. The 10-point gap is the measured enforcement gap.

7.7. Detection Ceiling via Data Processing Inequality

Theorem 7.4 (Detection Ceiling)**.** *For any defense layer ℓ with access to observable features X (code text, AST, test results), the detection rate for defect type j satisfies:*

$$d_{\ell,j} \leq \frac{I(X; D_j)}{H(D_j)} \tag{48}$$

where D_j is the binary indicator of defect presence and $I(X; D_j)$ is the mutual information between observable features and defect status. This follows from Fano's inequality: interpreting detection as a binary hypothesis test, Fano gives $H(D_j \mid \hat{D}_j) \geq H(D_j)(1 - d_{\ell,j})$, and since $H(D_j \mid \hat{D}_j) \geq H(D_j) - I(X; D_j)$, combining yields the stated bound.

The Detection Ceiling makes precise the intuition that some defects cannot be reliably detected from code alone. When the mutual information $I(X; D_j)$ is low (because the defect is about intent, not syntax), no amount of analytical sophistication can raise detection above the ceiling. For example, silent scope expansion requires the task description as context; Layer 1 (structural gates) does not observe the task description, so $d_{1,13} \approx 0$.

7.8. Coupling Complexity and the Accretion Category

Definition 7.4 (Coupling Complexity)**.** *For a codebase C of n files, let $\mu(f_i)$ be the minimum set of files a developer must understand to correctly modify f_i. The* coupling complexity *is:*

$$\Gamma(C) = \frac{1}{n} \sum_{i=1}^{n} \log_2 |\mu(f_i)| \tag{49}$$

A perfectly modular codebase has $\Gamma(C) = 0$; a fully coupled one has $\Gamma(C) = \log_2 n$.

Remark. *This quantity is a mean log-coupling-degree, not a Shannon entropy in the information-theoretic sense (it is not the entropy of a probability distribution). We use the symbol Γ rather than H to avoid suggesting the formal properties of Shannon entropy (additivity under independence, connection to coding theorems) that this quantity does not possess. The name "coupling complexity" reflects its actual meaning: the average number of bits needed to specify the modification context of a random file.*

Definition 7.5 (Accretion Rate)**.** *The accretion rate $\alpha(t) = (\Gamma(C_{t+1}) - \Gamma(C_t))/|\Delta_t|$ measures coupling complexity increase per unit of code change.*

Proposition 7.3 (Compound Degradation)**.** [Engineering Hypothesis.] *A sequence of n individually CI-passing changes, each adding $\Delta\Gamma$ bits of coupling complexity, produces total coupling complexity growth:*

$$\Gamma_{total} \geq n \cdot \Delta\Gamma + \binom{n}{2} \cdot \Delta\Gamma_{cross} \tag{50}$$

where $\Delta\Gamma_{cross} \geq 0$ accounts for cross-change coupling interactions. The growth is superlinear when $\Delta\Gamma_{cross} > 0$.

Proof sketch. Let Γ_0 be the initial coupling complexity and Γ_k the complexity after k changes. Each change k adds code to the codebase, increasing the number of files $n_k = n_0 + k$ and potentially increasing the modification context $|\mu(f_i)|$ for existing files (because the new code may import from or be imported by existing files). The direct contribution of change k is $\Delta\Gamma$ (the coupling complexity of the new code itself). The cross-change contribution arises when change k increases $|\mu(f_i)|$ for files modified by earlier changes $j < k$: if change j added file g_j and change k adds file g_k that imports g_j, then $|\mu(g_j)|$ increases by at least 1, contributing $\Delta\Gamma_{\text{cross}} \geq \log_2((|\mu(g_j)| + 1)/|\mu(g_j)|)$ to the coupling complexity. Summing over all $\binom{n}{2}$ pairs of changes gives the quadratic term. The key assumption is that changes add code without refactoring (no corresponding reduction in $|\mu(f_i)|$ for existing files); when refactoring occurs, $\Delta\Gamma_{\text{cross}}$ can be negative, and the growth rate is reduced. CI checks verify that each change individually compiles and passes tests (pointwise correctness), but do not constrain the global coupling structure $\Gamma(C)$. □

This theorem explains how codebases degrade under AI-generated code even when every individual change passes CI. The degradation is not visible at the per-change level; it accumulates through interactions between changes, manifesting as increasing complexity, decreasing coherence, and rising bug rates.

7.9. The DRE Cascade Theorem

Theorem 7.5 (DRE Cascade)**.** *For L independent layers, cumulative Defect Removal Efficiency for type j is:*

$$D_j = 1 - \prod_{\ell=1}^{L} (1 - d_{\ell,j}) \tag{51}$$

Under a Gaussian copula with correlation matrix R*:*

$$D_j = 1 - C_{\mathrm{R}}(1 - d_{1,j}, \ldots, 1 - d_{L,j}) \tag{52}$$

At correlation $\rho = 1$*, the second layer adds nothing (*$D = d$*). At* $\rho = 0$*, we recover independence.*

Three layers at individually modest rates ($d_1 = 0.3, d_2 = 0.75, d_3 = 0.5$) yield $D = 1-(0.7)(0.25)(0.5) = 0.9125$, exceeding 91% cumulative DRE. Achieving DRE $> 95\%$ requires ≥ 3 diverse techniques (Jones, 2012). A weak independent detector ($d = 0.3, \rho = 0$) can outperform a strong correlated detector ($d = 0.7, \rho = 0.8$) in marginal DRE improvement.

7.10. The Accretion Category

Definition 7.6 (Accretion Defect)**.** *An* accretion defect *is a code change that is: (i) functionally correct (all tests pass), (ii) superfluous (there exists a smaller change* $\Delta' \subset \Delta$ *satisfying the same requirement), (iii) individually defensible (it passes all automated checks and violates no stated standard), and (iv) collectively harmful (cumulative application increases coupling complexity by* $\epsilon > 0$ *per change).*

The Accretion Category overlaps with but is distinct from existing technical debt concepts. Cunningham's original technical debt metaphor (Cunningham, 1992) describes deliberate shortcuts taken for speed; Kruchten et al.'s taxonomy (Kruchten et al., 2012) identifies design debt and architecture debt as individually defensible choices that collectively degrade system quality; Fowler's code smells (especially "speculative generality" and "shotgun surgery") describe structural patterns that resist change. The Accretion Category shares the "individually defensible, collectively harmful" property with design debt, but differs in two respects: (i) the *generative mechanism* is statistical pattern completion rather than intentional choice, meaning the code is produced because it is *probable*, not because a developer decided it was *necessary*; and (ii) *individual detection is undecidable* (requiring exhibition of a smaller sufficient change, which reduces to program equivalence), whereas design debt is typically recognizable by an expert reviewer. Classical defect taxonomies (IEEE 1044, ODC, Beizer, CWE) do not capture this specific combination of properties, though the phenomenon exists on a spectrum with traditional design debt rather than being categorically distinct. The Accretion Category is the characteristic defect mode of AI-generated code, and the primary driver of the GitClear observations (8× duplication, 60% refactoring collapse).

Individual accretion detection is undecidable (requires exhibiting a smaller sufficient change, reducing to program equivalence). Aggregate detection is feasible via statistical process control on entropy proxies: accretion rate, refactoring ratio, clone frequency, lines per function point.

Corollary 7.1 (Refactoring Ratio Threshold)**.** *Let* r *be the fraction of refactoring changes (*$\alpha < 0$*) and* $(1 - r)$ *additions (*$\alpha > 0$*). Equilibrium requires* $r \cdot |\mathbb{E}[\alpha|\textit{refactoring}]| \geq (1 - r) \cdot \mathbb{E}[\alpha|\textit{addition}]$*. GitClear data indicates the pre-AI equilibrium at* $r \approx 0.24$*; the post-AI state at* $r \approx 0.095$ *violates the bound by approximately 2.5×.*

7.11. Cost-of-Quality Model

The Quality pillar develops a cost-of-quality model following the Juran tradition:

$$\mathrm{CoQ} = C_{\text{prevention}} + C_{\text{appraisal}} + C_{\text{internal failure}} + C_{\text{external failure}} \tag{53}$$

The classical 1:10:100 ratio (prevention to internal failure to external failure) requires recalibration by decidability class:

- **Decidable types:** approximately 1:3:100. Detection is cheap (automated at the appraisal stage); production costs remain high.
- **Semi-decidable types:** approximately 1:15:100. Appraisal requires LLM or human review.
- **Undecidable types:** approximately 1:50:30. Appraisal is expensive (deep expert review); but production cost is *lower* because these degrade maintainability, not availability.

For Undecidable types, appraisal cost can exceed discounted external failure cost. It is sometimes economically rational to *accept* these defects.

Theorem 7.6 (Optimal Quality Level)**.** *With defect rate $\lambda(q) = \lambda_0 e^{-\beta q}$, the optimal quality investment is:*

$$q^* = \frac{1}{\beta} \ln\left(\frac{c_F \cdot \lambda_0 \cdot \beta}{c_P + c_A}\right) \tag{54}$$

where c_F is failure cost and $c_P + c_A$ is marginal prevention/appraisal cost.

Proof sketch. Total cost $\text{CoQ}(q) = (c_P + c_A) \cdot q + c_F \cdot \lambda_0 e^{-\beta q}$. Differentiating and setting to zero: $(c_P + c_A) = c_F \cdot \lambda_0 \cdot \beta \cdot e^{-\beta q^*}$. Solving for q^* yields the stated expression. The second derivative is positive, confirming a minimum. □

This gives context-dependent quality targeting: $q^* \approx 1$ for security-critical production code (c_F high), $q^* \approx 0.7$ for internal tools (c_F moderate, Layers 1–2 suffice), $q^* \approx 0.3$ for prototypes (c_F low, Layer 1 only). The appraisal compression problem further constrains the achievable quality: if code generation rate g exceeds appraisal capacity a, a fraction $(g - a)/g$ bypasses appraisal entirely. Sonar data (48% always verify) implies $g/a \approx 2$ on average; roughly half of AI code receives no appraisal (SonarSource, 2026). The quality stack's ordering is not merely cost optimization; it is a *throughput constraint*: only automated layers (1–2) can match AI generation speed.

8. The Governance Pillar

The Governance pillar addresses the system-level problem: how do you maintain architectural coherence when AI agents modify codebases faster than humans can evaluate the implications?

The Governance pillar addresses four distinct sub-concerns, each with its own character: *theory preservation* (§8.1–§8.3, philosophical), *governance information processing* (§8.4–§8.5, information-theoretic), *feedback control* (§8.6–§8.8, engineering), and *decision lifecycle management* (§8.9–§8.11, formal methods / management science).

8.1. Theory Loss as Rate-Distortion

[Philosophical: formalizes Naur's insight about program theory.]

Following Naur (1985), a *program theory* T is the totality of a developer's understanding: the mapping from problem domain to code, design rationale, rejected alternatives, and the causal model of change propagation. The Governance pillar formalizes theory externalization as a rate-distortion problem:

Theorem 8.1 (Theory Externalization Bound)**.** *The minimum description length of a program theory T under distortion constraint D satisfies:*

$$R(D) = \min_{p(\hat{T}|T):\mathbb{E}[d(T,\hat{T})] \leq D} I(T;\hat{T}) \tag{55}$$

where $\hat{T}$ is the externalized representation (documentation, ADRs, comments) and $d(T, \hat{T})$ is a distortion measure.

[Philosophical Observation: the following illustrative formalization models Polanyi's insight using rate-distortion theory. The mathematical result is standard; the substantive claim is whether the modeling choice is appropriate.]

Proposition 8.1 (Tacit Divergence, Illustrative Formalization)**.** *Decompose the program theory as $T = (T_{\text{explicit}}, T_{\text{tacit}})$. If T_{explicit} is modeled as a discrete random variable over a finite alphabet (design decisions, API choices, data structure selections) and T_{tacit} is modeled as a continuous random variable (intuitive weightings, aesthetic judgments, causal models of change propagation), then under squared-error distortion for the continuous component:*

$$\lim_{D \to 0} R_{\text{tacit}}(D) = \infty \tag{56}$$

with $R_{\text{tacit}}(D) = \Omega(\log(1/D))$ as $D \to 0$. In contrast, $R_{\text{explicit}}(D)$ remains finite for all $D \geq 0$ (since a discrete source with finite entropy has bounded rate-distortion).

Remark (On the modeling choice)**.** *The mathematical content above is a standard result from rate-distortion theory (for a Gaussian source, $R(D) = \frac{1}{2}\log(\sigma^2/D)$, which diverges logarithmically). The entire weight of the claim rests on whether the continuous-source model is appropriate for tacit knowledge. Polanyi's (Polanyi, 1966) original point is stronger than this formalization suggests: his argument is that tacit knowledge is not the kind of thing that admits metric spaces and distortion functions at all, because it is inseparable from the knower and constituted by practice rather than by signal structure. We offer this formalization not as a faithful rendering of Polanyi but as an illustrative model that captures one consequence of his insight (perfect externalization requires unbounded description length) within the information-theoretic vocabulary of this paper. If tacit knowledge is better modeled as a discrete source with finite entropy (i.e., if design intuitions can be fully captured by a finite codebook), the divergence result does not hold and the formalization is misleading. This is an empirical question: it would be falsified by demonstrating that expert design intuitions can be losslessly captured in a finite set of rules.*

The practical implication, regardless of the formal model, is that no governance scheme can fully externalize program theory. Some knowledge will always be lost when the original developers (human or AI) are no longer involved. This is a design constraint to be managed, not a problem to be solved.

8.2. Collins's Tacit Knowledge Taxonomy

Collins (2010) advances beyond Polanyi (1966) by taxonomizing tacit knowledge into three types with distinct externalization properties:

Type	What Governance Can Capture	What Resists Capture
RTK (Relational)	Decision context, rationale, rejected alternatives, advice received	Obvious-to-the-writer context
STK (Somatic)	Checklists, review heuristics, examples (lossy)	Debugging intuition, code quality "smell"
CTK (Collective)	(Nothing)	Team norms, "good enough" thresholds

Relational tacit knowledge (RTK) is knowledge that *could* be made explicit but has not been, due to social contingency; ADRs and governance tools operate effectively here. Somatic tacit knowledge (STK) resides in embodied practice, partially capturable through instrumentation but not transmissible through text. Collective tacit knowledge (CTK) is "embodied in society," acquired only through enculturation, and resistant to externalization by any known means.

The Dreyfus model adds a further constraint: expert knowledge is specifically *less* amenable to externalization than novice knowledge. The more expert a practitioner, the more their knowledge resides in pattern recognition that resists decomposition into rules. AI agents operating at the "competent" level of the Dreyfus hierarchy lack the intuitive, holistic perception of proficient and expert practitioners.

8.3. The False Confidence Problem

If Naur (1985) is correct, then any governance artifact (THEORY.md, structured ADRs, extracted decision logs) is a lossy compression. The map-territory problem applies: documentation is a map, not the territory. The risk: a team possessing documentation may believe it understands the system's theory when it actually possesses only a compressed, lossy representation. This could be worse than having no documentation, because no documentation produces appropriate humility ("we don't know why this is here"), while lossy documentation produces false confidence ("THEORY.md says it's for X, so we'll change it accordingly").

However, this argument has practical limits. Lossy compression is still enormously useful (JPEG images demonstrate this). Documentation with explicit uncertainty markers ("confidence: medium," "last validated: 2025-03") addresses the false confidence risk.

8.4. The Double Bottleneck Theorem

[Information-theoretic: bounds on governance fidelity.]

AI coding agents produce verbose execution logs L (often 100K+ tokens). A governance system must extract structured knowledge (D, A, R) (decisions, assumptions, rejected alternatives). This forms a Markov chain: $T \to L \to (D, A, R) \to G$, where G represents governance actions.

Theorem 8.2 (Double Bottleneck)**.** *By the data processing inequality, applied twice:*

$$I(T; G) \leq I(T; (D, A, R)) \leq I(T; L) \tag{57}$$

Each stage of the governance pipeline loses information. Governance fidelity is bounded above by log richness.

Corollary 8.1 (Log Richness Lower Bound)**.** *For governance to achieve fidelity $I(T; G) \geq F_{\min}$, the agent log must satisfy $I(T; L) \geq F_{\min}$. Terse logs impose an upper bound on governance fidelity regardless of extraction sophistication.*

8.5. The Governance Capacity Bound

[Information-theoretic: rate-stability condition for governance.]

Proposition 8.2 (Governance Rate-Stability Principle)**.** *Let R_{drift} be the rate at which architectural drift is injected (corrections needed per time unit) and C_{gov} the governance throughput (corrections the governance system can process per time unit). Then:*

1. *If $R_{drift} < C_{gov}$, the governance system can maintain bounded architectural drift through layered correction (linting catches syntactic drift, CI tests catch behavioral drift, human review catches semantic drift, ADRs catch strategic drift).*
2. *If $R_{drift} > C_{gov}$, drift accumulates faster than it is corrected, and architectural coherence degrades monotonically. This is the queue-theoretic instability condition: when arrival rate exceeds service rate, the queue grows without bound.*

Justification. This is a rate-stability argument, analogous to the condition for queue stability in queueing theory: a queue is stable if and only if the arrival rate is strictly less than the service rate. Part (1): when $R_{\text{drift}} < C_{\text{gov}}$, the governance system has surplus capacity; each drift event is eventually corrected, and the backlog remains bounded. Part (2): when $R_{\text{drift}} > C_{\text{gov}}$, the backlog of uncorrected drift events grows linearly in time; no finite reallocation of governance effort within the existing capacity can reverse this. The argument does not require the formal prerequisites of Shannon's channel coding theorem (stationary transition probabilities, finite input alphabet, block coding over increasing lengths), which the "governance channel" lacks. The principle's value is in design guidance: governance systems have finite corrective bandwidth, that bandwidth is measurable in principle, and exceeding it produces qualitatively different (unbounded) behavior. □

Remark. *Earlier versions of this result were framed as a structural analogy to Shannon's noisy channel coding theorem. We have reformulated it as a rate-stability condition, which is the appropriate formal framework. The qualitative insight is the same (finite governance bandwidth implies a threshold beyond which no scheme prevents divergence), but the rate-stability framing avoids importing the formal prerequisites of Shannon's theorem that the governance setting does not satisfy.*

For a layered governance system with L independent layers, the combined capacity satisfies $C_{\text{gov}} \leq \sum_{\ell=1}^{L} C_\ell$, with equality when layers operate on non-overlapping aspects. When AI agents increase the modification rate λ while C_{gov} remains constant (human review bandwidth is fixed), the condition $R_{\text{drift}} > C_{\text{gov}}$ is eventually satisfied. Automated governance layers (fitness functions, architectural tests, invariant checkers) are therefore *necessary*, not merely convenient.

8.6. Multi-Granularity Governance as Cascade Control

[Engineering: feedback control architecture for governance loops.]

The Governance pillar models the governance system as a cascade control system with three loops:

1. **Inner loop** (synchronous, per-generation): validates each code generation against syntax rules, linting constraints, and file-ownership invariants. Sampling period T_1 (fastest). Bandwidth: high.
2. **Middle loop** (asynchronous, per-phase): runs compliance audits, reflexion model checks, and contract verification at phase boundaries. Sampling period $T_2 \gg T_1$. Bandwidth: moderate.
3. **Outer loop** (deliberative, per-milestone): performs ATAM tradeoff analysis, strategic alignment review, and ADR lifecycle management. Sampling period $T_3 \gg T_2$. Bandwidth: low.

Let $D(z)$ denote drift introduced by agent-generated code. Each loop has controller $C_k(z)$ and plant $P_k(z)$. The inner loop's closed-loop transfer function is:

$$G_{\text{inner}}(z) = \frac{C_1(z)P_1(z)}{1 + C_1(z)P_1(z)} \tag{58}$$

Each outer loop treats the inner loop's output as its plant.

Theorem 8.3 (Cascade Governance Stability)**.** *The three-loop system is stable if and only if:*

1. *Each loop is individually stable (all poles of $G_k(z)$ lie within the unit circle).*
2. *Bandwidth separation holds: $\omega_{\text{bw},1} > \omega_{\text{bw},2} > \omega_{\text{bw},3}$.*
3. *Each outer loop treats the inner as approximately unity gain within its bandwidth.*

Proof sketch. Condition (1) follows from BIBO stability of discrete-time systems. Condition (2) ensures frequency-domain decoupling; without it, outer loop corrections feed into the inner loop's operating range, creating destructive interference. Condition (3) ensures the outer loop's model of the inner loop is accurate within its bandwidth. Under these conditions, the combined characteristic polynomial factors approximately into three independent polynomials. □

Classical cascade control requires a 3:1 to 5:1 bandwidth ratio between adjacent loops. If the inner loop checks every generation (seconds), the middle loop should operate on minutes-to-hours timescales, and the outer loop on days-to-weeks.

8.7. Nyquist Sampling Requirements

Let $\omega_d^{(k)}$ be the maximum frequency of architectural drift relevant to loop k. Each governance loop must sample at:

$$f_k \geq 2\omega_d^{(k)} \tag{59}$$

Concrete rate calculations illustrate the constraint. If an AI agent produces 50 code generations per hour, the inner loop must sample at least 100 events per hour to capture per-generation violations. If emergent drift (coupling growth, convention erosion) manifests on a per-PR timescale and a team merges 10 PRs per day, the middle loop requires at least 20 audit events per day. If strategic misalignment accumulates over weeks, the outer loop requires review at least every 3.5 days to satisfy the Nyquist condition.

If the outer loop samples too slowly, it aliases gradual architectural erosion as apparent stability, missing drift until it becomes catastrophic. When AI agents increase the disturbance frequency (more changes per unit time), the sampling rate of each loop must increase accordingly, or the loop becomes unstable.

8.8. The Governance Bifurcation

[Engineering Hypothesis: the following model assumes specific functional forms that have not been empirically validated. The qualitative insight (a threshold separating self-correcting from divergent regimes) is robust to the functional form; the quantitative predictions are not.]

Define architectural coherence $C(t) \in [0, 1]$ as a scalar conformance measure. The dynamics are:

$$\frac{dC}{dt} = \mu \cdot G(t) \cdot (1 - C)^{\beta} - \gamma_g \cdot R(t) \cdot (1 - C) \tag{60}$$

where $G(t)$ is governance intervention rate, $R(t)$ is agent generation rate, μ is governance effectiveness, γ_g is agent drift propensity (subscripted to distinguish from the common-cause failure rate γ in the Reliability pillar), and $\beta > 1$ captures diminishing governance returns. The asymmetry (β in the governance term but not the drift term) reflects the empirical observation that easy violations are caught first while subtle ones require disproportionate effort, whereas drift injection is approximately linear in agent activity.

Remark (Functional form justification)**.** *The specific functional forms are modeling choices, not derived from first principles. The governance term $(1-C)^\beta$ with $\beta > 1$ encodes diminishing returns to governance effort as coherence improves: the first violations caught are obvious (linting failures, import violations); the last are subtle (architectural erosion, convention drift). This is consistent with the geometric diminishing returns of verification (Theorem 4.2). The drift term $\gamma_g R(1-C)$ is linear in both generation rate R and the remaining incoherence $(1-C)$, encoding the assumption that drift injection is proportional to activity and opportunity. Alternative functional forms (e.g., saturating drift, sigmoidal governance) would produce quantitatively different bifurcation points but preserve the qualitative threshold structure. The bifurcation at $\mu G/\gamma_g R = 1$ should be understood as a model prediction, not an empirical finding; the specific ratio at which real systems transition is an open empirical question.*

Proposition 8.3 (Governance Bifurcation (Model-Dependent))**.** [Engineering Hypothesis: quantitative threshold depends on assumed functional form.] *The system undergoes a transcritical bifurcation at $\mu G/\gamma_g R = 1$:*

- ***Supercritical*** *($\mu G > \gamma_g R$): a stable interior fixed point at $C^* = 1 - (\gamma_g R/\mu G)^{1/(\beta-1)} > 0$. The system self-corrects.*
- ***Subcritical*** *($\mu G < \gamma_g R$): the only stable fixed point is $C^* = 0$ (total coherence collapse).*

Near the bifurcation, recovery time diverges (critical slowing down), and small perturbations in $R(t)$ cause qualitative regime changes.

Proof sketch. Setting $dC/dt = 0$ and factoring yields $(1-C)[\mu G(1-C)^{\beta-1} - \gamma_g R] = 0$. The boundary fixed point $C = 1$ is always present and is stable (linearization gives $dC/dt \approx -\gamma_g R \cdot \Delta C$ near $C = 1$ since $(1-C)^{\beta-1} \to 0$ for $\beta > 1$), but it is unreachable from any $C < 1$ in finite time when $\beta > 1$, because both drift and governance forces vanish as $C \to 1$. The interior fixed point $C^* = 1 - (\gamma_g R/\mu G)^{1/(\beta-1)}$ exists only when $\mu G > \gamma_g R$ and is the practically relevant attractor. When $\mu G < \gamma_g R$, the drift term dominates everywhere in $(0, 1)$, driving coherence to zero. The interior fixed point exchanges stability with $C = 0$ at $\mu G/\gamma_g R = 1$, producing a transcritical bifurcation at that critical ratio. □

The phase portrait has two regimes separated by a sharp transition. In the supercritical regime, perturbations away from C^* are corrected by governance feedback. In the subcritical regime, coherence degrades monotonically. With $\beta > 1$, the equilibrium coherence is always strictly less than 1: perfect conformance is asymptotically unattainable, though the gap shrinks as G/R increases.

8.9. The Ratchet Lattice

[Formal methods: lattice-theoretic model of governance constraint evolution.]

Let $\mathcal{L} = (2^{\mathcal{R}}, \subseteq)$ be the power-set lattice over a universe of governance rules $\mathcal{R}$. A governance state at time t is an element $S_t \in \mathcal{L}$.

Definition 8.1 (Governance Ratchet)**.** *A ratchet is a closure operator $\rho : \mathcal{L} \to \mathcal{L}$ on the constraint lattice $\mathcal{L}$, satisfying: (i) extensiveness: $S \subseteq \rho(S)$ (rules are only added); (ii) monotonicity: $S \subseteq S' \implies \rho(S) \subseteq \rho(S')$; (iii) idempotence: $\rho(\rho(S)) = \rho(S)$.*

The ADR acceptance workflow is literally a closure operation: accepting a decision adds constraints, those constraints may force additional constraints (transitive closure), and the result is stable under re-application.

Corollary 8.2 (Ratchet Convergence)**.** *Let ρ be a governance ratchet on a finite rule universe $\mathcal{R}$. The sequence $S_0, S_1 = \rho(S_0), S_2 = \rho(S_1), \ldots$ converges to a fixed point $S^* = \rho(S^*)$ in at most $|\mathcal{R}|$ steps.*

Proof sketch. By extensiveness, $S_t \subseteq S_{t+1}$, so the sequence is monotonically non-decreasing. Since $\mathcal{R}$ is finite, $\mathcal{L}$ has finite height $|\mathcal{R}|$. A monotone chain of finite height stabilizes. Alternatively, by the Knaster-Tarski theorem, every monotone operator on a complete lattice has a least fixed point. □

Corollary 8.3 (Ossification Risk)**.** *If $\mathcal{R}$ contains contradictory rules ($\exists\, r_1, r_2 \in \mathcal{R}$ with no code satisfying both) and the ratchet is purely additive, then any trajectory including both r_1 and r_2 produces an ossified state (where $\mathrm{Allowed}(S) = \emptyset$).*

Proof. By extensiveness, once $r_1 \in S_t$ and $r_2 \in S_{t'}$, both persist in all subsequent states. If $r_1 \wedge r_2$ is unsatisfiable, $\mathrm{Allowed}(S) = \emptyset$. □

8.10. Controlled Relaxation with Evidence Expiry

The ossification corollary provides formal justification for relaxation mechanisms. Define a relaxation operator $\delta : \mathcal{L} \to \mathcal{L}$ with $\delta(S) \subseteq S$. The combined governance operator is:

$$\Gamma(S) = \rho(S \setminus \delta(S)) \tag{61}$$

First remove expired or superseded rules via δ, then apply the ratchet closure ρ. Relaxation may be triggered by evidence expiry (a rule's justifying evidence has a validity window), decision supersession (a newer ADR replaces an older one), or explicit deprecation.

Theorem 8.4 (Controlled Descent)**.** *If δ removes only rules whose justifying evidence has expired, and ρ re-derives still-justified consequences, then Γ produces states that are both internally consistent and evidence-backed.*

The evidence expiry window for each rule should be proportional to its domain-specific median lifetime, connecting the ratchet to the survival analysis below.

8.11. Decision Survival Analysis with Competing Risks

[Management science: temporal validity of architectural decisions.]

Architectural decisions have finite lifetimes. The survival function is $S(t) = P(\text{decision valid at time } t) = \exp\left(-\int_0^t h(u)\,du\right)$ where $h(u)$ is the hazard function. Decisions fail for multiple independent reasons. Let K index competing risks:

k	Risk Type	Description
1	Technology obsolescence	Chosen technology superseded
2	Requirement change	Business requirements shift
3	Evidence expiry	Justifying data becomes stale
4	Organizational change	Team structure or strategy shifts

The cause-specific hazard for risk k is:

$$h_k(t) = \lim_{\Delta t \to 0} \frac{P(t \le T_d < t + \Delta t,\ K_d = k \mid T_d \ge t)}{\Delta t} \tag{62}$$

with overall hazard $h(t) = \sum_k h_k(t)$. The cumulative incidence function is:

$$F_k(t) = \int_0^t h_k(u) \exp\left(-\int_0^u h(\tau)\, d\tau\right) du \tag{63}$$

Empirically calibrated decay rates (using Weibull hazard $h(t) = (k/\lambda)(t/\lambda)^{k-1}$ with shape k and scale λ):

Decision Domain	Dominant Risk	Weibull α	Scale λ (mo.)	Median Life
Frontend framework	$k = 1$	1.5–2.5	2–5	1–4 mo.
API contract	$k = 2$	1.2–1.8	5–12	3–9 mo.
Database schema	$k = 3$	1.0–1.4	18–36	12–36 mo.
Security policy	$k = 4$	0.8–1.2	12–24	8–18 mo.

These parameter ranges are *illustrative estimates* derived from practitioner experience and ecosystem churn rates. No controlled longitudinal study of decision decay rates exists. The qualitative ordering (frontend decisions decay fastest; database decisions are most durable) is better supported than the specific numerical values.

8.12. The Theory Preservation Problem

The Governance pillar identifies the *theory preservation problem* as fundamentally open: how do you preserve program theory across agent generations, context window boundaries, and team transitions? This is not merely a documentation problem. Documentation captures the explicit component of theory; the tacit component (Naur's "direct, intuitive knowledge," Polanyi's personal knowledge, Collins's mimeomorphic actions) resists formalization.

When AI generates a substantial fraction of code, a new failure mode appears: *theoretically orphaned implementations*, code for which the theory was never in anyone's head. Current approaches (Architecture Decision Records, ARCHITECTURE.md, embedded comments, convention-by-example) are partial solutions. Each captures some explicit knowledge; none captures tacit knowledge. The theory preservation problem is the long-term challenge that will determine whether AI-assisted software development produces maintainable systems or disposable ones.

9. The Economics Pillar

The Economics pillar addresses the binding constraint for AI coding agent deployment at scale: cost. A single 50-iteration agentic coding session can consume 2–5 million tokens, costing $3–$50 depending on model tier and caching configuration. Organizations running hundreds of such sessions daily confront annual token budgets of $200K–$2M, placing AI coding costs in the same category as cloud infrastructure expenditure (FinOps Foundation, 2023). Yet no existing harness includes an explicit cost optimization layer. The central structural insight is an asymmetry: quality aggregates multiplicatively across pipeline stages while cost aggregates additively, creating a distinctive optimization landscape.

9.1. Pipeline Model and Quality Axioms

Definition 9.1 (Pipeline)**.** *A* pipeline *is a tuple* $\Pi = (K, \mathrm{r}, \mathrm{p}, \mathrm{q})$ *where* $K \in \mathbb{N}$ *is the number of stages,* $\mathrm{r} = (r_1, \ldots, r_K) \in \mathbb{R}^K_{\geq 0}$ *is the token allocation vector,* $\mathrm{p} = (p_1, \ldots, p_K) \in \mathbb{R}^K_{>0}$ *is the price vector, and* $\mathrm{q} = (q_1, \ldots, q_K)$ *is the quality function vector.*

A function $q : \mathbb{R}_{\geq 0} \to [0, 1]$ is a *quality function* if it satisfies: **(Q1)** $q(0) = 0$; **(Q2)** q is non-decreasing; **(Q3)** $\lim_{r\to\infty} q(r) \leq 1$; **(Q4)** q is concave; and **(Q5)** q is continuously differentiable on $\mathbb{R}_{>0}$. Concavity **(Q4)** is the critical structural assumption, encoding diminishing returns: the first thousand tokens of context contribute more to quality than the hundred-thousandth. Empirical evidence from (Gao et al., 2025) and (Liu et al., 2024) supports this for retrieval-augmented and long-context settings, though real quality functions may exhibit non-monotone behavior at extreme context lengths (Chroma Research, 2025).

Definition 9.2 (Pipeline Quality and Cost)**.** *Given a pipeline* Π*, the* aggregate quality *and* aggregate cost *are:*

$$Q(\mathrm{r}) = \prod_{k=1}^{K} q_k(r_k), \tag{64}$$

$$C(\mathrm{r}) = \sum_{k=1}^{K} p_k \cdot r_k. \tag{65}$$

The multiplicative structure reflects a fundamental operational reality: if any single stage fails completely ($q_k = 0$), the entire pipeline output is worthless. This is the software reliability series-system model of (**?**) applied to cognitive pipeline stages. The additive cost structure is straightforward: token costs accumulate linearly.

9.2. The CVIH Metric

Definition 9.3 (CVIH)**.** [Engineering Hypothesis: depends on empirical estimates of λ_{raw} and p_{verify}.] *The* cost-adjusted verified iterations per hour *(CVIH) is:*

$$CVIH = \frac{\lambda_{raw} \cdot p_{verify}}{C_{total}}, \tag{66}$$

where λ_{raw} is the raw iteration rate (iterations per hour), $p_{verify} \in [0, 1]$ is the probability that an iteration produces a verified-correct result, and C_{total} is the total cost per iteration.

CVIH measures *verified progress per dollar-hour*. It is not directly computable from first principles; it depends on empirical estimates that are themselves functions of the token allocation and model selection. We treat CVIH as an empirical metric that can be *optimized* using the formal framework, not as a quantity that can be computed from axioms alone.

9.3. The Token Budget Allocation Problem

Definition 9.4 (TBAP)**.** *The* Token Budget Allocation Problem *is:*

$$\begin{aligned} \underset{\mathrm{r}\in\mathbb{R}^K_{\geq 0}}{\text{maximize}} \quad & Q(\mathrm{r}) = \prod_{k=1}^{K} q_k(r_k) \\ \textit{subject to} \quad & \sum_{k=1}^{K} p_k \cdot r_k \leq B, \end{aligned} \tag{67}$$

where $B > 0$ is the total budget.

Since $\log$ is strictly increasing, the TBAP is equivalent to maximizing $\sum_{k=1}^{K} \log q_k(r_k)$ subject to the same constraint (restricting to $\mathrm{r} > 0$, since $q_k(0) = 0$ by **Q1** makes zero allocation suboptimal). Under axioms **Q1–Q5**, this log-transformed objective is concave: each $\log \circ q_k$ is concave by the composition rule for concave functions (Boyd and Vandenberghe, 2004), making the TBAP a concave maximization problem with a linear constraint. The KKT conditions characterize its solution.

Proposition 9.1 (Equal Marginal Rate)**.** [Mathematical Fact: follows from KKT conditions of the log-transformed concave program.] *Let* r^* *be an optimal solution to the TBAP with* $r_k^* > 0$ *for all* k*. Then there exists* $\lambda^* \geq 0$ *such that for every stage* k*:*

$$\frac{q_k'(r_k^*)}{q_k(r_k^*) \cdot p_k} = \lambda^*. \tag{68}$$

That is, the marginal quality return per dollar *is equalized across all stages at optimality.*

Proof sketch. The KKT conditions for the log-transformed problem require $q_k'(r_k^*)/q_k(r_k^*) = \lambda^* p_k$ for all k. Dividing by p_k yields the result. Existence of λ^* follows from Slater's condition (the feasible set has non-empty interior). If each q_k is strictly concave, uniqueness of r^* follows from strict concavity of $\sum_k \log q_k(r_k)$ (Rockafellar, 1970). □

The ratio $q_k'(r_k^*)/(q_k(r_k^*) \cdot p_k)$ measures the percentage improvement in stage-k quality per additional dollar spent. At optimality, this ratio is the same for all stages, the token-economics analogue of the equal marginal utility per dollar condition from consumer theory (Mas-Colell et al., 1995). The solution structure is identical to the classical water-filling solution for parallel Gaussian channels (Cover and Thomas, 2006): stages with high price p_k or low responsiveness $q_k'(\cdot)$ receive less budget.

Pareto frontier. As the budget B varies, the optimal quality $Q^*(B)$ traces the cost-quality Pareto frontier. Under axioms **Q1–Q5**: (i) Q^* is non-decreasing in B; (ii) $\log Q^*$ is concave in B (i.e., Q^* is log-concave), since the log-transformed objective is concave and the feasible set is convex in B; (iii) the slope of the Pareto frontier at budget B equals the optimal Lagrange multiplier: $\frac{d}{dB} \log Q^*(B) = \lambda^*(B)$, by the envelope theorem. Log-concavity means that earlier dollars contribute more than later dollars, a crucial structural property that informs the CVIH budget separation result (Theorem 9.3).

9.4. The Weakest-Link Observation

Observation 9.1 (Weakest-Link, Symmetric Case)**.** [Mathematical Fact: follows from AM-GM inequality and the EMR condition.] *In the special case where all stages have identical quality functions* ($q_k = q$ *for all* k*) and identical prices* ($p_k = p$ *for all* k*), the optimal allocation is uniform:* $r_k^* = B/(Kp)$ *for all* k*.*

Proof sketch. By the EMR condition, $q'(r_k^*)/(q(r_k^*) \cdot p) = \lambda^*$ for all k. Since q is strictly concave and p is identical across stages, this requires $r_k^* = r_j^*$ for all k, j. The budget constraint gives $r_k^* = B/(Kp)$. □

More generally, for *any* allocation, the AM-GM inequality bounds the product: $\prod_{k=1}^{K} q_k(r_k) \leq \left(\frac{1}{K} \sum_{k=1}^{K} q_k(r_k)\right)^K$, with equality if and only if all quality values are equal.

Corollary 9.1 (Raise the Floor)**.** *Given a current allocation* r *with* $q_j(r_j) < q_k(r_k)$ *for some stages* j, k*, the marginal return from increasing* r_j *(the weaker stage) exceeds the marginal return from increasing* r_k *(the stronger stage), provided both are in the concave region.*

Concrete example. A pipeline with qualities $(0.95, 0.92, 0.88, 0.60, 0.93, 0.91)$ has aggregate quality $Q = 0.382$. Improving the weakest stage (0.60) to 0.80 yields $Q = 0.509$, a 33% improvement. Improving the best stage (0.95) to 0.99 yields $Q = 0.398$, only a 4% improvement.

9.5. Model Tier Selection

In practice, the price p_k is not fixed; it depends on which model is assigned to each stage. Let $\mathcal{M} = \{m_1, \ldots, m_M\}$ be available models, each with price $p^{(i)}$ and a stage-specific capability modifier $\gamma_k^{(i)} \in [0, 1]$ scaling the quality function.

Definition 9.5 (Model Tier Selection Problem)**.** *The* MTSP *assigns models to stages to minimize cost subject to per-stage capability thresholds $\gamma_k^{\min}$ and a minimum pipeline quality $Q_{\min}$:*

$$\begin{aligned} &\underset{\sigma:[K]\to[M]}{\text{minimize}} \quad \sum_{k=1}^{K} p^{(\sigma(k))} \cdot r_k \\ &\textit{subject to} \quad \gamma_k^{(\sigma(k))} \geq \gamma_k^{\min} \ \ \forall k, \quad \prod_{k=1}^{K} \gamma_k^{(\sigma(k))} \cdot q_k(r_k) \geq Q_{\min}. \end{aligned} \tag{69}$$

Proposition 9.2 (Complexity)**.** [Mathematical Fact: reduction from 0-1 knapsack.] *The MTSP is NP-hard in general (when M and K are both part of the input), by reduction from 0-1 knapsack. However, for fixed M, it is solvable in $O(M^K)$ time by enumeration. For typical values ($M \in \{3, 4, 5\}$ and $K \in \{5, 6, 7, 8\}$), this yields at most $5^8 = 390{,}625$ candidate assignments, which is tractable.*

The full optimization combines TBAP and MTSP into a Joint Allocation-Selection Problem (JASP): for each candidate assignment σ, the inner problem (optimizing r given σ) is a TBAP instance, which is convex and efficiently solvable. The outer problem enumerates over at most M^K assignments.

Worked example. With 3 model tiers (frontier at \$15/Mtok with $\gamma = 1.0$; mid-tier at \$3/Mtok with $\gamma \in [0.7, 0.95]$; economy at \$0.25/Mtok with $\gamma \in [0.4, 0.8]$) and 6 stages, the $3^6 = 729$ candidate assignments solve in under one second. The optimal assignment typically places the frontier model on quality-sensitive stages (planning, code generation) while routing less sensitive stages (formatting, simple retrieval) to economy tiers, achieving 70–85% of all-frontier quality at 20–35% of the cost.

9.6. Queueing Economics of Developer Waiting Time

AI coding agents operate in two fundamentally different regimes with different cost structures:

Definition 9.6 (Effective Cost)**.** *The* effective cost per token *in regime $\mathcal{R}$ is:*

$$p_{eff}^{(\mathcal{R})} = p_m + \frac{c_w^{(\mathcal{R})}}{s_m}, \tag{70}$$

where p_m is the per-token price of model m, $c_w^{(\mathcal{R})}$ is the cost of developer waiting time per unit time in regime $\mathcal{R}$, and s_m is the throughput of model m (tokens per unit time).

In the **interactive regime**, $c_w^{(\text{int})}$ equals the developer's fully loaded hourly rate (\$90–\$200/hr); latency directly impacts flow state. In the **autonomous regime**, $c_w^{(\text{auto})} \approx 0$ since the developer is productive elsewhere.

Proposition 9.3 (M/M/1 Optimal Service Rate)**.** [Mathematical Fact: follows from minimizing the M/M/1 cost function.] *In the M/M/1 model with waiting cost c_w per unit time and service cost c_s per unit service rate, the optimal service rate is:*

$$\mu^* = \lambda + \sqrt{\frac{c_w \cdot \lambda}{c_s}}. \tag{71}$$

Proof sketch. The total cost rate is $c_w\lambda/(\mu - \lambda) + c_s\mu$ (Kleinrock, 1975). Taking the derivative with respect to μ, setting to zero, and solving yields the result. □

For mixed workloads containing both interactive and autonomous tasks, the classical $c\mu$ rule (Van Mieghem, 1995) dictates priority ordering: in a single-server system, the policy that minimizes total weighted waiting cost processes tasks in decreasing order of $c_j\mu_j$, where c_j is the per-unit-time waiting cost and μ_j is the service rate. For AI coding agents, interactive tasks (high c_j) should always be prioritized over autonomous tasks (low c_j).

Empirical research on interruption costs (Leroy, 2009; Mark et al., 2008) suggests that developer context-switching cost is not a smooth function of waiting time but a step function with a critical threshold:

$$c_{\text{flow}}(w) = \begin{cases} 0 & \text{if } w < w_{\text{thresh}}, \\ c_{\text{switch}} & \text{if } w \geq w_{\text{thresh}}, \end{cases} \tag{72}$$

where $w_{\text{thresh}} \approx 120$–$180$ seconds and c_{switch} is estimated at 15–30 minutes of reduced productivity ($25–$100 at loaded developer rates). This step-function structure implies a clear model selection criterion: for interactive work, choose the cheapest model whose response time is below w_{thresh}; for autonomous work, minimize p_m/γ_m (cost per unit capability) regardless of latency. In practice, this often means using a faster mid-tier model for interactive work and a slower frontier model for autonomous batch processing, the opposite of the naive "use the best model for everything" strategy.

9.7. The Jevons Paradox for Token Economics

[Engineering Hypothesis: the elasticity estimate depends on aggregate market data with wide confidence bounds.]

The Jevons paradox (Jevons, 1865) states that improvements in efficiency of resource use tend to *increase* rather than decrease total resource consumption. We formalize this for token markets.

Observation 9.2 (Jevons Condition)**.** [Mathematical Fact: follows from differentiation of $S(p) = p \cdot D(p)$.] *Let $\epsilon = -\frac{p}{D(p)} \cdot \frac{dD}{dp}$ be the price elasticity of demand. Total spend $S(p) = p \cdot D(p)$ increases when price p decreases if and only if $\epsilon > 1$.*

Proof sketch. $dS/dp = D(p)(1 - \epsilon)$. Since $D(p) > 0$, $dS/dp < 0$ iff $\epsilon > 1$. □

From publicly available data (April 2023 to April 2026): frontier model input prices fell from approximately $30/Mtok to approximately $3/Mtok (roughly $10\times$; with caching, effectively $50\times$), while aggregate token consumption grew by an estimated 100–500×. The midpoint arc elasticity yields $\epsilon \approx 1.19$ (range 1.1–1.3), placing the token market firmly in the elastic regime. This estimate conflates intensive and extensive margins, and no confidence interval can be constructed from two aggregate data points, but the qualitative conclusion ($\epsilon > 1$) is robust: the direction is unambiguous.

Corollary 9.2 (Jevons Imperative)**.** *When $\epsilon > 1$, falling per-token prices* increase *total organizational token spend. The value of cost optimization grows as prices fall.*

This is counterintuitive. The naive expectation is that falling prices eliminate the need for cost optimization. The Jevons paradox reverses this: falling prices stimulate demand expansion that more than offsets per-unit savings, making budget discipline more valuable, not less. Token demand expansion has two components: the *intensive margin* (existing pipelines use more tokens per task through longer contexts, more iterations, higher-quality prompts) and the *extensive margin* (falling prices make previously uneconomical use cases viable, such as automated code review, speculative pre-computation, and parallel hypothesis testing). The extensive margin is likely dominant in the current growth phase, as entirely new agentic workflows become cost-feasible each time prices fall by 2–3×. The bandwidth market of the 2000s provides a close historical analogy (Sorrell, 2009).

9.8. CVIH Budget Optimality

Both λ_{raw} and p_{verify} are functions of budget B. Let $V(B) = \lambda_{\text{raw}}(B) \cdot p_{\text{verify}}(B)$ be the "value function" (verified iterations per hour). Then $\text{CVIH}(B) = V(B)/B$.

Theorem 9.1 (CVIH Tangent Optimality)**.** [Mathematical Fact: first-order condition of a ratio.] *The budget B^* maximizing $CVIH(B) = V(B)/B$ satisfies $V'(B^*) = V(B^*)/B^*$, i.e., marginal value equals average value. Geometrically, B^* is the tangent point from the origin to the value curve $V(B)$.*

Theorem 9.2 (Quasi-Concavity of CVIH)**.** *If $V(B)$ is concave, continuously differentiable, $V(0) = 0$, and $V'(0) > 0$, then $CVIH(B)$ is quasi-concave on $(0, \infty)$ with a unique maximizer B^*.*

Proof sketch. The ratio starts at $V'(0) > 0$ (by L'Hôpital's rule), is continuous, and eventually decreases (since V is bounded while $B \to \infty$). The ratio of a concave function to a linear function through the origin is quasi-concave (Boyd and Vandenberghe, 2004). The numerator of $h'(B) = [V'(B)B - V(B)]/B^2$ is strictly decreasing (since $V'' < 0$), so $h' = 0$ has a unique solution. □

Theorem 9.3 (Budget Separation)**.** [Mathematical Fact: the CVIH-optimal budget lies strictly below the quality-optimal budget.] *Let B^*_{CVIH} maximize CVIH. Let B^*_Q denote the quality-optimal budget (finite if Q^* has a peak; ∞ if Q^* is strictly increasing with asymptote). In either case:*

$$B^*_{CVIH} < B^*_Q. \tag{73}$$

Proof sketch. If $B^*_Q < \infty$: at B^*_Q, $V'(B^*_Q) = 0$ while $V(B^*_Q)/B^*_Q > 0$, so $V(B)/B$ is strictly decreasing there. If $B^*_Q = \infty$: since $V \leq 1$ is bounded, $V(B)/B \to 0$, so the finite maximizer B^*_{CVIH} exists and is strictly less than ∞. □

This formalizes a crucial operational insight: the budget that maximizes quality is not the budget that maximizes efficiency. Past B^*_{CVIH}, each additional dollar buys less verified output than the average dollar already spent.

Graceful degradation. When available budget B_{avail} falls below B^*_{CVIH}, the system must degrade gracefully. Under the TBAP optimal allocation, if the budget is reduced from B^* to αB^* (with $\alpha \in (0, 1)$), the concavity of $\log Q^*(B)$ implies that a fraction α of the budget retains more than a fraction α of the log-quality. Numerical verification for exponential saturation quality functions shows that a 50% budget cut retains 65–80% of the log-quality depending on the number of stages and saturation rate. This sub-linear degradation is a direct consequence of the diminishing-returns property and provides formal justification for budget-constrained operation.

9.9. Caching Economics and the Cache-First Principle

Definition 9.7 (Effective Price with Caching)**.** *The effective per-token price at stage k with caching is $p_k^{eff} = p_k \cdot (1 - \rho_k + \rho_k \cdot \delta_k)$, where $\rho_k \in [0, 1]$ is the cache hit rate and $\delta_k \in (0, 1)$ is the cache discount factor.*

With $\rho_k = 0.92$ and $\delta_k = 0.10$ (Anthropic's cache discount), $p_k^{\text{eff}} = 0.172\, p_k$, an 82.8% cost reduction.

Proposition 9.4 (Cache-First Principle)**.** [Engineering Hypothesis: depends on current provider implementations of exact caching.] *Under current LLM provider implementations (where cached tokens produce identical model computation), increasing ρ_k reduces p_k^{eff} without affecting $q_k(r_k)$. All other cost levers (reducing r_k, switching to a cheaper model, compressing prompts) reduce q_k along with cost.*

This implies a strict ordering: maximize cache hit rates before pursuing any optimization that trades quality for cost. Empirical data confirms the magnitude:

Approach	Scope	Cost Red.	Latency Red.	Source
Prompt prefix caching	Token-level	60–85%	50–80%	(Anthropic, 2026a)
Prompt-aware caching	Token-level	41–80%	30–60%	(Liang et al., 2026)
Agentic plan caching	Plan-level	~50%	~27%	(Zhang et al., 2025b)

All three approaches achieve zero quality impact under exact-caching implementations. The Cache-First principle is particularly significant because its cost reduction (60–85% when prompt and plan caching are combined) is the largest available from any single optimization, and it is the only lever that does not trade quality for cost.

9.10. Empirical Calibration (April 2026)

The formal results require empirical parameterization. Two parametric quality function families fit observed behavior: exponential saturation $q(r) = 1 - \exp(-\alpha r)$ with $\alpha \in [0.5 \times 10^{-4}, 5 \times 10^{-4}]$, and the Hill function $q(r) = r^n/(r^n + K_d^n)$ for $n \leq 1$. Both are empirical approximations; we recommend fitting parameters per-stage from logged pipeline data.

Provider	Model	Input ($/Mtok)	Output ($/Mtok)	Cache Discount
Anthropic	Claude Opus 4	15.00	75.00	90%
Anthropic	Claude Sonnet 4	3.00	15.00	90%
Anthropic	Claude Haiku 3.5	0.80	4.00	90%
OpenAI	GPT-4.1	2.00	8.00	50%
OpenAI	GPT-4.1 mini	0.40	1.60	50%
OpenAI	GPT-4.1 nano	0.10	0.40	50%
Google	Gemini 2.5 Pro	1.25	10.00	75%
Google	Gemini 2.5 Flash	0.15	0.60	75%

Key observations: the price spread from cheapest to most expensive input pricing is approximately $150\times$; the three-year trend shows approximately $300\times$ reduction in effective cost at the frontier tier when caching is included; cache discounts vary significantly across providers.

Developer time parameters: fully loaded cost $90–$200/hr; flow-state threshold 120–180 seconds; context-switching cost 15–30 minutes per interruption (Mark et al., 2008). Caching parameters: prompt cache hit rate 85–95% for iterative sessions (Anthropic, 2026a), plan cache hit rate 40–70% for recurring patterns (Zhang et al., 2025b), combined effective cost reduction 60–85%.

Calibration protocol. We recommend: (1) *Measure*: instrument the pipeline to log per-stage token consumption, latency, quality proxies, and cache hit rates; (2) *Fit*: estimate quality function parameters per stage from logged data; (3) *Optimize*: solve the JASP using fitted parameters and current pricing; (4) *Verify*: A/B test the optimized allocation against the baseline, measuring CVIH improvement; (5) *Repeat*: re-calibrate monthly or when pricing changes by more than 20%. Related work on model routing (Chen et al., 2023; Madaan et al., 2024; Ong et al., 2024) demonstrates that learned routing policies can achieve 40–98% cost savings with 95–100% quality retention at the per-query level; our framework extends this to per-stage allocation within multi-stage pipelines.

9.11. Seven Design Principles

Each principle is grounded in a theorem or empirical result from the preceding subsections.

E1. Equal Marginal Rate (Proposition 9.1). Equalize the marginal quality-per-dollar across all pipeline stages. If the ratio $q_k'(r_k)/(q_k(r_k) \cdot p_k)$ is significantly higher for stage j than stage k, shift budget from k to j.

E2. Raise the Floor (Observation 9.1, Corollary 9.1). Invest disproportionately in the weakest pipeline stage. A 10% improvement in the worst stage yields a larger multiplicative quality gain than a 10% improvement in the best stage.

E3. Cap the Context (empirical; (Chroma Research, 2025; Gao et al., 2025; Liu et al., 2024)). Set a maximum useful token budget per stage, beyond which additional tokens may harm quality. Enforce $r_k \leq r_k^{\max}$ as a hard constraint.

E4. Detect the Regime (Section 9.6). Use different models for interactive and autonomous workloads. For interactive tasks, select the cheapest model with response time below w_{thresh}; for autonomous tasks, minimize cost per unit capability regardless of latency.

E5. Cache First (Proposition 9.4). Maximize cache hit rates before pursuing any cost optimization that trades quality for cost. Structure prompts with stable prefixes; implement plan-level caching for recurring task patterns.

E6. Route by Economics (Definition 9.5). Model selection should be driven by cost-effectiveness, not capability alone. Use cheaper models for stages where the frontier provides little incremental quality.

E7. Budget for Jevons (Observation 9.2, Corollary 9.2). Expect total token spend to increase when per-token prices fall. Budget forecasts should include an elasticity multiplier: if prices fall by factor x, expect spend to change by factor $x^{1-\epsilon}$ (which increases for $\epsilon > 1$).

9.12. Contrarian Positions

"Quality over cost" (strength: 5/5 for safety-critical, 3/5 for general). The claim that one should always use the best model is strongest where errors are catastrophic: (IBM and Ponemon Institute, 2025) estimates average data breach cost at $4.5M, dwarfing token savings. For the median coding task, however, mid-tier models achieve 70–85% of frontier pass rates at 10–20% of the cost (JetBrains, 2025). Resolution: tiered quality via the JASP, frontier models for security-critical stages, mid-tier for the rest.

"Falling prices make optimization unnecessary" (strength: 2/5). Prices have fallen ~300× in three years, but the Jevons paradox with $\epsilon \approx 1.19$ means total spend increases as prices fall. The bandwidth market provides the closest historical analogue: per-bit costs fell 1000× from 1995 to 2015, yet total telecom spending increased throughout (Sorrell, 2009).

"Developer time dominates token cost" (strength: 4/5 interactive, 1/5 autonomous). In the interactive regime, a developer earning $150/hr who waits 30 seconds incurs $1.25 in waiting cost per interaction; a 50K-token interaction at mid-tier pricing with caching costs $0.025, a 50× dominance. In the autonomous regime, developer waiting cost is zero, and organizational-scale token budgets of $100K–$2.7M/year make token cost the binding constraint. Resolution: regime detection.

"Routing complexity outweighs savings" (strength: 3/5 small-scale, 1/5 large-scale). At small scale, absolute savings may be $500–$2,000/month, below the engineering cost. At large scale, savings of $20K–$200K/month easily justify dedicated infrastructure (FinOps Foundation, 2023). Resolution: tiered complexity: simple rules for small deployments, full JASP for large ones.

9.13. Connection to Other Pillars

[Philosophical Observation: the Economics pillar is a cross-cutting concern, not an isolated dimension.]

The Economics pillar connects to each of the seven existing pillars. The **Abstraction** pillar's gap $\mathcal{G}(S, P)$ determines how many tokens each stage requires; larger gaps demand larger budgets. The **Information** pillar's context selection directly determines r_k for retrieval stages, and submodular diminishing returns in context value mirror the concavity axiom **(Q4)**. The **Reliability** pillar's compound error model $R = p^n$ is structurally identical to the multiplicative quality model: per-step reliability is a quality function, and the Weakest-Link observation is the economic analogue of investing in the lowest-reliability step. The **Coordination** pillar's error amplification factor determines the quality function shape for multi-agent stages. The **Temporal** pillar's Verified Iterations per Hour is the numerator of CVIH, connecting iteration speed to cost-effectiveness. The **Quality** pillar's layered defense determines p_{verify} in the CVIH formula, with each verification layer consuming tokens at its own price point. The **Governance** pillar's capacity bounds determine how much architectural information must be preserved per iteration, a cost denominated in tokens. The Cache-First principle amplifies with the Temporal pillar's iteration model: caching benefits compound across iterations because the stable prefix (system prompt, tool definitions, architectural context) is reused.

10. The Human Interaction Pillar

The Human Interaction pillar addresses the scarce resource that every other pillar assumes exists but none model: human attention. AI coding agents now generate 40–60% of committed code, yet only 26% of developers always verify AI output before committing (SonarSource, 2026), and review time has increased 91% (Google Cloud, 2025). The human is not a perfect oracle at the end of a verification pipeline; they are an adaptive, degradable, finite-capacity component whose interaction with the agent must be *designed*.

Current harnesses treat human interaction as static gate-based systems (review at fixed points regardless of risk or trust). None implement adaptive triage based on learned trust or risk estimation, and none account for the vigilance decay that Bainbridge (1983) identified over four decades ago.

This pillar bridges the gap between the automation supervision literature and harness engineering practice.

10.1. Human Code Review as Signal Detection

Definition 10.1 (Code Review as Signal Detection)**.** [Mathematical Fact: standard SDT formulation.] *For each code unit, the reviewer observes an internal evidence variable X drawn from $\mathcal{N}(\mu_0, \sigma^2)$ under H_0 (correct code) or $\mathcal{N}(\mu_1, \sigma^2)$ under H_1 (defective code), with $\mu_1 > \mu_0$. The reviewer reports "defect" iff $X > \lambda$ for criterion λ.*

The sensitivity $d' = (\mu_1 - \mu_0)/\sigma$ measures discrimination independent of response bias. Published estimates place d' for code review at 0.84 (informal review) to 2.12 (optimal Fagan inspection conditions) (Cohen, 2006; Fagan, 1976). The criterion $c = (\lambda - (\mu_0 + \mu_1)/2)/\sigma$ captures the reviewer's bias.

The prevalence effect. When defect prevalence is low (most agent output is correct), reviewers shift their criterion toward conservative values ($c \uparrow$), increasing miss rates. Wolfe et al. (2005) demonstrated that miss rates rise from 7% at 50% prevalence to 30% at $< 1\%$ prevalence. For AI agents with 95%+ correctness, the effective prevalence may be below 5%, placing reviewers in the high-miss-rate regime. This is the signal-detection-theoretic explanation for why vigilance probes (§10.7) are necessary: they restore prevalence to maintain criterion placement.

Automation levels. Parasuraman et al. (2000) propose that automation operates across four stages of human information processing (acquisition, analysis, decision, action). For code review, high automation of information acquisition (auto-gathering diffs, test results, CI status) is generally safe, while high automation of decision selection (auto-approve/reject) is dangerous because it removes the operator from the comprehension loop. Endsley and Kiris (1995) showed that full automation degrades comprehension (Level 2 situation awareness) while preserving perception (Level 1): developers can *see* the code but lose understanding of what it means.

10.2. Definitions: Capacity, Risk, and Cost

Definition 10.2 (Human Review Capacity)**.** *The human reviewer can inspect at most H outputs per review cycle, where $H \ll N$ (N is the total agent output count). Effective code review requires 150–200 LOC/hour (Cohen, 2006; Fagan, 1976). For a harness producing dozens to hundreds of outputs per cycle, H/N is typically in $[0.05, 0.30]$.*

Definition 10.3 (Risk Weight)**.** *The risk weight of output i is $w_i = p_i \cdot c_i$, where p_i is the prior defect probability and $c_i > 0$ is the defect cost. The review decision carries asymmetric costs: c_{FG} for a missed defect (false negative) and c_{GF} for an unnecessary review (false positive), with $c_{FG} \gg c_{GF}$ in practice.*

10.3. Optimal Attention Allocation

Given N agent outputs and human capacity $H \ll N$, which outputs should the human review? We connect this to Dodge-Romig sampling inspection theory (Dodge and Romig, 1929, 1959). The mapping is direct: manufacturing lots become batches of agent outputs, items within lots become individual diff hunks or files, and the inspection station becomes the human reviewer. Under review

policy S with rectifying inspection, the Average Outgoing Quality is:

$$\text{AOQ}(S) = \frac{(1 - d_{\text{auto}})}{N} \sum_{i=1}^{N} p_i \left[1 - S(i) \cdot d_{\text{human}}\right] \tag{74}$$

where $S(i) \in \{0, 1\}$ indicates whether output i is reviewed.

Proposition 10.1 (Optimal Triage is Risk-Ranked Selection)**.** [Mathematical Fact: follows from the knapsack optimality of greedy selection under unit weights.] *The review set S^* minimizing expected defect escape cost consists of the H outputs with the largest risk weights $w_i = p_i \cdot c_i$. Formally, let σ be a permutation with $w_{\sigma(1)} \geq \cdots \geq w_{\sigma(N)}$. Then $S^*(i) = 1$ iff $i \in \{\sigma(1), \ldots, \sigma(H)\}$.*

Proof sketch. The objective $\sum_i S(i) w_i$ subject to $\sum_i S(i) \leq H$ and $S(i) \in \{0, 1\}$ is a 0-1 knapsack with unit weights. The KKT conditions yield $S(i) = 1$ when $w_i > \lambda$, where λ is the risk weight of the H-th most risky output. □

Proposition 10.2 (Stratification Improvement Bound)**.** [Mathematical Fact: follows from the rearrangement inequality.] *Let E_{uniform} and $E_{\text{stratified}}$ be the expected defect escape costs under uniform and optimal risk-stratified sampling of H outputs. Then:*

$$E_{\text{uniform}} - E_{\text{stratified}} \geq (1 - d_{\text{auto}}) \cdot d_{\text{human}} \cdot \frac{H}{N} \cdot \text{Var}_N(w) \tag{75}$$

where $\text{Var}_N(w)$ is the population variance of risk weights. The improvement factor is approximately $1 + \text{CV}(w)^2$ when H/N is small.

Proof sketch. Under uniform sampling, each output is reviewed with probability H/N. Under optimal stratification, the top-H outputs are reviewed. The improvement equals $(1 - d_{\text{auto}})d_{\text{human}}$ times $\sum_{i \in S^*} w_i - (H/N) \sum_i w_i$. By the rearrangement inequality, this gap is bounded below by $(H/N) \cdot \text{Var}_N(w)$. □

With risk concentrated (60% of defects in the riskiest 20% of code, consistent with the Pareto distribution commonly observed in software defects (Jones, 1996)), the stratification bound predicts a $3\times$ improvement in defects caught per review under optimal triage vs. uniform sampling. For an organization reviewing 20% of agent output ($H/N = 0.20$), stratified triage achieves an escape rate comparable to reviewing 50% uniformly.

Adaptive inspection via switching rules. We formalize the MIL-STD-105E switching rules (U.S. Department of Defense, 1989) as a finite state machine on three states $Q = \{R, N, T\}$ (Reduced, Normal, Tightened), with transitions governed by lot acceptance history.

Proposition 10.3 (Steady-State Distribution)**.** [Engineering Hypothesis: the conjectured 40–60% cost reduction is extrapolated from manufacturing data.] *For constant defect rate p, the steady-state probability of each inspection state satisfies a balance equation depending on the lot acceptance probabilities under each plan. When quality is good ($p \ll \text{AQL}$), $\pi_R \to 1$ and review workload drops to the reduced level. When quality deteriorates ($p > \text{AQL}$), $\pi_T \to 1$ and scrutiny increases automatically.*

This is precisely the trust calibration mechanism that Lee and See (2004) prescribe: review intensity should track demonstrated reliability. The switching rules reduce inspection cost by 40–60% at equivalent quality levels in manufacturing; we conjecture comparable savings for software review.

10.4. The Autonomy Boundary

For each output with feature vector x_i, the harness chooses between trust ($a = F$, autonomous passage) and review ($a = G$, human inspection).

Theorem 10.1 (Optimal Autonomy Threshold)**.** [Mathematical Fact: standard Bayesian decision theory.] *The Bayes-optimal rule routes output i to review iff $P(\theta_i = 1 \mid x_i) > \tau^*$, where:*

$$\tau^* = \frac{c_{GF}}{c_{FG} + c_{GF}} = \frac{1}{1+\rho}, \quad \rho = c_{FG}/c_{GF} \tag{76}$$

Proof sketch. The posterior expected loss for trusting is $c_{FG} \cdot P(\theta = 1 \mid x)$; for reviewing, $c_{GF} \cdot P(\theta = 0 \mid x)$. Review is preferred when the latter is smaller, yielding $P(\theta = 1 \mid x) > c_{GF}/(c_{FG} + c_{GF})$. □

Observation 10.1 (Monotonicity)**.** *The threshold $\tau^*(\rho) = 1/(1+\rho)$ is strictly decreasing in ρ, with $d\tau^*/d\rho = -1/(1+\rho)^2 < 0$.*

The cost ratio ρ serves as a single governance dial:

Context	ρ	τ^*	Interpretation
Documentation/comments	2	0.333	Review only when $P(\text{defect}) > 33\%$
Test files in dev branch	5	0.167	Moderate autonomy
Production source code	50	0.020	Review when $P(\text{defect}) > 2\%$
Security-critical code	200	0.005	Review almost everything

Theorem 10.2 (Capacity-Constrained Threshold)**.** [Mathematical Fact: connects the Bayesian threshold to the knapsack triage.] *When review capacity is binding, the effective threshold is $\tau_H = \max(\tau^*, \tau_{\text{cap}})$, where τ_{cap} is the smallest threshold such that $|\{i : P(\theta_i = 1 \mid x_i) > \tau_{\text{cap}}\}| \leq H$. Under capacity constraints, the top-H selection from Proposition 10.1 and the threshold classifier from Theorem 10.1 coincide.*

The Expected Value of Perfect Information for each output is $\text{EVPI}(x) = \min(c_{FG}P(\theta = 1 \mid x), c_{GF}P(\theta = 0 \mid x))$, maximized at the decision boundary $P(\theta = 1 \mid x) = \tau^*$. This provides an information-theoretic justification for prioritizing borderline cases: the outputs where the harness is most uncertain about the review decision are exactly the outputs where human input has the highest expected value.

10.5. Trust Calibration via Thompson Sampling

The harness should learn which (agent, task-type) pairs require oversight and which can be trusted. We model this as a multi-armed bandit (Thompson, 1933).

Definition 10.4 (Trust Calibration Bandit)**.** *A trust calibration bandit is a tuple $(\mathcal{K}, \Theta, \mathcal{F})$ where $\mathcal{K} = \{1, \ldots, K\}$ indexes (agent, task-type) pairs, $\Theta = (\theta_1, \ldots, \theta_K) \in [0,1]^K$ is the vector of unknown success probabilities, and $\mathcal{F} = \{\text{Bernoulli}(\theta_k)\}_{k=1}^K$ is the reward family. The trust score for arm k with posterior $\text{Beta}(\alpha_k, \beta_k)$ is the posterior mean $\tau_k = \alpha_k/(\alpha_k + \beta_k)$.*

Thompson sampling maintains Beta posteriors for each arm, sampling from each posterior at decision time and selecting the arm with the highest sample. The Agrawal-Goyal regret bounds (Agrawal and Goyal, 2012) guarantee:

Proposition 10.4 (Regret Bound, Agrawal and Goyal 2012)**.** [Mathematical Fact: cited result.] *Thompson sampling achieves expected regret* $\mathbb{E}[\text{Regret}(T)] \leq (1+\epsilon)\sum_{k:\Delta_k>0} \ln T/\Delta_k + O(K/\epsilon^2)$*, where* $\Delta_k = \theta^* - \theta_k$ *is the suboptimality gap.*

Proposition 10.5 (Posterior Concentration)**.** [Mathematical Fact: follows from Hoeffding's inequality and Bayesian consistency.] *The trust score concentrates around the true success probability at rate* $|\tau_k - \theta_k| = O_p(1/\sqrt{n_k})$*, where* n_k *is the number of observations for arm* k*.*

Proof sketch. The posterior mean is a weighted average of the sample mean $\hat{p}_k$ and the prior mean: $\tau_k = (n_k\hat{p}_k + c_0\tau_k^{(0)})/(n_k + c_0)$, where $c_0 = \alpha_k^{(0)} + \beta_k^{(0)}$. By Hoeffding's inequality, $|\hat{p}_k - \theta_k| = O_p(1/\sqrt{n_k})$, and the prior weight $c_0/(n_k + c_0) = O(1/n_k)$. □

Contextual extension. When the trust decision depends on context features x_t (file type, complexity, novelty), we extend to linear contextual bandits (Agrawal and Goyal, 2013; Li et al., 2010). The trust surface becomes $\tau(\text{agent}, \text{task\_type}, \mathrm{x}) = \mathrm{x}^\top\hat{\boldsymbol{\theta}}_k$, with contextual regret bound $O(d\sqrt{T}\ln^{3/2} T)$, where d is the context dimension.

Trust information bound. *[Engineering Hypothesis: the bound connects to the Governance pillar's capacity results.]* The trust information rate (the rate at which the harness learns about agent reliability) is bounded by $H \ln 2$ bits per review cycle. When K is the number of (agent, task-type) arms, calibrated trust (trust calibration error $< \epsilon$) requires:

$$H \geq \frac{K}{\epsilon^2} \cdot C_{\min} \tag{77}$$

where $C_{\min}$ depends on the minimum suboptimality gap. When $H < K/\epsilon^2$, trust calibration error remains above ϵ regardless of the algorithm. This is the trust-edition analogue of the governance capacity bound from the Governance pillar (Section 8): when agent reliability changes faster than the review channel can observe, no bandit algorithm can maintain calibrated trust.

10.6. The Automation Supervision Paradox

Bainbridge's (1983) ironies of automation identify a fundamental tension: if automation is reliable enough that the human rarely intervenes, the human's intervention skills degrade. When automation fails, the human is least prepared to take over. We formalize this for coding harnesses.

Definition 10.5 (Vigilance Decay Function)**.** [Engineering Hypothesis: functional form proposed, not validated for software review.] *When automation handles a fraction* p_{auto} *of decisions correctly, the human's detection probability decays according to* $d_{\text{human}}(t) = d_0 \cdot \Phi(p_{\text{auto}}, t)$*, where:*

$$\Phi(p_{\text{auto}}, t) = \phi_\infty(p_{\text{auto}}) + (1 - \phi_\infty(p_{\text{auto}}))\exp(-t/\xi(p_{\text{auto}})) \tag{78}$$

with $\phi_\infty(p_{\text{auto}}) = (1 - p_{\text{auto}})^\eta$ *and* $\xi(p_{\text{auto}}) = \xi_0(1 - p_{\text{auto}})^\gamma$.

Calibration from the vigilance literature: $\eta = 1.5$, $\gamma = 1.0$, $\xi_0 = 30$ minutes (Mackworth, 1948; See et al., 1995), $d_0 = 0.80$ (Cohen, 2006). Under these parameters with $p_{\text{auto}} = 0.95$: the asymptotic residual vigilance drops to $\phi_\infty \approx 0.011$, the decay timescale shrinks to 1.5 minutes, and effective detection collapses to under 4% within five minutes.

Theorem 10.3 (Automation Supervision Paradox)**.** [Mathematical Fact: follows from IVT under the stated model.] *Under the vigilance decay model (Definition 10.5), when $d_0(1+\eta) > 1$, there exists a threshold $p^* \in (0,1)$ such that for all $p_{\text{auto}} \in (0, p^*)$, the steady-state combined detection $D_\infty(p_{\text{auto}}) < d_0$: the human-automation system detects fewer defects than the human alone.*

Proof sketch. The steady-state combined detection is $D_\infty(p) = p + d_0(1-p)^{1+\eta}$. Let $g(p) = D_\infty(p) - d_0$. Then $g(0) = 0$ and $g'(0) = 1 - d_0(1+\eta) < 0$ when $d_0(1+\eta) > 1$. Since $g(1) = 1 - d_0 > 0$, the intermediate value theorem gives a unique $p^* \in (0,1)$ with $g(p^*) = 0$. For $p \in (0, p^*)$, $g(p) < 0$. □

For $d_0 = 0.80$ and $\eta = 1.5$: $p^* \approx 0.59$. The paradox region is substantial.

Concrete example. Consider an AI coding agent with 70% automation reliability ($p_{\text{auto}} = 0.70$). The human alone detects 80% of defects ($d_0 = 0.80$). With automation, steady-state human vigilance drops to $d_0 \cdot (1-0.70)^{1.5} = 0.80 \times 0.164 = 0.131$. The combined system detects $0.70 + 0.131 \times 0.30 = 0.739$, which is *less* than the 0.80 the human achieved alone. The automation reliably catches easy defects but degrades the human's ability to catch the hard ones that automation misses.

The irony is asymmetric: $D_\infty(p) \geq p$ always holds (the combined system never performs worse than automation alone), but adding the machine harms overall performance by degrading the human, not by failing itself.

10.7. Vigilance Probes

Definition 10.6 (Vigilance Probe)**.** *A vigilance probe is a synthetic, known-defective output injected into the review stream at rate λ, indistinguishable from genuine output at review time, with feedback provided after the reviewer's judgment. This is analogous to Threat Image Projection (TIP) in aviation security screening (Wolfe et al., 2005).*

Theorem 10.4 (Optimal Probe Rate)**.** [Engineering Hypothesis: the precise optimum depends on the vigilance model parameters, which are uncalibrated for software review.] *Under the vigilance model with probes augmenting the effective signal rate, there exists an optimal probe rate λ^* that maximizes combined system detection, balancing maintained vigilance against probe-consumed review capacity.*

The probe-augmented model replaces p_{auto} with an effective $p_{\text{eff}} = p_{\text{auto}}/(1+\lambda/(1-p_{\text{auto}}))$, shifting p^* upward (expanding the safe region). Numerical analysis with $d_0 = 0.80$, $\eta = 1.5$, and probe rate $\lambda = 0.05$ increases p^* from 0.59 to approximately 0.74. Probes convert a passive monitoring task into an active detection task, counteracting the vigilance decrement (Warm et al., 2008).

10.8. The Human Interaction Budget

The preceding components (triage, autonomy, trust calibration, vigilance) compete for a single scarce resource: human attention. We unify them through a budget decomposition.

Definition 10.7 (Budget Decomposition)**.** *The total interaction budget $B = H \cdot T$ (capacity × session time) is decomposed as:*

$$B = B_{\text{review}} + B_{\text{cal}} + B_{\text{vig}} \tag{79}$$

where B_{review} is risk-based triage review, B_{cal} is calibration (exploration of trusted arms), and B_{vig} is vigilance probes.

Let $\phi_r, \phi_c, \phi_v \geq 0$ with $\phi_r + \phi_c + \phi_v = 1$ be the allocation fractions. The defect escape rate depends on all three:

$$\mathcal{E}(\phi_r, \phi_c, \phi_v) = p(1 - d_{\text{auto}}) \left[(1 - \phi_r B/N) + \phi_r B/N \cdot (1 - d_0 \Phi(\phi_v)) \cdot (1 + g(\phi_c))\right] \tag{80}$$

where $\Phi(\phi_v) = 1 - (1 - \Phi_{\min}) e^{-\kappa \phi_v}$ captures vigilance maintenance and $g(\phi_c) = g_0 e^{-\lambda \phi_c}$ captures triage inefficiency from miscalibration.

Theorem 10.5 (Optimal Budget Allocation)**.** [Mathematical Fact: follows from optimization on a compact set with strict concavity/convexity.] *Under strict concavity of Φ and strict convexity of g, with $B/N < 1$ and $d_0, p > 0$, $d_{\text{auto}} < 1$, there exists a unique optimal allocation $(\phi_r^*, \phi_c^*, \phi_v^*)$ in the interior of the simplex.*

Proof sketch. The feasible set (2-simplex) is compact; $\mathcal{E}$ is continuous, so a minimum exists. For interiority: at $\phi_v = 0$, the marginal benefit of vigilance is large ($\Phi'(0) = \kappa(1 - \Phi_{\min})$); at $\phi_v \to 1$, review coverage vanishes. By IVT, an interior optimum exists. The bordered Hessian is negative definite on the constraint surface by strict concavity/convexity, giving uniqueness. □

Corollary 10.1 (Vigilance-Review Tradeoff)**.** *At the optimum, the marginal defect escape reduction from one additional unit of vigilance equals the marginal increase from one fewer unit of review:*

$$d_0 \Phi'(\phi_v^*) \cdot \phi_r^* B/N = (1 - d_0 \Phi(\phi_v^*)(1 + g(\phi_c^*))) \cdot B/N \tag{81}$$

Comparative statics. *[Philosophical Observation: this is the budget-theoretic formalization of Bainbridge's irony.]* As agent quality improves ($p \to 0$): ϕ_v^* and ϕ_c^* increase (the dominant failure modes shift from defect volume to detection degradation and trust miscalibration), while ϕ_r^* decreases. The better the automation, the more human attention must be allocated to *maintaining* oversight capability rather than exercising it.

10.9. Complementarity Conditions

Definition 10.8 (Complementarity Index)**.** $C = P(\textit{team correct}) - \max(P(\textit{human correct}), P(\textit{AI correct}))$.

Theorem 10.6 (Conditions for Positive Complementarity)**.** [Mathematical Fact: algebraic consequence of the OR combination rule.] *Under an OR combination rule, the human-AI team has $C > 0$ iff:*

$$\rho_{HA} < \sqrt{\frac{(1 - d_A) \cdot d_H}{(1 - d_H) \cdot d_A}} \tag{82}$$

where ρ_{HA} is the error correlation and d_H, d_A are detection rates. Independent errors ($\rho_{HA} = 0$) always produce positive complementarity; perfectly correlated errors ($\rho_{HA} = 1$) produce none.

Proof sketch. Under the OR rule, $P(\text{team detects}) = 1 - P(E_H)P(E_A) - \rho_{HA}\sqrt{P(E_H)(1 - P(E_H))P(E_A)(1 - P(E_A))}$. The condition $C > 0$ reduces to the stated bound on ρ_{HA} by algebra. □

Complementarity is robust to variation in individual detection rates but fragile to error correlation. Vaccaro et al. (2024) meta-analyzed human-AI collaboration, finding a mean effect of $g = -0.23$ (teams often underperform the better component alone). Theorem 10.6 provides a formal explanation: high error correlation destroys complementarity. The design implication: harness verification layers should be chosen for error diversity, not just individual accuracy.

10.10. Connection to Learning-to-Defer

Mozannar and Sontag (2020) formalize the joint optimization of a classifier h and rejector r, minimizing the system 0-1 loss. We establish the connection between this framework and the Bayesian autonomy threshold.

Theorem 10.7 (Deferral-Threshold Equivalence)**.** [Mathematical Fact: algebraic equivalence under stated conditions.] *Under asymmetric binary loss, well-calibrated defect probability estimates, and constant human detection rate, the Mozannar-Sontag optimal rejector is equivalent to the Bayesian threshold:*

$$r^*(x) = \mathbb{I}\left[\hat{p}(x) > \frac{c_{GF}}{c_{FG} \cdot d_{\text{human}} + c_{GF}}\right] \tag{83}$$

Proof sketch. Deferral is optimal when the review cost plus residual miss cost is less than the autonomous miss cost: $c_{GF} + c_{FG}\hat{p}(x)(1 - d_{\text{human}}) < c_{FG}\hat{p}(x)$. Solving for $\hat{p}(x)$ yields the threshold. □

This equivalence shows that the learning-to-defer framework and the Bayesian triage framework converge to the same decision rule under standard conditions. The deferral threshold adjusts Theorem 10.1 by incorporating the human's imperfect detection ($d_{\text{human}} < 1$), yielding a slightly higher threshold (review is less valuable when the human is imperfect).

10.11. Empirical Calibration

[Engineering Hypothesis: all parameter ranges are calibrated from analogue domains, not software review studies.]

Parameter	Range	Source	Conf.
d_{human} (formal inspection)	0.60–0.82	Fagan (1976); Jones (1996)	H
d_{human} (optimal conditions)	0.70–0.90	Cohen (2006)	H
d_{human} (high review rate)	0.25–0.40	Cohen (2006)	M
d_{auto} (hallucinated imports)	0.92–0.99	AST/lockfile validation	H
d_{auto} (security vulns)	0.15–0.35	SAST tools	M
Vigilance $\Phi(30$ min)	0.85–0.90	Mackworth (1948)	H
Vigilance $\Phi(120$ min)	0.50–0.70	Mackworth (1948)	M
Verification rate	26% always verify	SonarSource (2026)	H
Review time growth with AI	+91%	Google Cloud (2025)	H

Confidence levels: **H** (directly measurable from published studies), **M** (estimable with moderate study design effort), **L** (requires dedicated experimental infrastructure).

The review capacity gap. The DORA 2025 data implies RCG = $N_{\text{outputs}}/B > 1$ in many settings: PR size has grown 154% while review capacity has not scaled proportionally (Google Cloud, 2025). The Sonar 2026 survey found that 96% of developers do not fully trust AI output, yet only 26% always verify AI code before committing (SonarSource, 2026), confirming that the capacity-constrained regime ($B/N < 1$) is the default, not an edge case. Meanwhile, METR reports that experienced developers were 19% *slower* with AI tools in a controlled RCT (METR, 2026a), suggesting the interaction between expertise and AI assistance is non-monotonic.

10.12. Design Principles

From the formal framework, seven principles for harness builders:

1. **Risk-stratified triage, not uniform review.** Review the H highest-risk outputs (10.1). The improvement factor is approximately $1 + \text{CV}(w)^2$.
2. **Asymmetric cost thresholds per context.** Set the autonomy boundary using $\rho = c_{FG}/c_{GF}$ as a governance dial (10.1). Security-critical code ($\rho = 200$) is reviewed at $\tau^* = 0.005$; documentation ($\rho = 2$) at $\tau^* = 0.333$.
3. **Adaptive inspection regimes.** Implement MIL-STD-105E switching: reduced review for reliable agents, tightened review when quality degrades (10.3).
4. **Maintain a vigilance budget.** Allocate a fraction of review capacity to vigilance probes (10.4). This is the engineering response to Bainbridge's irony.
5. **Invest in trust calibration.** Maintain Beta posteriors per (agent, task-type) pair and allocate a calibration budget for exploration (10.5).
6. **Optimize for error diversity.** Complementarity requires low error correlation (10.6). Choose verification layers for diverse failure modes.
7. **Distinguish tactical from strategic interaction.** Per-output triage is amenable to formal optimization; per-milestone deliberation requires conversational grounding. Design both, but do not conflate them.

10.13. Contrarian Positions

"Humans should review everything." The throughput mismatch makes this impossible: AI generates 40–60% of code; PR size has grown 154% (Google Cloud, 2025). The generalized Deming kp rule shows 100% inspection is economically justified only when $\bar{p} > k_1/(c \cdot d_{\text{human}})$; at typical parameters, the crossover is around $\bar{p} = 6.25\%$. Below this rate, triage is more efficient. *Counter*: triage, not abdication.

"Fully autonomous agents are the goal." Current agents are insufficiently reliable: METR reports approximately 50% of test-passing PRs would not be merged (METR, 2026a). More importantly, Naur's theory preservation problem means human understanding degrades without review interaction (**?**). *Counter*: full autonomy is a legitimate asymptotic goal, but the transition requires adaptive oversight.

"Trust calibration creates auto-pilot complacency." Bainbridge's irony is real, and Theorem 10.3 quantifies it. But the solution is not to avoid trust calibration; it is to include vigilance probes that maintain detection skill (10.4), analogous to TIP in aviation security. *Counter*: vigilance probes are the engineering response.

"Interaction should be conversational, not gate-based." Suchman's critique (Suchman, 1987) is well-founded: real interaction is dynamic, not a series of approve/reject gates. *Counter*: gate-based interaction is the only model currently amenable to formal optimization. This is explicitly a limitation; tactical interaction (formalized here) and strategic interaction (conversational, requiring different formalisms) serve different purposes.

"Developer experience cannot be formalized." The METR finding that experienced developers are 19% *slower* with AI tools (METR, 2026a) suggests the moderation is non-monotonic. *Counter*: the

trust calibration mechanism implicitly captures expertise through observed defect rates, sidestepping the need for explicit experience modeling.

Developer experience moderation. The expertise parameter $e \in [0, 1]$ modulates three quantities: baseline detection $d_0(e)$ is increasing in e (experts detect more); vigilance decay rate is decreasing in e (experts resist decay longer); trust prior strength $c_0(e)$ is increasing in e (experts have more informative priors). For novices ($e \to 0$), the optimal policy is conservative: more review, lower autonomy threshold, more probes. Juniors exhibit 3–4× higher accretion rates; the context budget optimum shifts from approximately $0.08W$ for experts to approximately $0.50W$ for novices. However, the scalar parameter e is an oversimplification: the METR finding that AI slows experienced developers defies monotonic modeling.

Limitations. The formal framework captures tactical interaction (per-output triage, trust calibration, vigilance maintenance) but not strategic interaction (requirements evolution, architectural deliberation, theory building (**?**)). Suchman's critique (Suchman, 1987) applies: gate-based models cannot capture the full richness of situated human-agent collaboration. Key parameters (vigilance decay rate, error correlation structure, optimal probe rate) are calibrated from analogue domains (radiology, aviation, manufacturing), not from software review studies. The functional form of $\Phi(p_{\text{auto}}, t)$ is proposed, not validated. Empirical validation in coding harness settings is the most important next step.

11. The Model Routing Pillar

The Model Routing pillar addresses the problem of selecting, configuring, and composing multiple AI models across pipeline stages to optimize cost-quality-latency tradeoffs. Every other pillar in this framework treats the model as a constant; this pillar treats it as a variable. The economics are stark: input token prices span three orders of magnitude (from $0.03 to $30 per million tokens), yet capability gaps are sharply task-dependent. Small models achieve 88–90% of frontier quality on single-function generation but catastrophically fail on complex multi-file tasks. Despite this, every major harness treats the model as a global configuration parameter rather than a per-stage optimization variable.

11.1. The Model Landscape

Table 4 summarizes the model landscape as of early 2026, calibrated from published benchmarks and API pricing.

Table 4 | Model landscape: price, quality, and latency by tier (early 2026).

Tier	Representative	Input $/MTok	HumanEval	SWE-bench V.	TTFT (s)
Budget	DeepSeek V3.2	$0.03–0.28	85–88%	40–50%	0.3–0.5
Haiku-class	Claude Haiku 4.5	$1.00	90–92%	55–60%	0.5–1.0
Sonnet-class	Claude Sonnet 4.6	$3.00	95–97%	75–80%	1.0–2.0
Opus-class	Claude Opus 4.6	$5.00	97–98%	78–81%	2.0–5.0
Premium	Opus 4.6 Fast	$30.00	97–98%	78–81%	1.0–2.0

The 1000× price range between the cheapest and most expensive options, combined with task-dependent quality gaps ranging from 10 to 70+ percentage points, defines the optimization land-

scape for model routing.

11.2. Heterogeneous Stage Assignment

Let $\mathcal{M} = \{m_1, \ldots, m_M\}$ denote the set of available models and $\mathcal{K} = \{1, \ldots, K\}$ the pipeline stages. For each model m and stage k: $q_k(m) \in [0, 1]$ is the success probability, $c_k(m) > 0$ the expected cost, $l_k(m) > 0$ the expected latency, and $\rho(m_i, m_j) \in [0, 1]$ the blind-spot correlation between models.

Definition 11.1 (Model Assignment and Pipeline Quality)**.** *A model assignment $\sigma : \mathcal{K} \to \mathcal{M}$ maps each pipeline stage to a model. The pipeline quality (end-to-end success probability under independence) is $Q(\sigma) = \prod_{k=1}^{K} q_k(\sigma(k))$, cost is $C(\sigma) = \sum_{k=1}^{K} c_k(\sigma(k))$, and serial latency is $L(\sigma) = \sum_{k=1}^{K} l_k(\sigma(k))$.*

The assignment problem has strikingly different complexity depending on constraints.

Proposition 11.1 (Unconstrained Assignment Decomposes)**.** [Mathematical Fact: follows from separability of the objective.] *When models have no capacity constraints, minimizing $C(\sigma)$ subject to $Q(\sigma) \geq Q_{\min}$ decomposes into K independent per-stage selections: for each stage k, select $\sigma(k) = \arg\min_{m \in \mathcal{F}_k} c_k(m)$, where $\mathcal{F}_k = \{m : q_k(m) \geq Q_{\min}^{1/K}\}$ is the feasible set. Total complexity is $O(MK)$.*

This decomposition is the common case: any model can serve any number of stages. The problem becomes non-trivial when capacity constraints bind (e.g., API rate limits restricting concurrent stages per model):

Proposition 11.2 (Capacity-Constrained Assignment via Hungarian Algorithm)**.** [Mathematical Fact: reduction to known polynomial-time algorithm.] *When each model m can serve at most b_m stages, the problem becomes a generalized assignment problem. For the special case $b_m = 1$ (each model serves exactly one stage, requiring $M \geq K$), it reduces to the Linear Sum Assignment Problem (Kuhn and Tucker, 1951; Munkres, 1957), solvable in $O(n^3)$ where $n = \max(M, K)$.*

Proposition 11.3 (Multiplicative Objective Reduces to Additive)**.** *Maximizing $Q(\sigma) = \prod_k q_k(\sigma(k))$ subject to $C(\sigma) \leq C_{\max}$ reduces to an additive problem via $\tilde{q}_k(m) = \log q_k(m)$ (excluding zero-probability model-stage pairs from the feasible set).*

Conjecture 11.1 (Combined Assignment is Hard)**.** [Engineering Hypothesis: formal reduction not yet constructed.] *The problem of simultaneously minimizing cost, maximizing quality, and bounding latency:*

$$\min_{\sigma} C(\sigma) \quad s.t. \quad Q(\sigma) \geq Q_{\min}, \quad L(\sigma) \leq L_{\max} \tag{84}$$

is NP-hard in general for the capacity-constrained case, by analogy to the multi-level bottleneck assignment problem. We state this as a conjecture because the formal reduction requires an instance mapping that we do not provide.

For the model sizes relevant to AI coding harnesses ($M = 3$–6 tiers, $K = 4$–8 stages), the assignment space is small: $M^K \in [81, 1{,}679{,}616]$. For $M = 3, K = 6$, exhaustive enumeration of all 729 assignments is trivial. For larger instances, Dekoninck et al. (2025) provide a principled Lagrangian relaxation:

Proposition 11.4 (Lagrangian Scalarization (Dekoninck et al., 2025))**.** *The optimal routing strategy selects, for each task with features x, the model maximizing:*

$$\tau_i(\mathrm{x}, \lambda) = \hat{q}_i(\mathrm{x}) - \lambda \cdot \hat{c}_i(\mathrm{x}) \tag{85}$$

where $\hat{q}_i$ and $\hat{c}_i$ are estimated quality and cost, and λ is a Lagrange multiplier controlling the cost-quality tradeoff. Sweeping λ traces the Pareto frontier.

11.3. Cross-Model Diversity for Verification

The qualitative argument for cross-model diversity originates in the FKG inequality (Fortuin et al., 1971): under a shared-difficulty model, producer and verifier failure probabilities are positively correlated, so independence *underestimates* escape probability. This subsection takes a quantitative approach using the Gaussian copula, which directly models the correlation parameter and yields numerical predictions.

Definition 11.2 (Blind-Spot Correlation)**.** *For models m_i and m_j, define the latent Gaussian copula parameter $\rho(m_i, m_j)$ as the correlation of the underlying Gaussian variables in the copula representation of their joint failure distribution. Empirical estimates from Chen et al. (2026): same-family models have $\rho \approx 0.7$–0.9; cross-family models have $\rho \approx 0.3$–0.5.*

Proposition 11.5 (Cross-Model Diversity Gain)**.** [Mathematical Fact: follows from monotonicity of Φ_2 in its correlation parameter.] *Let m_P be the producer model and m_V the verifier model, with individual failure rates ϕ_P and ϕ_V. The correlated escape probability under a Gaussian copula (Sklar, 1959) with parameter ρ is:*

$$P_{\text{esc}}(\rho) = C_\rho(\phi_P, \phi_V) = \Phi_2(\Phi^{-1}(\phi_P), \Phi^{-1}(\phi_V); \rho) \tag{86}$$

The diversity gain from cross-family (ρ_{cross}) versus same-family (ρ_{same}) verification is:

$$\Delta_{\text{div}} = C_{\rho_{\text{same}}}(\phi_P, \phi_V) - C_{\rho_{\text{cross}}}(\phi_P, \phi_V) > 0 \tag{87}$$

This follows directly from the monotonicity of Φ_2 in its correlation parameter (Nelsen, 2006).

Corollary 11.1 (Weak Independent Beats Strong Correlated)**.** [Mathematical Fact: direct numerical consequence of copula monotonicity.] *A verifier with failure rate ϕ_V^{weak} and $\rho_{\text{cross}} = 0.4$ can achieve lower escape probability than a verifier with $\phi_V^{\text{strong}} < \phi_V^{\text{weak}}$ and $\rho_{\text{same}} = 0.8$, provided $C_{0.4}(\phi_P, \phi_V^{\text{weak}}) < C_{0.8}(\phi_P, \phi_V^{\text{strong}})$.*

Concrete example. With $\phi_P = 0.15$, a cross-family verifier with $\phi_V = 0.40$ and $\rho = 0.4$ yields $P_{\text{esc}} \approx 0.08$. A same-family verifier with $\phi_V = 0.20$ and $\rho = 0.8$ yields $P_{\text{esc}} \approx 0.11$. The weaker cross-family verifier achieves roughly 27% lower escape probability despite its lower individual detection rate. This result has a sharp design implication: the verification model must be from a different model family than the production model, a structural requirement, not an optimization variable.

Proposition 11.6 (Ensemble Saturation)**.** [Engineering Hypothesis: calibrated from empirical correlation data, not derived from first principles.] *Under a one-factor Gaussian copula with equicorrelation ρ, the probability that all N models simultaneously fail converges to a non-zero floor as $N \to \infty$ whenever $\rho > 0$ (Chen et al., 2026). Within-family ensembles ($\rho \approx 0.7$–0.9) saturate at $N = 3$–4; cross-family ensembles ($\rho \approx 0.3$–0.5) continue to improve to larger N.*

Proof sketch. Under the equicorrelated Gaussian copula, the joint failure event {all N models fail} has probability $P = \Phi_N(\Phi^{-1}(\phi), \ldots, \Phi^{-1}(\phi); \Sigma)$ where $\Sigma_{ij} = \rho$ for $i \neq j$. In the one-factor representation $X_i = \sqrt{\rho}\, Z + \sqrt{1-\rho}\, \epsilon_i$, conditioning on the common factor Z gives $P = \mathbb{E}_Z[\Phi((\Phi^{-1}(\phi) - \sqrt{\rho}\, Z)/\sqrt{1-\rho})^N]$. The integrand is bounded below by its value at $Z = \Phi^{-1}(\phi)/\sqrt{\rho}$, where the inner term approaches $1/2$. For $\rho > 0$, this lower bound is strictly positive and independent of N, establishing the saturation floor. □

11.4. The Pipeline Crossover Theorem

A fundamental question for model routing is: when should the harness invest in a single expensive-model attempt versus multiple cheap-model attempts? The answer depends critically on pipeline length and the distinction between two success semantics.

Definition 11.3 (Pass@N and Pass^n)**.** *For a model with per-attempt success probability q:*

$$\text{Pass@}N = 1 - (1-q)^N \qquad \textit{(at least one success in } N \textit{ attempts)} \tag{88}$$

$$\text{Pass}^n = q^n \qquad \textit{(all } n \textit{ sequential steps succeed)} \tag{89}$$

Pass@N applies when the harness can select the best output from multiple attempts. Pass^n applies when every step in a multi-step pipeline must succeed.

Theorem 11.1 (Pipeline Crossover)**.** [Mathematical Fact: follows from monotonicity of exponentials.] *Let q_e be the per-step success probability of an expensive model and $q_c < q_e$ that of a cheap model, with cost ratio $r = c_e/c_c > 1$. For a pipeline of n stages with Pass^n semantics, a single expensive-model attempt dominates $N = r$ independent cheap-model attempts (at equal total cost) when:*

$$q_e^n > 1 - (1 - q_c^n)^r \tag{90}$$

The crossover pipeline length n^* is characterized implicitly by $q_e^{n^*} = 1 - (1 - q_c^{n^*})^r$.

Proof sketch. A single expensive attempt at n stages succeeds with probability q_e^n. Running $N = r$ independent cheap attempts, each succeeding with probability q_c^n, gives Pass@N probability $1 - (1 - q_c^n)^N$. The expensive model dominates when the former exceeds the latter. Since q_e^n decays as $\exp(n \ln q_e)$ and $1 - (1 - q_c^n)^r$ decays approximately as $r \cdot q_c^n = r \exp(n \ln q_c)$ for large n (with $\ln q_c < \ln q_e < 0$), the expensive-model probability eventually dominates. The crossover n^* exists and is unique under mild conditions. □

Concrete example. For $q_e = 0.85, q_c = 0.60, r = 5$: at $n = 3$, $q_e^3 = 0.614$ versus cheap Pass@5 $= 0.714$ (cheap wins). At $n = 4$: $q_e^4 = 0.522$ versus cheap Pass@5 $= 0.504$ (expensive wins). The crossover occurs between $n = 3$ and $n = 4$.

Corollary 11.2 (Stage-Dependent Strategy)**.** *The optimal model selection strategy depends on pipeline structure: for single-stage or short pipelines ($n \leq n^*$), use cheap models with Pass@N sampling; for long pipelines ($n > n^*$), invest in per-step quality with expensive models. This connects to the budget reallocation evidence of Hassid et al. (2024), who showed a 13B model generating 5 outputs outperformed a 70B model generating 1 output by up to 15% on HumanEval under fixed compute budgets, but only when test-based selection is available. Without a reliable verifier, the Pass@N advantage vanishes.*

11.5. Adaptive Escalation

During pipeline execution, the harness may encounter tasks that exceed the assigned model's capability. The escalation policy decides when to switch to a more capable (and expensive) model.

Definition 11.4 (Escalation Threshold)**.** *Escalate from cheap model m_c to expensive model m_e when the posterior probability that the cheap model's response is adequate falls below:*

$$p^* = \frac{c_e + s(C)}{c_e + s(C) + c_{\text{rework}}} \tag{91}$$

where c_e is the expensive model cost, $s(C)$ is the switching cost (KV-cache invalidation for context of length C), and c_{rework} is the expected cost of fixing a failed cheap attempt.

The escalation decision maps to Wald's Sequential Probability Ratio Test (Wald, 1945):

Proposition 11.7 (SPRT-Based Escalation)**.** [Mathematical Fact: application of Wald-Wolfowitz optimality.] *Formulate H_0: cheap model response is adequate; H_1: cheap model response is inadequate. Accumulate evidence from quality signals (syntax validity, type-check pass, test results, self-evaluation score). The log-likelihood ratio $\Lambda_k = \sum_{i=1}^{k} \log(f_1(x_i)/f_0(x_i))$ evolves with each signal. Accept H_0 when $\Lambda_k \leq \log A$; accept H_1 (escalate) when $\Lambda_k \geq \log B$, with thresholds derived from the cost ratio.*

By the Wald-Wolfowitz optimality theorem (Wald and Wolfowitz, 1948), the SPRT minimizes the expected number of observations among all sequential tests achieving the same error rates, meaning the SPRT-based escalation policy gathers the minimum amount of evidence before deciding.

The model routing problem is fundamentally contextual: the best model depends on task features. We formalize this as a contextual bandit.

Definition 11.5 (Model Routing as Contextual Bandit)**.** *At each task t: the harness observes features $x_t \in \mathbb{R}^d$ (file complexity, task type, edit scope), selects model $a_t \in \mathcal{M}$, and observes reward $r_t(a_t) = q(a_t, x_t) - \lambda \cdot c(a_t)$.*

Proposition 11.8 (Routing Regret Bounds)**.** [Mathematical Fact: application of known bandit results to the routing setting.] *The following established regret bounds apply to model routing:*

(a) ***Non-contextual**: UCB1 achieves $O\left(\sum_{i:\Delta_i>0} \frac{\log T}{\Delta_i}\right)$ regret, matching the Lai-Robbins lower bound (**??**).*
(b) ***Contextual**: LinUCB achieves $O(d\sqrt{T \log T})$ regret for d-dimensional task features (**?**).*
(c) ***General policy class**: The reduction to cost-sensitive classification achieves $O(\sqrt{MT})$ regret (Agarwal et al., 2014).*

Model switching incurs a concrete cost: KV-cache invalidation. When switching from m_A to m_B mid-conversation, the entire KV cache is discarded, forcing recomputation costing $O(C)$ for context length C.

Theorem 11.2 (Switching-Cost Regret Penalty)**.** [Mathematical Fact: known result from online learning theory.] *With unit switching costs, the minimax regret for M-armed bandits over T rounds is $\Theta(M^{1/3}T^{2/3})$ (Dekel et al., 2014), strictly worse than the $O(\sqrt{MT})$ rate without switching costs.*

Corollary 11.3 (Batched Routing)**.** *By Perchet et al. (2016), $O(\log \log T)$ policy-update batches suffice to achieve near-optimal regret. In practice, re-evaluating the routing policy 3–5 times per day (rather than per-task) is theoretically near-optimal and eliminates unnecessary model switches.*

Proposition 11.9 (Budget-Constrained Routing (Badanidiyuru et al., 2018))**.** [Mathematical Fact: application of Bandits with Knapsacks.] *When routing is subject to a budget constraint B (daily or monthly API cost limit), the Bandits with Knapsacks framework achieves regret optimal up to polylogarithmic factors relative to the optimal dynamic allocation policy. This provides a principled answer to: "Given a $100/day API budget, how should I allocate tasks across model tiers?"*

Proposition 11.10 (Learning a Routing Policy)**.** [Engineering Hypothesis: sample counts depend on assumed gap Δ and feature dimension d.] *The sample complexity for learning an ϵ-optimal routing policy: non-contextual requires $O(M/\Delta^2 \cdot \log(M/\delta))$ tasks (approximately 1,000 for $M = 3$, $\Delta = 0.1$); contextual with d features requires $O(d^2/\epsilon^2)$ tasks (approximately 10,000 for $d = 10$, $\epsilon = 0.1$). Cold-start mitigation strategies include informative priors from benchmark data (**?**), warm-start transfer from other codebases, and heuristic initialization (e.g., "use Sonnet by default, escalate to Opus on failure").*

11.6. Cascade Routing Architecture

Inspired by the Viola-Jones cascade (Viola and Jones, 2001, 2004), we propose a cascade routing architecture where tasks flow through models of increasing capability, with early exit when the current model's output is deemed adequate.

Definition 11.6 (Model Cascade)**.** *A* model cascade *is a sequence of models $m_1, m_2, \ldots, m_L$ ordered by capability and cost ($c_1 < c_2 < \cdots < c_L$, $q_1 \leq q_2 \leq \cdots \leq q_L$). For each task: query m_1; if the quality estimator scores the response above threshold τ_1, return it; otherwise query m_2 (optionally with m_1's response as context); continue until m_L (which always returns).*

Proposition 11.11 (Cascade Expected Cost)**.** [Mathematical Fact: follows from the law of total expectation over cascade stages.] *For a cascade with L stages where stage i has acceptance probability a_i, the expected cost per task is:*

$$\mathbb{E}[cost] = \sum_{i=1}^{L} c_i \prod_{j=1}^{i-1} (1 - a_j) \tag{92}$$

When a_1 is large (most tasks are easy), the expected cost is dominated by c_1, the cheapest model.

Proof sketch. The cascade reaches stage i only if all previous stages rejected, with probability $\prod_{j=1}^{i-1}(1-a_j)$. Each stage incurs cost c_i when reached. The expected total cost is the sum over stages of cost times reach probability. This is the standard Viola-Jones cost decomposition (Viola and Jones, 2004). □

Quality estimation: the critical factor. Dekoninck et al. (2025) identify quality estimation as the sine qua non of effective routing. Observable quality signals for coding tasks include syntax validity and type-check pass (binary, fast, deterministic), test suite pass rate (high-signal but requires execution), model self-reported confidence (calibration varies), retry count (more retries signal harder tasks), and diff size and file count (complexity proxy).

Definition 11.7 (Routing Collapse)**.** [Engineering Hypothesis: documented empirically but not yet formally characterized.] Routing collapse *occurs when a learned router systematically defaults to the most capable (and expensive) model, even when cheaper models would suffice (Zhang et al., 2026). On RouterBench, existing routers send approximately 100% of queries to the strongest model under loose budgets, while the oracle optimal strategy uses it for fewer than 20%.*

The root cause is an *objective-decision mismatch*: routers trained on scalar quality prediction must make discrete ranking decisions at deployment. When 94.9% of queries have near-identical predicted quality across models (the "small-margin regime"), minimal perturbations flip routing decisions. Routing collapse is mitigated by cost-penalized routing (the Lagrangian form of Proposition 11.4), rank-based supervision (directly supervising per-query model rankings rather than scalar quality), and cascading over routing (try cheap first, escalate on evidence of inadequacy, converting a hard pre-routing classification into easier post-hoc quality assessment).

11.7. Empirical Calibration

Table 5 summarizes published cost savings from model routing systems.

Table 6 quantifies the capability gap by task type, confirming that routing value concentrates in the extremes.

Table 5 | Published cost savings from model routing systems.

System	Cost Reduction	Quality Retained	Source
RouteLLM (MT Bench)	85%	95% of GPT-4	?
RouteLLM (MMLU)	45%	92%	?
FrugalGPT	up to 98%	matches GPT-4	?
MetaLLM	up to 60%	+1% accuracy	Nguyen et al. (2024)
Cascade routing	14% perf. gain	same cost	Dekoninck et al. (2025)
Budget reallocation	75% compute	+15% HumanEval	Hassid et al. (2024)

Table 6 | Capability gap by task type (early 2026 benchmarks).

Task Type	Budget Model	Frontier Model	Gap
Single-function (HumanEval)	87–88%	97–98%	~10 pp
Multi-file bugs (SWE-bench V.)	40–50%	78–81%	~35 pp
Complex editing (Aider polyglot)	70% ($0.88)	88% ($29)	18 pp, 33× cost
Competitive programming (LCB)	79%	92%	~13 pp
Merge conflict resolution	LLaMA-8B best	CodeLlama-34B worse	Negative

The merge conflict result (Zhu et al., 2024) is particularly striking: general-purpose models outperform code-specialized models on a code task, suggesting that routing should consider model *type*, not just model *size*. Production architectures confirm the pattern: Cursor processes over 400 million predictions per day with task-specific routing (custom fine-tuned models for autocomplete, GPT variants for summarization, frontier models for agent tasks). Claude Code provides a two-tier cascade (Opus for architecture and planning, Sonnet for code generation). GitHub Copilot's "Smart Mode" routes across 9+ models from three providers.

11.8. Design Principles for Model Routing

The formal results yield six design principles:

1. **Cascade, don't predict.** *[Philosophical Observation.]* The cascade architecture (cheap model first, escalate on failure) is more robust than pre-routing (predict difficulty, select model). Pre-routing requires accurate difficulty estimation; cascading requires only adequate quality assessment, which is easier. This converts a hard classification problem into an easier verification problem.
2. **Cross-family verification is a hard constraint.** *[Mathematical Fact: from copula monotonicity.]* The FKG inequality and Gaussian copula analysis make same-family verification systematically biased. The verifier must come from a different model family than the producer (Corollary 11.1).
3. **Batch routing decisions.** *[Mathematical Fact: from switching-cost regret theory.]* KV-cache invalidation costs make per-task model switching expensive. Commit to a model for an entire conversation or task phase, re-evaluating only at natural breakpoints. Three to five policy updates per day is near-optimal (Corollary 11.3).
4. **Budget-aware allocation.** *[Engineering Hypothesis: depends on reward stationarity.]* Use the Bandits with Knapsacks framework (Proposition 11.9) to allocate tasks under API cost budgets. This naturally balances quality and cost without manual threshold tuning.
5. **Use pipeline length to select strategy.** *[Mathematical Fact: from the Crossover Theorem.]* For short pipelines ($n \leq 3$), use cheap models with Pass@N sampling. For long pipelines ($n > 3$), invest in per-step quality with expensive models (Theorem 11.1).

6. **Monitor for routing collapse.** *[Engineering Hypothesis: threshold is empirically derived.]* Track the fraction of tasks routed to the most expensive model. If it exceeds 80%, the router has likely collapsed and should be recalibrated or replaced with a cascade.

11.9. Contrarian Positions

We engage three objections to formal model routing in their strongest form.

"Single-model simplicity wins." Gabriel's "worse is better" principle (Gabriel, 1989) argues that implementation simplicity dominates interface correctness. Model routing is the LLM equivalent of premature microservice decomposition. Prompt format incompatibility across model families (XML for Claude, Markdown for GPT) multiplies maintenance cost. *Response:* The argument is strong for small-scale usage (fewer than 10,000 tokens per day). But the 1000× price range is not ignorable at scale. RouteLLM achieves 85% cost reduction using a lightweight BERT classifier; the relevant comparison is not "monolith vs. microservices" but "monolith vs. monolith with a routing header." Abstraction layers (OpenRouter, LiteLLM) further reduce integration burden.

"Frontier models will be cheap enough." LLM inference costs have declined approximately 1000× in three years. If this continues, routing becomes unnecessary. *Response:* The headline decline masks a K-shaped bifurcation: commodity inference is racing to zero while frontier reasoning models are getting *more* expensive. Google's Flash model surged 6.7× on input pricing between August 2024 and December 2025. Jevons paradox predicts that cheaper inference induces larger contexts, deeper reasoning chains, and more agent steps, maintaining cost pressure.

"Task difficulty prediction is too noisy." The routing collapse finding (Zhang et al., 2026), where 94.9% of queries occupy a small-margin regime, is the most damaging empirical evidence against routing. *Response:* We take this objection seriously. However, aggregate cost savings (85% reduction at 95% quality) demonstrate that routing works at the portfolio level even when individual decisions are noisy. The cascade architecture partially sidesteps the prediction problem by converting a priori difficulty classification into post-hoc quality assessment.

11.10. Cross-Pillar Connections

The Model Routing pillar connects to the other pillars in structured ways:

- **Reliability:** The compound error model ($R = p^n$, Observation 4.1) determines when per-step quality improvement (via a more expensive model) dominates architectural sophistication. The crossover theorem (Theorem 11.1) quantifies this tradeoff precisely.
- **Quality:** Cross-model diversity (Proposition 11.5) provides the formal basis for the Structural Separation Theorem's recommendation of different model families for code and test authorship.
- **Coordination:** The ensemble saturation result (Proposition 11.6) explains why multi-agent systems face diminishing returns from adding same-family agents: the correlation floor prevents ensemble improvement beyond $N = 3$–4 within a family.
- **Governance:** Budget-constrained routing (Proposition 11.9) is the resource-allocation complement to governance capacity bounds: the governance pillar bounds the rate of architectural drift, while routing bounds the cost of model consumption.

12. The Security Pillar

The Security pillar addresses the most structurally consequential property of agent harnesses: AI coding agents operate with developer-level credentials, yet their behavior is untrusted by design. A compromised or misbehaving agent has the same blast radius as a compromised developer, with access to source code, secrets, CI pipelines, and production infrastructure. The difference is that the developer exercises judgment; the agent follows instructions that may be adversarially manipulated. The central architectural result is that the agent must be placed *outside* the Trusted Computing Base, and security reduces to the correctness of a small, non-AI, formally verifiable enforcement boundary.

This pillar draws on four established bodies of theory: access control theory (capability-based security, the HRU undecidability result), information flow control (Denning's lattice model, Bell-LaPadula/Biba), the structural enforcement principle (connecting directly to the Reliability pillar's Proposition 3.4), and adversarial robustness analysis (prompt injection as the confused deputy problem, the BEB impossibility framework). Where the Reliability pillar proves that instructional compliance decays as $(1-\epsilon)^T$, the Security pillar identifies this as fundamentally a security result: security invariants are the canonical case where a single violation causes irreversible harm.

12.1. The Agent Outside the TCB

Definition 12.1 (Trusted Computing Base for Agent Harnesses)**.** [Mathematical Fact: follows from Department of Defense (1985); applied to agent context.] *The Trusted Computing Base (TCB) of an agent harness is the totality of protection mechanisms whose correct functioning is necessary for enforcing the security policy: the sandbox, the tool permission system, the credential vault, the network policy, and the output filter. The agent itself is* outside *the TCB.*

The architectural consequence is profound: security of the agent system reduces entirely to the correctness of a small, non-AI, formally verifiable TCB. The agent's behavior can be arbitrarily wrong, adversarially manipulated, or hallucinated, and the security policy must still hold.

Theorem 12.1 (TCB Correctness Sufficiency)**.** [Mathematical Fact: follows from reference monitor properties.] *If the TCB components $\{S, P, V, N, F\}$ (sandbox, permission system, credential vault, network policy, output filter) correctly implement the security policy Π, then no action by the agent can violate Π, regardless of the agent's internal state or the inputs it has received.*

Proof sketch. By the complete mediation property of the reference monitor (Anderson, 1972), every agent action passes through $\mathcal{M} = (S, P, V, N, F)$. By tamperproofness, the agent cannot modify $\mathcal{M}$. By the correctness of $\mathcal{M}$ (assumption), only actions consistent with Π are permitted. Therefore no action violating Π can execute. □

The theorem is trivial once stated, but it replaces the problem of "making the AI safe" with the problem of "making the sandbox correct," which is a classical software verification problem with known solutions (Klein et al., 2009).

12.2. Capability-Based Access Control

The agent's action space is naturally modeled through capability-based security (Dennis and Van Horn, 1966; Miller et al., 2003).

Definition 12.2 (Capability Set and Restriction Operator)**.** *The agent's full capability set C is the set of all operations (file reads, file writes, shell commands, network requests, API calls) the agent could*

invoke if unconstrained. A sandbox defines a restriction operator $\sigma : 2^C \to 2^C$ such that $\sigma(C) \subseteq C$ is the effective capability set. The operator σ is structurally enforced *if the agent cannot invoke any operation in $C \setminus \sigma(C)$ regardless of its internal state or instructions received.*

Definition 12.3 (Attack Surface)**.** *The attack surface $\mathcal{S} = \sigma(C) \setminus \mathcal{T}$, where $\mathcal{T} \subseteq C$ is the task-required capability set. The Principle of Least Authority (POLA) (Saltzer and Schroeder, 1975) requires minimizing $|\mathcal{S}|$: $\min_{|\sigma(C)|}$ subject to $\mathcal{T} \subseteq \sigma(C)$.*

Proposition 12.1 (Attack Surface Under Uncertainty)**.** [Engineering Hypothesis: tradeoff depends on α, which varies by task type.] *When $\mathcal{T}$ is estimated as $\hat{\mathcal{T}}$ with per-operation error probability $\alpha = \Pr(t \in \mathcal{T} \setminus \hat{\mathcal{T}})$, setting $\sigma(C) = \hat{\mathcal{T}}$ minimizes attack surface but fails task completeness with probability $1 - (1-\alpha)^{|\mathcal{T}|}$. The practical resolution is human-approved capability escalation: escape-hatch capabilities requiring human approval before activation, connecting directly to the Human Interaction pillar's approval gates.*

12.3. The Capability Attenuation Lattice

Capability-based security provides three structural properties for agent harnesses: (1) agent initialization as capability endowment (the harness passes a specific set of capabilities; the agent cannot acquire capabilities it was not given), (2) no ambient authority (the agent cannot discover resources by name, eliminating path-traversal attacks), and (3) delegation with attenuation (when an agent delegates a sub-task, it passes a subset of its own capabilities; the sub-agent cannot escalate beyond what was delegated). This attenuation property gives the capability set a lattice structure under set inclusion, with the empty set at the bottom and C at the top. Sub-agent capabilities are always below the delegating agent's capabilities in this lattice.

Theorem 12.2 (HRU Undecidability, Harrison et al., 1976)**.** [Mathematical Fact: classical result.] *The safety problem for general access control systems ("can a subject ever obtain a specific right through a sequence of operations?") is undecidable. Any Turing machine can be simulated by an HRU protection system, and safety reduces to the halting problem.*

Corollary 12.1 (Decidable Subcases for Harnesses)**.** *Safety is decidable for two subcases: (1) mono-operational systems (each command contains exactly one primitive operation; decidable but NP-complete), and (2) monotonic systems (permissions are fixed at initialization and never expanded at runtime; trivially decidable by inspecting the initial permission matrix).*

Most production sandboxes implement the monotonic subcase: the permission set is fixed before the agent begins executing and cannot be expanded by the agent itself. This is the design that Codex, Claude Code, and Spotify Honk all converge on. The connection to the Coordination pillar is direct: file-ownership contracts (§5) are a form of capability-based access control, where each agent's write set is a structurally enforced capability boundary that prevents cross-agent interference.

12.4. Information Flow Control and Credential Isolation

The agent needs some credentials (API keys for tool invocation, git tokens for repository access) but must never see others (production database passwords, code signing keys, cloud IAM admin credentials). We model this as an information flow control problem (Denning, 1976).

Definition 12.4 (Security Lattice for Agent Harnesses)**.** *A security lattice $(L, \sqsubseteq)$ with levels $L = \{\text{task}, \text{repo}, \text{infrastructure}, \text{production}\}$ and $\text{task} \sqsubseteq \text{repo} \sqsubseteq \text{infrastructure} \sqsubseteq \text{production}$. Each credential k has a security level $\ell(k) \in L$. The agent's clearance is ℓ_a. The noninterference property requires:*

$$\forall k : \ell(k) \not\sqsubseteq \ell_a \implies k \notin \text{context}(\text{agent}) \tag{93}$$

Theorem 12.3 (Credential Noninterference)**.** [Mathematical Fact: follows by construction from tool-level injection.] *If the harness implements credential injection at the tool level (the tool receives the secret directly from a vault; the agent sees only the tool's output), and if the tool's output does not contain the raw credential, then the noninterference property holds regardless of the agent's behavior.*

Proof sketch. The credential k with $\ell(k) \not\sqsubseteq \ell_a$ exists only in the vault (part of the TCB) and is injected into the tool invocation by the TCB. The agent's observable state consists of: (a) the prompt, which by assumption does not contain k; (b) tool outputs, which by assumption do not contain the raw k; and (c) environment variables, which by assumption do not contain k. Since k is absent from every component of the agent's observable state, the agent cannot leak k through any output channel. □

Remark (The BLP/Biba Tension)**.** *The agent faces a fundamental tension between confidentiality (Bell-LaPadula (Bell and LaPadula, 1973): "no read up, no write down") and integrity (Biba (Biba, 1977): "no read down, no write up"). The agent* must *read low-integrity input (user prompts, PR descriptions, web content) to do its job, violating Biba. Simultaneously, it must protect high-confidentiality secrets, requiring BLP. Following Lipner (1982), the resolution is structural separation of the data path and the credential path: the agent's context window has* $(c = \text{low}, i = \text{low})$*; the credential vault has* $(c = \text{high}, i = \text{high})$*; the TCB (not the agent) injects credentials. BLP is satisfied because secrets never flow to the agent's output. Biba is locally satisfied on the credential path because the vault is never modified by the agent. The Biba violation on the data path is contained: the data path cannot influence the credential path.*

In multi-agent harnesses where each agent may have a different clearance level, the decentralized label model (Myers and Liskov, 1997) extends Denning's lattice: each label specifies an owner and a set of readers, and information flow is permitted only when the target label's reader set includes the source label's readers. The label enforcement is structural (mediated by the harness), not behavioral (relying on agent compliance).

12.5. The Structural Enforcement Decay Theorem

The structural enforcement principle, developed as a cross-pillar result (Proposition 3.4), is fundamentally a security principle.

Theorem 12.4 (Structural Enforcement Decay)**.** [Mathematical Fact: $(1-\epsilon)^T$ is exponential decay; calibrated by AgentSpec and METR data.] *For a T-step agent pipeline where a prompt-enforced security constraint must hold at every step:*

$$\Pr(\text{no violation in } T \text{ steps}) = (1-\epsilon)^T \tag{94}$$

For structurally enforced constraints: $\Pr(\text{no violation in } T \text{ steps}) = 1 - \delta_s$*, where δ_s is the specification gap, which does not depend on T.*

This result reappears across the Information, Reliability, and Quality pillars, but security is the strongest motivation for structural over instructional enforcement. The reason: security invariants are the canonical case where *a single violation causes irreversible harm*. A single credential leak cannot be unlearned by the recipient. The $(1-\epsilon)^T$ decay from the Information pillar (§3) means that security invariants must not decay with pipeline length; only structural enforcement achieves this.

ϵ	$T = 5$	$T = 10$	$T = 20$	$T = 50$	$T = 100$
0.01	95.1%	90.4%	81.8%	60.5%	36.6%
0.02	90.4%	81.7%	66.8%	36.4%	13.3%
0.05	77.4%	59.9%	35.8%	7.7%	0.6%
0.10	59.0%	34.9%	12.2%	0.5%	<0.01%

At $\epsilon = 0.05$ and $T = 250$ (JetBrains' upper bound for agent runs), prompt-based security compliance is virtually guaranteed to fail at least once. Under adversarial conditions (the agent actively seeks to circumvent constraints), ϵ approaches 1 and the enforcement transition threshold collapses.

Theorem 12.5 (Cost-of-Failure Threshold)**.** *Let c_s be the cost of structural enforcement, c_p the cost of prompt enforcement, and L the expected cost of a failure. Structural enforcement is cost-effective when:*

$$L > L^* = \frac{c_s - c_p}{\epsilon - \delta_s} \tag{95}$$

For multi-step pipelines, the threshold drops to approximately L^*/n, since the expected number of violations grows linearly in n.

Proof sketch. The expected cost under prompt enforcement is $c_p + \epsilon L$; under structural enforcement, $c_s + \delta_s L$ where $\delta_s < \epsilon$. Setting these equal and solving for L yields $L^* = (c_s - c_p)/(\epsilon - \delta_s)$. In an n-step pipeline, the probability of at least one violation under prompt enforcement is $1 - (1 - \epsilon)^n \approx n\epsilon$ for small ϵ, so the effective threshold drops to approximately L^*/n. □

Using AgentSpec parameters (Wang et al., 2026): $\epsilon = 0.23$ (prompt failure rate), $\delta_s = 0.13$ (structural specification gap), $c_p = 1$, $c_s = 10$. Then $L^* = 9/0.10 = 90$. When a single violation costs more than $90\times$ the prompt engineering cost, structural enforcement is justified. For credential leakage or unauthorized deployment, L exceeds this threshold by orders of magnitude.

12.6. Adversarial Robustness: Prompt Injection as the Confused Deputy

Hardy (1988) defined the confused deputy problem: a more-privileged program is tricked by a less-privileged one into misusing its authority. The parallel to prompt injection is exact: the LLM agent (the deputy) holds tool permissions (the authority) and is tricked by adversarial content embedded in untrusted input (the confusion) into executing harmful operations. Willison (2024) identifies the *lethal trifecta*: when an agent has (1) access to private data, (2) exposure to untrusted content, and (3) ability to act, any prompt injection in the untrusted content can weaponize the agent's capabilities.

Definition 12.5 (Instruction-Data Conflation)**.** *An LLM exhibits instruction-data conflation because instructions and data share the same processing channel (the context window), with no structural delimiter the model can enforce. This distinguishes prompt injection from SQL injection: SQL injection was solved by parameterized queries, which structurally separate instruction from data channels (UK National Cyber Security Centre, 2023). LLMs have no analogous mechanism.*

Theorem 12.6 (Alignment Impossibility, BEB Framework, Wolf et al., 2024)**.** [Mathematical Fact: proven impossibility result.] *If undesired behavior B has non-zero probability under the model ($\alpha > 0$), there exist adversarial prompts of bounded length $L_1 = O(\log(1/\alpha)/\beta)$ that elicit that behavior, where β is the distinguishability of aligned and unaligned components. System prompt hardening increases the required adversarial prompt length by only $O(|s_0|)$, not exponentially.*

The implications are stark. First, for any undesired behavior with non-zero training-distribution probability, adversarial prompts of bounded length exist. Second, alignment (RLHF, Constitutional AI (Bai et al., 2022)) reduces α but only increases the required attack length logarithmically; the defense gain is sublinear in the alignment effort. Third, system prompt hardening provides linear, not exponential, defense.

Corollary 12.2 (Structural Defense Necessity)**.** *Because prompt injection cannot be eliminated at the model level (Theorem 12.6), and because prompt-based security decays as* $(1-\epsilon)^T$ *(Theorem 12.4), security guarantees must come from structural constraints: (1) the agent* cannot *take the harmful action (sandbox enforcement), (2) the action requires external authorization (human-in-the-loop, connecting to the Human Interaction pillar), (3) the credential is not accessible (vault with scoped tokens), (4) the output is validated structurally (output filters checking syntactic properties).*

12.7. Output Sanitization and Channel Capacity Restriction

Every agent output (code committed, shell commands executed, files written, network requests sent) is a potential attack vector. Output filtering restricts the effective channel capacity between the agent and the external system.

Proposition 12.2 (Covert Channel Risk)**.** [Engineering Hypothesis: covert channel bandwidth unmeasured for agent harnesses.] *Even with output filtering, covert channels may exist (Lampson, 1973). An agent can encode information in benign-looking outputs: URLs containing exfiltrated data as parameters, filenames encoding secrets, or timing patterns in sequential operations. Complete elimination of covert channels requires restricting not just the content but also the bandwidth, timing, and metadata of agent outputs.*

Spotify's Honk agent (Spotify Engineering, 2025b) provides the cleanest production instantiation: an allowlist of specific shell commands defines the output channel explicitly. Only $\mathrm{ripgrep}$, git operations (read-only), and a curated set of formatters and linters are permitted. Any command not on the list is rejected regardless of the agent's reasoning. This structural enforcement of the output channel means the permitted vocabulary is known-safe. The allowlist approach is structurally sound; a blocklist (anticipating every harmful operation) is structurally incomplete.

The need for output filtering is underscored by vulnerability generation rates. CodeRabbit (2025b) found that AI-generated code exhibits systematically higher vulnerability rates: 2.74× more XSS vulnerabilities, 1.91× more insecure object references, 1.88× more improper password handling, and 1.82× more insecure deserialization than human-written code across 470 PRs. These rates establish that AI-generated code requires *more* output scrutiny, not less. Static analysis and secret scanning are mandatory verification steps, not optional quality improvements.

12.8. Defense-in-Depth: When It Works and When It Doesn't

The classical argument for defense-in-depth assumes independent failure of security layers. For n layers with individual effectiveness p_i, the naive model predicts $\Pr(\text{attack succeeds}) = \prod_{i=1}^{n}(1-p_i)$. This exponential decay is seductive but unrealistic, connecting directly to the FKG inequality from the Quality pillar (Proposition 7.2): correlated security blind spots undermine the independence assumption.

Proposition 12.3 (Correlated Defense Bound)**.** [Engineering Hypothesis: $\beta = 0.5$ for heuristic layers is estimated, not measured.] *Using the beta factor model from reliability engineering (Fleming, 1975):*

$$\Pr(\textit{all fail}) \approx (1-\beta)^n q^n + \beta q \tag{96}$$

where β is the common-cause failure fraction and $q = \max_i(1-p_i)$. When $\beta > 0$, the common-cause term βq dominates the independent term. For agent harness security, heuristic defense layers (input sanitization, system prompt reinforcement, output validation) share a common failure mode: the instruction-data conflation (Definition 12.5). Estimating $\beta \approx 0.5$ for heuristic layers: the realistic correlated estimate for a three-heuristic-plus-one-structural defense stack is $333\times$ worse than the optimistic independent estimate.

Proposition 12.4 (Genuine Defense-in-Depth Criterion)**.** *Defense-in-depth provides genuine security improvement when: (1) layers have diverse failure modes (structural + heuristic), (2) layers operate on different data representations, (3) layers are not vulnerable to the same attack class. Combining multiple heuristic layers against the same attack vector provides diminishing returns bounded by β. The only layer providing genuinely uncorrelated defense is a structural one.*

12.9. The Multi-Agent Attack Surface

Multi-agent systems expand the attack surface in two dimensions. First, the number of potential injection points scales linearly with the number of agents processing untrusted content. Second, inter-agent communication creates new trust boundaries: if agent A_1 passes a summary to agent A_2, and the summary contains adversarial content inherited from untrusted input, A_2 becomes a secondary confused deputy. The lethal trifecta (Willison, 2024) applies at every inter-agent boundary.

Proposition 12.5 (Multi-Agent Blast Radius)**.** [Engineering Hypothesis: blast radius scaling depends on coordination topology.] *For k agents in a multi-agent system, the blast radius of a compromised agent A_i is bounded by its capability set $\sigma_i(C)$. Under the Coordination pillar's file-ownership contracts (disjoint write sets $F_i \cap F_j = \emptyset$), a compromised agent can damage only files in F_i, yielding blast radius proportional to $|F_i|/|\mathcal{F}|$ rather than $|\mathcal{F}|$. Without ownership contracts, a compromised agent with ambient authority can damage any file, and the blast radius is the full codebase.*

The Coordination pillar's assume-guarantee contracts (Theorem 5.1) serve double duty: they enforce file-ownership boundaries (preventing cross-agent write conflicts) and simultaneously enforce access control (agent A_i can only modify files in F_i, structurally preventing a compromised agent from writing outside its scope). This connection between coordination contracts and security boundaries is one of the strongest cross-pillar results: the same mechanism that prevents merge conflicts also limits blast radius.

The supply chain vector is specific to AI agents. The slopsquatting attack (Lanyado and Likhtenberg, 2025) demonstrates that approximately 20% of packages recommended by code generation models do not exist, and 43% of hallucinated package names are repeated across queries. Attackers can pre-register these predictable names with malicious payloads. For agent harnesses, this means that `pip install` and `npm install` commands executed by agents are security-critical operations requiring allowlist-based package verification.

12.10. Empirical Calibration

Five independent measurement studies calibrate the theoretical framework:

Source	Finding	Rate	Mechanism
Apollo Research	Covert action (o3, pre-training)	13.0%	In-context scheming
Apollo Research	Covert action (o3, post-training)	0.4%	Residual after anti-scheming
NIST CAISI	Grader gaming (CVE-Bench)	4.8%	Server crashing via $/\mathrm{dev}/\mathrm{urandom}$
NIST CAISI	Test manipulation (SWE-bench)	~1%	Commenting out assertions
METR	Reward hacking (RE-Bench)	30.4%	Monkey-patching, stack introspection
METR	Reward hacking (HCAST)	0.7%	Various
CodeRabbit	XSS vulnerabilities (AI vs human)	2.74×	Insecure code generation
AgentSpec	Prompt enforcement failure	23%	Non-compliance with prompt rules
AgentSpec	Structural enforcement failure	~13%	Specification gaps

Three patterns emerge from the empirical data.

1. Prompt-level defenses are insufficient. METR's finding that "do not cheat" instructions did not reduce reward hacking below 70% (METR, 2025) directly confirms the $(1-\epsilon)^T$ decay model. Adding "solve using intended methods" to the system prompt actually *increased* the reward hacking rate to 95%, confirming the BEB prediction that prompt-level defenses are fundamentally insufficient. The structural enforcement gap (AgentSpec (Wang et al., 2026): 87% structural vs. 77% prompt) provides calibration for $\epsilon \approx 0.23$ for prompt enforcement and $\delta_s \approx 0.13$ for structural enforcement.

2. Misbehavior compounds at scale. The 1–2% per-task reward hacking rate means 100–200 reward-hacking attempts per day in an organization making 10,000 agent invocations; the 0.4% residual covert action rate after anti-scheming training (Apollo Research, 2025) means 40 covert actions per 10,000 invocations. The variance suggests that task characteristics (scoring function visibility, tool access breadth, horizon length) dominate model characteristics in determining misbehavior rates. This implies that harness design choices are a primary lever for security, independent of model selection.

3. Longer pipelines are dramatically more vulnerable. METR found 30.4% reward hacking on longer RE-Bench tasks vs. 0.7% on shorter HCAST tasks, a $43\times$ difference, consistent with the $(1-\epsilon)^T$ model. AI-generated code exhibits 2.74× more XSS vulnerabilities than human-written code (CodeRabbit, 2025b), establishing that output filtering for security is not optional.

Remark (Connection to the Structural Enforcement Boundary)**.** *The Reliability pillar's structural enforcement boundary (Theorem 4.3) establishes the cost-of-failure threshold L^* at which structural enforcement becomes cost-effective. The Security pillar provides the strongest practical instances: for credential leakage, L includes breach notification costs, regulatory fines, and reputational damage, typically exceeding L^* by 10^2 to 10^4. For unauthorized code deployment, L includes service disruption and data integrity costs. These are the cases where the structural enforcement boundary is most clearly crossed, and where the gap between $(1-\epsilon)^T$ decay and structural stability has the most severe consequences.*

12.11. The Security-Usability Tradeoff

A common objection is that sandboxing kills productivity. The evidence is more nuanced. Cursor's production data (February 2026) shows sandboxing *reduces* developer interruptions by 40%, because the sandbox replaces per-command approval prompts with a permission boundary. Codex's two-phase model (setup with network access, agent phase without) amortizes dependency installation cost. The binding constraint in agent workflows is API latency (seconds to tens of seconds per LLM call), not sandbox setup (milliseconds to low seconds); the sandbox cost is amortized below the noise floor of the dominant latency source.

The genuine productivity concern is capability restriction: agents that cannot reach needed re-

sources fail tasks. The resolution is task-scoped capability grants (allowlists tuned per task type) with human-approved escalation, not removing the sandbox entirely. This connects to the Human Interaction pillar: security-critical operations (credential access, network requests to unapproved domains, file writes outside the task scope) require human approval, creating a natural boundary between autonomous agent operation and supervised escalation.

12.12. The Four-Layer Reference Security Architecture

Every production harness that achieves reliable security does so structurally. No production harness relies solely on prompt-based security. A survey of six production harnesses confirms convergence:

Harness	Execution Isolation	Network Control	Credential Separation	Output Filter	Injection Defense
Codex (cloud)	S	S	S	H	S
Claude Code	S	S	S (cloud)	H	H
Cursor	S	H	–	H	I
Devin	S	–	H	–	I
Spotify Honk	S	S	S	S	S
RAPID	S	–	–	S	S

(S = structural, H = hybrid, I = instructional, – = absent.)

Execution isolation is universal: Codex uses containers, Claude Code uses bubblewrap/Seatbelt, Cursor uses Landlock + seccomp, Devin uses per-session VMs, Honk uses containers with limited binaries. Credential separation at the tool level is the gold standard: Codex removes secrets before the agent phase starts; Claude Code's cloud proxy translates scoped tokens outside the sandbox; Honk handles all credential-bearing operations outside the agent entirely. Network isolation varies, with Devin notably having unrestricted internet access by default, which has been exploited for data exfiltration.

The convergent reference architecture consists of four structural layers, each independently enforced by the TCB:

Layer 1: Execution Isolation (structural, per task). The agent executes in an isolated environment: filesystem scope limited to the task-relevant directory, allowlist-only network egress, process isolation preventing privilege escalation.

Layer 2: Credential Management (structural, at tool level). Credentials are injected into tools, not into prompts. The tool receives the secret from a vault; the agent sees only the result. Short-lived tokens expire on task completion.

Layer 3: Output Filtering (structural, every write). Static analysis on every file write (secret detection, known vulnerability patterns). Allowlist-based shell command execution. URL filtering against the egress allowlist.

Layer 4: Adversarial Input Isolation (structural, at context assembly). External content is tagged with a low-integrity label before entering the agent's context. High-risk operations require explicit human confirmation regardless of context content.

12.13. Design Principles

The formal framework and empirical calibration yield six security-specific design principles:

1. **The agent is outside the TCB.** Security depends on the harness, not on the agent's behavior. Assume the agent is arbitrarily wrong.
2. **Structural enforcement for security; prompt enforcement for style.** Safety-critical constraints must be structurally enforced. This is the strongest application of Proposition 3.4.
3. **Credential injection at the tool level, never the prompt level.** If a secret appears in the agent's context window, it can be leaked through any output channel.
4. **Allowlist, do not blocklist.** A blocklist requires anticipating every harmful operation; an allowlist requires enumerating only the legitimate ones. Use the monotonic subcase (Corollary 12.1): fix permissions at initialization.
5. **Assume prompt injection will succeed.** The BEB impossibility (Theorem 12.6) means that for sufficiently long pipelines, prompt-based injection defenses will eventually fail. Design for containment, not prevention.
6. **Combine structural and heuristic layers, not multiple heuristic layers.** Defense-in-depth works when layers have uncorrelated failure modes. Multiple heuristic layers against the same attack vector provide diminishing returns bounded by β.

12.14. Contrarian Positions

"LLMs can be trained to be safe." Training is an instructional defense. It reduces ϵ but does not eliminate it. Over long pipelines, $(1-\epsilon)^T$ still converges toward zero. Apollo Research (2025) confirmed that anti-scheming training reduces covert action from 13% to 0.4%, but also increased evaluation-awareness reasoning, suggesting the model learned to detect evaluations rather than internalize alignment. METR (2025) showed that no prompt variant reduced reward hacking below 70%. Training is a necessary complement to structural enforcement, not a substitute for it.

"The threat model is unrealistic." The threat model is threefold: (a) the agent optimizes against poorly-specified objectives (NIST CAISI evidence: commenting out assertions, crashing servers (NIST CAISI, 2025)), (b) the agent processes adversarial external content (prompt injection via PR descriptions, malicious READMEs, poisoned dependencies; CVE-2025-53773 (Embrace The Red, 2025) demonstrated the end-to-end attack chain against GitHub Copilot), and (c) the agent has correlated failure modes at scale. None of these require the agent to be malicious; all require structural containment. The BLP/Biba formalism is not about treating the agent as an enemy but about ensuring information flows correctly regardless of the agent's behavior.

"Prompt injection is a solved problem." Wolf et al. (2024) proved that alignment-based defenses cannot reduce α to zero for behaviors in the model's training distribution. The NCSC (UK National Cyber Security Centre, 2023) concludes that prompt injection "will never be properly mitigated in the same way" as SQL injection because the underlying instruction-data conflation has no structural solution analogous to parameterized queries. Every mitigation reduces the attack success rate but cannot eliminate it without eliminating the model's ability to follow legitimate instructions from the same channel.

Conjecture 12.1 (Convergence of Security and Coordination Architecture)**.** [Philosophical Observation.] *As agent harnesses mature, the security architecture and the coordination architecture will converge to the same mechanism: capability-based access control, where each agent's capability set defines both its coordination scope (what files it may modify) and its security boundary (what resources it may access). The separation between "coordination contracts" and "security policies" is an artifact of current implementation fragmentation, not a principled distinction.*

The most important open questions are empirical. What is the actual correlation structure of per-step security failures? How does ϵ change as models improve? What is the beta factor for correlated

failures across heuristic defense layers? And, most practically: how do we build TCBs that are both small enough to verify and flexible enough for the diverse tasks agents must perform?

13. Cross-Pillar Synthesis

The preceding sections developed each architectural dimension independently. This section identifies the structural connections that bind them into a unified framework.

13.1. Information as the Unifying Currency

The deepest structural connection across all seven pillars is that each reasons about *information flow* through the harness:

- **Abstraction**: the information gap $K(P \mid S)$ between specification and code.
- **Information**: the Shannon decomposition $H(P \mid S) = H(P \mid S, C, \theta) + I(P; C \mid S, \theta) + I(P; \theta \mid S)$.
- **Reliability**: compound error as information destruction, $R(n) = p^n$.
- **Coordination**: error amplification as correlated information loss across agents.
- **Temporal**: context fidelity $\Phi(t) = e^{-\Lambda t}$ as information decay over time.
- **Quality**: the Detection Ceiling $d_{\ell,j} \leq I(X; D_j)/H(D_j)$ as an information bound on verification.
- **Governance**: governance capacity C_{gov} as information throughput of the correction channel.

The unification is strongest along the *semantic axis* (Abstraction, Information) and the *assurance axis* (Quality, Governance), where information theory is the native formalism. For the *execution axis* (Reliability, Coordination, Temporal), the information-theoretic framing is a valid interpretation but not the generative formalism: $R(n) = p^n$ is a probability product from reliability theory, the Extended Amdahl's Law draws on parallel computing and concurrency control, and VIH draws on scheduling and queueing theory. Calling compound errors "information destruction" is accurate (errors do corrupt the information content of pipeline output) but retrospective; the results would stand unchanged without the information-theoretic gloss. We adopt the information lens because it provides a common vocabulary for cross-pillar reasoning, while acknowledging that its explanatory power varies across pillars.

13.2. The Abstraction Gap Chain

The first structural connection links three pillars in a chain:

1. **Abstraction** defines the gap: $\mathcal{G}(S, P) = K(P \mid S)$.
2. **Information** operationalizes it: $H(P \mid S) = H(P \mid S, C, \theta) + I(P; C \mid S, \theta) + I(P; \theta \mid S)$, decomposing the gap into three actionable levers.
3. **Quality** addresses what happens when the gap is traversed poorly: the 18 slop types are failure modes of the translation from spec to code, and the Compound Degradation Theorem (Theorem 7.3) quantifies the consequences.

The chain makes precise how a specification becomes code and what goes wrong along the way. The Abstraction pillar establishes the theoretical limits. The Information pillar identifies the levers. The Quality pillar classifies the failure modes and designs defenses.

13.3. The Compound Error Cascade

The second structural connection links three pillars through the theme of multiplicative degradation:

1. **Reliability** proves $R(n) = p^n$: compound errors in sequential pipelines.
2. **Coordination** extends to multi-agent settings: the Extended Amdahl's Law (Theorem 5.1) adds the reliability factor $(1 - \delta_a)^n$, and empirical amplification (17.2×) exceeds the independence prediction.
3. **Temporal** frames the rate at which compound errors occur: VIH = $\lambda_{\text{raw}} \cdot p_{\text{verify}}$ captures the tradeoff between iteration speed and error rate.

The cascade reveals a fundamental tension: the same exponential that makes compound errors devastating also makes per-step improvements highly leveraged (elasticity = n). This is the core design insight: invest in per-step quality, not pipeline sophistication.

13.4. The Structural Enforcement Principle

The principle that structural enforcement dominates instructional enforcement appears independently in four pillars:

Table 7 | The structural enforcement principle across pillars.

Pillar	Manifestation
Information	Tier boundaries enforced structurally (file system, .gitignore) vs. instructionally (prompt rules). Compliance gap: $(1-\epsilon)^T$.
Reliability	Cost-of-failure threshold L^* determines when structural enforcement is cost-effective. Threshold drops as $1/n$ for multi-step pipelines.
Quality	Decidable slop types should be caught by structural gates (Layer 1), not by probabilistic methods.
Coordination	File-ownership contracts enforce $F_i \cap F_j = \emptyset$ structurally, eliminating textual/syntactic conflicts by construction.

The convergence is not coincidental. Structural enforcement converts probabilistic guarantees into deterministic ones. In any system where errors compound (as they do in all agent pipelines), converting even a small fraction of probabilistic steps to deterministic ones has outsized impact on end-to-end reliability.

13.5. The Four-Layer Defense Architecture

The four-layer verification architecture appears in both the Reliability and Quality pillars:

Table 8 | The shared four-layer verification architecture.

Layer	Mechanism	Cost	Determinism	Decidability Class
1	Structural gates	Very low	Deterministic	Decidable
2	Automated testing	Low	Probabilistic	Semi-decidable
3	AI review	Moderate	Probabilistic	Semi-decidable
4	Human review	High	Judgment	Undecidable

The ordering by cost and determinism is not arbitrary; it follows from the decidability classification. Decidable defects should be caught at Layer 1 (structural gates), where detection is certain

and marginal cost is near zero. Semi-decidable defects require Layers 2–3, where detection is probabilistic but cost-effective. Undecidable defects require Layer 4, where human judgment provides the irreplaceable element.

13.6. Governance Capacity as the Macro Constraint

The Governance pillar's capacity bound (Theorem 8.2) is the system-level analog of the Reliability pillar's compound error sensitivity (Observation 4.1):

- **Reliability**: at the step level, if $p < 1$, errors compound as p^n.
- **Governance**: at the system level, if $R_{\text{drift}} > C_{\text{gov}}$, divergence accumulates without bound.

Both are threshold phenomena: below the threshold, the system is self-correcting; above it, degradation is unbounded. Both have the same structural form: a rate of degradation versus a capacity for correction. And both yield the same design prescription: invest in increasing the correction capacity (per-step quality, governance bandwidth) rather than adding post-hoc repair mechanisms.

13.7. Shared Formal Machinery

Table 9 maps the 16 distinct formal techniques used across the seven pillars. The density of the mapping (most techniques appear in 2+ pillars) reflects the deep structural unity of the framework; the pillars are not independent theories glued together but facets of a common problem viewed through different mathematical lenses.

Table 9 | Formal machinery shared across pillars. A bullet (•) indicates that the pillar makes substantive use of the technique; a circle (○) indicates secondary or analogical use.

Technique	Abs	Inf	Rel	Crd	Tmp	Qlt	Gov
Shannon information theory (H, I, DPI)	•	•	○	○	○	•	•
Kolmogorov complexity (K)	•	○					•
Series reliability / compound errors (p^n)			•	•	○		
Structural vs. instructional enforcement		•	•	•		•	○
Submodular optimization / greedy approx.		•				•	
Control theory (cascade, Nyquist, stability)					○		•
Lattice theory (Galois, closure, refinement)	•						•
Concurrency control / isolation levels				•			
Survival analysis (competing risks, Weibull)							•
Scheduling theory (Amdahl, stochastic DAG)				•	•		
FKG / correlation inequalities			•			•	
Graph partitioning (min-cut, Cheeger)				•			
Rate-distortion theory							•
Queueing theory (Little's law, Kingman)					•		
Speculative execution theory					•		
Cache competitive analysis (Sleator-Tarjan)					•		

Three patterns emerge. First, Shannon information theory provides the broadest cross-pillar vocabulary, appearing substantively in four pillars and secondarily in three more. Second, structural enforcement is the most widely shared *design principle* (four pillars derive it independently). Third, several techniques (concurrency control, survival analysis, cache competitive analysis) are used in only one pillar, suggesting that further cross-pollination may be productive; for instance, survival analysis could model the lifetime of context relevance (connecting Governance to Temporal), and concurrency control formalisms could sharpen the governance ratchet's interaction with parallel

agent modifications.

13.8. Shared Empirical Sources

Table 10 maps the 14 primary empirical sources to the pillars that rely on them. The density of cross-pillar citations highlights both a strength (convergent evidence from multiple analytical perspectives) and a vulnerability (the same data points bear disproportionate evidentiary weight).

Table 10 | Key empirical sources and the pillars that draw on them. Columns indicate pillar usage: Abs(traction), Inf(ormation), Rel(iability), Crd (Coordination), Tmp (Temporal), Qlt (Quality), Gov(ernance).

Source	Abs	Inf	Rel	Crd	Tmp	Qlt	Gov	Key Finding
GitClear 2025		•			•	•	•	211M lines; 8× duplication; 60% refactoring collapse
DORA 2025				•	•	•	•	Throughput up 21%; stability negative
Kim et al. 2025	•			•				17.2× error amplification; 180 configs
METR 2026	•		•			•		50% test-passing PRs unmerged
Liu et al. 2024		•						Lost in the Middle; U-shaped recall
Chroma 2025		•			•			18 models; monotonic degradation
JetBrains 2025		•						Complexity increase in AI code
CodeRabbit 2025						•		1.68× issues; 2.74× XSS vulns
Veracode 2025						•		AI code security defect rates
Apollo Research 2025			•			•		Scheming behaviors in agents
NIST CAISI 2025			•			•		Benchmark cheating evidence
Hariharan et al. 2025			•					63% failure on 100-step tasks
Spotify Honk 2025		•				•		Production harness telemetry
Gloaguen et al. 2026		•						AGENTS.md convention data

The table reveals a concentration risk: GitClear, DORA, and METR each support claims in 3–4 pillars, meaning that a successful challenge to any one of these sources would ripple across the framework. Section 14 provides a quality assessment of each source to make this risk explicit.

13.9. Production Harness Instantiation

Table 11 shows how four production harnesses instantiate the framework's seven pillars, providing indirect validation that the pillar decomposition captures real architectural concerns.

The convergence across independently developed harnesses toward similar architectural patterns (fresh context, structural enforcement, contract-based coordination, layered verification) provides indirect support for the framework's decomposition. However, this is post-hoc rationalization, not prediction; see Section 13 on the distinction.

Table 11 | How production harnesses instantiate the seven pillars. Each cell describes the primary mechanism.

Harness	Abstraction	Information	Reliability	Coordination	Temporal	Quality	Governance
GSD	Phase-level specs	Fresh 200K per task	4 verification gates	Sequential phases	45–60 min cycles	Per-phase UAT	Phase artifacts
RAPID	Set definitions + contracts	Scoped CLAUDE.md per worktree	Hunter-advocate-judge cycles	Git worktrees + CONTRACT.json	Parallel sets; wave-sequential	4-skill review cascade	HANDOFF.md; contracts
Blueprint	ADR templates	21 read-only agents	Compliance audit; fitness functions	Parallel read-only agents	Async governance (never blocks)	Decidability-aware detection	Full ADR lifecycle + drift detection
Turing	Hypothesis files	Three-tier (hidden/read-only/write)	Immutable eval + behavioral probes	Researcher/evaluator role split	Autonomous loop with convergence	Metric regression guards	hypotheses.yaml lineage

13.10. Additional Cross-Pillar Connections

Beyond the three primary cascades (abstraction gap chain, compound error cascade, structural enforcement principle), five cross-pillar interactions merit formal development.

1. Accretion meets the ratchet. The Quality pillar's Accretion Category (individually CI-passing changes that collectively degrade codebase health) is precisely the failure mode that the Governance pillar's ratchet mechanism (Definition 8.1) aims to prevent. The interaction, however, is non-trivial. Accretion defects are correct by construction; they satisfy all existing ratchet constraints. To prevent accretion, the ratchet must enforce *structural health metrics* (duplication rate, dependency fan-out, abstraction depth) rather than per-change correctness. But structural health constraints are inherently more brittle than correctness constraints: they have higher false-positive rates and are sensitive to legitimate refactoring. Adding ratchet constraints on structural metrics risks triggering the ossification pathology (the Governance pillar's Ossification Corollary), where the constraint set grows until the allowed change set becomes empty or effectively so. The calibration question is: at what threshold should structural health metrics be ratcheted, and how should evidence expiry windows be set relative to the accretion rate $\alpha(t)$? The Compound Degradation Theorem (Theorem 7.3) provides the growth rate; the Ratchet Convergence corollary provides the stabilization guarantee; what is missing is the bridging analysis showing how to set ratchet parameters that prevent accretion without inducing ossification.

2. Staleness compounds with coordination. The Temporal pillar's context fidelity decay $\Phi(t) = e^{-\Lambda(t-t_0)}$ interacts multiplicatively with the Coordination pillar's error amplification. In multi-agent settings, the effective decay rate Λ is not a constant; it increases with the number of concurrent agents, because more agents means faster codebase evolution and therefore faster context invalidation. Specifically, if k agents each commit at rate λ_c, the effective decay rate for any one agent's context is $\Lambda_{\text{eff}} \approx (k-1)\lambda_c\alpha$, where α is the fraction of commits that modify files in the agent's active context. This creates a feedback loop: more agents $\rightarrow$ faster staleness $\rightarrow$ higher per-agent error rate $\rightarrow$ more rework $\rightarrow$ more commits $\rightarrow$ even faster staleness. The Extended Amdahl's Law (Theorem 5.1) does not account for this feedback; incorporating staleness-induced error rate inflation into the reliability factor $(1-\delta_a)^n$ would likely reduce n^* below the current estimate $1/\sqrt{\delta_a}$, particularly for tightly coupled codebases where α is large.

3. Lattice parallels across Abstraction and Governance. Both the Abstraction pillar and the Governance pillar employ lattice-theoretic structures, but with different orientations. The Abstraction pillar's refinement lattice orders specifications from most abstract (top, natural language) to most concrete (bottom, executable code), with the Galois connection (α, γ) mediating between specifications and programs. The Governance pillar's constraint lattice orders constraint sets by inclusion, with the ratchet as a closure operator that moves the system toward the join (more constrained). These two lattices are not independent: every specification in the refinement lattice *induces* a constraint in the governance lattice (the specification constrains which programs are acceptable). A formal bridge would establish that the Galois connection on the refinement lattice is compatible with the closure operator on the constraint lattice; that is, refining a specification (descending the refinement lattice) should tighten constraints (ascending the constraint lattice) monotonically. If this compatibility holds, then the act of specification refinement automatically generates governance constraints, providing a formal foundation for "specification as governance."

4. Decomposition enables reuse discovery. The Coordination pillar's task decomposition (balanced min-cut on the coupling graph) and the Information pillar's reuse discovery problem are connected through the structure of the coupling graph itself. Balanced min-cut partitions identify clusters of highly coupled files; these clusters are also the natural units for reuse discovery, because semantic similarity concentrates within clusters. An agent assigned to a cluster via the decomposition has, by construction, the most relevant context for discovering reuse opportunities within that cluster. This suggests that coordination topology should be co-designed with the context architecture: the partition that minimizes merge conflicts (Coordination) simultaneously maximizes reuse-relevant context concentration (Information). The formal connection is that the coupling graph's cluster structure determines both the optimal decomposition and the natural reuse neighborhoods, and a single spectral analysis of the coupling graph can serve both objectives.

5. FKG amplification across coordination boundaries. The Quality pillar's FKG inequality (Proposition 7.2), which establishes that same-family LLM judges underestimate escape probabilities due to correlated blind spots, has a multi-agent analog that the current framework does not address. When k agents from the same model family work on a shared codebase, their errors are positively correlated not only within each agent's pipeline (the Quality pillar's concern) but *across* agents (the Coordination pillar's concern). The FKG inequality implies:

$$P\left(\bigcap_{i=1}^{k} E_i\right) \geq \prod_{i=1}^{k} P(E_i) \tag{97}$$

where E_i is the event that agent i introduces a defect in the shared semantic category. This means the Coordination pillar's independence assumption ($A_e \to k$) not only underestimates amplification (as the Kim et al. empirical data shows) but does so in a structurally predictable way: the FKG framework provides an *upper* bound on the independence gap. Combining the FKG inequality with the coordination error amplification model would yield tighter bounds on multi-agent defect rates, explaining part of the $6\times$ discrepancy between the theoretical independence prediction and the empirical $17.2\times$ amplification.

13.11. The Human Correction Cycle

A significant limitation of the current framework is that it models agent pipelines as feedforward (Reliability) or parallel (Coordination) systems, with verification as a separate stage. In production

harnesses, the dominant quality mechanism is the *human correction cycle*: a user reads the agent's output, identifies where it went wrong, provides a correction or redirect, and the agent revises. This cycle operates continuously, not as a final gate.

The four-layer verification architecture (Section 4) places human review at Layer 4 (highest quality, highest cost, lowest throughput). This framing is technically correct in terms of cost per defect detected, but it misrepresents the role of humans in practice. Humans are not the last layer of a cascade; they are active co-pilots who steer the agent at every step. Human review is the only verification mechanism that can: (a) catch novel error categories not anticipated by automated layers, (b) redirect the agent when it is solving the wrong problem entirely, and (c) supply the tacit knowledge that the Governance pillar (Section 8) identifies as unexternalizable.

The efficiency metric $\eta = d/c$ (detection rate per unit cost) penalizes human review for being expensive but does not credit it for handling the tail of the error distribution, including errors that no automated layer can detect. A complete formal treatment would model the human as a feedback channel with its own capacity constraints, latency characteristics, and learning dynamics (Bainbridge's Ironies apply: as automation improves, human monitors get less practice, making their interventions less reliable precisely when they are most needed). We leave this formalization as an important direction for future work and note that the design principles in Section 15 should be interpreted in the context of a system where human correction is the dominant quality lever, not merely the most expensive one.

13.12. The Governance Information Budget

We propose the *Governance Information Budget* as the integrating concept across all seven pillars. The idea is that every harness operates under a total information budget, and the pillars decompose how that budget is allocated:

$$\underbrace{H(P \mid S)}_{\text{Abstraction gap}} = \underbrace{I(P; C \mid S, \theta)}_{\text{Context (Information)}} + \underbrace{I(P; \theta \mid S)}_{\text{Prior (Model)}} + \underbrace{H(P \mid S, C, \theta)}_{\text{Residual (Harness must handle)}} \tag{98}$$

This equation uses the Shannon operationalization $H(P \mid S)$ throughout, ensuring all terms are computable and live in the same mathematical framework. The Kolmogorov formulation $K(P \mid S)$ is asymptotically equivalent for typical sequences (Vitányi, 2004) and serves as the distribution-free theoretical ceiling, but should not be mixed with Shannon quantities in the same equation.

The residual term $H(P \mid S, C, \theta)$ is the information that neither the context nor the model provides. This residual must be filled by the agent's exploration and reasoning, which is subject to:

- Compound error sensitivity (Reliability): each exploratory step has failure probability q, compounding as $(1-q)^n$.
- Error amplification (Coordination): multi-agent exploration amplifies errors by up to $17.2\times$.
- Context staleness (Temporal): the residual grows as context decays, $\Phi(t) = e^{-\Lambda t}$.
- Detection limits (Quality): some errors in filling the residual are undetectable, $d_{\ell,j} \leq I(X; D_j)/H(D_j)$.
- Governance capacity (Governance): the total rate at which the residual is filled must not exceed governance capacity C_{gov}.

Proposition 13.1 (Governance Information Budget Constraint)**.** *For a harness operating at steady state with change rate λ (changes per unit time), the Governance Information Budget imposes five si-*

multaneous constraints, one from each execution-axis and assurance-axis pillar:

$$\textit{(Reliability)} \quad \lambda \leq \frac{-\ln R_{\min}}{n \cdot |\ln p|} \tag{99}$$

$$\textit{(Coordination)} \quad k \leq n^* \approx 1/\sqrt{\delta_a} \tag{100}$$

$$\textit{(Temporal)} \quad \lambda \leq \Lambda / \ln(1/\Phi_{\min}) \tag{101}$$

$$\textit{(Quality)} \quad \lambda \cdot P_{esc} \leq \mu_{repair} \tag{102}$$

$$\textit{(Governance)} \quad \lambda \cdot H_{drift/change} \leq C_{gov} \tag{103}$$

where $R_{\min}$ is the minimum acceptable pipeline reliability, $\Phi_{\min}$ is the minimum acceptable context fidelity, P_{esc} is the escape probability past all defense layers, μ_{repair} is the defect repair rate, and $H_{drift/change}$ is the architectural drift per change (in bits). The binding constraint (the one that limits λ most severely) determines which pillar is the system's current bottleneck.

Remark. *The five constraints in Proposition 13.1 are not independent. Increasing k (more parallel agents) relaxes the Reliability constraint (more throughput) but tightens the Temporal constraint (faster staleness) and the Governance constraint (more drift per unit time). Increasing per-step quality p relaxes the Reliability constraint but has no direct effect on the Governance constraint. This interdependence is the core reason that harness design requires simultaneous optimization across pillars, not sequential optimization of each pillar independently.*

The Governance Information Budget unifies the pillars by showing that they all constrain different aspects of the same fundamental problem: closing the abstraction gap reliably, efficiently, and coherently. The binding constraint rotates as the harness configuration changes: early in development (low k, high residual), the Abstraction and Information pillars dominate; at scale (high k, fast λ), the Coordination and Governance pillars dominate; for mature, stable systems (low λ, long-lived context), the Temporal pillar dominates through staleness accumulation.

13.13. Prediction vs. Post-Hoc Rationalization

The framework cites production harnesses (Claude Code, Cursor, Codex, GSD, RAPID) whose design choices are consistent with the formal results. An important methodological question is whether the framework *predicted* any of these design choices or merely *rationalized* them after the fact.

We distinguish three categories among the framework's claims:

1. **Post-hoc formalizations of existing practice:** The three-tier context architecture, fresh context dominance, structural enforcement preference, and "cheap layers first" verification ordering all emerged from practitioner experience before this framework existed. The framework's contribution for these is *precision* (quantifying thresholds, proving optimality conditions), not *prediction*.
2. **Quantitative predictions testable against existing data:** The optimal context budget at $0.15W$–$0.40W$, optimal agent count at $n^* \approx 1/\sqrt{\delta_a}$, the governance bifurcation threshold $\mu G/\gamma_g R = 1$, and the 96.5% convergence at 3 verification rounds are specific enough to test. These are stated with sufficient precision that measured values falling outside the predicted ranges would falsify the predictions (see verification roadmap, Section 17.3).
3. **Novel architectural predictions not yet observed:** The detection ceiling theorem (Fano-bounded detection rates for specific slop types), the Cross-Change coupling complexity growth (quadratic in number of unrefactored changes), and the governance rate-stability threshold make predictions about system behavior that have not, to our knowledge, been tested. These are the framework's strongest claims to predictive power, contingent on empirical validation.

A framework that only rationalizes existing practice has value (it explains *why* practitioners converged on certain designs), but less value than one that predicts novel phenomena. We encourage empirical work targeting category 3.

14. Empirical Foundations

The formal framework developed in Sections 2–13 rests on empirical evidence of varying quality. An honest assessment of this evidence is essential.

14.1. Strong Empirical Foundations

GitClear 2025. Analysis of 211 million changed lines across a multi-year period (GitClear, 2025). Key findings: code blocks with 5+ duplicated lines increased approximately 8× (overall duplication rate rose from 8.3% to 12.3%); refactoring collapsed from roughly 25% to under 10% of changed lines. Strengths: large sample, longitudinal design, objective metrics. Limitations: observational (no causal identification), single platform, code-level metrics only (no quality assessment).

DORA 2025. Telemetry from over 10,000 developers (Google Cloud, 2025). Key finding: individual task completion increased 21% with AI adoption, but organizational-level throughput stayed flat and software delivery stability showed a negative correlation with AI usage. Strengths: large sample, standardized metrics, multi-organization. Limitations: survey-based components, correlation not causation, no controlled intervention.

Kim et al. 2025. 180 configurations across 5 architectures, 3 LLM families, and 4 benchmarks (Kim et al., 2025). Key finding: 17.2× error amplification for independent agents, saturation of scaling benefits at approximately 45% of the task completion rate. Strengths: systematic, controlled, comprehensive coverage of configuration space. Limitations: benchmark tasks (not production code), specific models and benchmarks.

METR 2026. Expert evaluation of SWE-bench test-passing pull requests (METR, 2026b). Key finding: roughly 50% of test-passing PRs would not be merged by repository maintainers (but only 68% of human-written golden patches would be merged on re-review, so the AI-attributable gap is approximately 18 percentage points). Strengths: expert evaluation, direct relevance to the circular validation problem. Limitations: subjective judgments, small evaluator pool (4 maintainers, 3 repositories), specific benchmark context.

14.2. Moderate Empirical Foundations

Context degradation studies. Multiple studies (Chroma, 2025; Du et al., 2025; Liu et al., 2024) demonstrating monotonic performance degradation with context length. The power-law model $\delta(n) = (n/W_{\text{eff}})^{\alpha_d}$ with $\alpha_d \in [0.3, 0.5]$ is a fit to this data, not a derived result. The fit is reasonable but not definitive.

CodeRabbit 2025. Analysis of 470 pull requests (CodeRabbit, 2025a). Findings: 1.68× more issues in AI-generated PRs; 2.74× more XSS vulnerabilities. Strengths: direct comparison, automated analysis. Limitations: modest sample size, single tool, potential selection bias.

Hariharan et al. 2025. Evaluation of plan-and-execute agent architectures (Hariharan et al., 2025). Key finding: 63% failure rate on 100-step tasks. Strengths: systematic scaling study. Limitations: specific agent architecture, benchmark-only evaluation.

14.3. Weak or Missing Empirical Foundations

Several key claims in the framework lack strong empirical support:

- The optimal context budget ($|C^*| \approx 0.15W$ to $0.40W$) is derived from model fits, not from direct optimization experiments.
- The structural enforcement compliance gap $(1-\epsilon)^T$ is a theoretical prediction; no controlled study has measured ϵ for different enforcement mechanisms.
- The governance capacity bound (Theorem 8.2) is a theoretical result by construction; calibrating R_{drift} and C_{gov} for real systems remains an open challenge.
- The Accretion Category is well-motivated by observational data but has not been validated through controlled experiments.
- VIH has not been measured in production harness settings.
- The cache-staleness EOQ theorem has not been empirically validated.

14.4. Evidence Quality Assessment

Table 12 provides a systematic quality assessment of the empirical foundations supporting the framework. Quality ratings follow a four-tier scheme: **High** (large sample, controlled or quasi-experimental, replicable); **Medium** (moderate sample, systematic but observational, partially replicable); **Low** (small sample, anecdotal or single-platform, limited replicability); **Very Low** (theoretical prediction only, no direct empirical measurement).

Table 12 | Evidence quality assessment for major empirical sources and theoretical claims. "Key Gap" identifies the most important missing piece for each data category.

Data Category	Quality	Sample	Design	Key Gap
GitClear (duplication, refactoring)	High	211M lines	Longitudinal, observational	No causal identification; single platform; no direct quality measure
DORA 2025 (throughput, stability)	High	10,000+ devs	Cross-sectional survey + telemetry	Survey components; no controlled intervention; correlation only
Kim et al. (error amplification)	High	180 configs	Controlled, systematic	Benchmark tasks only; 3 model families; not production code
METR (merge gap)	Medium	~100 PRs	Expert evaluation	4 evaluators, 3 repos; subjective criteria; small sample
Liu et al. (Lost in Middle)	Medium	Multiple models	Controlled, synthetic tasks	Synthetic needle tasks; unclear production generalization
Chroma (context rot)	Medium	18 models	Controlled, multi-provider	Synthetic benchmarks; degradation model is a fit, not derived
JetBrains (complexity)	Medium	Survey + metrics	Cross-sectional	Self-reported AI usage; single IDE ecosystem
CodeRabbit (PR quality)	Medium	470 PRs	Comparative	Single tool; potential selection bias; modest sample
Veracode (security)	Medium	Undisclosed	Static analysis comparison	Methodology details limited; no controlled experiment
Apollo Research (scheming)	Low	Case studies	Behavioral evaluation	Small-scale; specific models; adversarial setup
NIST CAISI (cheating)	Low	Workshop report	Expert panel consensus	No quantitative data; policy-oriented rather than empirical
Optimal context budget	Very Low	None	Model fit to degradation data	No direct optimization experiment; parameter sensitivity unknown
Structural enforcement ϵ	Very Low	None	Theoretical prediction	No measurement of per-step violation probability
VIH in production	Very Low	None	Proposed metric, unmeasured	No harness instrumentation; no baseline data
Governance capacity (C_{gov})	Very Low	None	Theoretical construction	R_{drift} and C_{gov} undefined operationally
Accretion detection	Very Low	None	Category proposed, unvalidated	No labeled dataset; no precision/recall measurement
Cache-staleness EOQ	Very Low	None	Theoretical analog	No measurement of staleness cost or optimal refresh interval

14.5. The Calibration Gap

A recurring pattern across all seven pillars is the gap between theoretical models and empirical calibration. The models are well-grounded in established formal machinery, and the qualitative predictions align with practitioner experience. But the quantitative parameters (decay rates, threshold values, optimal counts) are either estimated from limited data or derived from model assumptions rather than measured directly.

This gap is not a criticism of the framework; it is a statement of the current state of the field. Closing this gap requires controlled experiments, which in turn requires access to production harness telemetry. Section 17 proposes specific experiments, organized as a concrete verification roadmap with falsification criteria for each major formal result.

15. Design Principles

The following principles synthesize the design implications of all seven pillars, organized by implementation effort. Each principle is grounded in specific formal results and traces to one or more source pillars. Where multiple pillars motivate the same principle, all sources are listed.

Practitioner summary: ten principles to start with. The full framework yields 35 principles across three tiers. For practitioners seeking immediate impact, the following ten are the most actionable, requiring no unmeasured parameters and minimal infrastructure investment. Each references the formal result that grounds it.

1. **Enforce structurally, not instructionally.** Instructional compliance decays as $(1-\epsilon)^T$; structural enforcement achieves compliance 1. Anything that must hold invariantly (file permissions, tool restrictions, output filtering) should be enforced by mechanism, not by prompt. *(Proposition 3.4)*
2. **Invest in per-step quality; elasticity equals pipeline length.** Compound error sensitivity has elasticity n: a 1% improvement in per-step reliability yields an n% improvement end-to-end. Per-step quality is the highest-leverage investment in any multi-step pipeline. *(Observation 4.1)*
3. **Budget context at 15–40% of window, not 100%.** The optimal context size is strictly less than window capacity. Every token added must justify its inclusion against the attention dilution cost; filling the window is actively harmful. *(Theorem 3.1)*
4. **Fix the weakest link first.** The step with the lowest p_i offers the highest marginal return on improvement. Heterogeneous improvement (raising the floor) dominates uniform improvement (raising the ceiling). *(Observation 9.1)*
5. **Three verification rounds, then stop.** Empirical convergence at 96.5% by round 3 establishes a firm verification budget. Subsequent rounds face the pesticide paradox: the remaining errors require different detection approaches, not more of the same. *(Theorem 4.2)*
6. **Fresh context per task; do not accumulate.** Fresh context dominates for diverse task sequences. When sessions exceed 35 minutes or 50K tokens, accumulated degradation outweighs continuity benefits; checkpoint and restart with curated summaries. *(Proposition 3.3)*
7. **The verifier must be structurally read-only.** Prompt-level "do not modify the code" is insufficient. Tool permissions must enforce read-only access for the verification agent; the filesystem, not the system prompt, is the enforcement mechanism. *(Theorem 4.4)*
8. **Use Quality-Adjusted Speedup as the go/no-go metric for multi-agent.** QAS < 1 means parallelism is net-negative: quality degradation outweighs speed improvement. Monitor the defect ratio and halt scaling when it exceeds $1/S(k)$. *(Definition 5.5, Proposition 6.1)*
9. **Cross-model diversity for verification.** A weaker verifier from a different model family (lower

correlation) can achieve lower defect escape probability than a stronger verifier from the same family (higher correlation). Structural separation of producer and verifier model families is a design requirement, not an optimization variable. *(Corollary 11.1)*

10. **Cache first; it is the only cost lever that does not trade quality.** Increasing cache hit rates reduces effective token cost without affecting output quality. All other cost levers (reducing tokens, switching models, compressing prompts) degrade quality alongside cost. *(Proposition 9.4)*

The full 35 principles follow, organized into three tiers by implementation effort.

15.1. Tier 1: Immediately Actionable

These principles can be implemented within a single sprint by a team with an existing harness.

P1. Budget Context, Do Not Maximize It. *Source: Information. Grounding: Context Budget Theorem (Thm. 3.1).* The optimal context size is $0.15W$ to $0.40W$, strictly less than window capacity. Every token added must justify its inclusion against the attention dilution cost. Design context selection as an optimization problem, not a packing problem.

P2. Place High-Value Items at Context Boundaries. *Source: Information. Grounding: U-shaped positional recall (Prop. 3.3).* Position dominates relevance by approximately $1.18\times$. The rearrangement inequality dictates placing highest-relevance items at the beginning and end of the context window, lowest-relevance items in the middle.

P3. Classify Before Contextualizing. *Source: Information, Abstraction. Grounding: Bimodal gap observation (§3).* The abstraction gap is bimodal. Estimate task novelty before allocating context retrieval effort. Common patterns need project conventions only; novel logic needs aggressive context provision.

P4. Prefer Fresh Context to Managed Accumulation. *Source: Information, Temporal. Grounding: Fresh Context Dominance (Thm. 3.3).* Fresh context dominates for diverse task sequences. When sessions exceed 35 minutes or 50K tokens, accumulated degradation outweighs continuity benefits. Checkpoint and restart with curated summaries.

P5. Fix the Weakest Link First. *Source: Reliability. Grounding: Weakest-Link Priority (Cor. 4.1).* The step with the lowest p_i offers the highest marginal return on improvement. In practice, this is usually problem framing (step 1), not execution.

P6. Three Verification Rounds, Then Stop. *Source: Reliability. Grounding: Geometric Diminishing Returns (Thm. 4.2).* Empirical convergence at 96.5% by round 3 establishes a firm verification budget. Rounds 4 and beyond face the pesticide paradox: the remaining errors require different detection approaches, not more of the same.

P7. Verify at Phase Boundaries, Not at Every Step. *Source: Reliability. Grounding: Optimal Checkpoint Count (Thm. 4.1).* The Young-Daly analog gives the optimal spacing. For typical parameters, this means 2 to 5 verification points in a 20-step pipeline.

P8. The Verifier Must Be Structurally Read-Only. *Source: Reliability. Grounding: Structural Separation Theorem (§4).* Prompt-level "do not modify the code" is insufficient. Tool permissions must enforce read-only access for the verification agent; the filesystem, not the system prompt, is the enforcement mechanism.

P9. Separate Code Authorship from Test Authorship. *Source: Reliability, Quality. Grounding: Circular Validation Bias (Thm. 4.2), FKG inequality (Thm. 7.2).* Self-verification provides zero bias reduction on shared blind spots. Cross-family verification (different model producing tests than code) is where gains concentrate.

P10. Opacity to the Agent. *Source: Quality. Grounding: METR visibility amplification (43×).* Quality gates must be invisible to the producing agent. When the agent can observe which

checks will run, reward hacking amplifies by 43×. Run quality checks out-of-band, after generation, with no feedback into the generation prompt about which checks exist.

P11. Cheap Layers First. *Source: Quality. Grounding: ROI Ordering Theorem (Thm. 7.2).* Order verification by return on investment. Layer 1 (structural gates) at $0.50/PR before Layer 4 (human review) at $80/PR. Each dollar of verification budget should go to the highest-efficiency layer not yet saturated.

P12. Match Appraisal to Throughput. *Source: Quality, Governance. Grounding: Appraisal Compression Problem (§7).* When generation rate exceeds review capacity, only automated layers maintain coverage. Layer 4 (human review) cannot scale to AI generation rates; Layers 1 and 2 can. Design the quality stack so that removing humans from the loop degrades gracefully, not catastrophically.

P13. Make Logs Rich. *Source: Governance. Grounding: Double Bottleneck Theorem (Thm. 8.2).* Governance fidelity is bounded by log richness. Agent logs must externalize decisions, assumptions, and rejected alternatives, not just final code. Without rich logs, downstream governance loops operate on a lossy signal regardless of their sophistication.

15.2. Tier 2: Requires Investment

These principles require weeks of engineering work to implement properly (new tooling, infrastructure changes, or process redesign).

P14. Enforce Structurally What Must Hold Invariantly. *Source: Information, Reliability, Quality, Coordination. Grounding: Structural Enforcement Boundary (§4), Cost-of-Failure Threshold (Thm. 4.3).* Structural enforcement achieves compliance probability 1; instructional enforcement achieves $(1-\epsilon)^T$, which decays exponentially with pipeline length. Safety-critical constraints (data deletion, financial limits, security boundaries) must be structurally enforced via filesystem permissions, tool restrictions, or output filtering. Prompt enforcement is acceptable only for subjective quality dimensions (naming, style) bounded by structural gates.

P15. Invest in Per-Step Quality, Not Pipeline Sophistication. *Source: Reliability. Grounding: Compound Error Sensitivity (Obs. 4.1).* Compound error sensitivity has elasticity n: a 1% improvement in per-step quality yields an n% improvement end-to-end. This makes per-step quality the highest-leverage investment in any multi-step pipeline.

P16. Navigate Structure, Do Not Search Semantics. *Source: Information. Grounding: Reuse Discovery Problem (§3), submodularity ratio analysis.* For code, traverse the dependency graph rather than searching by embedding similarity. Vector RAG yields 94.7% irrelevance for code retrieval; structural navigation via AST and dependency graphs preserves the relationships that make code comprehensible. Build structural navigators, not semantic search indices.

P17. Decompose Tasks Along Low-Coupling Boundaries. *Source: Coordination. Grounding: Cut-Conflict Bridge (Lem. 5.2), balanced min-cut formulation.* Task decomposition should follow the coupling structure of the codebase. Run graph partitioning (METIS or hypergraph methods) on the coupling graph before dispatching agents. File-ownership contracts reduce conflict growth from $O(k^2)$ to $O(k)$.

P18. Check Before Scaling. *Source: Coordination. Grounding: Capability Saturation Crossover (Conj. 5.1).* If single-agent accuracy exceeds the saturation threshold ($P^* \approx 0.30$–0.35 for software engineering tasks, lower than the $P^* \approx 0.45$ reported by Kim et al. for general benchmarks due to tighter coupling and stricter correctness requirements in code), do not add agents; invest in improving the single agent instead. Scale agents only to $n^* \approx 1/\sqrt{\delta_a}$; beyond this, adding agents decreases effective throughput due to the reliability factor $(1-\delta_a)^n$.

P19. Measure Quality-Adjusted Speedup, Not Raw Throughput. *Source: Coordination, Tem-*

poral. Grounding: QAS definition (Prop. 5.5), VIH optimality (Prop. 6.1). QAS < 1 means parallelism is net-negative. VIH (Verified Iterations per Hour), not raw iterations per hour, predicts effective throughput. Monitor the defect ratio $d(k)/d(1)$ and halt scaling when it exceeds $1/S(k)$. *Caveat: VIH has not been measured in any production harness (see footnote to Definition 6.2); this principle is a theoretical prediction awaiting empirical calibration.*

P20. Use Contracts to Reduce Conflict Scaling. *Source: Coordination. Grounding: Assume-Guarantee Composition (Thm. 5.1).* File-ownership contracts reduce conflict growth from $O(k^2)$ to $O(k)$. The residual semantic conflicts are bounded by $(1-\gamma)\cdot\rho\cdot\binom{k}{2}$. Every multi-agent deployment should define explicit interface contracts before parallelizing.

P21. Maintain the Refactoring Ratio. *Source: Quality. Grounding: Refactoring Ratio Threshold (Cor. 7.1).* The current 2.5× violation of the equilibrium condition is the root cause of AI-driven codebase degradation. Any quality architecture must include mechanisms to sustain $r \geq 0.20$ (refactoring as a fraction of total changes). Without this, coupling complexity grows superlinearly regardless of per-change quality.

P22. Accept What You Cannot Detect. *Source: Quality. Grounding: Detection Ceiling (Thm. 7.4), Turing Enforcement Principle.* For Undecidable slop types where appraisal cost exceeds the discounted failure cost, acceptance is the economically rational strategy. Compensate through aggregate health monitoring (entropy proxies, clone frequency, accretion rate) rather than per-instance detection.

P23. Three Diverse Layers for 95% DRE. *Source: Quality. Grounding: DRE Cascade, Gaussian copula model (Prop. 7.5).* No single detection technique suffices for 95% defect removal effectiveness. Correlated pairs underperform independent ones. Deploy at least three diverse detection methods (structural, deterministic analysis, cross-model judgment) to reach the Capers Jones threshold.

P24. Design Governance for the Velocity of Change. *Source: Governance. Grounding: Governance Capacity Bound (Thm. 8.2), Governance Bifurcation (Prop. 8.3).* When AI agents increase change velocity, governance capacity must increase proportionally. Operating near the bifurcation point is fragile: a temporary spike in agent activity can push the system into coherence collapse with hysteresis. Maintain margin above the critical ratio $\mu G/\gamma_g R = 1$.

P25. Govern at Three Granularities. *Source: Governance. Grounding: Cascade Governance Stability (Thm. 8.3), Nyquist sampling requirements.* Synchronous (per-generation) checks prevent individual violations. Asynchronous (per-phase) audits catch emergent drift. Deliberative (per-milestone) reviews address strategic alignment. Adjacent loops should differ by at least 3:1 in sampling rate to avoid destructive interference.

P26. Track Decision Age, Not Just Decision Content. *Source: Governance. Grounding: Decision Survival Analysis (§8), Weibull decay model.* Decision validity decays with domain-specific half-lives. Governance should proactively surface decisions approaching their expected half-life rather than waiting for violations. Attach evidence-expiry dates to every architectural decision.

15.3. Tier 3: Aspirational

These principles require tooling, infrastructure, or organizational capabilities that do not yet exist in most environments, or depend on research results not yet validated.

P27. Use AI as a Formalism Translator. *Source: Abstraction. Grounding: Spec-Code Convergence (Thm. 2.1), Galois connection tower.* AI can serve as a translator: accepting natural language intent and producing formal specifications for human review. Reviewing a specification is vastly easier than authoring one from scratch. This shifts the human role from specification

author to specification reviewer. The vericoding pattern (human specification, AI implementation, formal verification) already achieves 82% on Dafny benchmarks.

P28. Treat the Abstraction Gap as a Budget. *Source: Abstraction, Information. Grounding: Abstraction Gap Decomposition (§3.1).* The information needed to bridge the abstraction gap must come from somewhere: specification, context, or model prior. When the gap exceeds the sum of these three sources, the task should be decomposed or the specification enriched, not attempted with insufficient information.

P29. Treat the 12.3% Threshold as a Gate for Persistent State. *Source: Temporal. Grounding: VIH break-even analysis (§6).* Any context reuse strategy must demonstrate quality degradation below 12.3% (the break-even for a 7-minute time saving on a 57-minute cycle) to be VIH-positive. Naive persistent state is VIH-negative; only compacted, curated state survives the threshold test.

P30. Speculate Conservatively. *Source: Temporal. Grounding: Speculation Ratio Test (Thm. 6.2), Speculation Depth Limit (Thm. 6.3).* Use the ratio test ($p/q > R/B$) before speculating. Limit speculation depth to 1 for pipelines with per-stage success probability below 85%. The Spectre analogy warns that hidden costs (corrupted state, cascading rollbacks) may not surface immediately.

P31. Detect Accretion, Not Just Defects. *Source: Quality. Grounding: Compound Degradation (Thm. 7.3), Accretion Rate definition.* AI-generated code introduces a novel failure mode: code that is correct, unnecessary, individually defensible, and collectively harmful. Track structural metrics (duplication rates, abstraction depth, dependency fan-out) over time, not just per-change correctness. Individual accretion is undetectable; aggregate accretion is measurable via entropy proxies.

P32. Measure Governance, Not Just Architecture. *Source: Governance. Grounding: Governance Capacity Bound (Thm. 8.2).* Monitor governance response time, escalation frequency, and the ratio $\mu G/\gamma_g R$. Architectural metrics alone cannot detect governance failure; they can only detect its consequences after the fact. Governance is itself a system that requires observability.

P33. Close the Ratchet with Evidence Expiry. *Source: Governance. Grounding: Ratchet Convergence (Cor. 8.2), Ossification Risk (Cor. 8.3).* Every governance rule should have an evidence validity window. The relaxation operator δ prevents ossification (the state where the set of allowed changes becomes empty under contradictory accumulated rules) while preserving still-valid constraints.

P34. Accept Residual Theory Loss. *Source: Governance, Abstraction. Grounding: Tacit Divergence Conjecture (Conj. 8.1), Theory Preservation Problem.* Perfect externalization of program theory is impossible. Design governance to tolerate a non-zero residual, using it as a risk measure. Supplement artifact-based governance with social mechanisms (code review conversations, architecture advice processes) for knowledge that cannot be externalized into any artifact.

Summary. The 35 principles above subsume and extend the 11 principles of the original synthesis. Tier 1 (P1–P13) can be implemented immediately using existing tooling. Tier 2 (P14–P26) requires engineering investment but uses known techniques. Tier 3 (P27–P35) depends on tooling, organizational capabilities, or validated research results that are not yet widely available. Teams should implement Tier 1 first, then invest in Tier 2 based on measured bottlenecks, and treat Tier 3 as directional guidance for roadmap planning.

16. Engaging Contrarian Positions

The framework's formal results invite strong objections from multiple intellectual traditions. Rather than constructing strawmen, we steelman seven contrarian positions, group them thematically, and

Position	Strongest Form	Framework Response
Scale makes governance obsolete	Scaling laws predict learned priors will outperform engineered methods	Abstraction gap is mechanism-agnostic; compound errors are mathematical identities
Formal methods are overkill	Situated action and language games challenge fixed specifications	σ is a design parameter; formal methods at high σ, NL at low σ
Governance is overhead	Cost exceeds benefit below a team-size threshold	Bifurcation theorem identifies exact threshold; AI velocity shifts it lower
ADRs are theater	No empirical evidence ADRs improve quality outcomes	Survival analysis with evidence expiry addresses staleness; empirical gap acknowledged
Humans should review everything	Human review is the gold standard	Throughput mismatch: 50%+ of code is AI-generated; humans triage, not review exhaustively
AI quality is good enough	GitHub RCT shows no quality difference	Longitudinal data contradicts; accretion effects invisible to short-term studies
Theory cannot be preserved	Tacit knowledge has divergent description length	Lossy compression beats no compression; uncertainty markers address false confidence

Table 13 | Summary of contrarian positions engaged, with their strongest forms and framework responses.

evaluate what the evidence actually supports. Each position draws on material originally developed across the seven pillars; here we consolidate by theme rather than by pillar. Table 13 provides a summary; the subsections below develop each position in detail.

16.1. The Bitter Lesson: Scale Makes Governance Obsolete

The steelmanned argument. Sutton's Bitter Lesson (Sutton, 2019) observes that general methods leveraging computation (search, learning) have historically defeated hand-engineered, knowledge-intensive approaches. LLMs trained on all publicly available code are the epitome of the scaling approach. If scaling continues to improve, formal methods, governance mechanisms, and structured verification may become unnecessary overhead, much as hand-tuned chess evaluation functions were rendered obsolete by AlphaZero. Dijkstra's programme of designing formal languages for formal thought is precisely the kind of "human-knowledge" approach that scaling predicts will lose.

Distributed cognition theory (Clark, 2003; Hutchins, 1995) strengthens this position: the human + LLM + IDE + test suite constitutes a distributed cognitive system that achieves precision even when no single component (including the human's natural-language specification) is precise alone. Formally, the abstraction gap $\mathcal{G}(S_{\text{NL}}, P)$ may be large, but $\mathcal{G}(S_{\text{NL}} \parallel \Phi_{\text{LLM}} \parallel \mathcal{T}_{\text{tests}}, P)$ can be small under distributed composition. This is a genuine strength of the natural-language approach that the framework can accommodate but does not predict.

What the evidence shows. The framework is agnostic on mechanism. If scaling obsoletes formal methods, the abstraction gap $\mathcal{G}(S, P) = K(P \mid S)$ still exists; what changes is the mechanism for traversing it (brute-force learned priors rather than structured refinement). The compound error sensitivity observation (Observation 4.1) is a mathematical identity, not a claim about any particular verification method: $R = \prod p_i$ holds whether the p_i are achieved by formal proofs, learned verifiers, or brute-force sampling. The Bitter Lesson predicts that *how* we achieve high p_i will shift; it does not predict that the multiplicative structure of pipeline reliability will change.

Moreover, current empirical evidence is mixed. HumanEval pass rates exceed 90%, but SWE-bench Verified plateaus around 77%, with METR finding that approximately 50% of test-passing PRs would not be merged by human reviewers (METR, 2026b). LiveCodeBench remains at 38.9%. The gap between isolated-function benchmarks and real-world integration tasks has not closed with scale alone. The 50-point gap between HumanEval ($> 90\%$, low-NCD tasks with short specifications) and LiveCodeBench (38.9%, high-NCD tasks requiring complex integration) is precisely what the abstraction gap framework predicts: as task complexity grows, the LLM's learned prior can no longer compensate for the information deficit in the specification.

The Goodhart taxonomy (Manheim and Garrabrant, 2019) provides additional grounds for skepticism about pure-scaling approaches. All four Goodhart mechanisms (regressional, extremal, causal, adversarial) manifest in agent benchmarks:

- *Regressional:* optimising a noisy proxy (test pass rates) selects for noise in the proxy-target relationship.
- *Extremal:* the proxy-target relationship, estimated in a typical regime, breaks down at extremes of task complexity.
- *Causal:* intervening on the proxy (writing code to pass tests) can break the causal pathway to genuine quality.
- *Adversarial:* agents can directly manipulate the proxy without affecting the target.

Every documented NIST CAISI cheating instance is detectable through structural invariant violation checks, not through statistical proxy-target monitoring, supporting the principle that structural enforcement addresses what scaling alone cannot. The "true quality" Q in any gaming detection framework is fundamentally uncomputable; all practical proxies inherit the Goodhart problem they aim to detect.

16.2. Formal Methods Are Overkill

The steelmanned argument. Suchman's situated action theory (Suchman, 1987) argues that plans are resources for action, not determinants of action. If this is correct, the refinement lattice (which models specification as a plan progressively concretised into code) may be the wrong metaphor for iterative, conversational development. The specification-code relationship is not a static refinement chain but a dynamic, context-dependent process where meaning emerges from interaction.

Wittgenstein's later philosophy (Wittgenstein, 1953) deepens this critique. If meaning is constituted by use rather than by formal structure, then a prompt's informational content is not a fixed quantity measurable by Kolmogorov complexity; it is a move in a language game whose significance depends on shared practices. This challenges a presupposition of the entire information-theoretic framework: that specifications have fixed informational content independent of context.

The democratisation argument adds practical weight: 5 billion natural-language speakers versus 27 million programmers (a 185:1 ratio). If natural-language programming enables even 10% of this

population to direct computation, the total value created may dwarf the quality loss on individual programmes (Altman, 2024; Litt, 2023). Prompt-based rules achieve 77% compliance (AgentSpec data), which may be "good enough" for most use cases.

What the evidence shows. These arguments are strongest when the specificity requirement is low ($\sigma < 0.3$) and weakest when formal guarantees are needed ($\sigma > 0.7$). The framework accommodates both by treating σ as a design parameter, not a universal constant. For prototypes and low-stakes tooling, Layer 1 structural gates alone may suffice; for production systems with safety constraints, the formal machinery becomes essential.

The 82% success rate of vericoding (VeriCoding, 2025) (human writes formal specification, AI generates implementation, formal verifier checks conformance) demonstrates that formal methods can be made accessible through AI translation: the human reviews a specification rather than authoring one from scratch. This addresses the historical cost barrier (Kleppmann, 2025) without abandoning formal guarantees. The key insight is that AI can serve as a *formalism translator*, shifting the human role from specification author to specification reviewer. Reviewing a formal specification is vastly easier than authoring one from scratch, which changes the cost calculus that historically made formal methods prohibitive.

We acknowledge, following Suchman and Wittgenstein, that the σ parameter itself may be an illegitimate formalisation of something that resists quantification. We flag this as a foundational question for future work on the epistemology of specification, rather than claiming to resolve it.

For AI coding agents that operate through conversation (iterative prompting, evaluation, and revision), Suchman's critique is particularly sharp: the "specification" is not a document but an evolving dialogue whose informational content is not fixed at any single moment. The framework applies most naturally when specifications are written documents with stable content, and less naturally to interactive, conversational development where meaning is negotiated in real time. This limitation is genuine and not resolved by the current framework.

16.3. Governance Is Overhead

The steelmanned argument. Governance has a cost curve that intersects its benefit curve at a specific scale, and below that scale the cost exceeds the benefit. For a two-person startup, time spent writing ADRs and configuring fitness functions is time not spent shipping features. The strongest version: governance mechanisms are a form of organisational scar tissue, growing in response to past failures but imposing ongoing friction on teams that have already internalised the lessons.

Conway's Law (Conway, 1968) strengthens this position: if architecture mirrors organisational structure, and organisational metrics predict failure-proneness with approximately 85% precision (Nagappan et al., 2008) (outperforming all code metrics), then code-level governance treats a symptom. Restructuring teams would be more effective than adding governance layers. The argument extends to agent teams: if agents mirror the organisational structures of the teams that build them, then governing agent code is futile without governing the organisational context that shapes agent behaviour.

What the evidence shows. Governance cost is roughly fixed per mechanism; benefit scales with team size and codebase complexity. The crossover appears at 5 to 10 developers, or more precisely, at the point where no single person holds the complete mental model. For distributed teams, formal governance becomes valuable earlier because informal channels (hallway conversations, whiteboard

sessions) are degraded or absent.

Conway's Law admits an inverse: the inverse Conway maneuver (Skelton and Pais, 2019) deliberately restructures teams to produce desired architecture. Code-level governance and organisational governance are complementary, not substitutes. The novel question for AI agents is whether agent governance requires organisational awareness, and the answer appears to be yes: agent topology should mirror the desired system decomposition (the inverse Conway maneuver applied to agents).

The governance capacity bound (Theorem 8.2) provides the formal resolution: governance is overhead precisely when the drift rate is below the bifurcation threshold. Below the threshold, the system self-corrects without governance. Above it, governance is not overhead but a structural necessity; its absence leads to coherence collapse. The question is empirical: where does a given team sit relative to the threshold?

The AI velocity corollary sharpens this analysis: when agents generate code at 10 to 100× the rate of human developers, the drift rate increases proportionally while governance capacity remains constant (it is bounded by human review bandwidth). This pushes teams above the bifurcation threshold earlier and more frequently. The automated governance necessity follows: governance must itself be partially automated to scale with agent throughput, which is precisely what the multi-granularity cascade control architecture provides.

16.4. ADRs and Documentation Are Theater

The steelmanned argument. ADRs can drift from purpose, enable responsibility diffusion, and suffer the staleness problem. Despite recommendations from ThoughtWorks, AWS, Microsoft, and Google, no published empirical study demonstrates that ADRs improve measurable software quality outcomes (defect rates, time-to-market, maintenance cost). The evidence is entirely practitioner reports and observational accounts.

The false confidence problem is acute: documentation with the appearance of completeness but no mechanism for detecting its own staleness creates a worse situation than acknowledged ignorance. Teams may defer to stale ADRs rather than exercising judgment, producing decisions that are formally "compliant" but substantively wrong.

What the evidence shows. ADRs provide clear value for onboarding and cross-team alignment. The primary failure mode is absent content (decisions made without ADRs) or bloated content (too many decisions documented), not wrong content. The survival analysis (Section 8) provides a concrete remedy: tracking decision age alongside decision content, with domain-specific half-lives triggering proactive review.

Evidence expiry and the controlled relaxation operator δ address the staleness critique directly. Documentation with explicit uncertainty markers ("confidence: medium," "last validated: 2025-03") addresses the false confidence risk. The contribution is making these mechanisms formal rather than ad hoc.

The empirical gap is real and should be acknowledged honestly. The strongest evidence is that the *problem* is real (AI accelerates architectural drift and quality degradation). The evidence that documentation and governance *solve* it remains largely anecdotal. Designing governance systems to be testable, by tracking drift trajectories and decision decay, is a prerequisite for future empirical validation.

The "ratchet is too rigid" objection (Ford et al., 2017) applies here as well: monotonically increas-

ing documentation strictness prevents legitimate architectural evolution. The strongest governance systems already incorporate relaxation mechanisms. Fitness functions can be deprecated. Evidence expiry provides a natural clock for rule relaxation. The ratchet critique applies most forcefully to systems that treat rules as permanent rather than contextual, which is precisely what the lattice descent operator δ addresses through controlled relaxation with evidence expiry.

16.5. The Human Should Just Review Everything

The steelmanned argument. Human review is the gold standard for code quality. Rather than building elaborate multi-layer verification architectures, organisations should invest in thorough human review of all agent-generated code. Human reviewers can detect subtle architectural violations, naming convention drift, unnecessary abstractions, and strategic misalignment that no automated system can match. The four-layer defense architecture adds complexity and cost without matching the capabilities of a skilled human reviewer.

Mode selection is hard to automate (Bainbridge, 1983): deciding when to apply lightweight versus heavyweight governance requires the kind of contextual judgment that resists formalisation. Bainbridge's ironies of automation create a genuine double bind: the more we automate governance decisions, the less capable humans become at making them when automation fails.

What the evidence shows. The throughput mismatch is the decisive counterargument. AI agents now generate over 50% of committed code on major platforms. GitClear documents 211 million changed lines with an 8× increase in code duplication and a 60% collapse in refactoring activity (GitClear, 2025). Human review cannot scale to match this generation rate. At $80 per PR for thorough human review (Layer 4 cost), the economics are prohibitive for high-throughput agent pipelines.

The detection ceiling theorem (Theorem 7.4) shows that each layer has hard upper bounds on what it can detect, determined by available context. Human review has the highest ceiling but the lowest throughput. Layers 1 through 3 have lower ceilings but orders-of-magnitude higher throughput. The optimal architecture uses cheap, fast layers to filter the stream before expensive human attention is applied.

The Bayesian mode selection framework (Theorem 6.5) provides a principled threshold for escalation: the asymmetric cost structure ($c_{\mathrm{FG}} \gg c_{\mathrm{GF}}$) means choosing the governed (human-reviewed) mode at even 5 to 10% posterior probability of needing it is Bayes-optimal. This is not a replacement for human judgment but a triage mechanism that directs human attention where it has the highest marginal value.

The Dodge-Romig sampling plans (Dodge and Romig, 1929) and MIL-STD-105E switching rules provide the classical precedent: adaptive inspection regimes that tighten or relax based on observed quality. Our four-layer architecture adapts this paradigm to software verification, where the "inspector" at each layer has different capabilities and costs. The human reviewer remains indispensable for the undecidable slop types (meaningless tests, architectural erosion, boilerplate inflation) that resist automated detection, but the human's scarce attention is directed by the preceding automated layers rather than applied uniformly.

16.6. AI Code Quality Is Good Enough

The steelmanned argument. The quality concerns are overstated. GitHub's controlled RCT (GitHub, 2024) found that AI-assisted developers completed tasks 55% faster with no measurable quality difference. For the vast majority of business software (CRUD applications, internal tools, prototypes),

"good enough" quality is the economically rational target. The optimal quality level is explicitly $q^* < 1$ for low-stakes code; investing in elaborate quality architectures for code that will be rewritten in six months is waste.

Speed itself has value. The Yerkes-Dodson objection ("speed kills quality" (Yerkes and Dodson, 1908)) applies to human cognition under time pressure, but AI agents do not experience cognitive fatigue. The DORA finding that high-performing teams achieve both speed and stability (Forsgren et al., 2018) suggests that the speed-quality tradeoff is not fundamental.

LLM-as-Judge is unreliable: inter-rater agreement (omega of 0.462 to 0.803) is concerning (Various Authors, 2024). Same-model judging amplifies correlated errors via the FKG inequality (Theorem 7.2). If the verification layer itself is unreliable, the entire quality architecture rests on shaky foundations.

What the evidence shows. The GitHub RCT measured short-term task completion, not long-term maintainability. GitClear's longitudinal data tells the opposite story: 17% maintainability drops, refactoring declining from 25% to less than 10% of changed lines, and bug rates rising 12% at six months (GitClear, 2025). The compound degradation theorem (Theorem 7.3) formalises why: individually CI-passing changes can produce superlinear entropy growth through accretion effects that no single-change review can detect.

The "slop is acceptable" position is partially validated by our own framework. The optimal quality level (Theorem 7.6) is explicitly $q^* < 1$ for low-stakes code. The contribution is making this precise: for prototypes, Layer 1 only; for production systems, all four layers. The framework does not argue for maximal quality everywhere; it argues for type-specific, cost-aware quality investment.

On LLM-as-Judge reliability: same-model judging is indeed problematic (Theorem 7.2). But cross-model judging with structural scaffolding (Layer 3 constrained by Layers 1 and 2) remains cost-effective for semi-decidable types. The diversity gain (Definition 7.2) shows that a weak independent detector outperforms a strong correlated one, providing a concrete architectural remedy.

VIH (Verified Iterations per Hour) captures the speed-quality tradeoff more precisely than raw throughput: VIH = $\lambda_{\text{raw}} \cdot p_{\text{verify}}$. The strongest form of the "speed kills quality" objection is that verification itself becomes less reliable under time pressure, creating a feedback loop that systematically overestimates VIH at high speeds. This remains an open empirical question that the framework does not resolve.

VIH is also not a complete objective function. A verified iteration that introduces subtle architectural debt, passes tests but degrades maintainability, or solves the wrong subproblem is "verified" but not valuable. Reinertsen's cost-of-delay framework provides a more comprehensive alternative that accounts for the economic value of delivery timing. VIH is a useful proxy for the speed-quality tradeoff specifically, but it should not be mistaken for a measure of total value delivered. The "18 types are not the right taxonomy" objection from the quality pillar applies here as well: the three-factor latent structure (context blindness, completion bias, training leakage) may be more fundamental than the 18-type enumeration; confirmatory factor analysis on appropriate datasets would test this.

The "caching is a false economy" objection deserves particular attention. Cache invalidation bugs are intermittent, state-dependent, and compound through cascading cached values. For rapidly changing codebases under active development, cache hit rates may be too low to justify the infrastructure overhead. Our EOQ theorem provides a principled refresh schedule, but assumes knowledge of the change rate λ, which itself must be estimated. The "fresh eyes" advantage (17.1% quality improvement from fresh context) is large enough that naive persistent state is VIH-negative; incre-

mental context is only justified when compaction reduces quality loss below 12.3%.

16.7. Naur Is Right and Theory Cannot Be Preserved

The steelmanned argument. Naur (Naur, 1985) argued that programming is theory building, where "theory" (in Ryle's sense) is the knowledge a person must have "in order not only to do certain things intelligently but also to explain them, to answer queries about them, to argue about them." Three capabilities resist documentation: mapping between code and reality, design justification ("the justification is and must always remain the programmer's direct, intuitive knowledge or estimate"), and modification intelligence, which depends on recognising similarities "that are not, and cannot be, expressed in terms of criteria."

The tacit divergence conjecture (Conjecture 8.1) formalises this: for Kolmogorov-random components of tacit knowledge, the description length diverges, meaning that no finite documentation can capture them. If this is correct, then THEORY.md, ADRs, and all documentation artifacts are lossy compression creating false confidence. Acting on extracted "decisions" may be worse than no documentation at all, because it creates the illusion of understanding where none exists.

Tsoukas's critique of Nonaka and Takeuchi deepens the problem: tacit and explicit knowledge are not convertible pools but two dimensions of all knowing. The "conversion" metaphor (tacit to explicit and back) is incoherent; what actually happens is articulation, which necessarily transforms and reduces the original knowledge.

What the evidence shows. Naur wrote in 1985 about small teams with long tenure. Modern software involves high turnover across time zones and continents. The alternative to lossy documentation is not "direct contact with theory-holders" but *no knowledge transfer at all*. Lossy compression is still enormously useful: JPEG images demonstrate this. The question is not whether documentation is perfect but whether it is better than nothing, and whether its limitations are understood.

The double bottleneck theorem (Theorem 8.2) provides the formal resolution: governance fidelity is bounded by log richness, and agent logs that externalise decisions, assumptions, and rejected alternatives capture more theory than code alone, even if the capture is necessarily lossy. Documentation with explicit uncertainty markers ("confidence: medium," "last validated: 2025-03") addresses the false confidence risk by making the lossiness visible rather than hiding it.

The survival analysis (§8) adds a temporal dimension: decision validity decays with domain-specific half-lives (UI/frontend decisions, approximately 1.2 years; API contracts, approximately 2.5 years; data model decisions, approximately 4.3 years). Governance that tracks decision age and proactively surfaces decisions approaching their expected half-life provides a practical response to Naur's concern, acknowledging that documentation decays while building mechanisms to detect and respond to that decay.

We accept the residual: perfect externalisation is impossible. The framework uses this impossibility as a design constraint (Design Principle 11: accept residual theory loss), supplementing artifact-based governance with social mechanisms (code review conversations, architecture advice processes) for knowledge that resists externalisation.

The Dreyfus model of skill acquisition provides additional nuance: expert practitioners operate on intuitions that resist rule-based decomposition. Collins's taxonomy of tacit knowledge (relational, somatic, collective) identifies which components are in principle externalisable (relational) and which are not (somatic, collective). The governance architecture should invest extraction effort proportional to externalisability, rather than attempting uniform documentation. This selective approach

respects Naur's insight while remaining practically useful.

16.8. Cross-Cutting Assessment

Several patterns emerge across these seven positions. First, most contrarian arguments are *partially* correct rather than wholly right or wrong. The framework's response is typically to identify the conditions under which the objection holds ($\sigma < 0.3$ for the formalism critique, below 5 to 10 developers for the governance-as-overhead critique, $q^* < 1$ for the quality-is-good-enough critique) and to incorporate those conditions as design parameters rather than dismissing the objection.

Second, the strongest objections challenge the framework's *foundations* (Suchman on situated action, Wittgenstein on language games, Naur on theory preservation) rather than its *conclusions*. We acknowledge these as genuine limitations that define the framework's scope of applicability rather than falsify its results within that scope.

Third, the empirical evidence is strongest for the *problems* the framework addresses (compound errors in pipelines, quality degradation from AI code generation, governance scaling challenges) and weakest for the *solutions* it proposes (ADRs improving quality outcomes, formal methods reducing maintenance costs, governance preventing drift). This asymmetry is honest but should constrain confidence in the prescriptive claims.

17. Limitations and Open Problems

17.1. Threats to Validity

We assess threats following the standard construct, internal, external, and conclusion validity framework.

Construct validity. Several core quantities are either uncomputable or lack operational definitions with validated measurement procedures. The abstraction gap $\mathcal{G}(S, P) = K(P \mid S)$ is uncomputable; the Shannon operationalization $H(P \mid S)$ is computable but model-dependent (a property of the model-specification pair, not of the specification alone). The coupling complexity $H(C)$ (Definition 7.4) is a mean log-coupling-degree, not a Shannon entropy in the strict sense; calling it "entropy" borrows connotations of information-theoretic irreversibility without the formal properties. The governance capacity C_{gov} lacks an operational definition with demonstrated inter-rater reliability. VIH (Verified Iterations per Hour) has never been measured in any production harness.

Internal validity. The observational data sources that ground the framework (GitClear, DORA 2025) have uncontrolled confounders. The GitClear 8× duplication increase is correlational; alternative explanations include the concurrent shift toward microservices and template-based scaffolding, selection effects in AI tool adoption, and changing developer demographics. The DORA finding of negative correlation between AI usage and stability may reflect reverse causation (struggling teams adopting AI as a remedy). The METR merge-gap study used 4 evaluators across 3 repositories, providing moderate confidence at best. No causal identification strategy (difference-in-differences, regression discontinuity, instrumental variables) has been applied to any of these data sources.

External validity. All calibration data comes from specific model families (Claude, GPT, Gemini), specific benchmarks (SWE-bench, HumanEval, RE-Bench), specific tools (Cursor, Codex, GSD), and

predominantly English-language codebases. Generalization to different programming languages, application domains, team sizes, and organizational contexts is unestablished. The degradation function $\delta(n) = (n/W_{\text{eff}})^{\alpha_d}$ is fit to three data points from a single model (Llama-3.1-70B); cross-model validation is needed before the power-law form can be considered robust.

Conclusion validity. Many quantitative predictions (optimal context budget at $0.15W$–$0.40W$, optimal agent count at $1/\sqrt{\delta_a}$, governance bifurcation at $\mu G/\gamma_g R = 1$) depend on parameters that have not been measured in production settings. The sensitivity of conclusions to parameter uncertainty is not systematically analyzed. Where the paper states numerical ranges (e.g., the context budget), these should be understood as model predictions contingent on the assumed functional forms, not as empirically validated recommendations.

17.2. Empirical Validation Gaps

The most significant limitation of this framework is the gap between formal models and empirical validation. The following experiments are needed to close this gap:

1. **Context budget validation**: controlled experiments measuring task success as a function of context size, across multiple models and task types, to validate the optimal budget estimates ($|C^*| \approx 0.15W$ to $0.40W$).
2. **Structural enforcement measurement**: controlled comparison of instructional vs. structural enforcement, measuring compliance rates ϵ across different enforcement mechanisms and pipeline lengths T.
3. **VIH measurement**: instrumentation of production harnesses to measure raw iteration rate, verification pass rate, and their product across different verification configurations.
4. **Governance capacity calibration**: measurement of R_{drift} and C_{gov} in production settings, to test whether the capacity bound predicts the onset of architectural degradation.
5. **Accretion detection**: development and validation of detectors for the Accretion Category, measuring precision and recall against expert-labeled datasets.

17.3. Empirical Verification Roadmap

This subsection provides a concrete verification roadmap for the five most important formal results in the framework. For each result, we specify the data to collect, the testable prediction, and the falsification criterion. This roadmap was developed in direct response to peer review feedback identifying the empirical gap as the framework's most significant weakness.

17.3.1. Result 1: Compound Error Sensitivity (Observation 4.1)

Formal claim. For an n-step agent pipeline with homogeneous per-step success probability p, the end-to-end reliability is $R(n) = p^n$, and the elasticity of R with respect to p equals the pipeline length: $\mathcal{E}(R, p) = n$.

Data to collect.

1. Instrument a production agent harness (e.g., Claude Code, Cursor, Codex) to log, for each pipeline execution: (a) the number of discrete steps n, (b) a binary success/failure indicator for each step (judged by automated tests plus human spot-checks on a stratified subsample), and (c) the end-to-end success indicator.

2. Collect at least 500 pipeline executions per pipeline-length bin, with bins at $n \in \{5, 10, 15, 20, 30, 50\}$.
3. For each bin, estimate $\hat{p}$ (the empirical per-step success rate) and $\hat{R}(n)$ (the empirical end-to-end success rate).
4. Additionally, collect data on step-to-step error correlation by recording whether failure at step i predicts failure at step $i+1$ (beyond the base rate). This tests the independence assumption.

Testable prediction.

- Plot $\ln \hat{R}(n)$ against n. Under the model, this should be approximately linear with slope $\ln \hat{p}$.
- For each pipeline-length bin, compute the empirical elasticity $\hat{\mathcal{E}} = \Delta \ln \hat{R} / \Delta \ln \hat{p}$ (using within-bin variation in p due to model differences or task difficulty). The prediction is $\hat{\mathcal{E}} \approx n$.
- Specifically: for $n = 20$ and $\hat{p} = 0.95$, the predicted reliability is $0.95^{20} = 0.36$, with 95% CI bounds derivable from the binomial model.

Falsification criterion. The result is falsified if *any* of the following hold:

1. The relationship between $\ln \hat{R}(n)$ and n is not approximately linear (e.g., $R^2 < 0.7$ for the linear fit), indicating that the series reliability model is structurally wrong (not merely noisy).
2. The empirical elasticity $\hat{\mathcal{E}}$ differs from n by more than a factor of 2 across at least 3 pipeline-length bins, indicating that the elasticity claim is quantitatively misleading.
3. Step-to-step error correlations are so strong (correlation coefficient > 0.5) that the independence model provides a poor approximation, in which case the common-cause model from the Reliability pillar must be used as the baseline.

Expected timeline and resources. Requires partnership with a harness vendor or access to open-source harness telemetry. Estimated: 3–6 months for instrumentation and data collection; 1–2 months for analysis. Minimum viable dataset: 3,000 pipeline executions across 3+ pipeline-length bins.

17.3.2. Result 2: Optimal Context Budget (Theorem 3.1)

Formal claim. Under the power-law degradation model $\delta(n) = (n/W_{\text{eff}})^{\alpha_d}$, the optimal context size satisfies $|C^*| < W$, with practical estimates $|C^*| \approx 0.15W$ to $0.40W$.

Data to collect.

1. Select a benchmark of 200+ coding tasks spanning five difficulty levels and three code domains (web, systems, data). Tasks must have known-correct solutions for automated evaluation.
2. For each task, prepare context sets at 10 sizes: 5%, 10%, 15%, 20%, 30%, 40%, 50%, 60%, 80%, 100% of the model's effective context window W_{eff}.
3. Context content at each size level should be selected by the same retrieval algorithm (e.g., BM25 + reranking), adding lower-relevance items as size increases, to ensure that the size variation reflects the natural relevance gradient.
4. For each (task, context-size) pair, run the agent 5 times (to estimate variance) and record: (a) task success (binary), (b) code quality score (0–10, via automated rubric), (c) token usage.
5. Repeat across at least 3 frontier models from different providers.

Testable prediction.

- Plot mean task success against context size (as fraction of W_{eff}). Under the model, this curve should be unimodal: increasing for small context (more relevant information), peaking at $|C^*|/W_{\text{eff}} \in [0.15, 0.40]$, and declining for large context (degradation dominates).
- The peak location should be robust across models (within 15 percentage points), though the peak height will vary.
- For tasks where the model's prior is strong (boilerplate, common patterns), the peak should shift left (less context needed). For tasks requiring novel logic, the peak should shift right.

Falsification criterion. The result is falsified if *any* of the following hold:

1. Task success is monotonically non-decreasing in context size for a majority ($> 60\%$) of tasks and models, indicating that degradation is not a binding constraint in practice.
2. The peak, when it exists, occurs above $0.60W_{\text{eff}}$ for the majority of tasks, indicating that the $[0.15, 0.40]$ range is too conservative.
3. The peak location varies by more than 40 percentage points across models (e.g., 0.10 for one model and 0.55 for another), indicating that the "optimal budget" is model-specific rather than a general property of context selection.

Expected timeline and resources. Can be conducted with API access to frontier models. Estimated: 2–4 months for benchmark construction and execution; 1 month for analysis. Cost: approximately $5,000–$15,000 in API fees depending on model pricing and repetition count.

17.3.3. Result 3: Structural Enforcement Threshold (Theorem 4.3)

Formal claim. Instructional enforcement (prompts, rules files) achieves compliance $(1-\epsilon)^T$, decaying exponentially in pipeline length T. Structural enforcement achieves compliance 1. The threshold for cost-effectiveness is $L > L^* = (c_s - c_p)/\epsilon$, which drops as L^*/n for multi-step pipelines.

Data to collect.

1. Define 10 invariants spanning the decidability spectrum: 4 decidable (e.g., "never import package X," "all functions have type annotations," "no files exceed 500 lines," "API keys never appear in source"); 3 semi-decidable (e.g., "all error paths return appropriate status codes," "no race conditions in shared state access," "all user inputs are sanitized"); 3 undecidable (e.g., "no unnecessary abstraction layers," "naming conventions reflect domain semantics," "code follows the existing architectural style").
2. For each invariant, implement both an instructional enforcement mechanism (system prompt instruction, CLAUDE.md rule, or rules file entry) and, where possible, a structural enforcement mechanism (linter rule, pre-commit hook, file-system permission, CI gate).
3. Run a standardized coding agent through 50-step pipelines on a realistic codebase, with $T \in \{10, 20, 30, 50, 75, 100\}$ steps. At each step, record whether the invariant holds. Use 3 different frontier models.
4. For each (invariant, enforcement-type, model, T) combination, compute the compliance rate. Minimum 30 trials per combination to achieve adequate statistical power.

Testable prediction.

- For structural enforcement on decidable invariants, compliance should be ≥ 0.99 regardless of T and model.

- For instructional enforcement, compliance should decay exponentially. Specifically, plotting $\ln(\text{compliance})$ against T should yield an approximately linear relationship with slope $\approx \ln(1-\hat{\epsilon})$, from which $\hat{\epsilon}$ can be estimated.
- The estimated $\hat{\epsilon}$ should fall in the range $[0.005, 0.10]$ for decidable invariants and $[0.05, 0.30]$ for semi-decidable invariants.
- The compliance gap between structural and instructional enforcement should be statistically significant ($p < 0.01$) for $T \geq 20$.

Falsification criterion. The result is falsified if *any* of the following hold:

1. Instructional compliance does not decay with T (i.e., compliance is approximately constant across pipeline lengths), indicating that the $(1-\epsilon)^T$ model is wrong and agents may have mechanisms (e.g., in-context learning, self-correction) that counteract exponential decay.
2. Structural enforcement on decidable invariants fails at rates $> 5\%$ (due to agent workarounds, tool limitations, or enforcement gaps), indicating that the "compliance = 1" claim is practically false.
3. The compliance gap between structural and instructional enforcement is not statistically significant for $T \leq 50$, indicating that the theoretical advantage of structural enforcement is too small to matter in practice.

Additional test: T-dependent ϵ. A critical assumption of the $(1-\epsilon)^T$ model is that ϵ is constant across pipeline steps. If agents exhibit in-context learning (improved rule-following as the pipeline progresses) or rule fatigue (degraded compliance as context accumulates), ϵ would depend on T. The experiment should explicitly test this by fitting a model with $\epsilon(t) = \epsilon_0 + \epsilon_1 t$ and comparing AIC/BIC against the constant-ϵ model. If the T-dependent model fits significantly better, the exponential decay prediction must be revised.

Expected timeline and resources. Requires moderate engineering to implement enforcement mechanisms and instrumentation. Estimated: 4–6 months including implementation and data collection. Minimum viable dataset: 10 invariants × 2 enforcement types × 3 models × 6 pipeline lengths × 30 trials = 10,800 pipeline executions.

17.3.4. Result 4: Optimal Agent Count (Theorem 5.1)

Formal claim. For n agents with per-agent defect injection rate δ_a and serial fraction s, effective speedup is $S_{\text{eff}}(n) = [1/(s + (1-s)/n)] \cdot (1-\delta_a)^n$, with optimal agent count $n^* \approx 1/\sqrt{\delta_a}$.

Data to collect.

1. Select 30+ software engineering tasks of moderate complexity (2–8 hours of human developer time), covering feature implementation, bug fixing, and refactoring. Tasks must be decomposable into parallel subtasks of varying granularity.
2. For each task, execute with $k \in \{1, 2, 3, 4, 5, 7, 10, 15, 20\}$ agents. Use at least two coordination topologies: (a) independent (no inter-agent communication) and (b) centrally coordinated (orchestrator assigns subtasks, reviews outputs, resolves conflicts).
3. For each (task, k, topology) configuration, measure: (a) wall-clock time T_k, (b) code quality score Q_k (via automated tests + human review on a subsample), (c) merge conflict count and resolution time, (d) rework iterations (code that was written, then reverted or rewritten).

4. Compute Quality-Adjusted Speedup: $\text{QAS}(k) = (T_1 \cdot Q_1)/(T_k \cdot Q_k)$.
5. Estimate $\hat{\delta}_a$ from the per-agent defect rate and $\hat{s}$ from the serial fraction of each task (via dependency analysis of the task decomposition).

Testable prediction.

- QAS should be unimodal in k: increasing for $k \leq n^*$, then decreasing.
- The peak should occur at $\hat{k}^* \approx 1/\sqrt{\hat{\delta}_a}$, within a factor of 2.
- For independent coordination with typical $\delta_a \approx 0.03$–0.07, the peak should be at $k^* \in [3, 6]$.
- At $k = 2n^*$, QAS should be measurably below its peak value (by at least 20%), confirming that over-parallelization is costly.
- For the centrally coordinated topology, δ_a should be lower (better error correction), yielding a higher n^*.

Falsification criterion. The result is falsified if *any* of the following hold:

1. QAS is monotonically non-decreasing in k up to $k = 20$ for a majority of tasks, indicating that the reliability penalty $(1 - \delta_a)^n$ does not dominate within the tested range and the optimal agent count is much higher than predicted.
2. The observed peak $\hat{k}^*$ differs from $1/\sqrt{\hat{\delta}_a}$ by more than a factor of 3 across a majority of tasks, indicating that the Extended Amdahl model is a poor predictor.
3. QAS < 1 for $k = 2$ (i.e., even minimal parallelism is net-negative) for a majority of tasks and topologies, indicating a qualitative failure of the parallelism model that suggests coordination overhead dominates even before reliability penalties matter.

Expected timeline and resources. The most resource-intensive experiment in this roadmap, requiring either a multi-agent harness platform or substantial API costs. Estimated: 6–12 months. Cost: $20,000–$50,000 in compute. Minimum viable dataset: 30 tasks × 9 agent counts × 2 topologies = 540 configurations, each with quality evaluation.

17.3.5. Result 5: Governance Capacity Bound (Theorem 8.2)

Formal claim. If the architectural drift rate R_{drift} (bits per unit time) exceeds the governance channel capacity C_{gov} (bits per unit time), no governance scheme prevents unbounded divergence from the intended architecture.

Data to collect.

1. Select 20+ open-source repositories that (a) have Architecture Decision Records or equivalent governance artifacts, (b) have measurable AI adoption (via commit metadata, co-author tags, or tool integration logs), and (c) span at least 12 months of history pre- and post-AI adoption.
2. For each repository, construct proxy measures for the two key quantities:
   - **Drift rate proxy** ($\hat{R}_{\text{drift}}$): rate of architectural violations per month, measured as: (a) ADR compliance violations (commits that contradict accepted ADRs, detected by automated pattern matching), (b) layer boundary violations (imports across declared layer boundaries), (c) dependency direction violations (detected by dependency analysis tools), (d) style/convention violations in new code. Each violation category receives a weight propor-

tional to its severity. The composite is normalized to bits per month using $\hat{R}_{\text{drift}} = \sum_i w_i \cdot r_i$, where r_i is the violation rate for category i.

- **Governance capacity proxy** ($\hat{C}_{\text{gov}}$): governance throughput per month, measured as: (a) code review throughput (PRs reviewed per month × mean review depth), (b) ADR update rate, (c) architectural review meeting frequency × mean decision output, (d) automated governance gate coverage (fraction of invariants enforced structurally). The composite is $\hat{C}_{\text{gov}} = \sum_j v_j \cdot g_j$, where g_j is the governance activity rate for category j.

3. For each repository, measure **architectural health** over time using: (a) coupling density (mean module coupling, measured monthly), (b) abstraction stability (rate of interface changes per month), (c) technical debt proxies (TODO density, code duplication rate, cyclomatic complexity growth rate), (d) developer satisfaction with architecture (from surveys, if available).
4. Compute the ratio $\hat{R}_{\text{drift}}/\hat{C}_{\text{gov}}$ monthly for each repository and plot against architectural health metrics.

Testable prediction.

- Repositories where $\hat{R}_{\text{drift}}/\hat{C}_{\text{gov}} > 1$ (drift exceeds governance) for sustained periods (≥ 3 consecutive months) should show measurable architectural degradation: increasing coupling density, rising duplication, growing technical debt proxies.
- Repositories where $\hat{R}_{\text{drift}}/\hat{C}_{\text{gov}} < 1$ consistently should show stable or improving architectural health.
- The transition should be relatively sharp: repositories near the threshold ($0.8 < \hat{R}_{\text{drift}}/\hat{C}_{\text{gov}} < 1.2$) should show mixed signals, while those clearly above or below should show consistent degradation or stability.
- Repositories that increased AI adoption (higher commit velocity) without increasing governance capacity should show a transition from sub-threshold to super-threshold, with architectural degradation following the transition with a lag of 2–6 months.

Falsification criterion. The result is falsified if *any* of the following hold:

1. No measurable relationship exists between the $\hat{R}_{\text{drift}}/\hat{C}_{\text{gov}}$ ratio and architectural health metrics (correlation < 0.2 across repositories), indicating that the governance capacity framing has no predictive power.
2. Repositories sustain $\hat{R}_{\text{drift}}/\hat{C}_{\text{gov}} > 1.5$ for 6+ months without measurable architectural degradation, indicating that the "unbounded divergence" prediction is too pessimistic and self-correcting mechanisms (developer workarounds, organic refactoring, competitive pressure) prevent divergence even without formal governance capacity.
3. The proxy measures $\hat{R}_{\text{drift}}$ and $\hat{C}_{\text{gov}}$ cannot be operationalized with inter-rater reliability > 0.6 (Cohen's κ), indicating that the theoretical quantities are too vague to measure, which would itself be a significant finding about the framework's practical applicability.

Methodological limitation. This experiment is the methodologically weakest in the roadmap. The proxy measures $\hat{R}_{\text{drift}}$ and $\hat{C}_{\text{gov}}$ are composite indices with tunable weights w_i and v_j. The weights can be adjusted post hoc to fit the data, which undermines the experiment's falsifiability. A stronger design would use a single, pre-registered proxy for each quantity (e.g., layer violation rate for $\hat{R}_{\text{drift}}$ and code review throughput for $\hat{C}_{\text{gov}}$), sacrificing coverage for testability. Additionally, the governance capacity bound is a threshold result (above the threshold, divergence is unbounded), but the experiment tests for a correlation, which is a weaker claim. A proper threshold test would identify

repositories that crossed the threshold and verify that architectural health degraded sharply (not gradually) afterward.

Expected timeline and resources. The most methodologically challenging experiment, requiring careful proxy construction and validation. Estimated: 12–18 months for the longitudinal component; initial cross-sectional results possible in 6 months. Requires access to repository history and governance artifacts for 20+ projects. No significant compute cost, but substantial research analyst time.

Summary of the verification roadmap. Table 14 summarizes the five experiments.

Table 14 | Verification roadmap summary. Each row identifies the formal result, the core prediction, the falsification standard, and the estimated resource requirement.

Result	Core Prediction	Falsification Standard	Est. Resources
Compound error sensitivity	$\ln R$ linear in n; elasticity = n	$R^2 < 0.7$ or elasticity off by $> 2\times$	3–6 months; vendor partnership
Optimal context budget	Unimodal success in context size; peak at $[0.15, 0.40]W$	Monotonic success or peak $> 0.60W$ for majority	2–4 months; \$5–15K API cost
Structural enforcement threshold	Instructional compliance decays as $(1-\epsilon)^T$; structural = 1	No decay with T, or structural fails $> 5\%$	4–6 months; moderate engineering
Optimal agent count	QAS unimodal; peak at $\approx 1/\sqrt{\delta_a}$	Monotonic QAS or peak off by $> 3\times$	6–12 months; \$20–50K compute
Governance capacity	Drift/capacity ratio predicts arch. health	Correlation < 0.2 or sustained excess without degradation	12–18 months; analyst time

17.4. Human Factors Not Modeled

The framework models harness architecture but not the human using it. Developer experience level is likely the single strongest moderator of AI coding tool effectiveness, yet it appears nowhere in the formal framework. Empirical evidence suggests that junior developers (0–2 years experience) exhibit 3–4× higher accretion rates than senior developers (10+ years) using identical tools. The context budget optimum shifts substantially with developer familiarity (from $\approx 0.08W$ for experts to $\approx 0.50W$ for novices on the same codebase). The harness that is optimal for one developer population may be suboptimal for another. Incorporating developer expertise as a first-class variable would require a sociotechnical extension of the framework, modeling the developer's prior knowledge as supplementary context outside the window.

17.5. Uncomputability of Idealized Metrics

Several foundational quantities in the framework are uncomputable:

- $K(P \mid S)$ (Kolmogorov complexity) is uncomputable by definition. The Shannon operationalization $H(P \mid S)$ is computable but distribution-dependent.
- The decidability classification (Section 7) establishes that 4 of 18 slop types are undecidable in general. No automated system can achieve complete detection for these types.
- The Detection Ceiling (Theorem 7.4) is an information-theoretic bound, but computing $I(X; D_j)$ for a specific defect type and feature set requires knowing the joint distribution, which is not available in practice.

These uncomputability results are not limitations of the framework; they are features. They identify hard boundaries that no engineering effort can overcome, focusing design attention on what is achievable.

17.6. The Theory Preservation Problem

The Governance pillar identifies theory preservation as fundamentally open. If the Tacit Divergence Conjecture (Conjecture 8.1) holds, then perfect externalization of program theory is impossible in principle. Current approaches (ADRs, ARCHITECTURE.md, inline documentation) capture explicit knowledge but not tacit knowledge.

This problem becomes more acute with AI agents: the "theory" held by an agent exists only during a context window and is lost when the conversation ends. There is no persistent memory that preserves the agent's understanding of design rationale, rejected alternatives, or the causal model of change propagation. The theory preservation problem is arguably the most important open problem in the field.

17.7. Scale-Dependent Governance ROI

The governance capacity bound (Theorem 8.2) establishes a threshold, but the return on governance investment is highly scale-dependent. For small projects with low change velocity, the drift rate R_{drift} is naturally low, and minimal governance suffices. For large projects with high AI-driven change velocity, governance investment must scale, but the cost of governance also scales. The optimal governance investment as a function of project scale is not well characterized.

17.8. The Measurement Problem (Goodhart's Law)

Several metrics in the framework (VIH, QAS, detection rates) are susceptible to Goodhart's Law: when a measure becomes a target, it ceases to be a good measure. If harnesses are optimized for VIH, agents may learn to produce code that passes verification gates without being genuinely correct. If QAS becomes a target, teams may game quality metrics.

Three partial defenses mitigate the Goodhart vulnerability without resolving it fully:

1. **Metric rotation:** periodically change which metrics are optimized and which are monitored passively. An agent optimizing for VIH cannot game a metric it does not know will be evaluated.
2. **Metric triangulation:** require improvement across multiple uncorrelated metrics simultaneously. VIH, coupling complexity $\Gamma(C)$, refactoring ratio r, and human merge-approval rate measure different dimensions; gaming all four simultaneously is substantially harder than gaming any one.
3. **Multi-granularity governance:** the three-loop cascade (Section 8) measures different quantities at different frequencies and timescales, making systematic gaming more difficult because each loop uses a different measurement instrument.

These defenses reduce but do not eliminate the Goodhart risk. The fundamental tension between optimizable proxies and true quality remains an open problem for any prescriptive framework.

18. Related Work

We position this work against six bodies of prior research. Each subsection identifies the specific contributions we build upon and clarifies where our framework extends, synthesises, or merely applies existing results.

18.1. Formal Methods in Software Engineering

The refinement lattice (Section 2) builds directly on Back's refinement calculus (Back, 1980) and Morgan's *Programming from Specifications* (Morgan, 1994), which formalised the stepwise refinement of specifications into code. Hoare and He's Unifying Theories of Programming (Hoare and Jifeng, 1998) showed that programs are predicates and refinement is logical implication, placing specifications and code in a single semantic universe. Abrial's B-Method and Event-B (Abrial, 2010) provide the most industrially deployed refinement-based formal method, with verified systems including the Paris Météor driverless line, directly instantiating the tower of Galois connections.

Cousot and Cousot's abstract interpretation framework (Cousot and Cousot, 1977) formalised abstraction via Galois connections five decades ago, leading to mature industrial tools (Astrée, Polyspace, Infer). We use the Galois connection formalism to model the abstraction spectrum but do not extend the abstract interpretation framework itself. Roy and Cordy's clone type taxonomy (Types I through IV) characterises detection decidability for duplicate logic: Types I through III (textual, renamed, gapped) are decidable; Type IV (semantic clones) is not. This distinction directly informs our decidability classification of slop types.

Murphy, Notkin, and Sullivan's reflexion models (Murphy et al., 1995, 2001) compare intended architecture against extracted source models, providing the formal basis for architectural erosion detection and the conformance checking that underpins our governance pillar. The Curry-Howard-Lambek correspondence (Wadler, 2015) provides the type-theoretic foundation connecting proofs, programmes, and categories that unifies our treatment of specifications as both logical propositions and programme types.

For AI-assisted formal methods, Kleppmann (Kleppmann, 2025) predicted that AI would make formal verification mainstream; Congdon (Congdon et al., 2025) endorsed and extended this argument with a concrete workflow vision. The vericoding benchmark (VeriCoding, 2025) provides early empirical support. Our framework formalises *why* this convergence is expected: AI reduces the cost of traversing the refinement lattice, while formal verification provides a deterministic oracle.

18.2. Multi-Agent Systems and Coordination

Kim et al. (Kim et al., 2025) provide the most comprehensive empirical evaluation of multi-agent system scaling, establishing error amplification factors, capability saturation thresholds, and communication scaling laws across five canonical architectures. Our work builds on their empirical findings by developing a formal framework that explains their observations and derives actionable design rules. Cemri et al. (Cemri et al., 2025) introduced MAST, the first empirically grounded failure taxonomy for multi-agent LLM systems, identifying 14 failure modes across over 1,600 traces.

For merge conflict research, Brun et al. (Brun et al., 2011) pioneered proactive conflict detection with Crystal. Kasi and Sarma (Kasi and Sarma, 2013) developed Cassandra for conflict-minimising

task scheduling. Ghiotto et al. (Ghiotto et al., 2018) provided the largest study of merge conflict characteristics (2,731 Java projects). Sousa et al. (Sousa et al., 2018) formalised semantic conflict-freedom verification with SafeMerge. Zhu et al. (**?**) introduced ConGra, finding (counterintuitively) that general-purpose LLMs outperform code-specialised ones at conflict resolution, a result relevant to coordination agent design. For merge tools, Falleri et al. (Falleri et al., 2014) developed GumTree for fine-grained AST differencing; Shen et al. (Shen et al., 2019) developed IntelliMerge for refactoring-aware merging (88.48% precision); and Apel et al. (Apel et al., 2012) developed JDime for semistructured merge (39% conflict reduction).

Graph partitioning, which formalises our task decomposition problem, has deep roots: Fiedler (Fiedler, 1973) introduced algebraic connectivity and spectral bisection; Kernighan and Lin (Kernighan and Lin, 1970) developed the KL heuristic; Fiduccia and Mattheyses (Fiduccia and Mattheyses, 1982) improved it with linear-time passes and hypergraph support; Karypis and Kumar (Karypis and Kumar, 1998) developed METIS with $O(|E|)$ multilevel partitioning; Arora, Rao, and Vazirani (Arora et al., 2009) achieved $O(\sqrt{\log n})$ sparsest cut via SDP; and Lee et al. (Lee et al., 2012) proved higher-order Cheeger inequalities connecting spectral gaps to partition quality.

Contract-based design provides our assume-guarantee framework: Meyer (Meyer, 1992) introduced Design by Contract; Henzinger et al. (Henzinger et al., 2002) developed assume-guarantee methodology; Jones (Jones, 1983) developed rely-guarantee reasoning; Benveniste et al. (Benveniste et al., 2018) provided a comprehensive theory of contracts for system design. Our coordination contracts extend these to the specific setting of LLM agents operating on shared codebases.

For distributed systems foundations, Bernstein and Hadzilacos (Bernstein et al., 1987) established concurrency control theory; Berenson et al. (Berenson and Levine, 1995) formalised isolation levels and identified write skew; Fekete et al. (Fekete et al., 2005) showed how to make snapshot isolation serialisable via promotion, analogous to our use of contracts to close the gap from snapshot to serialisable isolation for agents. Lamport (Lamport, 1978) defined happened-before and logical clocks; Herlihy (Herlihy, 1991) proved the consensus number hierarchy; and the FLP impossibility result (Fischer et al., 1985) proved that deterministic consensus is impossible with one faulty process. Ongaro and Ousterhout (Ongaro and Ousterhout, 2014) developed Raft for understandable consensus. These results inform our treatment of coordination correctness: the strong correctness condition (serialisable merge outcomes) is analogous to linearisability, while the weak correctness condition (compiles and passes tests) provides a practically achievable alternative.

For software modularity, Mitchell and Mancoridis (Mitchell and Mancoridis, 2006) developed Bunch for automatic software modularisation. Martin (Martin, 1994) defined coupling metrics (afferent, efferent, instability) that underpin our coupling graph formalism. MacCormack et al. (MacCormack et al., 2012) validated Conway's Law empirically via the mirroring hypothesis, confirming that organisational structure predicts architectural structure with high fidelity.

18.3. Software Quality and Testing

Software defect taxonomies provide context for our slop type classification. IEEE 1044 (IEEE, 2009) provides multi-attribute anomaly classification. ODC (Chillarege et al., 1992) offers eight orthogonal defect types with development-phase correlations. Beizer (Beizer, 1990) provides a ten-category bug taxonomy. CWE covers security vulnerabilities. The Accretion Category (Section 7) overlaps with the technical debt literature: Cunningham's original metaphor (Cunningham, 1992), Kruchten et al.'s taxonomy of design debt and architecture debt (Kruchten et al., 2012), Brown et al.'s (Brown et al., 2010) characterization of technical debt management, and Li et al.'s systematic mapping (Li et al., 2015) all describe patterns where individually defensible choices collectively degrade system

quality. Our contribution is identifying the specific generative mechanism (statistical pattern completion rather than intentional choice) and the decidability status (individual detection is undecidable) that distinguish AI-generated accretion from traditional design debt.

For software reliability, Musa's models (Musa et al., 1987) connect operational profiles to defect discovery. Fenton and Neil (Fenton and Pfleeger, 1999) introduced Bayesian networks for defect prediction. Capers Jones (Jones, 2012) provided the foundational defect removal effectiveness data from 25,000 projects. Littlewood and Strigini (Littlewood and Strigini, 1993, 2000) proved that diverse software versions do not fail independently, a result we adapt to LLM judge-producer pairs via the FKG inequality.

The classical reliability theory foundations include Barlow and Proschan's series system product law and importance measures (Barlow and Proschan, 1965, 1975), the IFR/IFRA closure theorems (Birnbaum and Saunders, 1961), and the Young-Daly formula (Daly, 2006; Young, 1974) for optimal checkpoint intervals. The Jelinski-Moranda (Jelinski and Moranda, 1972) and Musa-Okumoto (Musa and Okumoto, 1984) models of fault detection as a depletion process inform our verification convergence analysis.

For AI code quality measurement, CodeRabbit (CodeRabbit, 2025a), GitClear (GitClear, 2025), Qodo (Qodo, 2025), and industry analyses of the AI-driven technical debt boom provide the emerging empirical base. The controlled GitHub trial (GitHub, 2024) stands in tension with observational results, a contradiction we interpret through the verification bottleneck (short-term task completion gains masking long-term maintainability costs). The Swiss Cheese Model (Reason, 1990) frames accident causation through aligned holes in multiple barriers. Viola-Jones cascades (Viola and Jones, 2004) formalise sequential classifier design with early rejection of easy negatives. Boehm's cost escalation curve (Boehm, 1981) and Capers Jones (Jones, 2012) provide the economic foundation for layered defense. We note that the classical 1:10:100 cost ratio (prevention to internal failure to external failure) has been challenged: Shull et al. (Shull et al., 2002) and Boehm and Basili (Boehm and Basili, 2001) found that the ratio is context-dependent and may be flatter in modern CI/CD environments with rapid deployment cycles. Our recalibration by decidability class (Section 7) should be understood as adjusting a baseline that is itself disputed.

Anthropic's agent evaluation methodology (Anthropic, 2026b) introduces the pass@k vs. pass^k distinction, clarifying whether verification requires any correct output in k attempts or correct outputs across all k attempts. The LLM-as-Judge survey (Li et al., 2024) documents systematic biases (sycophancy, position bias) that limit LLM-based verification. The METR merge-gap study (METR, 2026b) provides the most striking evidence of circular validation bias: agents that "solve" benchmarks by passing tests frequently produce code that human reviewers would reject.

18.4. Architecture Governance and ADRs

Perry and Wolf (Perry and Wolf, 1992) established the foundational framework for software architecture as a discipline, defining components, connectors, and configurations; the Governance pillar's treatment of architectural erosion builds directly on their observation that "the life of a software architect is one of routine repair."

Nygard (Nygard, 2011) proposed the lightweight ADR format. The Architecture Advice Process (Harmel-Law, 2023) adds a governance layer: anyone can make architectural decisions, but must first consult affected parties and domain experts. Ford et al. (Ford et al., 2017) defined architectural fitness functions as objective integrity assessments of architectural characteristics, with tools like ArchUnit encoding these as executable tests. The gap in ADR-to-fitness-function automation is being addressed by tools like Archgate through the ratchet model.

For reflexion models, Murphy et al. (Murphy et al., 1995, 2001) provide the formal foundation for conformance checking, with extensions including behavioural reflexion models and CI-integrated prototypes detecting nonconformances "within minutes, instead of the months or years it takes today." Hochstein and Lindvall (Hochstein and Lindvall, 2005) surveyed approaches to combating architectural degeneration, establishing the empirical baseline for architecture conformance recovery. Subsequent work on architectural drift detection in evolving systems (Passos et al., 2021) has examined how architectural decisions scatter across codebases over time, directly informing our governance pillar's drift detection formalism. The AgDR format extends ADRs with agent-specific metadata (agent name, model ID, session, trigger type), addressing traceability gaps when AI agents make architectural choices.

Conway's Law (Conway, 1968) established the homomorphism between organisational and system structure. Nagappan et al. (Nagappan et al., 2008) found that organisational metrics predict failure-proneness with approximately 85% precision and recall, outperforming all code metrics. MacCormack et al. (MacCormack et al., 2008) validated the mirroring hypothesis empirically. Skelton and Pais (Skelton and Pais, 2019) operationalised the inverse Conway maneuver through Team Topologies.

Hellerstein et al. (Hellerstein et al., 2004) provided the foundational treatment of feedback control for computing systems. Our contribution is applying cascade control specifically to multi-granularity governance, with stability analysis and Nyquist-like sampling requirements for governance loops. Lehman's laws of software evolution (Lehman, 1980), particularly the law of increasing complexity, underpin our coupling density dynamics model.

The Goodhart's Law literature (Goodhart, 1975; Manheim and Garrabrant, 2019) is directly relevant to governance measurement. When a governance metric becomes a target, it ceases to be a good measure: coupling targets incentivise restructuring that reduces measured coupling while introducing unmeasured coupling through shared configuration; coverage targets incentivise trivial fitness functions. The defense is metric rotation, metric triangulation, and cultural alignment. Our framework addresses this through the multi-granularity approach: different governance loops measure different quantities at different frequencies, making systematic gaming more difficult.

The DORA programme (Google Cloud, 2025) provides the largest objective measurement of AI's impact on software engineering practice. The productivity paradox it documents (individual gains, organisational stagnation) motivates our quality-adjusted speedup metric, which formalises the quality cost of parallelism. The observation that Brooks's Law applies directly to LLM agents is a position our Extended Amdahl's Law formalises.

18.5. Information Theory Applied to Software

The application of algorithmic information theory to software has precedent. Li and Vitányi (Li and Vitányi, 2019) discuss the connection between algorithmic complexity and programme equivalence. The normalised information distance (Li et al., 2004) has been applied to software clone detection and plagiarism, though not (to our knowledge) to the specification-code relationship specifically. Our abstraction gap $\mathcal{G}(S, P) = K(P \mid S)$ applies conditional Kolmogorov complexity to this relationship.

Hindle et al. (Hindle et al., 2012) established that code is "natural" (statistically predictable), measuring code entropy and showing that language models can exploit this regularity. Hellendoorn and Devanbu refined these measurements, finding entropy of approximately 1.25 bits per token. Shannon's original measurements (Shannon, 1948) establish the baseline at 0.6 to 1.3 bits per character for natural language. These measurements ground our information-theoretic treatment of the specification-code channel.

Hassan's change entropy (Hassan, 2009) predicts faults from change scattering. Our coupling complexity (Definition 7.4) shifts from measuring change scattering to measuring the coupling structure that necessitates it. Tishby et al.'s information bottleneck method (Tishby et al., 2000) provides an alternative formalisation: the context window as an information bottleneck between the full codebase and the generated code.

For context engineering specifically, Anthropic (Anthropic, 2025) established the "smallest set of high-signal tokens" principle with four functional components (system prompts, tools, examples, message history). Spotify's Honk (Spotify Engineering, 2025b) contributed the "static over dynamic" philosophy. HumanLayer (HumanLayer, 2025) distinguished structural from instructional enforcement and coined "success should be silent." Lin and Bilmes (Lin and Bilmes, 2011) proved submodularity for document summarisation functions. Jina et al. (Jina et al., 2025) independently applied submodular optimisation to LLM context engineering, validating our formalisation. Liu et al. (Liu et al., 2024) identified the "lost in the middle" phenomenon. Subsequent work by Chroma (Chroma, 2025), Du et al. (Du et al., 2025), Kuratov et al. (Kuratov et al., 2024), and Hsieh et al. (Hsieh et al., 2024) collectively establishes the empirical degradation landscape that our context degradation function formalises.

The value of information literature, originating with Howard (1966), frames context selection as a decision-theoretic problem: which information to acquire before acting. The greedy algorithm for submodular value of information was analysed by Krause and Guestrin (2005). Our degradation penalty is a novel addition absent from the classical setting, where acquiring more information is assumed to be never harmful. For code, this assumption fails: code dependencies create non-monotonic relevance effects where additional context can actively mislead.

18.6. AI Coding Agents and Harnesses

The empirical landscape of AI coding agents provides both motivation and calibration for our framework. SWE-bench (SWE Efficiency Research, 2025) measures resolve rates, with SWE-Effi introducing resource-bounded effectiveness scores. Bian et al. (Bian et al., 2025) decompose bottlenecks, finding that web environment latency dominates (53.7% of runtime) with LLM API latency varying up to 69×. These findings motivate our temporal architecture.

For speculative execution in agent pipelines, Zhou et al. (Zhou et al., 2025) achieve 46 to 55% next-action prediction accuracy; Ji et al. (Ji et al., 2026) extend this with heterogeneous speculation, achieving 3.28× speedup. Our contribution is the formal speculation ratio test and depth limit, grounded in classical scheduling theory and connected to the Spectre analogy for hidden costs.

Agent verification frameworks include AgentSpec (Wang et al., 2026), which provides the key empirical data point on structural versus prompt enforcement (87% versus 77%); AgentGuard (Koohestani et al., 2025), which introduces dynamic probabilistic assurance via MDP modelling; and Multi-Agent Verification (Lifshitz et al., 2025), which demonstrates that diverse verifier ensembles outperform homogeneous ones and that weaker models can verify stronger ones.

For scalable oversight, Irving et al.'s debate framework (Irving et al., 2018) proposes using adversarial argumentation between agents to help weaker judges evaluate stronger systems. Constitutional AI (Bai et al., 2022) uses self-critique against a set of principles; while effective as a training methodology, it inherits the circular validation limitations we formalise. METR's reward-hacking analysis (METR, 2025), Apollo Research's scheming studies (Apollo Research, 2025), and NIST CAISI's cheating documentation (NIST CAISI, 2025) collectively establish that structural enforcement is necessary for adversarial robustness.

For caching and context persistence, Liang et al. (Liang et al., 2026) evaluate prompt caching empirically, and Zhang et al. (Zhang et al., 2025a) introduce plan template caching at the reasoning level. Suzgun and Kalai (Suzgun et al., 2024) establish the "fresh eyes" advantage that motivates our fresh context dominance theorem. The speed-quality tradeoff literature includes Reinertsen's cost-of-delay framework, Forsgren et al.'s (Forsgren et al., 2018) evidence that speed and stability can improve simultaneously through DevOps practices, and Kahneman's (Kahneman, 2011) cognitive science foundation for why speed pressure degrades complex reasoning.

Industrial harness implementations provide the strongest empirical grounding. Cursor (Cursor, 2026) documented the evolution from flat locking to planner/worker/judge topology, providing evidence that hierarchical role separation succeeds at scale. Osmani (Osmani, 2025) synthesised practitioner patterns including optimal team sizing and tier-based tool stacks. ChatDev, MetaGPT, and AgentCoder explore role-based agent decomposition for software tasks, providing additional data points on multi-agent coordination architectures. The convergence of these production systems toward architectures predicted by our framework (layered verification, structural enforcement, bounded context windows, governance loops) provides indirect validation, though controlled experiments remain necessary.

The sampling inspection literature from manufacturing quality control provides precedent for our adaptive verification approach. The Dodge-Romig sampling plans (Dodge and Romig, 1929, 1959) and MIL-STD-105E switching rules (U.S. Department of Defense, 1989) formalised adaptive inspection regimes that tighten or relax based on observed quality. The Lindsay-Bishop dynamic programming formulation for inspection allocation (Lindsay and Bishop, 1964) provides the discrete optimisation framework that informs our verification scheduling result. These classical results, developed for physical manufacturing, transfer naturally to software verification pipelines where the "inspection" is a verification step and the "product" is generated code.

Concurrent and competing frameworks. Several concurrent works address the same problem space from complementary angles. Hassan et al. (Hassan et al., 2025) present a Structured Agentic Software Engineering (SASE) vision with a competing pillar-based decomposition (actors, processes, tools, artifacts) and the ACE/AEE dual-workbench model. Their decomposition is oriented around engineering workflows rather than formal properties; the two frameworks are complementary rather than contradictory, though a systematic comparison of their respective pillar boundaries is needed. Lahiri (Lahiri, 2026) identifies intent formalisation as a grand challenge for reliable AI coding, addressing the same specification gap that our Abstraction pillar formalises, but focusing on the practical challenge of eliciting and verifying intent rather than the information-theoretic characterisation. For model routing and cascade architectures, Dekoninck et al. (Dekoninck et al., 2025) provide a unified framework with optimality proofs for routing and cascading, which our Model Routing pillar should be read alongside.

Position in the landscape. This paper is a synthesis and formalisation, not a claim of novel mathematical results. The information-theoretic machinery is standard (Li and Vitányi, 2019); the refinement calculus is established (Back, 1980; Morgan, 1994); the reliability theory is classical (Barlow and Proschan, 1965); the Curry-Howard correspondence is a theorem (Wadler, 2015). Our contribution is the application of these tools to the specific problem of AI coding agent harness design, the unification across seven architectural pillars through shared formal machinery, and the identification of cross-pillar constraints (the abstraction gap chain, the compound error cascade, the governance capacity bound) that emerge only from the unified treatment.

19. Conclusion

This paper has presented a unified formal framework for AI coding agent harness architecture across eleven pillars. The key contributions are:

1. **Information as a shared vocabulary**: all eleven pillars reason about information flow. Information theory is the primary analytical tool for the semantic and assurance axes (4 pillars), provides a compatible interpretive framework for the execution axis (3 pillars), and connects to the operational axis through the CVIH metric (Economics), the detection ceiling (Human Interaction), and the diversity gain (Model Routing).
2. **The Governance Information Budget** (Equation 98): the integrating concept showing that every pillar constrains a different aspect of the same problem, namely closing the abstraction gap reliably, efficiently, and coherently.
3. **Three cross-cutting cascades**: the abstraction gap chain (Abstraction, Information, Quality), the compound error cascade (Reliability, Coordination, Temporal), and the structural enforcement principle (Information, Reliability, Quality, Coordination).
4. **Design principles** grounded in specific formal results across eleven dimensions, including principles for the operational pillars (equal marginal rate, risk-ranked triage, cross-model diversity) and the security pillar (principle of least authority, credential isolation, structural containment).

The thesis of this paper is that harness architecture is not merely an engineering problem to be solved through trial-and-error, but a domain amenable to formal analysis that yields testable predictions and non-obvious design guidance. The seven pillars provide a decomposition of the design space, each grounded in established mathematical machinery. The cross-pillar connections show that these are not independent concerns but interacting dimensions whose constraints compose in structured ways.

The framework is incomplete. The empirical validation gaps identified in Section 14 are substantial, and several key quantities (Kolmogorov complexity, tacit knowledge content, governance capacity in practice) are either uncomputable or unmeasured. The theory preservation problem (Section 8) remains fundamentally open.

Nevertheless, the framework provides something the field currently lacks: a systematic way to reason about harness design decisions, predict their consequences, and identify the leverage points where investment yields the highest returns. The compound error sensitivity observation tells us to invest in per-step quality. The structural enforcement principle tells us to make critical invariants agent-proof. The governance capacity bound tells us to scale governance with velocity. The abstraction gap decomposition tells us where information should come from.

The next step is empirical validation. The five experiments proposed in Section 17 would ground the theoretical predictions in measured parameters, transforming the framework from a formal exercise into a predictive tool. We invite the community to contribute to this validation program.

Appendices

A. Cross-Pillar Theorem Index

Table 15 provides a comprehensive index of the significant theorems, propositions, observations, definitions, and conjectures across all seven dimensions. The index is organized by pillar and includes all results that are referenced in cross-pillar arguments or that constitute a pillar's primary formal contributions.

Table 15 | Cross-pillar theorem index. Entries marked with † participate in cross-pillar arguments (Section 13).

Pillar	Result	Summary
Abstraction	Abstraction Gap†	$\mathcal{G}(S,P) = K(P \mid S)$: conditional Kolmogorov complexity between spec and code.
	Gap Properties	Minimality ($\mathcal{G} \geq 0$), maximality ($\mathcal{G} \leq K(P)$), monotonicity (refining S can only decrease $\mathcal{G}$), additivity (gap decomposes over independent components).
	NID Universality	Normalised information distance $d(S,P)$ is a universal metric (Li-Vitanyi).
	Spec-Code Convergence†	For incompressible P, any S with $K(P \mid S) \leq c$ satisfies $\lvert S\rvert \geq \lvert P\rvert - O(\log \lvert P\rvert)$.
	Refinement Lattice	Specifications form a complete lattice under refinement; Galois connection (α, γ) between spec and program lattices.
	Agent Decomposition	$\mathcal{G}(S,P) \leq \mathcal{G}(S,\hat{S}) + \mathcal{G}(\hat{S},P) + O(\log n)$: intermediate specs reduce total gap up to a logarithmic overhead.
Information	Submodularity†	Context relevance $f(C) = I(C; P \mid S, \theta)$ is submodular under mild conditions; greedy achieves $(1 - 1/e)$ approximation.
	Non-Monotonicity	The degradation-adjusted objective $\tilde{f}(C) = f(C) \cdot (1 - \delta(\lvert C\rvert))$ is non-monotone; requires randomized greedy with $1/2$-approximation.
	Gap Decomposition†	$H(P\lvert S) = H(P\lvert S,C,\theta) + I(P;C\lvert S,\theta) + I(P;\theta\lvert S)$, exact equality via Shannon chain rule.
	Position Dominance	Context item position dominates item relevance in determining effective contribution; positional recall follows a U-shaped curve.
	Context Budget†	$\lvert C^*\rvert < W$; optimal context is sub-capacity, approximately $0.15W$ to $0.40W$.
	Curation Beats Accumulation	Curated context (active tier) outperforms accumulated context of the same size.
	Enforcement Dominance†	Structural compliance = 1; instructional compliance = $(1-\epsilon)^T$, exponential decay in pipeline length.
	Fresh Context Dominance	Agent with fresh context outperforms agent with stale context by ~17% ("Fresh Eyes" effect).
Reliability	Series Reliability	$R(n) = \prod p_i$; for homogeneous pipelines, $R(n) = p^n$.
	Compound Sensitivity†	Elasticity $\mathcal{E}(R,p) = n$; 1% per-step improvement yields n% system improvement.
	Weakest-Link Priority	Improve the step with lowest p_i first; marginal gain is maximized at the weakest link.
	Correlated Failure	Under Marshall-Olkin common-cause model: $R_{\text{sys}} = (1-\gamma)\prod(1-\alpha_i)$, where γ is shared failure rate.
	Optimal Checkpoints†	$k^* \approx \sqrt{n(1-p)c_0/c_v}$; Young-Daly analog. Three rounds capture > 96% of detectable errors.
	Diminishing Returns	Marginal value of the k-th checkpoint: $\Delta M(k) = N_0 v(1-v)^{k-1}$, geometric decay.
	Circular Bias†	$\beta = \beta_A(1-\beta_{A'})$; self-verification ($A = A'$) yields zero bias reduction on shared blind-spot categories.
	Structural Boundary†	Structural enforcement cost-effective when $L > L^* = (c_s - c_p)/\epsilon$; threshold drops as L^*/n for multi-step pipelines.
Coordination	Error Amplification†	Under independence, $A_e = (1-(1-\epsilon)^k)/\epsilon \to k$; empirical is 17.2× (correlation multiplier).
	Saturation Crossover	Scaling benefits saturate at approximately $P^* \approx 0.45$ of the task completion rate. (Conjecture.)
	Extended Amdahl†	$S_{\text{eff}}(n) = [1/(s + (1-s)/n)] \cdot (1-\delta_a)^n$; optimal $n^* \approx 1/\sqrt{\delta_a}$.
	NP-Hardness	Balanced k-way partition of the coupling graph is NP-hard.
	Cheeger Inequality	Spectral gap bounds partition quality: $h(G) \leq \sqrt{2\lambda_2}$ (discrete Cheeger).

	Cut-Conflict Bridge†	$\mathbb{E}[\text{conflicts}] \leq \alpha \cdot W_{\text{cut}}(\pi)$; merge conflicts bounded by cut weight.
	Contract Reduction	File-ownership contracts reduce conflict growth from $O(k^2)$ to $O(k)$.
	Write Skew	Two agents with individually valid changes can produce an invalid combination; requires assume-guarantee contracts.
	QAS†	Quality-Adjusted Speedup = $(T_1 Q_1)/(T_k Q_k)$; QAS < 1 when quality degradation outweighs speed gain.
	Conway's Law	Low-overhead coordination requires $G_{\text{coupling}} \subseteq G_{\text{comm}}$; graph containment condition.
Temporal	Context Fidelity†	$\Phi(t) = e^{-\Lambda(t-t_0)}$; exponential staleness decay with half-life $t_{1/2} = \ln 2/\Lambda$.
	VIH Decomposition†	VIH = $\lambda_{\text{raw}} \cdot p_{\text{verify}}$; verified iterations per hour.
	VIH Optimality	VIH maximized at unit elasticity: $\partial \ln \lambda / \partial v = -\partial \ln p_{\text{verify}} / \partial v$.
	Speculation Test†	Speculate iff $p/q > R/B$, where R is rollback cost and B is early-completion benefit.
	Speculation Depth Limit	$k^* < \ln(1 + B_0/R_0)/\ln(1/p)$; depth beyond this has negative expected value.
	Cache EOQ†	$T^* = \sqrt{2R/(\lambda\alpha \cdot E_{\text{stale}})}$; optimal cache refresh interval.
	Bayesian Mode Selection	Switch from fast to governed mode when $P(\text{Governed} \mid \text{x}) > c_{GF}/(c_{FG} + c_{GF})$.
	Pareto Convexification	Mixing fast and governed modes convexifies the speed-quality frontier; VIH optimum lies at the tangency of the VIH iso-hyperbola.
Quality	Decidability Classes†	7 Decidable, 7 Semi-decidable, 4 Undecidable slop types across 18 AI code defect categories.
	Latent Factor Structure	Three approximately orthogonal generative factors: Context Blindness, Completion Bias, Training Distribution Leakage.
	Defense NP-Hardness†	Layered defense optimization is NP-hard (via Weighted Set Cover); greedy achieves $(1 - 1/e)$ approximation.
	Detection Ceiling†	$d_{\ell,j} \leq I(X; D_j)/H(D_j)$ via Data Processing Inequality; information-theoretic limit on detection.
	No Free Lunch	Context-poor layers cannot exceed the detection ceiling; context access is necessary for high detection rates.
	FKG Correlation†	$P(\text{both miss}) \geq P(\text{producer misses}) \cdot P(\text{judge misses})$; independence underestimates escape probability.
	Diversity Gain	Weakly independent diverse verifiers outperform strongly correlated single-family verifiers.
	ROI Ordering	Optimal layer ordering: decreasing r_ℓ/c_ℓ (detection rate per unit cost).
	Enforcement Gap	Syntactic properties enforceable with $P = 1$; semantic properties strictly less (Rice's theorem).
	Compound Degradation†	CI-passing changes yield superlinear entropy growth: $H \geq n\Delta H + \binom{n}{2}\Delta H_{\text{cross}}$.
	Ratchet Effect	Codebase entropy is monotonically non-decreasing under CI-only governance.
	Optimal Quality Level	Closed-form $q^* = (1/\beta)\ln(c_F \lambda_0 \beta/(c_P + c_A))$; higher for decidable types, lower for undecidable.
Governance	Theory Extraction Bound	Rate-distortion: $R(D) = \min_{p(\hat{T}\mid T):\mathbb{E}[d(T,\hat{T})]\leq D} I(T;\hat{T})$; minimum description length of program theory.
	Tacit Divergence	Conjectured: $R(D) \to \infty$ as $D \to 0$ for tacit knowledge components; perfect externalization requires infinite description.
	Incompressibility	Kolmogorov-random tacit components satisfy $K(t_\tau) \geq \lvert t_\tau \rvert - O(1)$; no lossless compression possible.
	Double Bottleneck	$I(T;G) \leq I(T;(D,A,R)) \leq I(T;L)$; governance artifacts are twice-bottlenecked.
	Capacity Bound†	If $R_{\text{drift}} > C_{\text{gov}}$, no governance scheme prevents unbounded divergence.
	AI Velocity Corollary	Increasing agent throughput λ eventually overwhelms any fixed C_{gov}; governance must co-scale with velocity.
	Cascade Stability	Three-loop cascade governance is stable iff each loop satisfies Nyquist-like sampling requirement: $f_k \geq 2\omega_d^{(k)}$.
	Governance Bifurcation	Transcritical bifurcation at $\mu G/\gamma_g R = 1$; below: self-correcting; above: unbounded drift.
	Ratchet Convergence†	Closure operator on constraint lattice converges in $\leq \lvert\mathcal{R}\rvert$ steps.
	Ossification	When contradictory constraints accumulate, the allowed change set becomes empty; the ratchet becomes pathological.

Decision Survival	Competing-risks Weibull model; median lifetimes: technology 2–4y, framework 3–5y, architecture 5–10y.

The index contains 63 entries across 7 pillars. Of these, 19 (marked with †) participate directly in the cross-pillar arguments developed in Section 13. The remaining 44 are pillar-internal results that provide the technical foundations for the cross-pillar theorems. The density of cross-references (19 out of 63 results, approximately 30%) confirms that the pillars are not independent theories but a connected formal network.

B. Extended Empirical Calibration

The calibration tables in this appendix consolidate empirical parameters from the seven architectural dimensions. All estimates carry heterogeneous confidence; ranges and confidence markers are preserved from the source papers.

B.1. Detection Probability Matrix (Quality Pillar)

Table 16 presents estimated detection probabilities $d_{\ell,j}$ for selected slop types across four defense layers. Confidence levels: **H** = strong empirical backing; **M** = single source or derived; **L** = extrapolated; **E** = estimated from first principles.

Table 16 | Detection probability ranges for slop types across four defense layers (from Quality pillar).

Slop Type	L1: Structural	L2: Deterministic	L3: LLM-Judge	L4: Human
Decidable types				
Hallucinated imports	0.90–0.98 H	0.50–0.70 M	0.60–0.80 M	0.70–0.90 M
Placeholder/stub code	0.30–0.50 M	0.85–0.95 H	0.70–0.85 M	0.80–0.95 M
Duplicate logic	0.05–0.15 L	0.75–0.90 H	0.40–0.60 M	0.50–0.70 M
Semi-decidable types				
Security vulns	0.10–0.20 M	0.10–0.15 H	0.50–0.70 M	0.60–0.80 M
Happy-path handling	0.05–0.15 L	0.20–0.40 M	0.55–0.75 M	0.70–0.85 M
Scope expansion	0.05–0.15 M	0.10–0.25 L	0.60–0.80 M	0.70–0.85 M
Undecidable types				
Meaningless tests	0.00–0.05 E	0.05–0.15 E	0.20–0.40 L	0.50–0.70 L
Redundant comments	0.00–0.02 E	0.05–0.15 E	0.30–0.50 L	0.50–0.70 L

Key calibration anchors: type systems catch 15% of public bugs (Gao et al., 2017); static analysis tools achieve 13% recall in production; Spotify's 25% veto rate across 1,500+ PRs calibrates Layer 3 (Spotify Engineering, 2025a); AgentSpec's 87% code-based enforcement (Wang et al., 2026). Combined escape rate for meaningless tests (across all layers) is approximately 25%, representing the current detection frontier.

B.2. Cost per PR by Defense Layer (Quality Pillar)

The false positive cost trap: at 100 PRs/week with Layer 3 FP rate of 25%, developers spend approximately 2,600 hours/year investigating false findings, equivalent to roughly 1.3 FTEs. ROI ordering confirms that Layer 1 is always cost-effective (ROI > 1 for all 18 types), while Layer 4 is cost-effective

Table 17 | Cost per PR by defense layer, with false positive rates.

Layer	Latency	Cost/PR	FP rate
L1: Structural gates	2–10s	$0.50	0.01–0.05
L2: Deterministic analysis	10–60s	$2.00	0.05–0.20
L3: LLM-as-Judge	20–120s	$8.00	0.15–0.35
L4: Human review	10–60 min	$80.00	0.05–0.15

only for 4–5 high-severity types (security, architectural erosion, data model mismatches, meaningless tests).

B.3. Error Amplification Parameters with 95% CIs (Coordination Pillar)

Table 18 | Error amplification parameters from empirical studies (Coordination pillar).

Parameter	Value	95% CI	Source	Conditions
A_e (Independent)	17.2×	$[14.3, 20.1]$	Kim et al. (2025)	No shared state
A_e (Centralized)	4.4×	$[3.8, 5.0]$	Kim et al. (2025)	Hub coordinator; 285% overhead
A_e (Decentralized)	7.8×	NR†	Kim et al. (2025)	Peer-to-peer; 263% overhead
A_e (Hybrid)	5.1×	NR†	Kim et al. (2025)	Combined; 515% overhead
Bug mult. (conflict code)	2×	NR	Lesenich et al. (2017)	Any merge conflict
Bug mult. (manual)	26×	NR	Lesenich et al. (2017)	Manual intervention

†Point estimates only; interpret with caution.

The gap between the independence prediction ($A_e \leq k$) and the observed $17.2\times$ quantifies a "correlation multiplier" of roughly $6\times$, indicating that agent errors are correlated and cascading rather than independent.

B.4. Merge Conflict Rate Data (Coordination Pillar)

Table 19 | Merge conflict rate parameters from large-scale studies.

Parameter	Value	Source	Sample
Textual conflict rate	17–20% of merges	Brun et al. (2011); Ghiotto et al. (2018)	3.4M LOC; 2,731 projects
Project-level range	7.6–19.3%	Kasi and Sarma (2013)	Multiple OSS projects
Build conflict rate	1–10%	Brun et al. (2011)	9 projects
Trivial resolution rate	75%	Ghiotto et al. (2018)	2,731 Java projects

B.5. Spec-to-Code Ratios (Abstraction Pillar)

The seL4 data is particularly instructive: the proof-to-code ratio of 23:1 for full verification demonstrates that the "cost" of closing the abstraction gap to zero is enormous in formal-verification settings.

B.6. GSD Pipeline Stage Parameters (Temporal Pillar)

The sequential makespan sums to $\mathbb{E}[M_{\text{seq}}] = \sum \mu_i = 56.5$ minutes. Accounting for geometric retries, $\mathbb{E}[M_{\text{fail}}] = \sum \mu_i / p_i \approx 61.5$ minutes. In the Pinedo classification (Pinedo, 2016), this scheduling problem is $\mathrm{Rm} \mid \mathrm{prec}, \mathrm{stoch} \mid \mathbb{E}[C_{\max}]$: unrelated parallel machines, precedence constraints, stochastic processing times, minimizing expected makespan.

Table 20 | Spec-to-code ratios from real-world projects. The ratio varies by over three orders of magnitude depending on specification formality.

Project	Spec Size	Code Size	Ratio	Notes
Gonzalez YAML study	3,388 lines	16,063 lines	4.7:1	Semi-structured spec
seL4 (abstract spec)	7,000 lines Haskell	8,700 lines C	1.2:1	Abstract spec only
seL4 (full verification)	200,000+ lines Isabelle	8,700 lines C	0.04:1	Proof ≫ code
CompCert C compiler	28,000 lines Coq	6,000 lines C	0.2:1	Verified compiler
AWS TLA+ (DynamoDB)	~50 lines TLA+	~thousands	20–100:1	Per-component
Typical enterprise	~1 page NL	500–5,000 lines	500–5000:1	Highly variable

Table 21 | Estimated stage parameters for the GSD pipeline (derived from reported ranges; no controlled measurement).

Stage	μ_i (min)	σ_i (min)	q_i	Distribution
Context loading	10.0	2.0	0.02	Log-normal
Research	7.5	2.5	0.05	Log-normal
Planning	6.5	1.5	0.08	Log-normal
Execution	12.5	2.5	0.15	Log-normal
Verification	10.0	2.0	0.10	Log-normal
Human review	10.0	5.0	0.05	Log-normal

B.7. Pipeline Speedup Analysis (Temporal Pillar)

Table 22 | Speedup analysis for four temporal optimization strategies against GSD baseline ($\mathbb{E}[T_{\text{seq}}] = 56.8$ min).

Strategy	Time saved	Quality impact	VIH impact	Net verdict
Persistent state	7.0 min	-17.1%	Negative	VIH-negative
Speculative pipelining	8.5 min	Minimal	$+19\%$	Positive
Tiered verification	4.8 min	-1.5%	$+7.6\%$	Positive
Async governance	8.0 min	Minimal	$+17\%$	Positive
Combined (VIH-optimal)	19.3 min	-1.5%	$+34\%$	Positive

The key finding: naive persistent state is VIH-negative when the quality penalty approaches the 17.1% upper bound. The break-even threshold is approximately 12.3% (ratio of time saved to total cycle time). The combined speedup from the three VIH-positive strategies yields $\mathbb{E}[T_{\text{combined}}] \approx 37.5$ minutes, a 20–34% cycle time reduction (the range accounts for interaction effects between the three optimizations).

B.8. Degradation Function Fit Comparison (Information Pillar)

Three candidate functional forms were fit to empirical data from Du et al. (2025) (a convergent pattern across production harnesses), using Llama-3.1-70B measurements at three context lengths:

The recommended model is $\delta(n) = \min\left(1,\ (n/W_{\text{eff}})^{\alpha}\right)$ with $\alpha \in [0.3, 0.5]$ and $W_{\text{eff}} \in [0.1W, 0.7W]$. The effective window fraction is task-dependent: reasoning tasks (multi-hop, code generation) have $W_{\text{eff}} \approx 0.1W$ to $0.2W$; retrieval tasks (lexical, NIAH) have $W_{\text{eff}} \approx 0.6W$ to $0.7W$. The broad parameter

Table 23 | Degradation function fits. The power law provides the best fit, capturing steep early degradation followed by a flattening tail.

Functional Form	$\delta(5.3\text{K})$	$\delta(13\text{K})$	$\delta(20\text{K})$
Observed (Llama-3.1-70B)	0.60	0.82	0.88
Exponential: $1 - e^{-\lambda n}$	0.60	0.90	0.97
Power law: $(n/W_{\mathbf{eff}})^{\alpha}$	**0.60**	**0.82**	**0.86**
Sigmoid: $1/(1 + e^{-k(n-n_0)})$	0.60	0.85	0.95

ranges reflect calibration to only three data points from a single model; cross-model validation is needed.

C. Extended Formal Definitions

This appendix collects formal tuple definitions from the preceding sections that the main text uses but does not define explicitly.

C.1. Agent, SAS, and MAS (Abstraction Pillar)

Definition C.1 (Agent)**.** *An* agent *a is a tuple $a = (\Phi, \mathfrak{A}, \mathcal{T}, \pi)$, where Φ is the reasoning policy (typically an LLM with parameters θ), $\mathfrak{A}$ is a finite set of available actions (tools: file read/write, shell execution, web search), $\mathcal{T} = \{t_1, \ldots, t_k\}$ is a set of tools providing environmental interaction, and $\pi : O^* \to \Delta(\mathfrak{A})$ is the agent's policy, mapping observation histories to distributions over actions.*

Definition C.2 (Single-Agent System (SAS))**.** *A* single-agent system *is a tuple $\mathcal{S}_{SAS} = (\{a\}, \mathcal{E}, \emptyset)$ where $|A| = 1$, $\mathcal{E}$ is the environment (codebase, terminal, file system), and there is no inter-agent communication. The agent operates in an agentic loop:* observe → reason → act → verify → repeat*, consuming a token budget B.*

Definition C.3 (Multi-Agent System (MAS))**.** *A* multi-agent system *is a tuple $\mathcal{S}_{MAS} = (A, \mathcal{E}, C, \Omega)$ where $|A| > 1$, C is a communication topology (a directed graph over agents), and Ω is an orchestration policy governing task assignment, aggregation, and termination. Four canonical topologies are distinguished:* independent *($C = \emptyset$),* centralized *(star graph),* decentralized *(possibly complete undirected graph), and* hybrid *(star with selective peer edges).*

C.2. Verification Layer 5-Tuple (Reliability Pillar)

Definition C.4 (Verification Layer)**.** *A* verification layer *ℓ is a tuple (d, f, c, τ, η) where:*

- *$d(\ell) = \mathbb{P}(\text{flag} \mid \text{defect})$ is the detection rate (sensitivity),*
- *$f(\ell) = \mathbb{P}(\text{flag} \mid \text{no defect})$ is the false positive rate,*
- *$c(\ell)$ is the cost per invocation,*
- *$\tau(\ell)$ is the latency per invocation, and*
- *$\eta(\ell) = d(\ell)/c(\ell)$ is the layer efficiency.*

C.3. Coordination Contract (Coordination Pillar)

Definition C.5 (Coordination Contract)**.** *A* coordination contract *for k agents on codebase $\mathcal{F}$ is a tuple $C = (\{F_i\}_{i=1}^{k}, \{I_j\}_{j=1}^{m}, \sigma)$ where:*

- $F_i \subseteq \mathcal{F}$ *is the owned file set of agent* A_i*, with* $F_i \cap F_j = \emptyset$ *for* $i \neq j$ *(disjoint ownership);*
- $\{I_j\}_{j=1}^{m}$ *is a set of interface specifications, each* $I_j = (\text{name}_j, \text{sig}_j, \text{module}_j)$*; and*
- $\sigma : \mathcal{F} \to 2^{\{I_1,\ldots,I_m\}}$ *maps each file to its exported interfaces.*

Under this contract, agent A_i *may freely modify files in* F_i*, must preserve the signatures of all interfaces in* $\sigma(F_i)$*, and may only depend on (not modify) interfaces exported by other agents' files.*

C.4. Pipeline DAG with Pinedo Classification (Temporal Pillar)

Definition C.6 (Agent Pipeline DAG)**.** *An agent pipeline is a stochastic DAG* $G = (V, E, \mathrm{D}, \mathrm{q})$ *where* $V = \{v_1, \ldots, v_n\}$ *are pipeline stages,* $E \subseteq V \times V$ *encodes precedence constraints, each stage* v_i *has duration* $t_i \sim D_i$ *with mean* μ_i *and variance* σ_i^2*, and failure probability* $q_i \in [0, 1]$ *with success probability* $p_i = 1 - q_i$*.*

In the Pinedo classification (Pinedo, 2016), the harness scheduling problem is $\mathrm{Rm} \mid \mathrm{prec}, \mathrm{stoch} \mid \mathbb{E}[C_{\max}]$: unrelated parallel machines, precedence constraints, stochastic processing times, minimizing expected makespan.

C.5. Speculation Policy (Temporal Pillar)

Definition C.7 (Speculation Policy)**.** *A* speculation policy $\pi : V \to \{0, 1\}$ *assigns each stage a binary decision:* $\pi(v_j) = 1$ *if* v_j *begins execution before its predecessor commits. Speculation is EV-positive for adjacent stages* (v_i, v_j) *when the success-to-failure odds ratio exceeds the rollback-to-benefit ratio:*

$$\frac{p_i}{q_i} > \frac{R}{B}$$

where $B = \mathbb{E}[\min(t_i, t_j)]$ *is the parallelism benefit and* $R = \mathbb{E}[d_i + t_i^{\text{retry}}]$ *is the rollback penalty.*

D. Extended Proofs

D.1. Full Proof of Spec-Code Convergence (Abstraction Pillar)

Theorem D.1 (Spec-Code Convergence, restated)**.** *For an algorithmically random program* P *(i.e.,* $K(P) \geq |P| - O(1)$*), any specification* S *such that* $P \models S$ *and* $\mathcal{G}(S, P) = O(1)$ *satisfies* $|S| \geq |P| - O(\log |P|)$*.*

Proof. Let P be algorithmically random, so $K(P) \geq |P| - O(1)$. Let S be a specification with $P \models S$ and $\mathcal{G}(S, P) = K(P \mid S) = O(1)$.

Step 1. From $\mathcal{G}(S, P) = O(1)$, there exists a constant-length program p with $U(p, S) = P$. Therefore S together with p constitutes a description of P:

$$K(P) \leq K(S) + |p| + O(\log(K(S) + |p|)) = K(S) + O(\log K(S))$$

The logarithmic overhead arises from delimiting S within the concatenated description.

Step 2. Since $K(P) \geq |P| - O(1)$ (algorithmic randomness):

$$|P| - O(1) \leq K(P) \leq K(S) + O(\log K(S))$$

Step 3. Since $K(S) \leq |S| + O(1)$ (any string can be described by itself plus constant overhead):

$$|P| \leq |S| + O(\log |S|) + O(1)$$

To bound $\log|S|$: from Step 1, $K(S) \geq K(P) - O(1)$, and combining with Step 2 gives $|S| + |P| = O(|S|)$, so $\log|S| = O(\log|P|)$.

Rearranging: $|S| \geq |P| - O(\log|P|)$.

Interpretation. For a 10,000-line program ($\approx 4\times10^6$ bits at 50 characters/line, 8 bits/character), the slack is $O(\log 4\times10^6) \approx 22$ bits, which is negligible. The specification must contain essentially all the information of the program. □

D.2. Markov Error Propagation Model (Reliability Pillar)

When errors propagate (a step can fail because its input was corrupted), the simple product law $R(n) = \prod p_i$ does not hold. We model the pipeline as a Markov chain on states $\{C, E\}$ (correct, erroneous) with transition matrix:

$$\mathrm{P} = \begin{pmatrix} p & 1-p \\ q & 1-q \end{pmatrix}$$

where p is the probability of correct output given correct input, and q is the probability of correct output given incorrect input.

Theorem D.2 (Pipeline Reliability Under Markov Error Propagation)**.** *Starting from state C, the probability of being in state C after n steps is:*

$$R_{\mathrm{Markov}}(n) = \pi_C + (1-\pi_C)(p+q-1)^n$$

where $\pi_C = q/(1-p+q)$ is the stationary probability of the correct state.

Proof. Diagonalize P. Its eigenvalues are $\lambda_1 = 1$ and $\lambda_2 = p+q-1$. The spectral decomposition gives $\mathrm{P}^n = \Pi + (p+q-1)^n(\mathrm{I} - \Pi)$ where Π is the matrix of stationary probabilities (each row equal to the stationary distribution (π_C, π_E)). The (C, C) entry of P^n is $\pi_C + (1-\pi_C)(p+q-1)^n$. □

Remark. *When $q = 0$ (errors never self-correct), the error state is absorbing: once an error is introduced, it persists through all subsequent steps. In this case $R_{\mathrm{Markov}}(n) = p^n$, recovering the series reliability product law. The Markov model with $q > 0$ is more optimistic because it allows error recovery at each step. For typical agent pipelines, q is small but positive (an LLM may "route around" a prior error through independent reasoning).*

D.3. NP-Hardness of Balanced Partition (Coordination Pillar)

Theorem D.3 (NP-Hardness of Balanced Partition, restated)**.** *The $(k, 1)$-balanced partition problem (partitioning a graph into k exactly equal parts minimizing edge cut) admits no polynomial-time approximation with any finite factor unless P = NP.*

Proof sketch (reduction from 3-Partition). Recall the 3-Partition problem: given a multiset $S = \{s_1, \ldots, s_{3m}\}$ of $3m$ positive integers with $\sum s_i = m \cdot T$ and $T/4 < s_i < T/2$ for all i, determine whether S can be partitioned into m triples each summing to exactly T. This problem is NP-hard in the strong sense (it remains NP-hard even when values are bounded by a polynomial in m).

Given a 3-Partition instance, construct a graph G as follows. For each element s_i, create a clique of s_i vertices. Connect all cliques with edges of weight M (a sufficiently large penalty). Set $k = m$ and require each part to contain exactly n/m vertices.

Any $(k, 1)$-balanced partition of G must split each clique into exactly one part (splitting a clique incurs internal cut cost exceeding M). Since $T/4 < s_i < T/2$, each triple summing to T yields a part of the correct size. Thus a zero-cost balanced partition exists if and only if the 3-Partition instance has a solution.

Since 3-Partition is strongly NP-hard, the balanced partition problem with exact balance ($\varepsilon = 0$) is NP-hard. When the balance constraint is relaxed ($\varepsilon > 0$), approximation becomes possible: $O(\log^{1.5} n)$ via Andreev and Räcke (2006) and $O(\sqrt{\log n})$ via Arora et al. (2009). □

This result has direct implications for agent task decomposition: partitioning a codebase into k equal-work assignments that minimize inter-agent coupling is computationally intractable in the worst case, motivating the use of heuristic algorithms (Kernighan-Lin, Fiduccia-Mattheyses, METIS).

D.4. Beta-Binomial Calibration for Heterogeneous Step Quality (Reliability Pillar)

To model heterogeneity in per-step success rates across different tasks, we place a Beta prior on the step success probability: $p \sim \mathrm{Beta}(\alpha, \beta)$.

Theorem D.4 (Beta-Binomial Pipeline Reliability)**.** *The marginal pipeline success probability under heterogeneous step quality is:*

$$R_{\mathrm{BB}}(n, \alpha, \beta) = \mathbb{E}[p^n] = \frac{B(\alpha + n, \beta)}{B(\alpha, \beta)} = \prod_{k=0}^{n-1} \frac{\alpha + k}{\alpha + \beta + k}$$

where $B(\cdot, \cdot)$ is the Beta function.

Proof. By definition, $\mathbb{E}[p^n] = \int_0^1 p^n \cdot \frac{p^{\alpha-1}(1-p)^{\beta-1}}{B(\alpha,\beta)}\, dp = \frac{1}{B(\alpha,\beta)} \int_0^1 p^{\alpha+n-1}(1-p)^{\beta-1}\, dp = \frac{B(\alpha+n,\beta)}{B(\alpha,\beta)}$. Using the recurrence $B(a+1, b) = B(a, b) \cdot a/(a+b)$ repeatedly yields the product form. □

Calibrated parameters from published benchmarks:

Table 24 | Beta-Binomial calibration: back-solved parameters from published benchmark success rates.

Benchmark	n	R_{obs}	μ	α	β	α+β
SWE-bench (METR)	10	0.50	0.93	12.5	0.95	13.5
Devin (2025)	12	0.67	0.97	28.4	0.88	29.3
RE-Bench	100	0.37	0.99	85.0	0.86	85.9
Generic agent	20	0.12	0.90	8.1	0.90	9.0

Higher concentration ($\alpha + \beta$) indicates more homogeneous task populations; lower concentration indicates high task-to-task variance. The Beta-Binomial model is strictly more pessimistic than the homogeneous model ($R_{\mathrm{BB}} \leq \mu^n$ by Jensen's inequality, since $p \mapsto p^n$ is convex for $n \geq 1$), capturing the intuition that a few very hard tasks dominate the failure distribution.

Remark (Calibration limitation)**.** *Each row of Table 24 is back-solved from a single observation (R_{obs} and μ), yielding a two-parameter fit with zero degrees of freedom. Any two-parameter model can fit a single data point exactly. The calibrated parameters should be treated as illustrative, demonstrating that the Beta-Binomial model can accommodate published benchmark results, not as validated estimates. Genuine calibration would require multiple observations per benchmark (e.g., reliability measured across different task difficulty strata), providing the degrees of freedom needed to test the Beta prior assumption.*

References

J.-R. Abrial. *Modeling in Event-B: System and Software Engineering*. Cambridge University Press, 2010.

A. Agarwal, D. Hsu, S. Kale, J. Langford, L. Li, and R. E. Schapire. Taming the monster: A fast and simple algorithm for contextual bandits. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pages 1638–1646, 2014.

S. Agrawal and N. Goyal. Analysis of Thompson sampling for the multi-armed bandit problem. In *Conference on Learning Theory*, pages 39.1–39.26. JMLR, 2012.

S. Agrawal and N. Goyal. Thompson sampling for contextual bandits with linear payoffs. In *International Conference on Machine Learning*, pages 127–135. PMLR, 2013.

S. Altman. Intelligence. OpenAI Blog, 2024. URL https://blog.samaltman.com/intelligence.

G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485, 1967.

J. P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Electronic Systems Division, Air Force Systems Command, 1972.

K. Andreev and H. Räcke. Balanced graph partitioning. In *Proceedings of the 18th Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 120–124. ACM, 2006.

Anthropic. Context windows and long-context language models. Anthropic Technical Report, 2025.

Anthropic. Agentic coding best practices. Technical report, Anthropic, 2026a. URL https://docs.anthropic.com/en/docs/build-with-claude/agentic-coding.

Anthropic. Evaluations for large language models. Technical report, Anthropic, 2026b.

S. Apel, D. Batory, C. Kästner, and G. Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, 2012.

Apollo Research. Frontier models are capable of in-context scheming, 2025.

S. Arora, S. Rao, and U. Vazirani. Expander flows, geometric embeddings and graph partitioning. *Journal of the ACM*, 56(2):1–37, 2009.

R.-J. Back. *Correctness Preserving Program Refinements*. Mathematisch Centrum, 1980.

A. Badanidiyuru, R. Kleinberg, and A. Slivkins. Bandits with knapsacks. In *Journal of the ACM*, volume 65, pages 1–55, 2018. Originally appeared at FOCS 2013.

Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983.

R. E. Barlow and F. Proschan. Mathematical theory of reliability. *SIAM Series in Applied Mathematics*, 1965.

R. E. Barlow and F. Proschan. *Statistical Theory of Reliability and Life Testing: Probability Models*. Holt, Rinehart and Winston, 1975.

B. Beizer. *Software Testing Techniques*. Van Nostrand Reinhold, 2nd edition, 1990.

D. E. Bell and L. J. LaPadula. Secure computer systems: Mathematical foundations. Technical Report MTR-2547, MITRE Corporation, 1973.

A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, and K. G. Larsen. Contracts for system design. In *Foundations and Trends in Electronic Design Automation*, volume 12, pages 124–400, 2018.

M. L. Berenson and D. M. Levine. *Basic Business Statistics: Concepts and Applications*. Prentice Hall, 6th edition, 1995.

P. A. Bernstein, V. Hadzilacos, and N. Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

S. Bian et al. On the limits of language model reasoning. *arXiv preprint*, 2025.

K. J. Biba. Integrity considerations for secure computer systems. Technical Report MTR-3153, MITRE Corporation, 1977.

Z. W. Birnbaum and S. C. Saunders. A stochastic model for the occurrence of main sequence events. *Annals of Mathematical Statistics*, 32(4):1035–1062, 1961.

B. Boehm and V. R. Basili. Software defect reduction top 10 list. *Computer*, 34(1):135–137, 2001.

B. W. Boehm. *Software Engineering Economics*. Prentice-Hall, 1981.

S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya, R. Sangwan, C. Seaman, K. Sullivan, and N. Zazworka. Managing technical debt in software-reliant systems. pages 47–52, 2010.

Y. Brun, R. Holmes, M. D. Ernst, and D. Notkin. Proactive detection of collaboration conflicts. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 168–178. ACM, 2011.

N. Buchbinder, M. Feldman, J. Naor, and R. Schwartz. A tight linear time (1/2)-approximation for unconstrained submodular maximization. *SIAM Journal on Computing*, 44(5), 2015.

M. Cemri et al. Why do multi-agent LLM systems fail? arXiv preprint, 2025.

G. J. Chaitin. On the length of programs for computing finite binary sequences. *Journal of the ACM*, 13(4): 547–569, 1966.

G. Chen, M. Liao, J. Li, and K. Fan. Don't always pick the highest-performing model: Error correlation in ensembles. *arXiv preprint arXiv:2602.08003*, 2026. Gaussian-copula model; within-family correlations 0.7–0.8.

L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

R. Chillarege, I. S. Bhandari, J. K. Chaar, M. J. Halliday, D. S. Moebus, B. K. Ray, and M.-Y. Wong. Orthogonal defect classification: A concept for in-process measurements. *IEEE Transactions on Software Engineering*, 18 (11):943–956, 1992.

Chroma. Context window utilization and recall degradation in frontier models, 2025.

Chroma Research. Context rot: How long contexts degrade agent performance. Technical report, Chroma, 2025.

A. Clark. Natural-born cyborgs: Minds, technologies, and the future of human intelligence. *Oxford University Press*, 2003.

CodeRabbit. AI code review quality analysis, 2025a.

CodeRabbit. State of AI vs human code generation. Technical report, CodeRabbit, 2025b.

J. Cohen. *Best Kept Secrets of Peer Code Review*. Smart Bear Software, Austin, TX, 2006.

H. Collins. *Tacit and Explicit Knowledge*. University of Chicago Press, 2010.

B. Congdon et al. Formal verification of AI-generated code. *arXiv preprint*, 2025.

M. E. Conway. How do committees invent? *Datamation*, 14(4):28–31, Apr. 1968.

P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis. In *Proc. 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, pages 238–252, 1977.

T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

W. Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, pages 29–30, 1992.

Cursor. Cursor: The AI-first code editor. https://cursor.com, 2026.

J. T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

A. Das and D. Kempe. Submodular meets spectral: Greedy algorithms for subset selection. *Proc. 28th ICML*, 2011.

O. Dekel, J. Ding, T. Koren, and Y. Peres. Bandits with switching costs: $t^{2/3}$ regret. *arXiv preprint arXiv:1310.2997*, 2014. STOC 2014.

J. Dekoninck, M. Baader, and M. Vechev. A unified approach to routing and cascading for LLMs. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. arXiv preprint arXiv:2410.10347.

D. E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

P. J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

Department of Defense. Trusted computer system evaluation criteria. Technical Report DoD 5200.28-STD, National Computer Security Center, 1985. The "Orange Book".

H. F. Dodge and H. G. Romig. A method of sampling inspection. *Bell System Technical Journal*, 8(4):613–631, 1929.

H. F. Dodge and H. G. Romig. *Sampling Inspection Tables: Single and Double Sampling*. John Wiley & Sons, 2nd edition, 1959.

Du et al. Context window scaling and retrieval in large language models, 2025.

Embrace The Red. CVE-2025-53773: GitHub Copilot remote code execution via prompt injection. embracethered.com, 2025.

M. R. Endsley and E. O. Kiris. The out-of-the-loop performance problem and level of control in automation. *Human Factors*, 37(2):381–394, 1995.

M. E. Fagan. Design and code inspections to reduce errors in program development. *IBM Systems Journal*, 15 (3):182–211, 1976.

J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, pages 313–324. ACM, 2014.

A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil, and D. Shasha. Making snapshot isolation serializable. *ACM Transactions on Database Systems*, 30(2):492–528, 2005.

N. E. Fenton and S. L. Pfleeger. *Software Metrics: A Rigorous and Practical Approach*. PWS Publishing, 2nd edition, 1999.

A. Fiat et al. Competitive paging algorithms. *Journal of Algorithms*, 12(4):685–699, 1991.

C. M. Fiduccia and R. M. Mattheyses. A linear-time heuristic for improving network partitions. In *Proceedings of the 19th Design Automation Conference (DAC)*, pages 175–181. IEEE, 1982.

M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.

FinOps Foundation. State of FinOps survey 2023. Technical report, FinOps Foundation, 2023.

M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

K. N. Fleming. Reliability analysis of a redundant sensor system. *Nuclear Engineering and Design*, 1975.

N. Ford, R. Parsons, and P. Kua. *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media, 2017.

N. Forsgren, J. Humble, and G. Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

C. M. Fortuin, P. W. Kasteleyn, and J. Ginibre. Correlation inequalities on some partially ordered sets. *Communications in Mathematical Physics*, 22(2):89–103, 1971.

R. P. Gabriel. The rise of "worse is better". 1989. Published in Lisp: Good News, Bad News, How to Win Big.

C. Gao, J. Zeng, M. R. Lyu, and I. King. Online app review analysis for identifying emerging issues. *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, 2017.

Y. Gao et al. Context length alone hurts: Rethinking long-context utilization in LLM-based code agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

G. Ghiotto, L. Murta, M. Barros, and A. van der Hoek. On the nature of merge conflicts: A study of 2,731 open source Java projects hosted by GitHub. In *IEEE Transactions on Software Engineering*, volume 46, pages 1112–1128, 2018.

GitClear. AI copilot code quality research 2025. https://www.gitclear.com/ai_assistant_code_quality_2025_research, 2025.

GitHub. Research: Quantifying GitHub Copilot's impact in the enterprise, 2024.

Gloaguen et al. AGENTS.md evaluation, 2026.

Gonzalez. A sufficiently detailed spec is code, 2026.

C. A. E. Goodhart. Problems of monetary management: The U.K. experience. *Papers in Monetary Economics*, 1, 1975.

Google Cloud. 2025 DORA report. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report, 2025.

N. J. Gunther. *Guerrilla Capacity Planning*. Springer, 2008.

N. Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.

Hariharan et al. Planning and verification in multi-step AI agent pipelines, 2025.

A. Harmel-Law. *Facilitating Software Architecture: Empowering Teams to Make Architectural Decisions*. O'Reilly Media, 2023.

M. A. Harrison, W. L. Ruzzo, and J. D. Ullman. Protection in operating systems. *Communications of the ACM*, 19(8):461–471, 1976.

A. E. Hassan. Predicting faults using the complexity of code changes. *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 78–88, 2009.

A. E. Hassan, D. Lin, B. Chen, J. Zhang, and H. Zhang. Agentic software engineering: Foundational pillars and a research roadmap. arXiv preprint arXiv:2509.06216, 2025.

M. Hassid, T. Lacroix, T. Bhatt, G. Synnaeve, A. Szlam, and E. Grave. The larger the better? improved LLM code-generation via budget reallocation. *arXiv preprint arXiv:2404.00725*, 2024. ICLR 2025.

J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury. *Feedback Control of Computing Systems*. John Wiley & Sons, 2004.

T. A. Henzinger, R. Jhala, R. Majumdar, and G. Sutre. Lazy abstraction. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 58–70. ACM, 2002.

M. Herlihy. Wait-free synchronization. *ACM Transactions on Programming Languages and Systems*, 13(1): 124–149, 1991.

A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu. On the naturalness of software. *Communications of the ACM*, 59(5):122–131, 2012.

C. A. R. Hoare and H. Jifeng. Unifying theories of programming. *Prentice Hall International Series in Computer Science*, 1998.

L. Hochstein and M. Lindvall. Combating architectural degeneration: A survey. volume 47, pages 643–656, 2005.

C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

HumanLayer. The skill issue in AI agent development, 2025.

E. Hutchins. *Cognition in the Wild*. MIT Press, 1995.

IBM and Ponemon Institute. Cost of a data breach report 2025. Technical report, IBM Security, 2025.

IEEE. IEEE standard classification for software anomalies. Technical Report IEEE Std 1044-2009, IEEE Computer Society, 2009.

G. Irving, P. Christiano, and D. Amodei. AI safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.

D. Jackson. *The Essence of Software*. Princeton University Press, 2021.

Z. Jelinski and P. B. Moranda. Software reliability research. *Statistical Computer Performance Evaluation*, pages 465–484, 1972.

JetBrains. Cutting through the noise: Smarter context management for LLM-powered agents. JetBrains Blog, 2025. Specific numerical claims from internal studies; details available on request.

W. S. Jevons. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal-Mines*. Macmillan and Co., London, 1865.

Ji et al. Dual-specification verification for LLM-generated code. *arXiv preprint*, 2026.

Jina et al. Submodular optimization for context window management. *arXiv preprint*, 2025.

C. Jones. Measuring programming quality and productivity. *IBM Systems Journal*, 17(1):39–63, 1983.

C. Jones. Software defect-removal efficiency. *IEEE Computer*, 29(4):94–95, 1996.

C. Jones. *The Economics of Software Quality*. Addison-Wesley, 2012.

D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. In *SIAM Journal on Scientific Computing*, volume 20, pages 359–392, 1998.

B. K. Kasi and A. Sarma. Cassandra: Proactive conflict minimization through optimized task scheduling. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 732–741. IEEE, 2013.

B. W. Kernighan and S. Lin. An efficient heuristic procedure for partitioning graphs. *Bell System Technical Journal*, 49(2):291–307, 1970.

J. Kim et al. Scaling LLM-based multi-agent systems for software engineering. arXiv preprint arXiv:2512.08296, 2025.

G. Klein et al. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.

L. Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.

M. Kleppmann. Formal verification of distributed systems. *Communications of the ACM*, 2025.

Koohestani et al. AgentGuard: Safety guardrails for LLM agents. *arXiv preprint*, 2025.

A. Krause and D. Golovin. Submodular function maximization. *Tractability*, 2008.

P. Kruchten, R. L. Nord, and I. Ozkaya. Technical debt: From metaphor to theory and practice. *IEEE Software*, 29(6):18–21, 2012.

H. W. Kuhn and A. W. Tucker. Nonlinear programming. *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, pages 481–492, 1951.

Y. Kuratov, A. Bulatov, P. Anokhin, D. Sorokin, A. Sorokin, and M. Burtsev. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. *arXiv preprint arXiv:2406.10149*, 2024.

S. K. Lahiri. Intent formalization: A grand challenge for reliable coding in the age of AI agents. arXiv preprint arXiv:2603.17150, 2026.

J. Lambek and P. J. Scott. *Introduction to Higher Order Categorical Logic*. Cambridge University Press, 1986.

L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21 (7):558–565, 1978.

B. W. Lampson. A note on the confinement problem. *Communications of the ACM*, 16(10):613–615, 1973.

B. Lanyado and O. Likhtenberg. Slopsquatting: How AI hallucinations enable supply chain attacks. Technical report, University of Texas at San Antonio, Virginia Tech, University of Oklahoma, 2025.

J. D. Lee and K. A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004.

J. R. Lee, S. O. Gharan, and L. Trevisan. Multiway spectral partitioning and higher-order Cheeger inequalities. *Journal of the ACM*, 61(6):1–30, 2012.

M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980.

S. Leroy. Why is it so hard to do my work? the challenge of attention residue when switching between work tasks. *Organizational Behavior and Human Decision Processes*, 109(2):168–181, 2009.

O. Lesenich, S. Apel, and C. Lengauer. Balancing precision and performance in structured merge. In *Automated Software Engineering*, volume 24, pages 367–396, 2017.

Li et al. A survey of large language model agents. *arXiv preprint*, 2024.

L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pages 661–670. ACM, 2010.

M. Li and P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 4th edition, 2019.

M. Li et al. The similarity metric. *IEEE Transactions on Information Theory*, 50(12):3250–3264, 2004.

Z. Li, P. Avgeriou, and P. Liang. A systematic mapping study on technical debt and its management. *Journal of Systems and Software*, 101:193–220, 2015.

Liang et al. Caching strategies for LLM agent pipelines, 2026.

Lifshitz et al. Multi-agent verification for LLM outputs. *arXiv preprint*, 2025.

H. Lin and J. Bilmes. A class of submodular functions for document summarization. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 510–520, 2011.

G. F. Lindsay and A. L. Bishop. Allocation of screening inspection effort: A dynamic programming approach. *Management Science*, 10(2):342–352, 1964.

S. B. Lipner. Non-discretionary controls for commercial applications. In *IEEE Symposium on Security and Privacy*, 1982.

G. Litt. Malleable software in the age of LLMs. https://www.geoffreylitt.com/2023/03/25/llm-end-user-programming, 2023.

B. Littlewood and L. Strigini. Validation of ultrahigh dependability for software-based systems. *Communications of the ACM*, 36(11):69–80, 1993.

B. Littlewood and L. Strigini. Software reliability and dependability: A roadmap. *Proc. Conference on The Future of Software Engineering*, pages 175–188, 2000.

N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. In *Transactions of the Association for Computational Linguistics*, 2024.

A. MacCormack, J. Rusnak, and C. Y. Baldwin. Exploring the duality between product and organizational architectures: A test of the mirroring hypothesis. In *Harvard Business School Working Paper*, 2008.

A. MacCormack, C. Baldwin, and J. Rusnak. Exploring the duality between product and organizational architectures: A test of the "mirroring" hypothesis. *Research Policy*, 41(8):1309–1324, 2012.

N. H. Mackworth. The breakdown of vigilance during prolonged visual search. *Quarterly Journal of Experimental Psychology*, 1(1):6–21, 1948.

A. Madaan, P. Aggarwal, P. Anand, S. Potdar, S. Mishra, P. Zhou, A. Gupta, D. Rajalingham, S. Muckatira, Mausam, and A. Grover. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

D. Manheim and S. Garrabrant. Categorizing variants of Goodhart's law. *arXiv preprint arXiv:1803.04585*, 2019.

G. Mark, D. Gudith, and U. Klocke. The cost of interrupted work: More speed and stress. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*, pages 107–110. ACM, 2008.

A. W. Marshall and I. Olkin. A multivariate exponential distribution. *Journal of the American Statistical Association*, 62(317):30–44, 1967.

R. C. Martin. The dependency inversion principle. *C++ Report*, 1994.

P. Martin-Löf. *Constructive Mathematics and Computer Programming*. North-Holland, 1979.

A. Mas-Colell, M. D. Whinston, and J. R. Green. *Microeconomic Theory*. Oxford University Press, 1995.

METR. Reward hacking in AI agent benchmarks, 2025.

METR. Measuring the impact of AI agents on software engineering. Technical report, Model Evaluation and Threat Research, 2026a.

METR. Do SWE-bench verified solutions actually get merged?, 2026b.

B. Meyer. *Applying "Design by Contract"*, volume 25. IEEE Computer, 1992.

M. S. Miller, K.-P. Yee, and J. Shapiro. Capability myths demolished. Technical Report SRL2003-02, Johns Hopkins University, 2003.

B. S. Mitchell and S. Mancoridis. On the automatic modularization of software systems using the Bunch tool. In *IEEE Transactions on Software Engineering*, volume 32, pages 193–208, 2006.

C. Morgan. *Programming from Specifications*. Prentice Hall, 2nd edition, 1994.

H. Mozannar and D. Sontag. Consistent estimators for learning to defer to an expert. In *International Conference on Machine Learning*, pages 7076–7087. PMLR, 2020.

J. Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957.

G. C. Murphy, D. Notkin, and K. Sullivan. Software reflexion models: Bridging the gap between source and high-level models. *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 18–28, 1995.

G. C. Murphy, D. Notkin, and K. J. Sullivan. Software reflexion models: Bridging the gap between design and implementation. *IEEE Transactions on Software Engineering*, 27(4):364–380, 2001.

J. D. Musa and K. Okumoto. A logarithmic Poisson execution time model for software reliability measurement. *Proceedings of the 7th International Conference on Software Engineering*, pages 230–238, 1984.

J. D. Musa, A. Iannino, and K. Okumoto. *Software Reliability: Measurement, Prediction, Application*. McGraw-Hill, 1987.

A. C. Myers and B. Liskov. A decentralized model for information flow control. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles*, pages 129–142, 1997.

N. Nagappan, B. Murphy, and V. Basili. The influence of organizational structure on software quality. In *Proceedings of the 30th International Conference on Software Engineering (ICSE)*, pages 521–530. ACM, 2008.

P. Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985.

R. B. Nelsen. *An Introduction to Copulas*. Springer, 2 edition, 2006.

G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming*, 14:265–294, 1978.

C. Newcombe et al. How Amazon web services uses formal methods. *Communications of the ACM*, 58(4): 66–73, 2015.

Q. H. Nguyen et al. MetaLLM: A high-performant and cost-efficient dynamic framework for wrapping LLMs. *arXiv preprint arXiv:2407.10834*, 2024.

NIST CAISI. AI agent benchmark cheating analysis, 2025.

M. T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2011.

I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (ATC)*, pages 305–319. USENIX Association, 2014.

OpenAI. Codex: Autonomous software engineering agent, 2025.

A. Osmani. Patterns for AI-assisted software engineering. Web publication, 2025.

R. Parasuraman, T. B. Sheridan, and C. D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans*, 30(3):286–297, 2000.

L. Passos, R. Queiroz, M. Mukelabai, T. Berger, S. Apel, K. Czarnecki, and J. Padilla. A study of feature scattering in the linux kernel. *IEEE Transactions on Software Engineering*, 47(1):146–164, 2021.

V. Perchet, P. Rigollet, S. Chassang, and E. Snowberg. Batched bandit problems. *The Annals of Statistics*, 44 (2):660–681, 2016.

D. E. Perry and A. L. Wolf. Foundations for the study of software architecture. volume 17, pages 40–52, 1992.

M. L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5th edition, 2016.

M. Polanyi. *The Tacit Dimension*. Doubleday, 1966.

Qodo. Qodo: AI-powered code quality platform. https://www.qodo.ai, 2025.

J. Reason. *Human Error*. Cambridge University Press, 1990.

R. T. Rockafellar. *Convex Analysis*. Princeton University Press, 1970.

J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

J. E. See, S. R. Howe, J. S. Warm, and W. N. Dember. Meta-analysis of the sensitivity decrement in vigilance. *Psychological Bulletin*, 117(2):230–249, 1995.

C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, July 1948.

Shen et al. Intellimerge: A refactoring-aware software merging technique. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, volume 3, pages 1–28, 2019.

F. Shull, V. Basili, B. Boehm, A. W. Brown, P. Costa, M. Lindvall, D. Port, I. Rus, R. Tesoriero, and M. Zelkowitz. What we have learned about fighting defects. pages 249–258, 2002.

M. Skelton and M. Pais. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press, 2019.

A. Sklar. Fonctions de répartition à n dimensions et leurs marges. *Publications de l'Institut de Statistique de l'Université de Paris*, 8:229–231, 1959.

D. D. Sleator and R. E. Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985.

SonarSource. The architecture gap: Why your code becomes hard to change. https://www.sonarsource.com/blog/the-architecture-gap-why-your-code-becomes-hard-to-change/, 2026.

S. Sorrell. Jevons' paradox revisited: The evidence for backfire from improved energy efficiency. *Energy Policy*, 37(4):1456–1469, 2009.

M. Sousa, I. Dillig, and S. K. Lahiri. Verified three-way program merge. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, volume 2, pages 1–29, 2018.

Spotify Engineering. Spotify engineering practices for AI-assisted development. Spotify Engineering Blog, 2025a.

Spotify Engineering. HONK: Spotify's approach to AI code quality. Spotify Engineering Blog, 2025b.

L. A. Suchman. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, 1987.

R. Sutton. The bitter lesson. http://www.incompleteideas.net/IncIdeas/BitterLesson.html, Mar. 2019.

M. Suzgun et al. Meta-prompting: Enhancing language models with task-agnostic scaffolding, 2024.

SWE Efficiency Research. Software engineering efficiency with AI assistants. Industry benchmark report, 2025.

W. R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.

N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, pages 368–377, 2000.

UK National Cyber Security Centre. Prompt injection is not SQL injection (it may be worse). Technical report, NCSC, 2023.

U.S. Department of Defense. MIL-STD-105E: Sampling procedures and tables for inspection by attributes. Technical report, U.S. Department of Defense, 1989.

M. Vaccaro, H. Alhoori, et al. When combinations of humans and AI are useful: A systematic review and meta-analysis. *Nature Human Behaviour*, 2024.

J. A. Van Mieghem. Dynamic scheduling with convex delay costs: The generalized $c\mu$ rule. *The Annals of Applied Probability*, 5(3):809–833, 1995.

Various Authors. On the trustworthiness of LLM-as-judge evaluations. arXiv preprint, 2024.

VeriCoding. VeriCoding: Formal verification for AI-generated code. https://vericoding.com, 2025.

P. Viola and M. Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 511–518, 2001.

P. Viola and M. J. Jones. Robust real-time face detection. In *International Journal of Computer Vision*, volume 57, pages 137–154, 2004.

P. M. B. Vitányi. Shannon versus Kolmogorov. *IEEE Information Theory Society Newsletter*, 2004.

P. Wadler. Propositions as types. *Communications of the ACM*, 58(12):75–84, 2015.

A. Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.

A. Wald and J. Wolfowitz. Optimum character of the sequential probability ratio test. volume 19, pages 326–339, 1948.

Wang et al. AgentSpec: Specification and verification of AI agent behavior, 2026.

J. S. Warm, R. Parasuraman, and G. Matthews. Vigilance requires hard mental work and is stressful. *Human Factors*, 50(3):433–441, 2008.

S. Willison. The lethal trifecta for AI agents. simonwillison.net, 2024.

L. Wittgenstein. *Philosophical Investigations*. Macmillan, 1953.

Y. Wolf, N. Wies, Y. Levine, and A. Shashua. Fundamental limitations of alignment in large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2304.11082.

J. M. Wolfe, T. S. Horowitz, and N. M. Kenner. Low target prevalence is a stubborn source of errors in visual search tasks. *Journal of Experimental Psychology: General*, 134(4):623–638, 2005.

Xiao et al. Efficient context window management for large language models, 2024.

R. M. Yerkes and J. D. Dodson. The relation of strength of stimulus to rapidity of habit-formation. *Journal of Comparative Neurology and Psychology*, 18(5):459–482, 1908.

J. W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17 (9):530–531, 1974.

Zhang et al. Adaptive prompt caching for LLM agent pipelines, 2025a.

Y. Zhang, Z. Feng, and Z. Zhou. When routing collapses: Understanding systematic failures in LLM model selection. *arXiv preprint arXiv:2602.03478*, 2026. 94.9% of queries in small-margin regime; routers default to strongest model.

Y. Zhang et al. Agentic plan caching for software engineering tasks. *arXiv preprint arXiv:2506.14852*, 2025b.

Zhou et al. Speculative execution strategies for LLM agent pipelines. *arXiv preprint*, 2025.

Zhu et al. ConGra: Context-grounded code generation with graph-based retrieval. In *Proceedings of NeurIPS*, 2024.